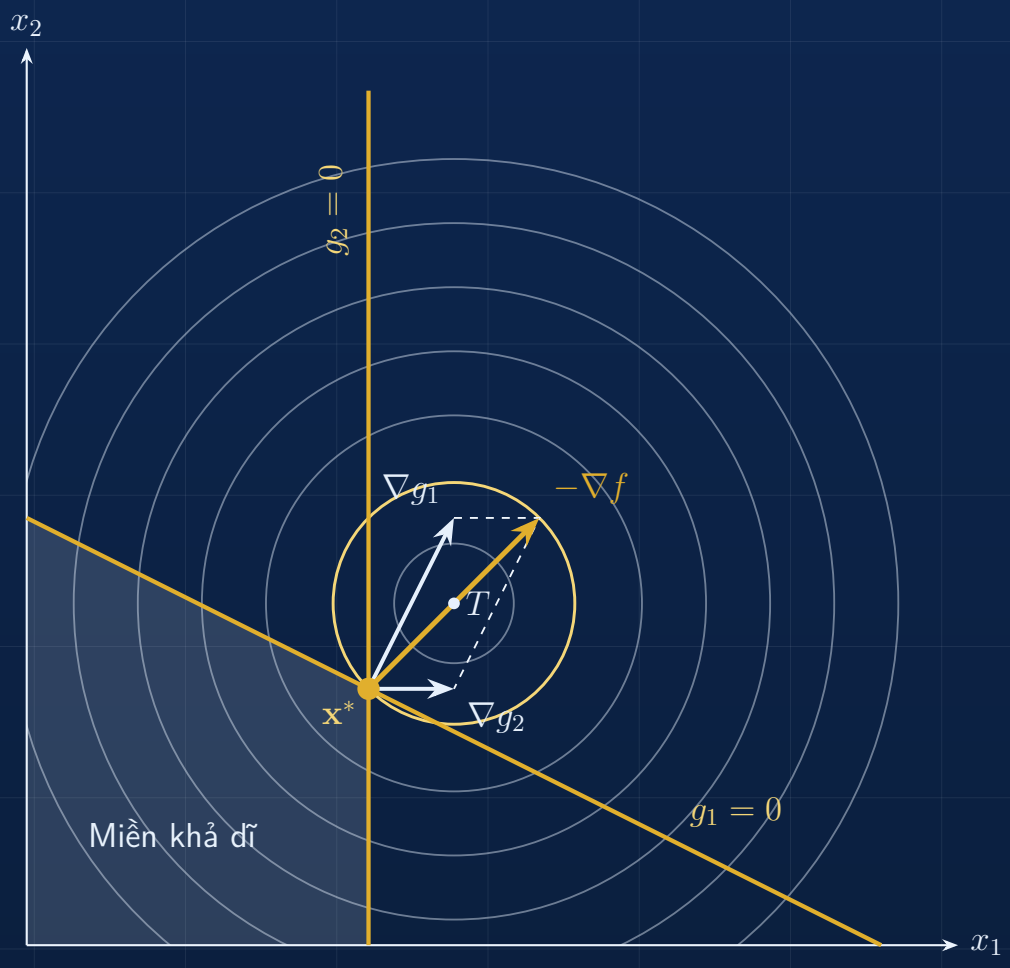
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG TRÌNH VÀ MÔI TRƯỜNG

TỐI ƯU KẾT CẤU

LÝ THUYẾT VÀ ỨNG DỤNG TRONG CÔNG TRÌNH CẦU



GS.TS. BÙI TIẾN THÀNH
ThS. TRƯƠNG THÀNH CHUNG

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG TRÌNH VÀ MÔI TRƯỜNG

TỐI ƯU KẾT CẤU

LÝ THUYẾT VÀ ỨNG DỤNG TRONG CÔNG TRÌNH CẦU

GS.TS. BÙI TIẾN THÀNH
ThS. TRƯƠNG THÀNH CHUNG

MỤC LỤC

LỜI NÓI ĐẦU 7

CHƯƠNG 1. NHẬP MÔN TỐI ƯU KẾT CẤU 9

1.1. Giới thiệu 9

1.2. Phát biểu tổng quát của bài toán tối ưu kết cấu 10

1.3. Công thức đồng thời và công thức lồng ghép của bài toán tối ưu kết cấu . . 11

1.4. Phân loại bài toán tối ưu 11

1.4.1. Tối ưu hóa kích thước 11

1.4.2. Tối ưu hóa hình dạng 12

1.4.3. Tối ưu hóa cấu trúc 12

1.5. Bài tập 14

CHƯƠNG 2. TỐI ƯU HÓA KẾT CẤU BẰNG PHƯƠNG PHÁP ĐỒ THỊ. 15

2.1. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất 15

2.2. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và ổn định . 16

2.3. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và chuyển vị 18

2.4. Tối ưu trọng lượng dầm công-xôn chịu ràng buộc chuyển vị 20

2.5. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc ứng suất 22

2.6. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc độ cứng 30

2.7. Bài tập 32

CHƯƠNG 3. CƠ SỞ TOÁN HỌC QUY HOẠCH LỒI 35

3.1. Cực trị địa phương và cực trị toàn cục 35

3.2. Tính lồi 36

3.2.1. Định nghĩa tập lồi và hàm lồi 36

3.2.2. Tính chất của bài toán tối ưu lồi 37

3.2.3. Kiểm tra tính lồi của hàm khả vi 37

3.3. Điều kiện tối ưu KKT của bài toán tối ưu hóa có ràng buộc 38

3.4. Đối ngẫu Lagrange 44

3.5. Bài tập 49

CHƯƠNG 4. CÁC THUẬT TOÁN XẤP XỈ LỒI TƯỜNG MINH TUẦN TỰ 54

4.1. Quy trình giải tổng quát 54

4.2. Quy hoạch tuyến tính tuần tự SLP 54

4.3. Quy hoạch toàn phương tuần tự SQP 55

4.4. Tuyến tính hóa lồi CONLIN 55

4.5. Phương pháp tiệm cận động MMA 56

4.6. Bài tập 59

CHƯƠNG 5. TỐI ƯU ĐỘ CỨNG GIÀN 61

5.1. Thiết lập bài toán tối ưu độ cứng giàn 61

5.1.1. Công thức lồng ghép 61

5.1.2. Phân tích độ nhạy 62

5.2. Tối ưu độ cứng giàn sử dụng phương pháp tiêu chuẩn tối ưu 63

5.2.1. Thiết kế đồng ứng suất 63

5.2.2. Phương pháp tiêu chuẩn tối ưu OC 64

5.3. Tối ưu độ cứng giàn sử dụng phương pháp tiệm cận động 67

5.4. Bài tập 67

CHƯƠNG 6. TỐI ƯU CẤU TRÚC 72
6.1. Phát biểu bài toán tối ưu cấu trúc 72
6.2. Thuật toán tối ưu cấu trúc SIMP 73
6.3. Bài tập 76
CHƯƠNG 7. TỐI ƯU HÓA ĐA MỤC TIÊU 77
7.1. Giới thiệu 77
7.2. Phát biểu bài toán đa mục tiêu 78
7.3. Khái niệm tối ưu Pareto 79
7.4. Một số cách tiếp cận trong tối ưu hóa đa mục tiêu 81
7.4.1. Phương pháp tổng trọng số (weighted sum method) 81
7.4.2. Phương pháp ràng buộc epsilon (epsilon-constraint method) 81
7.4.3. Phương pháp tiến hóa đa mục tiêu 81
7.5. Ví dụ minh họa 82
7.6. Bài tập 85
CHƯƠNG 8. THUẬT TOÁN TỐI ƯU METAHEURISTIC 87
8.1. Giới thiệu 87
8.2. Thuật toán di truyền 88
8.2.1. Nguyên lý cơ bản 88
8.2.2. Quy trình vận hành 88
8.3. Thuật toán luyện kim mô phỏng 89
8.3.1. Nguyên lý cơ bản 89
8.3.2. Quy trình vận hành 89
8.4. Ví dụ minh họa 90
8.5. Bài tập 95
CHƯƠNG 9. ỨNG DỤNG TỐI ƯU KẾT CẤU TRONG CÔNG TRÌNH CẦU 99
9.1. Bài toán tối ưu trong thiết kế và khai thác công trình cầu 99
9.1.1. Đặc điểm của bài toán tối ưu công trình cầu 99
9.1.2. Các ví dụ trong chương và phương pháp áp dụng 100
9.2. Tối ưu dầm thép chữ I liên hợp theo TCVN 11823:2017 100
9.2.1. Mô tả bài toán và số liệu 100
9.2.2. Phát biểu bài toán tối ưu 101
9.2.3. Kiểm chứng mô hình tính toán 102
9.2.4. Lời giải bằng SQP và thuật toán tiến hóa 103
9.2.5. Phân tích nghiệm tối ưu 105
9.3. Tối ưu bộ giảm chấn khối lượng cho cầu đường sắt cao tốc 106
9.3.1. Cộng hưởng của cầu khi đoàn tàu chạy qua và yêu cầu của tiêu chuẩn 107
9.3.2. Mô hình hệ cầu – bộ giảm chấn chịu đoàn tàu HSLM-A 107
9.3.3. Phản ứng của cầu khi chưa có bộ giảm chấn 109
9.3.4. Tối ưu tham số bộ giảm chấn 109
9.3.5. Đánh đổi giữa khối lượng bộ giảm chấn và gia tốc mặt cầu 111
9.4. Hiệu chỉnh mô hình cầu giàn thép đường sắt Nam Ô từ dữ liệu đo 113
9.4.1. Cầu Nam Ô và dữ liệu đo dao động 113
9.4.2. Mô hình phần tử hữu hạn ban đầu 113
9.4.3. Hiệu chỉnh mô hình như một bài toán tối ưu 114
9.4.4. Kết quả hiệu chỉnh và thảo luận 116

9.5. Nhận xét chung 118
9.6. Bài tập 119
TÀI LIỆU THAM KHẢO 120
PHỤ LỤC CODE 124

LỜI NÓI ĐẦU

Cuốn sách này trình bày các cơ sở và phương pháp tối ưu hóa kết cấu trong kỹ thuật xây dựng, từ các bài toán minh họa bằng phương pháp đồ thị đến các thuật toán tối ưu hiện đại. Nội dung được biên soạn theo hướng ngắn gọn, hệ thống và thực hành, giúp người đọc nắm được mô hình toán học, điều kiện ràng buộc và cách triển khai tính toán cho các dạng kết cấu thường gặp.

Trong thiết kế kết cấu truyền thống, kỹ sư thường bắt đầu từ một phương án sơ bộ, sau đó kiểm tra và hiệu chỉnh nhiều lần để thỏa mãn các yêu cầu về độ bền, độ cứng, ổn định và khả năng thi công. Cách tiếp cận này phụ thuộc nhiều vào kinh nghiệm và thường chỉ cho một phương án chấp nhận được, chứ chưa chắc là phương án hiệu quả nhất. Tối ưu hóa kết cấu cung cấp một khuôn khổ toán học và tính toán để biến quá trình hiệu chỉnh đó thành một bài toán có mục tiêu rõ ràng, có ràng buộc cụ thể và có tiêu chí đánh giá định lượng.

Mục tiêu của cuốn sách là giới thiệu các khái niệm nền tảng và các phương pháp tính thường dùng trong tối ưu hóa kết cấu, đồng thời liên hệ trực tiếp với các bài toán cơ học kết cấu quen thuộc như hệ giàn, dầm, khung và bài toán tối ưu cấu trúc. Tài liệu không chỉ trình bày công thức toán học, mà còn nhấn mạnh ý nghĩa kỹ thuật của các biến thiết kế, hàm mục tiêu, ràng buộc và nghiệm tối ưu. Nhiều ví dụ được xây dựng theo hướng từng bước để người đọc thấy rõ cách chuyển từ mô hình kết cấu sang mô hình tối ưu, cách diễn giải nghiệm và cách kiểm tra điều kiện tối ưu.

Cuốn sách hướng đến sinh viên ngành kỹ thuật xây dựng, học viên cao học, kỹ sư thiết kế và những người quan tâm đến ứng dụng tối ưu hóa trong cơ học kết cấu. Người đọc nên có kiến thức cơ bản về cơ học kết cấu, sức bền vật liệu, đại số tuyến tính và giải tích. Tuy nhiên, các khái niệm tối ưu hóa quan trọng đều được nhắc lại ở mức cần thiết để tài liệu có thể dùng như một phần nhập môn cho chủ đề thiết kế tối ưu.

Cấu trúc cuốn sách được sắp xếp theo trình tự từ trực quan đến tổng quát. Chương 1 giới thiệu bức tranh tổng quan về tối ưu hóa kết cấu, các loại bài toán tối ưu thường gặp và vai trò của tối ưu hóa trong quy trình thiết kế. Chương 2 trình bày phương pháp đồ thị thông qua các bài toán giàn và dầm đơn giản, giúp người đọc hình dung miền khả dĩ, đường ràng buộc và vị trí nghiệm tối ưu. Chương 3 hệ thống hóa các cơ sở toán học của quy hoạch lồi, điều kiện tối ưu KKT và phương pháp đối ngẫu Lagrange. Chương 4 giới thiệu các thuật toán xấp xỉ lồi tường minh tuần tự như SLP, SQP, CONLIN và MMA, là nền tảng cho nhiều phương pháp tối ưu kết cấu quy mô lớn.

Chương 5 tập trung vào bài toán tối ưu độ cứng của hệ giàn, trong đó phương pháp tiêu chuẩn tối ưu và phương pháp tiệm cận động được trình bày cùng ví dụ tính toán và các bài tập trên những hệ giàn chuẩn. Chương 6 mở rộng sang tối ưu cấu trúc (tối ưu topo), đặc biệt là phương pháp SIMP và các vấn đề thực tế như lọc độ nhạy, hiện tượng ô cờ và cập nhật biến thiết kế. Chương 7 giới thiệu tối ưu hóa đa mục tiêu, khái niệm Pareto, phương pháp tổng trọng số, phương pháp ràng buộc epsilon và cách diễn

giải đánh đổi giữa các tiêu chí thiết kế. Chương 8 trình bày nguyên lý của các thuật toán tối ưu metaheuristic, tập trung vào thuật toán di truyền và thuật toán luyện kim mô phỏng; các ví dụ tối ưu dầm công-xôn và khung thép được giải bằng thuật toán luyện kim mô phỏng, kèm các bài tập kết cấu có nhiều biến và ràng buộc phức tạp. Chương 9 vận dụng các phương pháp trên vào ba bài toán của công trình cầu có số liệu công bố hoặc theo tiêu chuẩn hiện hành: tối ưu dầm thép chữ I liên hợp theo TCVN 11823:2017, tối ưu bộ giảm chấn khối lượng cho cầu đường sắt cao tốc dưới các đoàn tàu chuẩn HSLM-A, và hiệu chỉnh mô hình phần tử hữu hạn cầu giàn thép đường sắt Nam Ô từ dữ liệu đo dao động. Cuối mỗi chương đều có phần bài tập để người đọc tự kiểm tra và mở rộng nội dung đã học.

Trong quá trình biên soạn, nhóm tác giả tham khảo chủ yếu các giáo trình và chuyên khảo [7, 9, 10, 24, 31, 35, 40] cùng các tài liệu tiếng Việt [2, 3]. Trình tự trình bày cùng nhiều ví dụ và bài tập của các Chương 1–5 bám theo giáo trình của Christensen và Klarbring [10]; nguồn gốc của các bài toán giàn chuẩn ở Chương 5 và Chương 8 được ghi ngay tại phần bài tập tương ứng.

Mã nguồn đi kèm được cung cấp nhằm hỗ trợ người đọc tự kiểm tra lại các ví dụ và mở rộng thành các chương trình tính riêng. Các đoạn mã không nhằm thay thế phần lý thuyết, mà đóng vai trò minh họa cách triển khai thuật toán, cách tổ chức dữ liệu kết cấu và cách biểu diễn kết quả. Mã nguồn của các ví dụ và hình vẽ được in trong phần Phụ lục code ở cuối sách; riêng các tệp dữ liệu giàn dài chỉ được in phần đầu và phần cuối, còn hai hàm MMA của Svanberg [46] dùng trong Chương 5 không được in lại. Toàn bộ mã nguồn, các tệp dữ liệu và bản PDF của sách được cập nhật tại `https://github.com/chungntu/structural-optimization-book-code`. Người đọc được khuyến khích thay đổi tham số, điều kiện ràng buộc hoặc mô hình kết cấu để hiểu rõ hơn ảnh hưởng của từng giả thiết đến nghiệm tối ưu.

Mặc dù đã cố gắng trình bày nội dung một cách mạch lạc và kiểm tra các công thức, hình vẽ cũng như mã nguồn, cuốn sách khó tránh khỏi thiếu sót. Các góp ý của người đọc về nội dung, ký hiệu, ví dụ và cách trình bày sẽ rất hữu ích cho việc hoàn thiện tài liệu trong các lần biên soạn tiếp theo.

Nhóm tác giả

Hà Nội, tháng 10 năm 2026

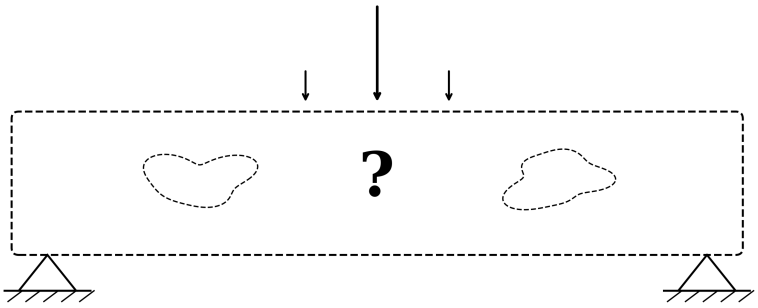
CHƯƠNG 1

NHẬP MÔN TỐI ƯU KẾT CẤU

Chương này cung cấp cái nhìn tổng quan về các khái niệm và thuật ngữ nền tảng trong tối ưu hóa kết cấu, từ cách mô tả bài toán thiết kế, biến thiết kế, hàm mục tiêu và ràng buộc đến các dạng tối ưu thường gặp trong cơ học kết cấu. Nội dung chương là cơ sở để người đọc hiểu cách một yêu cầu thiết kế kỹ thuật được chuyển thành mô hình toán học có thể giải bằng các phương pháp tối ưu hóa ở các chương sau.

1.1. Giới thiệu

Một kết cấu (structure) có thể hiểu đơn giản là một hệ thống vật liệu được bố trí nhằm mục đích chịu tải trọng. Theo đó, tối ưu hóa kết cấu (Structural Optimization – SO) là quá trình sắp xếp vật liệu hoặc tinh chỉnh hệ thống đó sao cho khả năng chịu tải đạt mức "tốt nhất" (Hình 1.1; mã nguồn vẽ hình tại Code 1.1 trên GitHub).



Hình 1.1. Bài toán tối ưu hóa kết cấu.

Khái niệm "tốt nhất" ở đây cần được định lượng cụ thể thông qua một hàm mục tiêu (objective function), chẳng hạn như tối thiểu hóa trọng lượng để kết cấu nhẹ nhất, tối đa hóa độ cứng để kết cấu ít biến dạng nhất hoặc tối đa hóa độ bền để kết cấu ổn định và an toàn nhất. Tuy nhiên, quá trình tìm kiếm phương án tối ưu này không diễn ra tự do mà bị giới hạn bởi các ràng buộc (constraints) về ứng suất, chuyển vị hoặc kích thước hình học nhằm đảm bảo tính khả thi và an toàn cho công trình.

Tối ưu hóa là một mắt xích quan trọng trong quy trình thiết kế tổng thể gồm 4 bước [10]:

- Chức năng (function): xác định mục đích sử dụng của sản phẩm. Ví dụ với một cây cầu: cần xác định chiều dài, bề rộng, số lượng làn xe và các loại tải trọng dự kiến tác động lên nó.
- Thiết kế ý tưởng (conceptual design): lựa chọn giải pháp kết cấu sơ bộ. Đối với cây cầu, kỹ sư cần quyết định nên sử dụng hệ kết cấu nào: cầu giàn (truss), cầu

treo (suspension) hay cầu vòm (arch)...
- Tối ưu hóa (optimization): từ phương án ý tưởng đã chọn và các giới hạn về công năng, tiến hành tinh chỉnh để sản phẩm đạt hiệu quả cao nhất. Với cầu, mục tiêu thường là giảm thiểu chi phí, gián tiếp thực hiện qua việc sử dụng lượng vật liệu tối thiểu mà vẫn đảm bảo an toàn.
- Chi tiết (details): hoàn thiện thiết kế dựa trên các yếu tố phi kỹ thuật như thị trường, xã hội hoặc thẩm mỹ (ví dụ: lựa chọn màu sắc hoặc các chi tiết trang trí).

1.2. Phát biểu tổng quát của bài toán tối ưu kết cấu

Trong mọi bài toán tối ưu hóa kết cấu, ba thành phần cơ bản sau đây luôn được xác định:
- Hàm mục tiêu f: là hàm dùng để đánh giá mức độ "tốt" của một phương án thiết kế. Với mỗi thiết kế khả dĩ, hàm mục tiêu trả về một giá trị số phản ánh chất lượng của thiết kế đó. Thông thường, bài toán được xây dựng theo dạng cực tiểu hóa, trong đó giá trị nhỏ hơn tương ứng với thiết kế tốt hơn. Trong tối ưu kết cấu, hàm mục tiêu thường biểu diễn khối lượng, chuyển vị theo một phương nhất định, năng lượng biến dạng hoặc chi phí sản xuất.
- Biến thiết kế x: là hàm hoặc vector mô tả các đặc trưng của thiết kế và có thể thay đổi trong quá trình tối ưu hóa. Biến thiết kế có thể đại diện cho hình học của kết cấu hoặc cho sự lựa chọn vật liệu. Khi mô tả hình học, biến thiết kế có thể liên quan đến các tham số nội suy phức tạp của hình dạng, hoặc đơn giản là diện tích tiết diện của một thanh hay chiều dày của một tấm.
- Biến trạng thái y: với một kết cấu xác định, tương ứng với một thiết kế x cho trước, biến trạng thái là hàm hoặc vector biểu diễn phản ứng của kết cấu. Trong cơ học kết cấu, phản ứng thường bao gồm chuyển vị, ứng suất, biến dạng hoặc nội lực.

Một bài toán tối ưu hóa kết cấu tổng quát có thể được phát biểu như sau:

$$(\mathbb{SO})\begin{cases}\text{Cực tiểu hóa } f(x,y) \text{ theo các biến } x \text{ và } y,\\ \text{Thỏa mãn các điều kiện} \begin{cases}\text{Ràng buộc về ứng xử theo } y,\\ \text{Ràng buộc về thiết kế theo } x,\\ \text{Ràng buộc cân bằng.}\end{cases}\end{cases} \tag{1.1}$$

Ba nhóm ràng buộc thường gặp trong bài toán tối ưu kết cấu bao gồm:
- Ràng buộc ứng xử (behavioral constraints): là các ràng buộc đặt lên biến trạng thái y. Chúng thường được biểu diễn dưới dạng $g(y) \leqslant 0$, trong đó g là hàm mô tả một đại lượng phản ứng của kết cấu, chẳng hạn như chuyển vị tại một vị trí theo một phương xác định.
- Ràng buộc thiết kế (design constraints): là các ràng buộc liên quan trực tiếp đến biến thiết kế x, ví dụ như giới hạn hình học, giới hạn tiết diện hoặc giới hạn vật liệu.
- Ràng buộc cân bằng (equilibrium constraint): trong các bài toán rời rạc tự nhiên hoặc các bài toán đã được rời rạc hóa tuyến tính, điều kiện cân bằng của kết cấu có dạng $\mathbf{K}(x)\mathbf{u} = \mathbf{F}(x)$, trong đó $\mathbf{K}(x)$ là ma trận độ cứng của kết cấu, $\mathbf{u}$ là vector chuyển vị, và $\mathbf{F}(x)$ là vector ngoại lực.

1.3. Công thức đồng thời và công thức lồng ghép của bài toán tối ưu kết cấu

Trong công thức (1.1), các biến thiết kế x và biến trạng thái y được xem là độc lập. Cách phát biểu bài toán theo dạng này thường được gọi là công thức đồng thời (simultaneous formulation – sf), trong đó điều kiện cân bằng được giải đồng thời với bài toán tối ưu hóa.

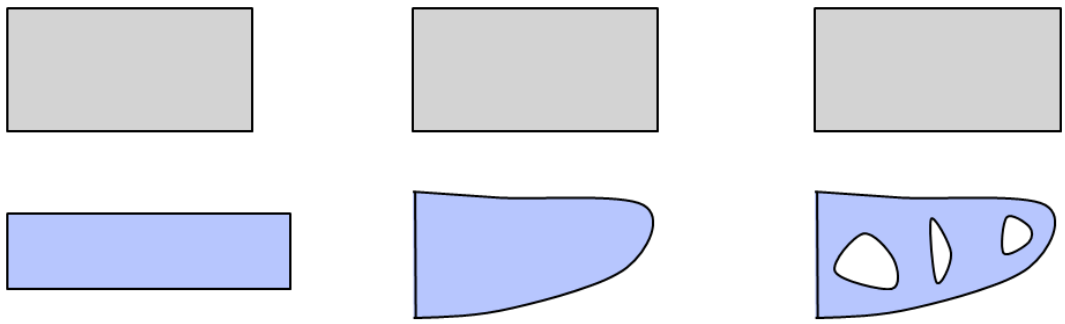
Tuy nhiên, trong nhiều trường hợp thực tế, bài toán trạng thái có nghiệm duy nhất đối với mỗi thiết kế x. Chẳng hạn, nếu ma trận độ cứng $\mathbf{K}(x)$ là khả nghịch với mọi x, ta có thể xác định chuyển vị dưới dạng hàm của thiết kế: $\mathbf{u} = \mathbf{u}(x) = \mathbf{K}(x)^{-1}\mathbf{F}(x)$. Khi đó, bằng cách xem $\mathbf{u}(x)$ như một hàm đã biết, điều kiện cân bằng có thể được loại bỏ khỏi bài toán ($\mathbb{SO}$), và hàm này được thay thế cho biến trạng thái. Bài toán tối ưu khi đó có thể được viết lại dưới dạng:

$$(\mathbb{SO})_{\text{nf}} \quad \begin{cases} \min\limits_{x} f(x, \mathbf{u}(x)) \\ \text{s.t. } g(x, \mathbf{u}(x)) \leqslant 0, \end{cases} \tag{1.2}$$

trong đó ký hiệu "s.t." (subject to) biểu thị các ràng buộc, và giả thiết rằng toàn bộ các ràng buộc ứng xử và ràng buộc thiết kế đều có thể được viết gộp dưới dạng $g(x, \mathbf{u}) \leqslant 0$. Cách phát biểu này được gọi là công thức lồng ghép (nested formulation – nf).

1.4. Phân loại bài toán tối ưu

Các bài toán tối ưu hóa kết cấu có thể được phân thành ba nhóm chính: tối ưu hóa kích thước (sizing optimization), tối ưu hóa hình dạng (shape optimization) và tối ưu hóa cấu trúc hay tối ưu topo (topology optimization) [9, 10], như minh họa trên Hình 1.2.



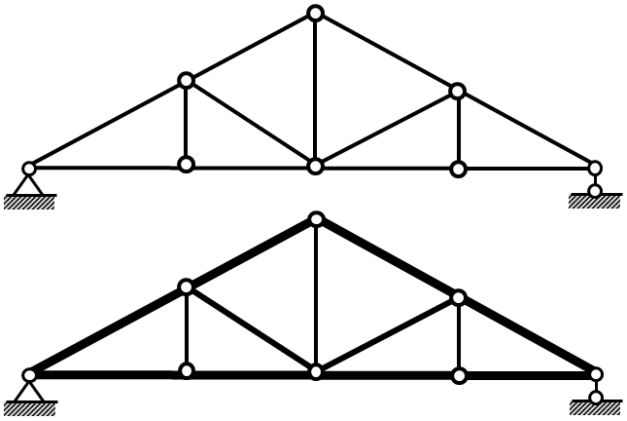
(a) Tối ưu kích thước (b) Tối ưu hình dạng (c) Tối ưu cấu trúc

Hình 1.2. Ba cấp độ của bài toán tìm bố cục tối ưu (lay-out optimization).

1.4.1. Tối ưu hóa kích thước

Tối ưu hóa kích thước là dạng bài toán phổ biến nhất trong thiết kế kết cấu, trong đó biến thiết kế đại diện cho các thông số kích thước đặc trưng của cấu kiện, ví dụ như diện tích mặt cắt ngang của thanh giàn, tiết diện dầm/cột, hoặc chiều dày của tấm vỏ. Hình học tổng thể và dạng của mô hình thiết kế được giữ nguyên (không thay đổi lưới phần tử). Quá trình tối ưu chỉ thực hiện việc tăng hoặc giảm các kích thước cục bộ. Mục tiêu là xác định tiết diện hợp lý nhất cho các cấu kiện dưới tác dụng của tải trọng thiết kế. Kết quả tối ưu phải thỏa mãn các ràng buộc về độ bền, độ cứng, độ ổn định, cũng như các tiêu chí về khả năng chế tạo và hiệu quả kinh tế. Một ví dụ

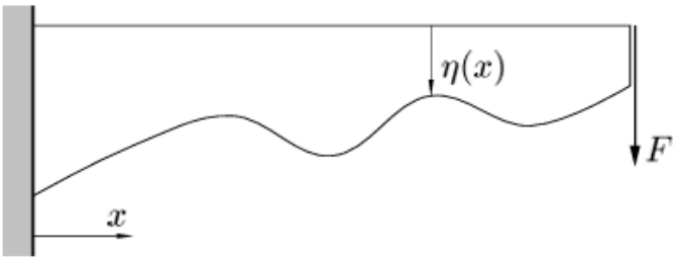
điển hình là bài toán tìm diện tích tiết diện tối ưu cho các thanh của kết cấu giàn (Hình 1.3) để chịu lực tốt nhất với trọng lượng nhẹ nhất.



Hình 1.3. Tối ưu hóa kích thước các thanh giàn.

1.4.2. Tối ưu hóa hình dạng

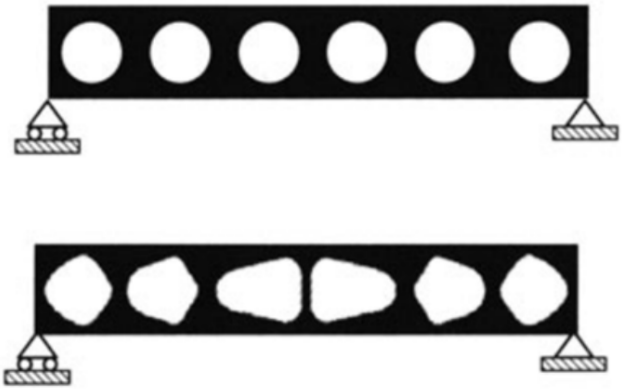
Trong bài toán tối ưu hóa hình dạng, biến thiết kế được sử dụng để mô tả đường bao hình học của một miền kết cấu, và quá trình tối ưu thực chất là việc xác định các tọa độ biên lý tưởng bằng cách dịch chuyển chúng theo những phương phù hợp. Một nguyên tắc cốt lõi của phương pháp này là tính liên kết (connectivity) và cấu trúc (topology) của hệ thống luôn được bảo toàn; nghĩa là hình dáng các biên hoặc lỗ rỗng có thể thay đổi để chịu lực tốt hơn, nhưng số lượng lỗ và cấu trúc tổng thể vẫn giữ nguyên. Các ví dụ điển hình cho bài toán này bao gồm tối ưu hóa hình dạng dầm (Hình 1.4), tối ưu hóa hình dáng các lỗ giảm trọng lượng trên dầm (Hình 1.5) hay thiết kế nhíp xe ô tô (Hình 1.6).



Hình 1.4. Tối ưu hóa hình dạng dầm.

1.4.3. Tối ưu hóa cấu trúc

Tối ưu hóa cấu trúc là cấp độ tổng quát và linh hoạt nhất trong thiết kế kết cấu, cho phép thay đổi đồng thời cả biên dạng bên ngoài lẫn sự phân bố vật liệu bên trong miền thiết kế. Nguyên lý cốt lõi của phương pháp này là xác định sự tồn tại của vật liệu tại mỗi điểm trong không gian thiết kế: đối với kết cấu rời rạc (như hệ giàn), biến thiết kế là diện tích mặt cắt ngang, và khi giá trị này về 0, thanh tương ứng sẽ bị loại bỏ hoàn toàn khỏi hệ thống (Hình 1.7); đối với kết cấu liên tục (như tấm hoặc khối rắn), mật độ vật liệu tại mỗi phần tử được xem là biến số, thường nhận giá trị nhị phân (0 là lỗ rỗng, 1 là vật liệu đặc). Ví dụ điển hình là bài toán tìm bố cục cho dầm

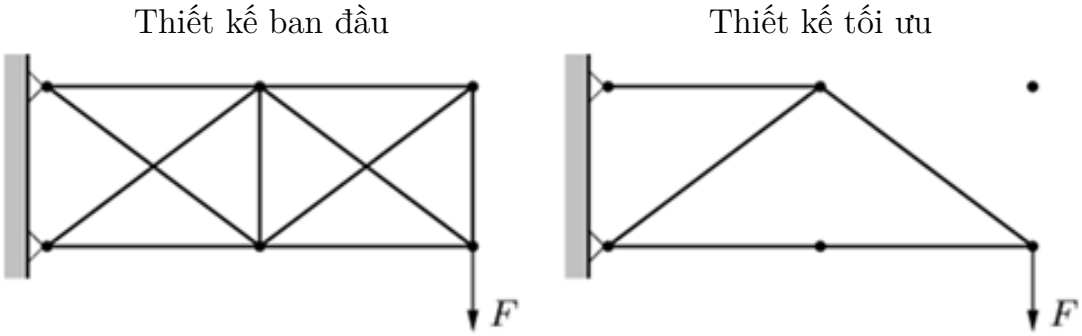


Hình 1.5. Tối ưu hóa hình dạng lỗ rỗng trong thanh dầm đơn giản.

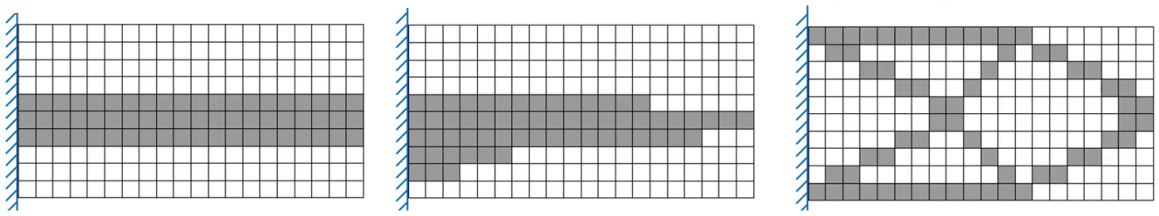


Hình 1.6. Tối ưu hóa hình dạng nhíp xe chịu lực tập trung giữa nhịp.

công-xôn trong miền thiết kế hình chữ nhật (lưới 20×10) với lượng vật liệu giới hạn (Hình 1.8); thuật toán sẽ tự động phân bố các phần tử vật liệu để tạo ra kết cấu có độ cứng lớn nhất, hình thành nên các lỗ rỗng đặc trưng bên trong mà các phương pháp tối ưu hóa kích thước hay hình dạng truyền thống không thể thực hiện được.



Hình 1.7. Tối ưu cấu trúc hệ giàn.



(a) Thiết kế 1 (b) Thiết kế 2 (c) Thiết kế 3

Hình 1.8. Các phương án thiết kế dầm công-xôn với mục tiêu cực đại hóa độ cứng khi sử dụng 60 phần tử vật liệu (ô vuông).

1.5. Bài tập

Bài 1.1: Một thanh thẳng chiều dài L, mô-đun đàn hồi E, khối lượng riêng ρ, được treo thẳng đứng và chịu lực kéo $F > 0$ ở đầu tự do (bỏ qua trọng lượng bản thân của thanh). Cần chọn diện tích tiết diện A sao cho khối lượng thanh nhỏ nhất, trong khi ứng suất không vượt quá σ_0 và độ giãn dài không vượt quá δ_0. Hãy chỉ ra biến thiết kế, biến trạng thái, hàm mục tiêu và từng nhóm ràng buộc (ứng xử, thiết kế, cân bằng); viết bài toán dưới dạng công thức đồng thời (1.1) và dạng công thức lồng ghép (1.2), rồi tìm nghiệm tối ưu.

Bài 1.2: Xếp các bài toán sau vào một trong ba nhóm tối ưu kích thước, tối ưu hình dạng hoặc tối ưu cấu trúc, và nêu rõ biến thiết kế trong mỗi trường hợp: (a) chọn chiều dày bản sàn bê tông cốt thép; (b) tìm đường cong trục của một vòm cầu; (c) chọn tiết diện cột thép từ một danh mục thép hình; (d) xác định tọa độ các nút cánh trên của một giàn mái; (e) quyết định giữ lại hay loại bỏ từng thanh trong một lưới thanh dày đặc; (f) tìm phân bố vật liệu trong một tấm giá đỡ chịu tải.

Bài 1.3: Chọn một công trình quen thuộc (chẳng hạn khung nhà xưởng một tầng hoặc cầu dầm nhịp nhỏ). Mô tả bốn bước của quy trình thiết kế đã nêu ở Mục 1.1 cho công trình đó; ở bước tối ưu hóa, hãy đề xuất ít nhất hai hàm mục tiêu có thể sử dụng và các ràng buộc tương ứng, đồng thời giải thích vì sao các hàm mục tiêu này có thể mâu thuẫn với nhau.

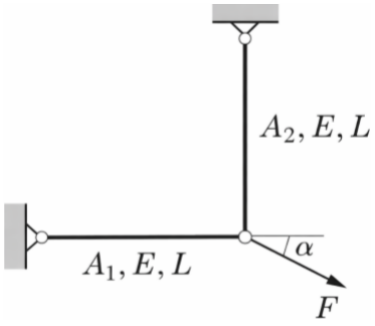
CHƯƠNG 2

TỐI ƯU HÓA KẾT CẤU BẰNG PHƯƠNG PHÁP ĐỒ THỊ

Chương này giới thiệu một số ví dụ minh họa cho bài toán tối ưu kích thước của thanh và dầm sử dụng phương pháp đồ thị. Phương pháp này có thể áp dụng cho các bài toán đơn giản, có ít biến thiết kế; các phương pháp tổng quát hơn sẽ được trình bày trong các chương tiếp theo. Các ví dụ trong chương được trình bày theo [10, Chương 2].

2.1. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất

Hãy xem xét một hệ giàn hai thanh như minh họa tại Hình 2.1. Hai thanh này có cùng chiều dài L và mô-đun đàn hồi E. Hệ chịu tác dụng của một lực tập trung F tại nút tự do, hợp với phương ngang một góc α ($0 \leqslant \alpha \leqslant 90°$). Mục tiêu của bài toán là tìm phương án thiết kế sao cho trọng lượng của hệ là nhỏ nhất (tối thiểu hóa trọng lượng) nhưng vẫn phải thỏa mãn các điều kiện bền về ứng suất.



Hình 2.1. Tối ưu hóa trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất.

Chúng ta chọn diện tích mặt cắt ngang của hai thanh, A_1 và A_2, làm các biến thiết kế. Hàm mục tiêu (trọng lượng tổng của giàn) được xác định như sau:

$$f(A_1, A_2) = (A_1 + A_2)\rho L \tag{2.1}$$

trong đó ρ là khối lượng riêng của vật liệu. Vì ρ và L là các hằng số dương, việc tối thiểu hóa trọng lượng tương đương với việc tối thiểu hóa tổng diện tích mặt cắt ngang $(A_1 + A_2)$, với điều kiện hiển nhiên:

$$A_1 \geqslant 0, \quad A_2 \geqslant 0 \tag{2.2}$$

Nội lực và ứng suất trong các thanh có thể được xác định trực tiếp thông qua các phương trình cân bằng lực tại nút:

$$\begin{aligned} F\cos\alpha - \sigma_1 A_1 &= 0 \\ F\sin\alpha - \sigma_2 A_2 &= 0 \end{aligned} \tag{2.3}$$

Để đảm bảo an toàn cho kết cấu, trị tuyệt đối của ứng suất trong mỗi thanh không được vượt quá giới hạn cho phép σ_0:

$$|\sigma_i| \leqslant \sigma_0, \quad i = 1, 2 \tag{2.4}$$

Kết hợp phương trình cân bằng và điều kiện bền, ta chuyển hóa các ràng buộc ứng suất thành các ràng buộc trực tiếp lên biến thiết kế:

$$A_1 \geqslant \frac{F\cos\alpha}{\sigma_0}, \qquad A_2 \geqslant \frac{F\sin\alpha}{\sigma_0} \tag{2.5}$$

Từ các phân tích trên, bài toán tối ưu hóa ban đầu được đưa về dạng rút gọn như sau:

$$(\mathbb{SO})^1_{\text{nf}} \quad \begin{cases} \min\limits_{A_1, A_2} \ (A_1 + A_2) \\ \text{s.t.} \quad \begin{cases} A_1 \geqslant \frac{F\cos\alpha}{\sigma_0} \\ A_2 \geqslant \frac{F\sin\alpha}{\sigma_0} \end{cases} \end{cases} \tag{2.6}$$

Dễ dàng nhận thấy hàm mục tiêu đạt giá trị nhỏ nhất khi các biến thiết kế chạm ngưỡng giới hạn dưới của chúng. Nghiệm tối ưu là:

$$A_1^* = \frac{F\cos\alpha}{\sigma_0}, \qquad A_2^* = \frac{F\sin\alpha}{\sigma_0} \tag{2.7}$$

Lời giải bằng đồ thị của bài toán $(\mathbb{SO})^1_{\text{nf}}$ được trình bày ở Hình 2.2.

2.2. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và ổn định

Xét hệ giàn như Hình 2.3, gồm hai thanh tiết diện tròn vuông góc nhau, cùng chiều dài L và mô-đun đàn hồi E. Lực F tác dụng tại nút tự do với góc nghiêng 45° so với phương ngang. Tìm diện tích mặt cắt ngang A_1, A_2 của hai thanh để trọng lượng giàn là nhỏ nhất. Kết cấu phải thỏa mãn điều kiện bền và điều kiện ổn định (buckling).

Lực dọc trong hai thanh có độ lớn bằng nhau nhưng trái dấu. Thanh 1 chịu kéo và thanh 2 chịu nén với giá trị lực dọc: $N = \frac{F}{\sqrt{2}}$. Cả hai thanh đều không được vượt quá giới hạn ứng suất cho phép σ_0:

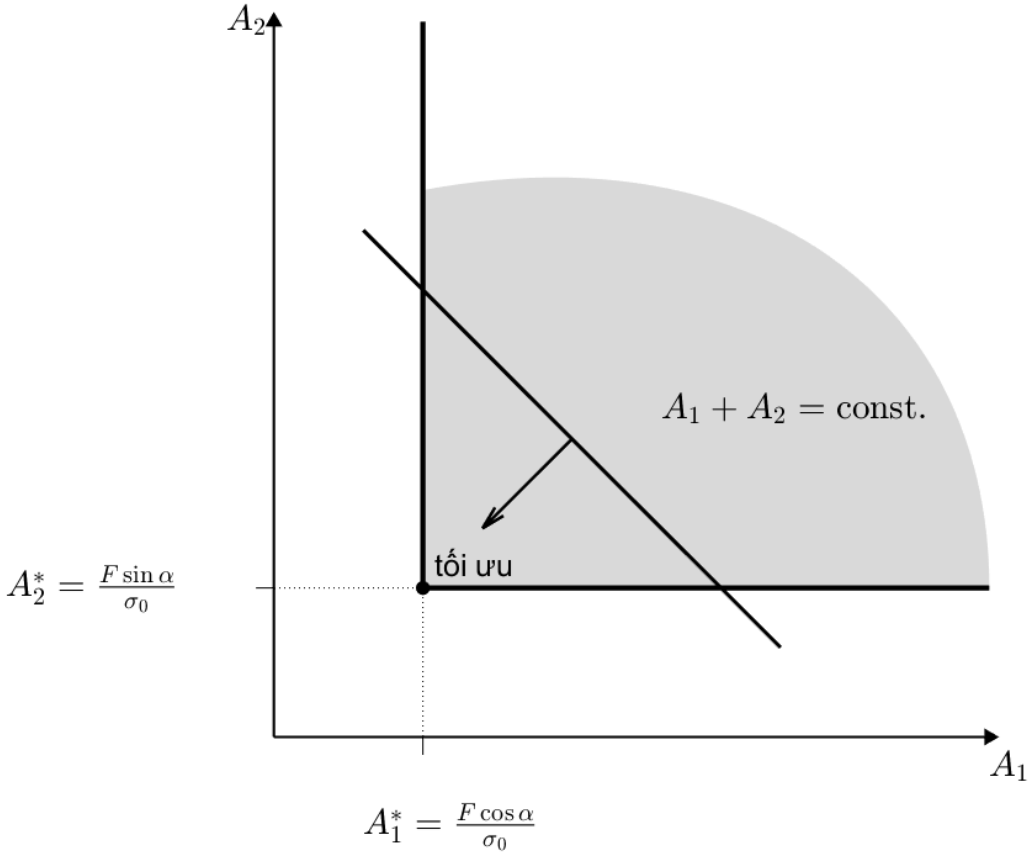
$$A_1 \geqslant \frac{F}{\sqrt{2}\sigma_0} \text{ và } A_2 \geqslant \frac{F}{\sqrt{2}\sigma_0} \tag{2.8}$$

Ràng buộc về ổn định chỉ áp dụng cho thanh 2 vì thanh này chịu nén. Công thức lực tới hạn Euler:

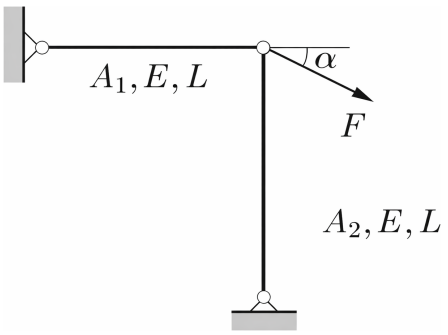
$$P_c = \pi^2 \frac{EI}{L^2} \tag{2.9}$$

trong đó mô-men quán tính của tiết diện tròn là $I = \frac{A^2}{4\pi}$.

Hệ số an toàn về ổn định được chọn bằng 4, tức là lực tới hạn phải lớn gấp 4 lần lực



Hình 2.2. Lời giải bằng đồ thị của bài toán $(\mathbb{SO})^1_{\text{nf}}$.



Hình 2.3. Tối ưu hóa trọng lượng hệ giàn hai thanh chịu ràng buộc điều kiện bền và ổn định.

nén thực tế: $P_c \geqslant 4 \times$ lực nén trong thanh 2.

$$\frac{\pi^2 E(A_2^2/4\pi)}{L^2} \geqslant 4\frac{F}{\sqrt{2}} \tag{2.10}$$

Rút gọn biểu thức trên, ta có ràng buộc phi tuyến đối với A_2:

$$A_2^2 \geqslant \frac{16FL^2}{\sqrt{2}\pi E} \tag{2.11}$$

Bài toán được tóm tắt dưới dạng công thức giải tích như sau:

$$(\mathbb{SO})^2_{\text{nf}} \quad \begin{cases} \min\limits_{A_1,A_2} (A_1 + A_2) \\ \text{s.t.} \begin{cases} A_1 \geqslant \frac{F}{\sqrt{2}\sigma_0} \\ A_2 \geqslant \frac{F}{\sqrt{2}\sigma_0} \\ A_2 \geqslant \sqrt{\frac{16FL^2}{\sqrt{2}\pi E}} \end{cases} \end{cases} \tag{2.12}$$

Việc xác định ràng buộc nào trong số các ràng buộc trên đóng vai trò ràng buộc hoạt động (active constraint), tức là ràng buộc mà tại đó dấu bằng xảy ra, sẽ phụ thuộc vào tương quan giữa các tham số vật lý. Để minh họa, hãy xem xét một trường hợp cụ thể với các thông số giả định sau:

$$\sigma_0 = \frac{E}{100}, \qquad \sqrt{\frac{F}{\sigma_0}} = \frac{L}{4} \tag{2.13}$$

Khi thay các giá trị này vào hệ điều kiện của bài toán $(\mathbb{SO})^2_{\text{nf}}$, ta thu được các giới hạn cụ thể cho tiết diện:

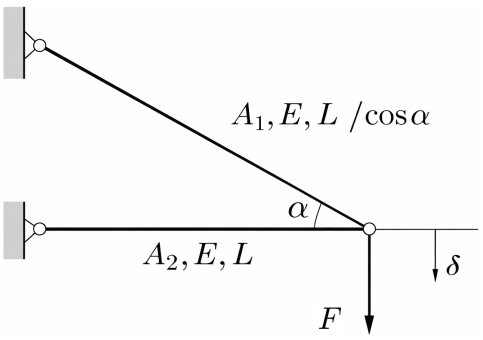
$$A_1 \geqslant \frac{L^2}{16\sqrt{2}}, \qquad A_2 \geqslant \frac{L^2}{16\sqrt{2}}, \qquad A_2 \geqslant \frac{L^2}{10\sqrt{\sqrt{2}\pi}} \tag{2.14}$$

Tiếp theo, ta so sánh hai giới hạn dưới áp dụng cho thanh chịu nén. Dễ thấy rằng ràng buộc thứ ba sẽ chi phối thiết kế của thanh 2. Kết quả cuối cùng là:

$$A_1^* = \frac{L^2}{16\sqrt{2}} \approx 0{,}044L^2, \qquad A_2^* = \frac{L^2}{10\sqrt{\sqrt{2}\pi}} \approx 0{,}047L^2 \tag{2.15}$$

2.3. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và chuyển vị

Xét hệ giàn hai thanh như Hình 2.4, góc $\alpha = 30°$. Tìm diện tích mặt cắt ngang A_1, A_2 để trọng lượng giàn là nhỏ nhất. Ứng suất không vượt quá giới hạn σ_0 và chuyển vị đứng tại đầu mút δ không được vượt quá giá trị cho phép $\delta_0 = \sigma_0 L/E$.



Hình 2.4. Tối ưu hóa trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và chuyển vị.

Hàm mục tiêu (trọng lượng của giàn) được xác định như sau:

$$f(A_1, A_2) = \rho L \left(\frac{2}{\sqrt{3}} A_1 + A_2 \right) \tag{2.16}$$

Sử dụng phương pháp phân tích kết cấu, ta thiết lập được mối quan hệ giữa chuyển vị đầu mút và diện tích tiết diện như sau (người đọc tự chứng minh):

$$\delta = \frac{FL}{E} \left(\frac{8}{\sqrt{3} A_1} + \frac{3}{A_2} \right) \tag{2.17}$$

Ràng buộc chuyển vị $\delta \leqslant \delta_0$ trở thành:

$$\frac{8}{\sqrt{3} A_1} + \frac{3}{A_2} \leqslant \frac{\sigma_0}{F} \tag{2.18}$$

Do tính chất phi tuyến của ràng buộc (2.18) đối với các biến A_1 và A_2, việc tìm kiếm lời giải tối ưu trở nên phức tạp. Nhằm đơn giản hóa bài toán, ta thực hiện chuyển đổi các biến sang dạng không thứ nguyên:

$$x_1 = \frac{2F}{\sigma_0 A_1}, \qquad x_2 = \frac{\sqrt{3} F}{\sigma_0 A_2} \tag{2.19}$$

Khi đó ràng buộc chuyển vị trở nên tuyến tính:

$$\frac{4}{\sqrt{3}} x_1 + \sqrt{3} x_2 \leqslant 1 \tag{2.20}$$

Ràng buộc ứng suất $\sigma_1 = 2F/A_1 \leqslant \sigma_0$, $\sigma_2 = \sqrt{3}F/A_2 \leqslant \sigma_0$ trở thành $x_1 \leqslant 1$, $x_2 \leqslant 1$. Bỏ qua hằng số dương $\sqrt{3}\rho LF/\sigma_0$ trong hàm mục tiêu, bài toán tối ưu hóa được phát biểu như sau:

$$(\mathbb{SO})^3_{\text{nf}} \quad \begin{cases} \min\limits_{x_1, x_2} \hat{f}(x_1, x_2) = \frac{4}{3x_1} + \frac{1}{x_2} \\ \text{s.t.} \quad \begin{cases} \frac{4}{\sqrt{3}} x_1 + \sqrt{3} x_2 \leqslant 1 \\ 0 < x_1 \leqslant 1, \ 0 < x_2 \leqslant 1. \end{cases} \end{cases} \tag{2.21}$$

Bài toán này được minh họa trong Hình 2.5, từ đó chúng ta kết luận rằng ràng buộc (2.20) là hoạt động. Chúng ta viết (2.20) dưới dạng đẳng thức và giải tìm x_2 để thu được:

$$x_2 = \frac{1}{\sqrt{3}} - \frac{4}{3} x_1 \tag{2.22}$$

Thế phương trình này vào hàm mục tiêu $\hat{f}(x_1, x_2)$, ta đưa bài toán về hàm một biến theo x_1. Lấy đạo hàm và cho bằng 0 để tìm cực trị:

$$x_1 = \pm \left(\frac{1}{\sqrt{3}} - \frac{4}{3} x_1 \right) \tag{2.23}$$

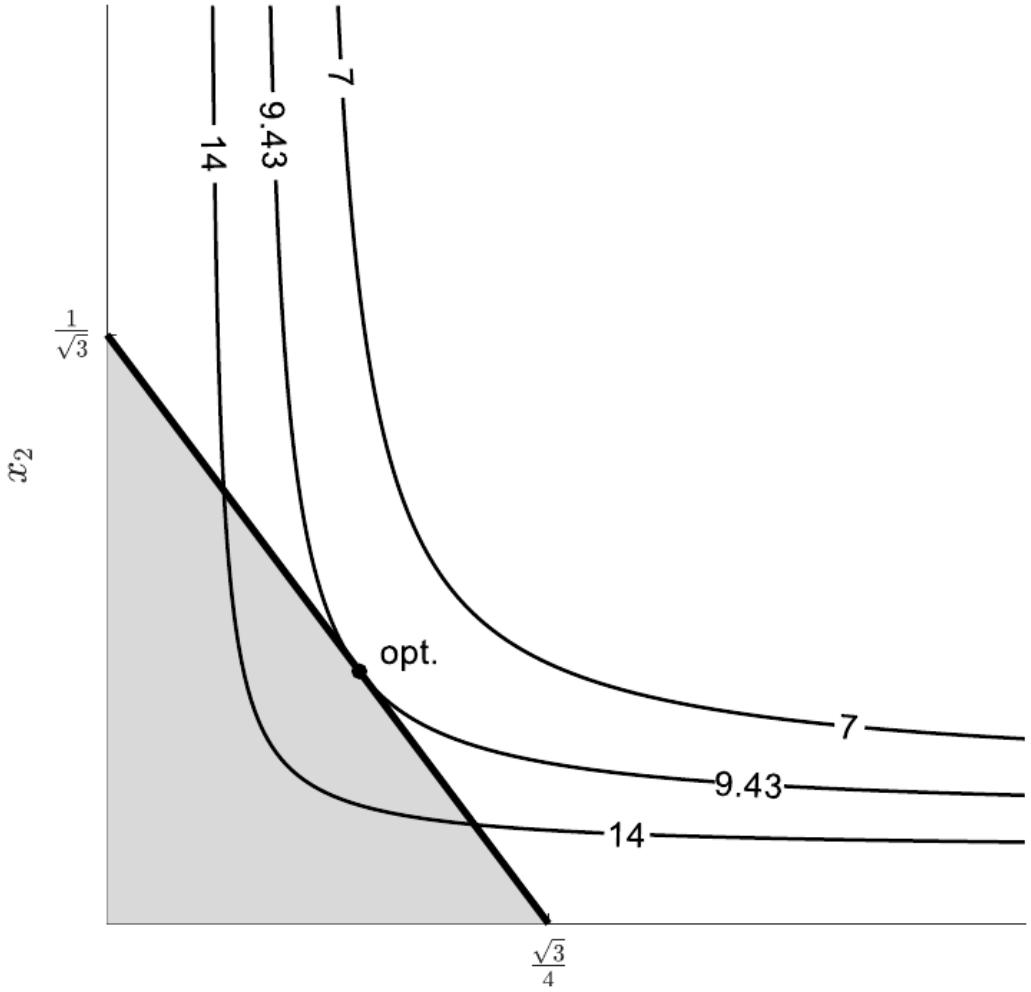
Giải phương trình trên với dấu "+" (dấu "−" cho $x_1 = \sqrt{3} > 1$ và $x_2 = -\sqrt{3} < 0$, vi phạm ràng buộc), ta thu được nghiệm tối ưu:

$$x_1^* = \frac{\sqrt{3}}{7}, \quad x_2^* = \frac{\sqrt{3}}{7}. \tag{2.24}$$

Chuyển đổi ngược lại về biến diện tích, ta có thiết kế tối ưu:

$$A_1^* = \frac{14F}{\sqrt{3}\sigma_0} \approx \frac{8{,}1F}{\sigma_0}, \quad A_2^* = \frac{7F}{\sigma_0} \tag{2.25}$$

Lời giải bằng đồ thị của bài toán $(\mathbb{SO})^3_{\text{nf}}$ được trình bày ở Hình 2.5.



Hình 2.5. Minh họa bài toán tối ưu $(\mathbb{SO})^3_{\text{nf}}$.

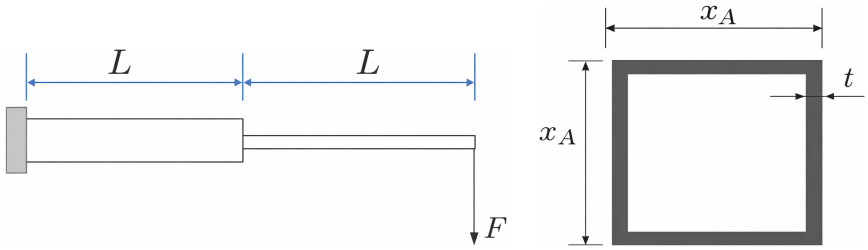
2.4. Tối ưu trọng lượng dầm công-xôn chịu ràng buộc chuyển vị

Xét dầm công-xôn ngàm tại đầu trái và chịu lực tập trung $F > 0$ tại đầu phải (Hình 2.6). Dầm gồm N đoạn có chiều dài L bằng nhau với tiết diện hình hộp rỗng. Giả sử chiều dày thành t không đổi, trong khi kích thước cạnh x_A thay đổi theo từng đoạn. Các đoạn được đánh số $A = 1, \ldots, N$ bắt đầu từ đầu tự do (đoạn 1 ở đầu mút chịu lực, đoạn N ở ngàm).

Với giả thiết $t \ll x_A$, mô-men quán tính I_A được xấp xỉ bằng công thức:

$$I_A = \frac{x_A^4}{12} - \frac{(x_A - 2t)^4}{12} \approx \frac{2tx_A^3}{3} \tag{2.26}$$

Mục tiêu là cực tiểu hóa trọng lượng dầm sao cho chuyển vị đầu mút δ không vượt quá giới hạn δ_0. Hàm mục tiêu trọng lượng được viết dưới dạng:



Hình 2.6. Tối ưu hóa trọng lượng dầm công-xôn N đoạn chịu ràng buộc chuyển vị (hình vẽ với $N = 2$).

$$f(x_1, \ldots, x_N) = 4L\rho t \sum_{A=1}^{N} x_A \tag{2.27}$$

Áp dụng nguyên lý cộng tác dụng, chuyển vị tổng cộng δ tại đầu tự do là tổng đóng góp chuyển vị từ mỗi đoạn (khi coi các đoạn khác là tuyệt đối cứng). Với giả thiết biến dạng bé ($\sin\theta_A \approx \theta_A$), ta có hệ thức hình học:

$$\delta^{(A)} = \delta_A + (A-1)L\theta_A \tag{2.28}$$

trong đó δ_A và θ_A là độ võng và góc xoay tại đầu phải của đoạn A, được xác định theo lý thuyết dầm cơ bản:

$$\delta_A = \frac{M_A L^2}{2EI_A} + \frac{F_A L^3}{3EI_A}, \quad \theta_A = \frac{M_A L}{EI_A} + \frac{F_A L^2}{2EI_A} \tag{2.29}$$

với mô-men uốn tại đầu phải của đoạn A là $M_A = (A-1)LF$ và lực cắt $F_A = F$.

Thay thế các biểu thức trên vào phương trình tổng quát, ta thu được công thức chuyển vị tường minh:

$$\delta = \frac{3FL^3}{2Et} \sum_{A=1}^{N} \left(A^2 - A + \frac{1}{3}\right) \frac{1}{x_A^3} \tag{2.30}$$

Xét trường hợp đơn giản với 2 đoạn, bài toán tối ưu hóa $(\mathbb{SO})^4_{\text{nf}}$ được phát biểu như sau:

$$(\mathbb{SO})^4_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} f(x_1,x_2) = C_1(x_1+x_2) \\ \text{s.t.} \begin{cases} \frac{1}{x_1^3} + \frac{7}{x_2^3} \leqslant C_2 \\ x_1 > 0, \quad x_2 > 0, \end{cases} \end{cases} \tag{2.31}$$

trong đó $C_1 = 4\rho Lt$ và $C_2 = \frac{2\delta_0 Et}{FL^3}$.

Giải bài toán ta thu được nghiệm tối ưu:

$$x_1^* = \left(\frac{1+7^{1/4}}{C_2}\right)^{1/3}, \quad x_2^* = 7^{1/4}\left(\frac{1+7^{1/4}}{C_2}\right)^{1/3} \tag{2.32}$$

Xét bài toán ngược lại, hoán đổi vai trò của hàm mục tiêu và ràng buộc (không phải bài toán đối ngẫu Lagrange ở Chương 3): cực tiểu hóa chuyển vị đầu mút với ràng

buộc về tổng trọng lượng W cho trước:

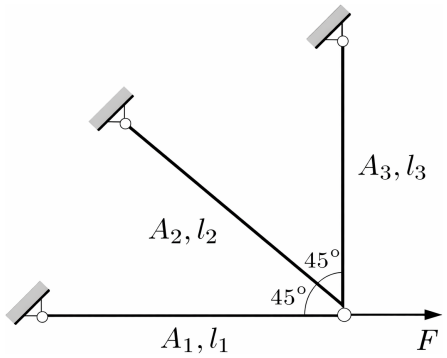
$$\begin{cases} \min\limits_{x_1,x_2} \frac{1}{x_1^3} + \frac{7}{x_2^3} \\ \text{s.t.} \begin{cases} C_1(x_1 + x_2) \leqslant W \\ x_1 > 0,\ x_2 > 0 \end{cases} \end{cases} \tag{2.33}$$

Bài toán này có thể được giải theo cách tương tự như $(\mathbb{SO})^4_{\text{nf}}$. Ta tìm được lời giải:

$$x_1^{**} = \frac{W}{C_1}\left(\frac{1}{1+7^{1/4}}\right), \quad x_2^{**} = \frac{W}{C_1}\left(\frac{7^{1/4}}{1+7^{1/4}}\right). \tag{2.34}$$

2.5. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc ứng suất

Xét một hệ giàn ba thanh như Hình 2.7, chịu tác dụng của lực F. Các thanh có cùng mô-đun đàn hồi E. Chiều dài các thanh lần lượt là $l_1 = l_2 = L$ và $l_3 = L/\beta$ (với $\beta = 1$), diện tích tiết diện $A_1 = A_3$. Bài toán đặt ra là tìm các biến thiết kế A_1 và A_2 để tối thiểu hóa trọng lượng của hệ giàn, đồng thời thỏa mãn các ràng buộc về ứng suất.



Hình 2.7. Tối ưu hóa trọng lượng hệ giàn ba thanh chịu ràng buộc ứng suất.

Tổng trọng lượng của giàn được xác định bởi:

$$f(A_1, A_2) = \rho_1 L A_1 + \rho_2 L A_2 + \rho_3 \frac{L}{\beta} A_1 = L\left(\rho_1 + \frac{\rho_3}{\beta}\right) A_1 + L\rho_2 A_2 \tag{2.35}$$

trong đó ρ_i là khối lượng riêng của vật liệu các thanh. Các biến thiết kế phải không âm: $A_1 \geqslant 0, A_2 \geqslant 0$.

Do hệ giàn là siêu tĩnh, nội lực trong các thanh phụ thuộc vào độ cứng của chúng. Giải bài toán cân bằng dựa trên chuyển vị, ta thu được công thức tính chuyển vị tại điểm đặt lực như sau:

$$\begin{aligned} u_x &= \frac{FL}{EA_1}\left(\frac{2\beta A_1 + A_2}{2\beta A_1 + (1+\beta)A_2}\right), \\ u_y &= \frac{FL}{EA_1}\left(\frac{A_2}{2\beta A_1 + (1+\beta)A_2}\right). \end{aligned} \tag{2.36}$$

và ứng suất trong các thanh như sau:

$$\begin{aligned}
\sigma_1 &= \frac{F}{2\beta A_1 + (1+\beta)A_2}\left(2\beta + \frac{A_2}{A_1}\right) \\
\sigma_2 &= \frac{\sqrt{2}F\beta}{2\beta A_1 + (1+\beta)A_2} \\
\sigma_3 &= -\frac{F\beta\frac{A_2}{A_1}}{2\beta A_1 + (1+\beta)A_2}
\end{aligned} \tag{2.37}$$

Từ kết quả trên, dễ thấy thanh 1 và 2 chịu kéo, còn thanh 3 chịu nén. Do đó, các điều kiện bền cần kiểm tra là $\sigma_1 \leqslant \sigma_1^{max}$, $\sigma_2 \leqslant \sigma_2^{max}$ và $-\sigma_3 \leqslant \sigma_3^{max}$.

Lưu ý rằng thanh 2 có thể có diện tích bằng 0 mà kết cấu vẫn tĩnh định; ngược lại, do giả thiết $A_1 = A_3$, thanh 1 và thanh 3 luôn có cùng diện tích và không thể bằng 0 vì khi đó kết cấu trở thành hệ biến hình. Các ràng buộc ứng suất được viết lại cụ thể:

Thanh 1:

$$\frac{F(2A_1 + A_2)}{2A_1(A_1 + A_2)} \leqslant \sigma_1^{max} \tag{2.38}$$

Thanh 2 (chỉ xét khi $A_2 > 0$):

$$\frac{F}{\sqrt{2}(A_1 + A_2)} \leqslant \sigma_2^{max} \tag{2.39}$$

Thanh 3:

$$\frac{FA_2}{2A_1(A_1 + A_2)} \leqslant \sigma_3^{max} \tag{2.40}$$

Tổng hợp lại, ta có phát biểu bài toán tối ưu hóa đầy đủ như sau:

$$(\mathbb{SO})^5_{\text{nf}} \quad \begin{cases} \min\limits_{A_1, A_2} (\rho_1 A_1 + \rho_2 A_2 + \rho_3 A_1)L \\ \text{s.t.} \begin{cases} \frac{F(2A_1+A_2)}{2A_1(A_1+A_2)} - \sigma_1^{\max} \leqslant 0 \\ \frac{F}{\sqrt{2}(A_1+A_2)} - \sigma_2^{\max} \leqslant 0 \quad \text{nếu } A_2 > 0 \\ \frac{FA_2}{2A_1(A_1+A_2)} - \sigma_3^{\max} \leqslant 0 \\ A_1 > 0, \quad A_2 \geqslant 0. \end{cases} \end{cases} \tag{2.41}$$

Để minh họa tính chất đa dạng của lời giải tối ưu trong kết cấu, các phần tiếp theo sẽ khảo sát bài toán này dưới 5 trường hợp khác nhau của vật liệu và giới hạn bền.

Trường hợp a: $\rho_1 = 2\rho_0, \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_2^{\max} = \sigma_3^{\max} = \sigma_0$.

Với các thông số vật liệu $\rho_1 = 2\rho_0$, $\rho_2 = \rho_3 = \rho_0$ và giới hạn ứng suất $\sigma^{max} = \sigma_0$, ta sử dụng các biến không thứ nguyên:

$$x_1 = \frac{A_1\sigma_0}{F}, \quad x_2 = \frac{A_2\sigma_0}{F} \tag{2.42}$$

Bài toán tối ưu hóa $(\mathbb{SO})^{5a}_{\text{nf}}$ được phát biểu như sau:

$$(\mathbb{SO})^{5a}_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} 3x_1 + x_2 \\ \text{s.t.} \quad \begin{cases} \frac{2x_1+x_2}{2x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_1) \\ \frac{1}{\sqrt{2}(x_1+x_2)} - 1 \leqslant 0 \quad \text{nếu } x_2 > 0 \quad (\sigma_2) \\ \frac{x_2}{2x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_3) \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.43}$$

Quan sát Hình 2.8, ta thấy tại điểm tối ưu, chỉ có ràng buộc σ_1 là hoạt động, trong khi các ràng buộc khác đều không hoạt động. Do đó, ta xét phương trình đẳng thức của σ_1:

$$2x_1 + x_2 - 2x_1(x_1 + x_2) = 0 \tag{2.44}$$

Biến đổi để biểu diễn x_2 theo x_1:

$$x_2 = \frac{2x_1(x_1 - 1)}{1 - 2x_1} \tag{2.45}$$

Thế vào hàm mục tiêu, bài toán quy về việc cực tiểu hóa hàm số theo biến x_1. Việc tìm điểm dừng dẫn đến phương trình bậc hai:

$$8x_1^2 - 8x_1 + 1 = 0 \tag{2.46}$$

Giải phương trình, ta có hai nghiệm:

$$x_1^* = \frac{1}{2} \pm \frac{\sqrt{2}}{4} \tag{2.47}$$

Để thỏa mãn điều kiện $x_2 > 0$ khi thế vào (2.45), ta loại nghiệm dấu trừ và chọn nghiệm dấu cộng. Kết quả các biến tối ưu không thứ nguyên là:

$$x_1^* = \frac{2 + \sqrt{2}}{4}, \quad x_2^* = \frac{\sqrt{2}}{4} \tag{2.48}$$

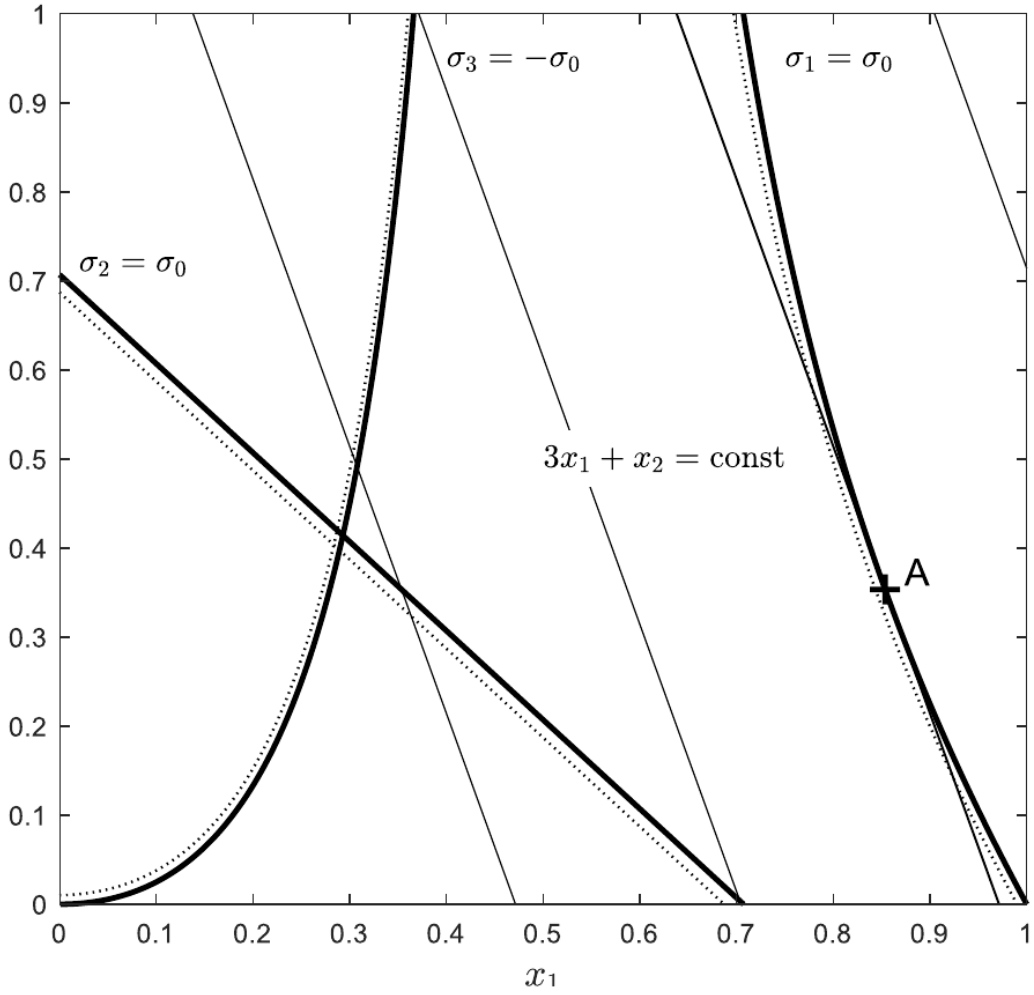
Chuyển đổi ngược về các biến diện tích tiết diện thực tế (A_1, A_2) ta có diện tích tiết diện tối ưu:

$$A_1^* = \frac{F}{2\sigma_0}\left(1 + \frac{1}{\sqrt{2}}\right), \quad A_2^* = \frac{F}{2\sqrt{2}\sigma_0} \tag{2.49}$$

Trường hợp b: $\rho_1 = \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$.

Sử dụng cùng các biến không thứ nguyên như trường hợp trước, chúng ta có thể viết bài toán là:

$$(\mathbb{SO})^{5b}_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} \quad 2x_1 + x_2 \\ \text{s.t.} \quad \begin{cases} \frac{2x_1+x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_1) \\ \frac{1}{\sqrt{2}(x_1+x_2)} - 1 \leqslant 0 \quad \text{nếu } x_2 > 0 \quad (\sigma_2) \\ \frac{x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_3) \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.50}$$



Hình 2.8. Trường hợp a. Đường liền nét đậm đi kèm với đường nét chấm biểu thị một ràng buộc; vùng nằm cùng phía với đường nét chấm so với đường đậm không thuộc miền khả dĩ. Các đường liền nét mảnh là các đường đẳng trị, nghĩa là tất cả các điểm nằm trên cùng một đường mảnh sẽ cho cùng một giá trị hàm mục tiêu. Điểm A là lời giải của bài toán.

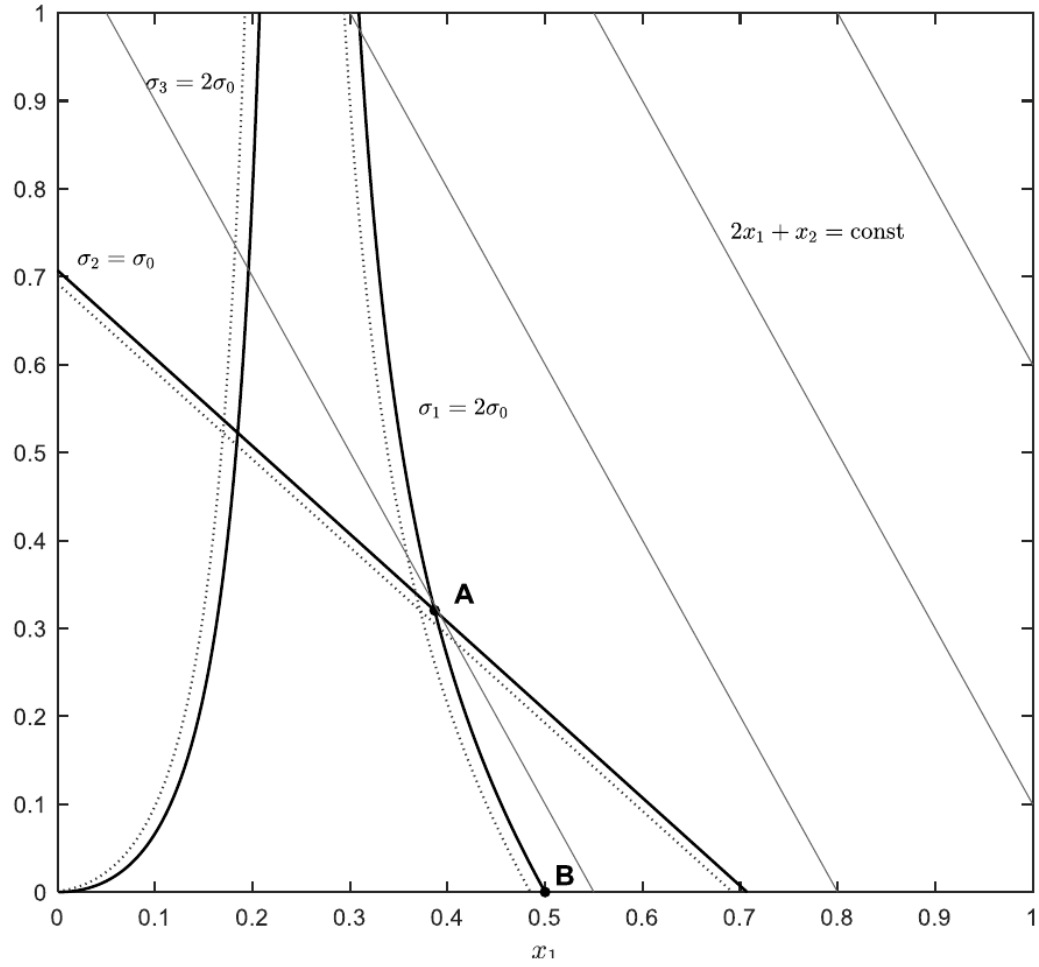
Quan sát Hình 2.9, thoạt nhìn lời giải tối ưu dường như nằm tại điểm A, là giao điểm của hai đường ràng buộc σ_1 và σ_2. Tuy nhiên, cần lưu ý rằng ràng buộc σ_2 chỉ có hiệu lực khi $x_2 > 0$.

Khi $x_2 = 0$ (loại bỏ thanh 2), ràng buộc σ_2 không còn hiệu lực; dựa vào đồ thị, ta thấy điểm B nằm trên đường cong ràng buộc σ_1 (tại vị trí $x_2 = 0$) mới chính là điểm mang lại trọng lượng nhỏ nhất. Tọa độ điểm này được xác định bằng cách thay $x_2 = 0$ vào phương trình ràng buộc σ_1 đang hoạt động:

$$2x_1 - 4x_1^2 = 0 \tag{2.51}$$

Phương trình trên cho kết quả $x_1^* = 1/2$ (loại bỏ nghiệm $x_1^* = 0$ vì đây không phải là một phương án thiết kế hợp lệ). Khi chuyển đổi trở lại các biến ban đầu, lời giải tối ưu là:

$$A_1^* = \frac{F}{2\sigma_0}, \quad A_2^* = 0 \tag{2.52}$$



Hình 2.9. Trường hợp b. Điểm B là lời giải tối ưu.

Trường hợp c: $\rho_1 = (2\sqrt{2}-1)\rho_0, \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$.

Trong trường hợp này, khối lượng riêng của thanh số 1 được tăng lên $\rho_1 = (2\sqrt{2}-1)\rho_0 \approx 1{,}83\rho_0$ so với $\rho_1 = \rho_0$ ở trường hợp b. Sự thay đổi này dẫn đến việc hàm mục tiêu biến đổi, trong khi các ràng buộc của bài toán vẫn giữ nguyên như $(\mathbb{SO})^{5b}_{\text{nf}}$. Bài toán tối ưu hóa mới được thiết lập như sau:

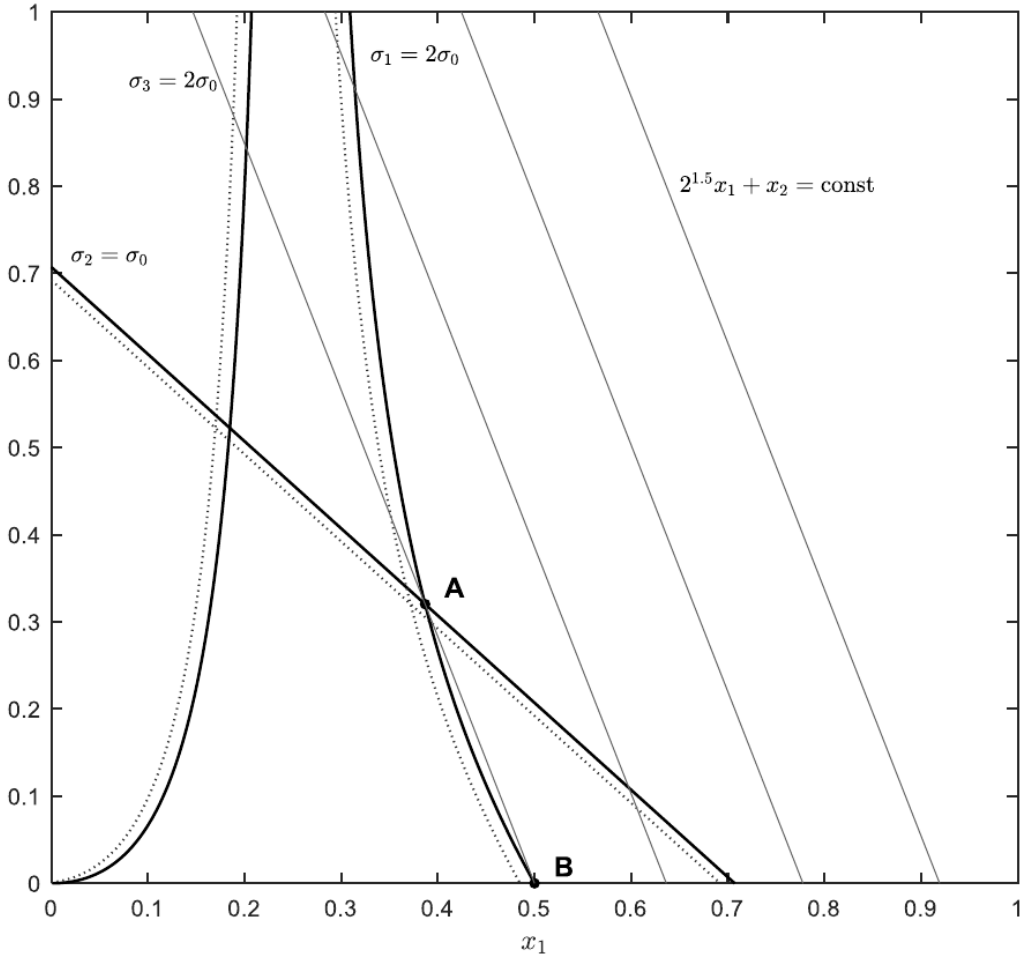
$$(\mathbb{SO})^{5c}_{\text{nf}} \quad \begin{cases} \min_{x_1,x_2} 2\sqrt{2}x_1 + x_2 \\ \text{s.t. các ràng buộc trong } (\mathbb{SO})^{5b}_{\text{nf}}, \end{cases} \tag{2.53}$$

Bài toán được minh họa trực quan trong Hình 2.10. Quan sát đồ thị, việc xác định lời giải tối ưu chỉ bằng mắt thường trở nên mơ hồ; không rõ liệu điểm tối ưu thực sự nằm tại giao điểm A của hai đường biên ràng buộc (σ_1 và σ_2), hay tại điểm B (nơi $x_2 = 0$, tương ứng với thiết kế loại bỏ thanh 2).

Tại điểm A: đây là giao điểm của hai ràng buộc đang hoạt động. Tọa độ của A được xác định bằng cách giải hệ phương trình khi các bất đẳng thức σ_1 và σ_2 trở thành đẳng thức:

$$x_1^* = \frac{4+\sqrt{2}}{14}, \quad x_2^* = \frac{6\sqrt{2}-4}{14} \tag{2.54}$$

Tại điểm B: tương ứng với thiết kế không có thanh 2, ta có $x_1^{**} = 1/2, \quad x_2^{**} = 0.$



Hình 2.10. Trường hợp c. Điểm A và B là lời giải tối ưu.

Một kết quả thú vị xuất hiện khi thay các giá trị trên vào hàm mục tiêu: cả hai điểm A và B đều mang lại cùng một giá trị hàm mục tiêu. Điều này đồng nghĩa với việc bài toán tồn tại song song hai lời giải tối ưu toàn cục. Khi chuyển đổi trở lại các biến thiết kế ban đầu, hai phương án tối ưu được biểu diễn như sau: phương án 1, tương ứng với điểm A, có $A_1^* = \frac{F}{\sigma_0}\left(\frac{4+\sqrt{2}}{14}\right)$ và $A_2^* = \frac{F}{\sigma_0}\left(\frac{6\sqrt{2}-4}{14}\right)$; phương án 2, tương ứng với điểm B, có $A_1^{**} = \frac{F}{2\sigma_0}$ và $A_2^{**} = 0$.

Cả hai phương án thiết kế này đều cho cùng một trọng lượng tối ưu của hệ là:

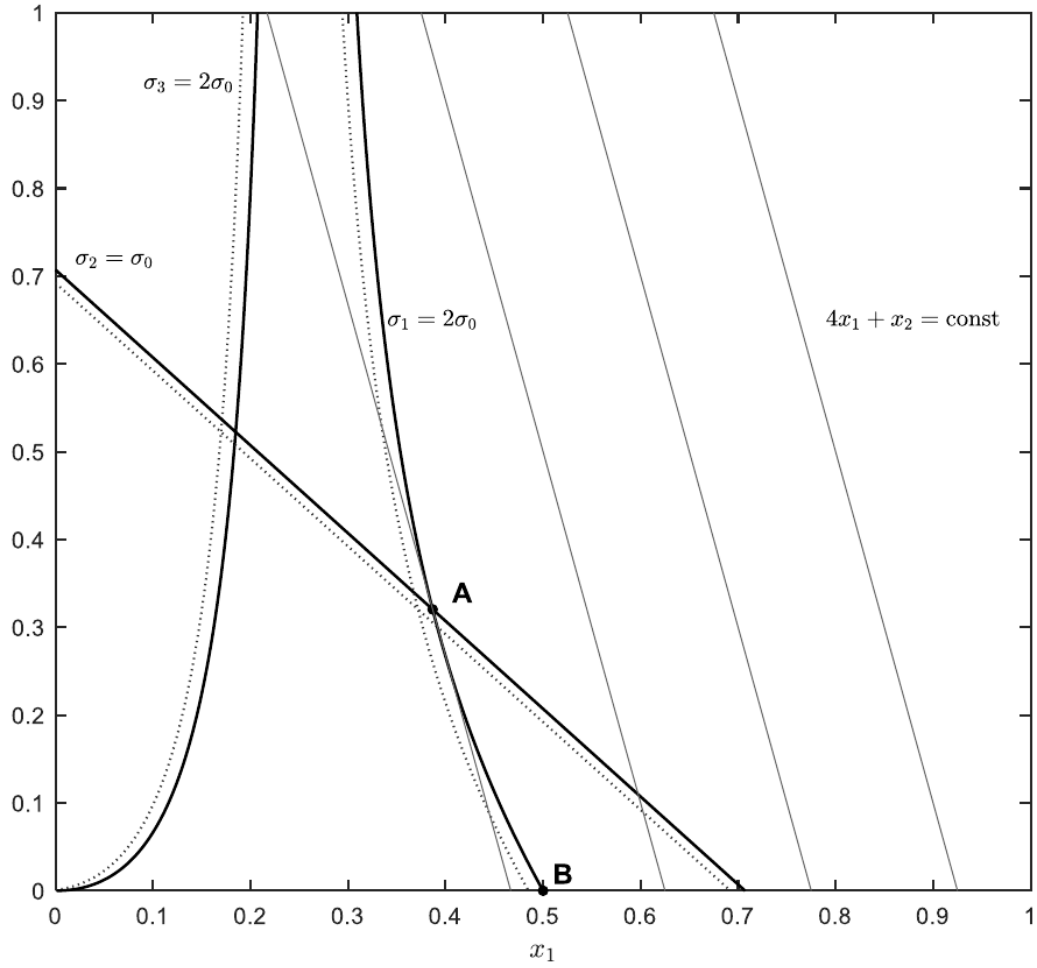
$$W_{min} = \frac{\sqrt{2}FL\rho_0}{\sigma_0} \tag{2.55}$$

Trường hợp d: $\rho_1 = 3\rho_0, \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$.

Tiếp tục khảo sát ảnh hưởng của khối lượng riêng, trong trường hợp này, ta giả định khối lượng riêng của thanh số 1 tăng lên đáng kể, cụ thể là gấp 3 lần so với ban đầu ($\rho_1 = 3\rho_0$). Sự thay đổi này làm biến đổi hàm mục tiêu, trong khi tập hợp các ràng buộc vẫn giữ nguyên như trong mô hình $(\mathbb{SO})^{5b}_{\text{nf}}$. Bài toán tối ưu hóa lúc này được thiết lập lại như sau:

$$(\mathbb{SO})^{5d}_{\text{nf}} \quad \begin{cases} \min_{x_1,x_2} 4x_1 + x_2 \\ \text{s.t. các ràng buộc trong } (\mathbb{SO})^{5b}_{\text{nf}}. \end{cases} \tag{2.56}$$

Quan sát đồ thị minh họa Hình 2.11, ta dễ dàng nhận thấy lời giải tối ưu nằm tại giao điểm của hai đường cong ràng buộc σ_1 và σ_2. Điều này chỉ ra rằng cả hai ràng buộc ứng suất này đều đang ở trạng thái hoạt động tại điểm tối ưu.



Hình 2.11. Trường hợp d. Điểm A là lời giải tối ưu.

Đáng chú ý, tọa độ của điểm tối ưu này hoàn toàn trùng khớp với kết quả điểm A mà chúng ta đã tính toán được trong trường hợp c. Khi chuyển đổi trở lại các biến thiết kế ban đầu, lời giải tối ưu là:

$$A_1^* = \frac{F}{\sigma_0}\left(\frac{4+\sqrt{2}}{14}\right), \quad A_2^* = \frac{F}{\sigma_0}\left(\frac{6\sqrt{2}-4}{14}\right). \tag{2.57}$$

Tương ứng với phương án thiết kế này, trọng lượng tối ưu của hệ kết cấu được xác định bởi công thức:

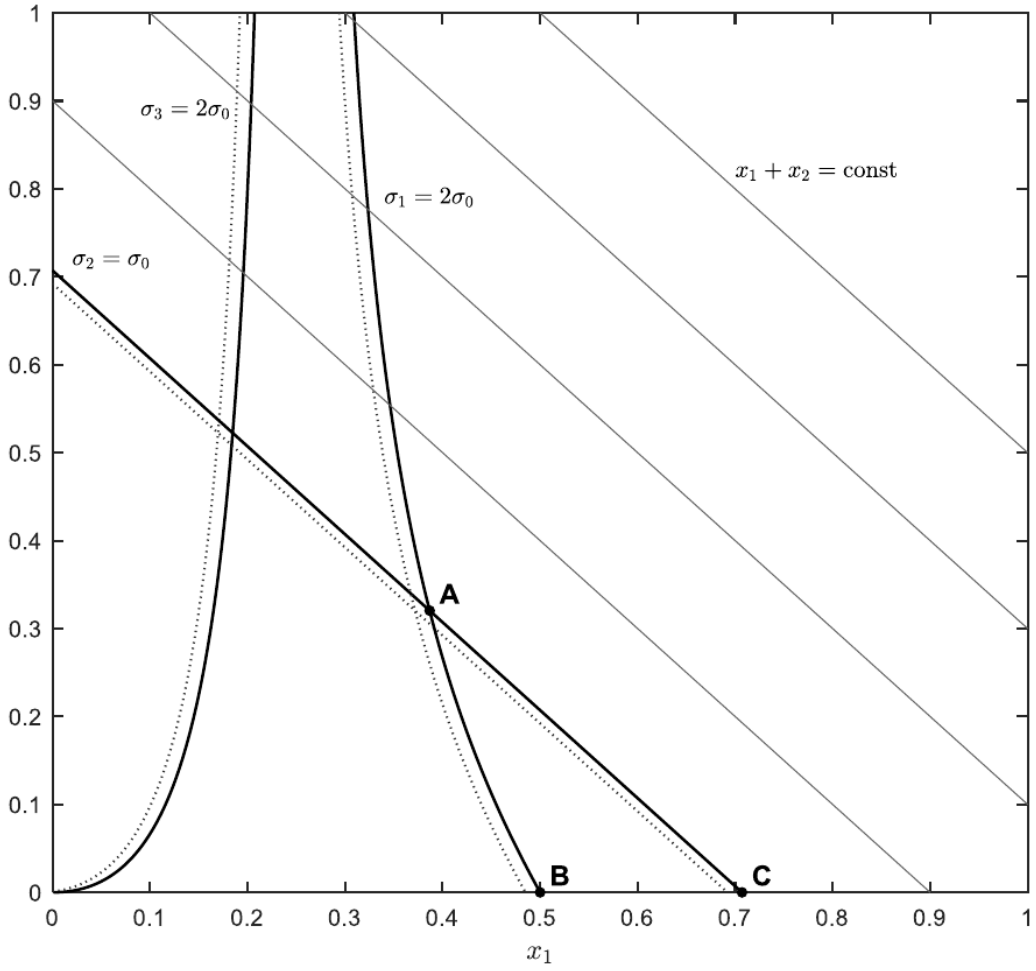
$$W_{min} = \frac{FL\rho_0}{\sigma_0}\left(\frac{6+5\sqrt{2}}{7}\right) \tag{2.58}$$

Trường hợp e: $\rho_1 = \rho_3 = \rho_0, \rho_2 = 2\rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$.

Cuối cùng, chúng ta xét trường hợp e bằng cách điều chỉnh lại mô hình của trường hợp b. Cụ thể, khối lượng riêng của thanh số 2 được tăng lên gấp đôi, trong khi các thông số khác giữ nguyên. Khi sử dụng các biến không thứ nguyên tương tự như trước, bài toán tối ưu hóa được thiết lập như sau:

$$(\mathbb{SO})_{\text{nf}}^{5e} \quad \begin{cases} \min\limits_{x_1,x_2} \quad x_1 + x_2 \\ \text{s.t.} \quad \begin{cases} \frac{2x_1+x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_1) \\ \frac{1}{\sqrt{2}(x_1+x_2)} - 1 \leqslant 0 \quad \text{nếu } x_2 > 0 \quad (\sigma_2) \\ \frac{x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_3) \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.59}$$

Quan sát Hình 2.12, điểm lời giải tối ưu nằm tại vị trí B. Tại điểm này, kết cấu tối ưu đạt được bằng cách loại bỏ hoàn toàn thanh số 2. Khi chuyển đổi trở lại các biến ban đầu, lời giải tối ưu là:



Hình 2.12. Trường hợp e. Điểm B là lời giải tối ưu.

$$A_1^* = \frac{F}{2\sigma_0}, \quad A_2^* = 0 \tag{2.60}$$

Trọng lượng tối ưu tương ứng của hệ là:

$$W_{min} = \frac{FL\rho_0}{\sigma_0} \tag{2.61}$$

Một điểm đáng chú ý là kết quả này hoàn toàn trùng khớp với lời giải của trường hợp b. Nguyên nhân là thanh số 2 không tham gia vào cấu trúc tối ưu, nên việc thay đổi khối lượng riêng của nó (dù tăng gấp đôi) chỉ làm thay đổi hệ số của x_2 trong hàm mục tiêu (hàm $2x_1 + 2x_2$, được viết gọn thành $x_1 + x_2$ trong (2.59)), không ảnh hưởng đến giá trị tối ưu và các ràng buộc đang hoạt động tại B.

2.6. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc độ cứng

Trong phần này, chúng ta tiếp tục bài toán cực tiểu hóa trọng lượng của hệ giàn ba thanh đã khảo sát trước đó, nhưng thay vì ràng buộc về ứng suất, chúng ta sẽ áp đặt ràng buộc về độ cứng. Cụ thể, yêu cầu đặt ra là chuẩn Euclid của vector chuyển vị $\mathbf{u}$ không được vượt quá một giới hạn $\delta_0 > 0$ cho trước, tức là $\mathbf{u}^T\mathbf{u} \leqslant \delta_0^2$.

Xét trường hợp cụ thể với tham số $\beta = 1/10$, tức giả định rằng thanh số 3 có chiều dài gấp 10 lần các thanh 1 và 2. Thay giá trị này vào các biểu thức (2.36), ta xác định được vector chuyển vị $\mathbf{u}$:

$$\mathbf{u} = \frac{FL}{EA_1(2A_1 + 11A_2)} \begin{pmatrix} 2A_1 + 10A_2 \\ 10A_2 \end{pmatrix}. \tag{2.62}$$

Khi đó, ràng buộc độ cứng được triển khai cụ thể như sau:

$$\mathbf{u}^T\mathbf{u} = \frac{F^2L^2(4A_1^2 + 200A_2^2 + 40A_1A_2)}{E^2A_1^2(2A_1 + 11A_2)^2} \leqslant \delta_0^2 \tag{2.63}$$

Giả sử khối lượng riêng của tất cả các thanh đều là ρ_0, hàm mục tiêu (trọng lượng W của hệ) được xác định bởi:

$$W = \rho_0 L(11A_1 + A_2) \tag{2.64}$$

Để thuận tiện cho việc tính toán, ta đưa vào các biến thiết kế không thứ nguyên:

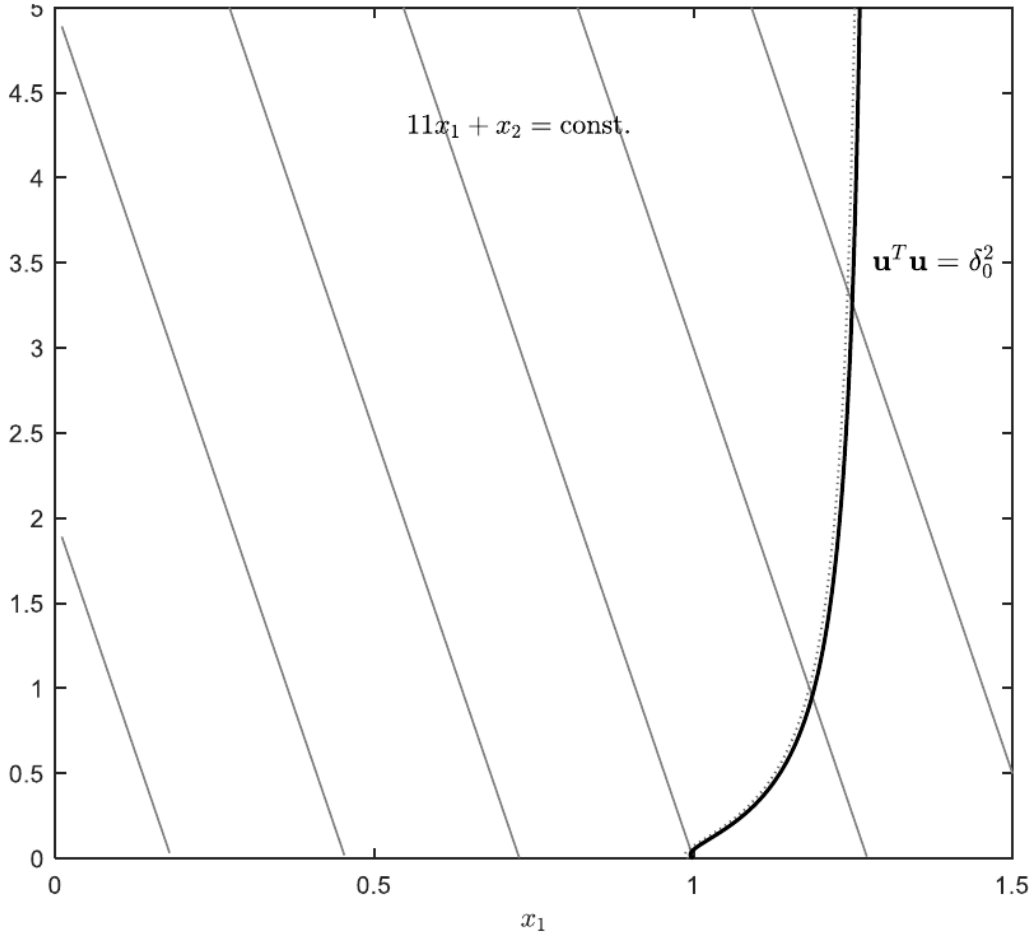
$$x_i = \frac{E\delta_0}{FL}A_i, \quad i = 1,\ 2 \tag{2.65}$$

Bài toán tối ưu hóa được thiết lập lại theo các biến này như sau:

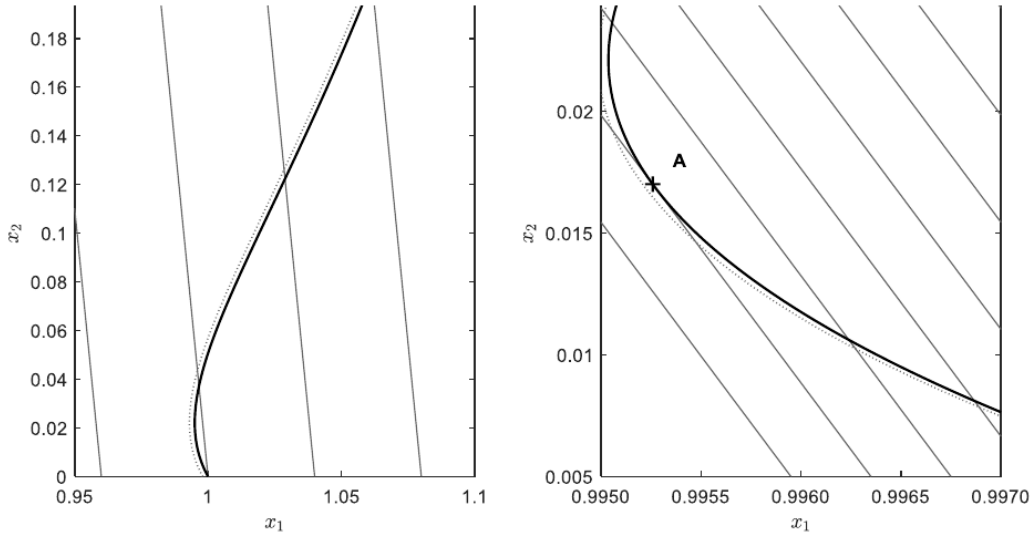
$$(\mathbb{SO})^6_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} & 11x_1 + x_2 \\ \text{s.t.} & \begin{cases} \frac{4x_1^2+200x_2^2+40x_1x_2}{x_1^2(2x_1+11x_2)^2} - 1 \leqslant 0 \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.66}$$

Bài toán được minh họa trong Hình 2.13. Thoạt nhìn, lời giải tối ưu dường như nằm tại điểm $x_1 = 1, x_2 = 0$. Tuy nhiên, hình phóng to ở Hình 2.14 cho thấy nhận định này không chính xác.

Để tìm nghiệm thực sự, ta giả định ràng buộc độ cứng là ràng buộc hoạt động. Từ phương trình ràng buộc này, ta biểu diễn x_2 theo x_1, sau đó thế vào hàm mục tiêu để giải bài toán tối ưu hóa một biến đối với x_1. Kết quả tính toán cho ra nghiệm tối ưu là:



Hình 2.13. Minh họa bài toán $(\mathbb{SO})^6_{\text{nf}}$.



Hình 2.14. Điểm A là lời giải tối ưu bài toán $(\mathbb{SO})^6_{\text{nf}}$.

$$x_1^* = 0{,}995, \qquad x_2^* = 0{,}0169 \tag{2.67}$$

Từ (2.65), diện tích tối ưu là $A_i^* = x_i^* FL/(E\delta_0)$, tức $A_1^* \approx 0{,}995\, FL/(E\delta_0)$ và $A_2^* \approx 0{,}0169\, FL/(E\delta_0)$.

Mã nguồn MATLAB vẽ các hình minh họa lời giải bằng đồ thị trong chương này được cung cấp trên GitHub: Code 2.1 (Hình 2.2), Code 2.2 (Hình 2.5), Code 2.3 (Hình 2.8), Code 2.4 (Hình 2.9), Code 2.5 (Hình 2.10), Code 2.6 (Hình 2.11), Code 2.7 (Hình 2.12), Code 2.8 (Hình 2.13) và Code 2.9 (Hình 2.14).

2.7. Bài tập

Các Bài 2.1–2.5 được phỏng theo các bài tập Chương 2 trong [10]; Bài 2.6 phỏng theo một ví dụ trong [40, Chương 1], số liệu đã được quy đổi sang hệ SI.

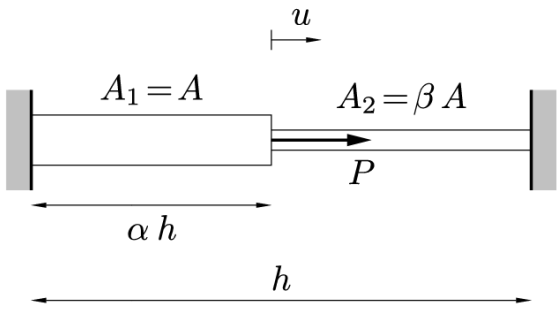
Bài 2.1: Điều gì xảy ra nếu $F < 0$ trong ví dụ ở Mục 2.1?

Bài 2.2: Kiểm chứng các bước suy ra nghiệm (x_1^*, x_2^*) và (x_1^{**}, x_2^{**}) trong Mục 2.4.

Bài 2.3: Nếu chiều dài của thanh thứ hai trong ví dụ ở Mục 2.5 thay đổi, thì cấu trúc tối ưu của hệ giàn cũng thay đổi: diện tích tối ưu của thanh thứ hai bằng 0 khi $l_2 \geqslant L$ với $\beta = 1$, $\rho_i = \rho_0$ và $\sigma_i^{\max} = \sigma_0$, $i = 1, 2, 3$. Hãy kiểm chứng điều này cho một trường hợp đặc biệt, ví dụ $l_2 = \sqrt{2}L$.

Bài 2.4: Nghiệm của ví dụ ở Mục 2.5 thay đổi như thế nào nếu ứng suất nén cho phép lớn nhất nhỏ hơn ứng suất kéo cho phép?

Bài 2.5: Độ cứng của hệ giàn hai thanh chịu lực $P > 0$ trong Hình 2.15 được cực đại hóa bằng cách cực tiểu hóa chuyển vị $\mathbf{u}$ của nút tự do. Mô-đun đàn hồi của cả hai thanh là E. Thể tích của giàn không được phép vượt quá giá trị V_0. Tổng chiều dài của các thanh là h, trong đó thanh 1 có chiều dài αh, với α là một số nằm trong khoảng từ $\alpha_{\min}$ đến $\alpha_{\max}$. Diện tích mặt cắt ngang của các thanh lần lượt là $A_1 = A$ và $A_2 = \beta A$, với β là một số không âm.



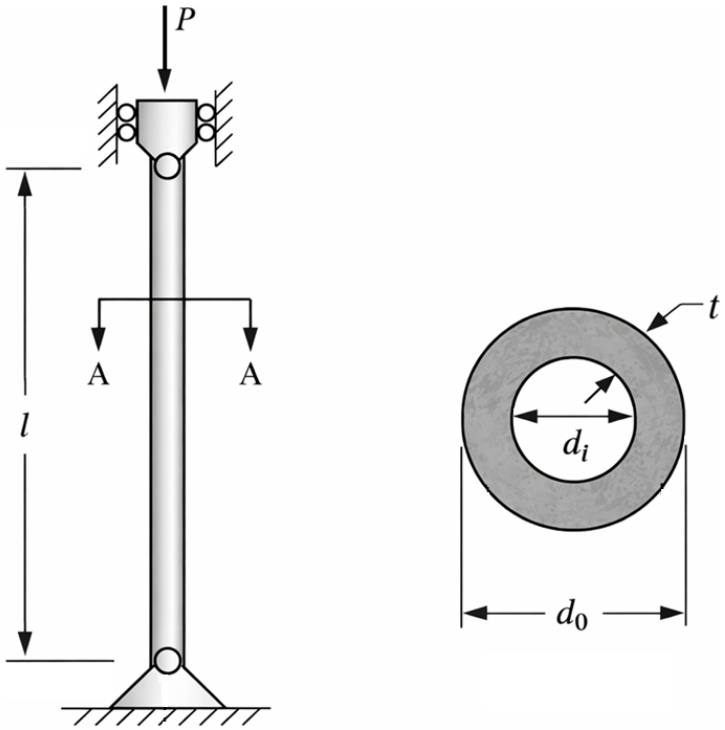
Hình 2.15. Hệ giàn hai thanh chịu lực P (Bài 2.5).

a) Chứng minh rằng bài toán có thể được lập công thức dưới dạng bài toán quy hoạch toán học sau:

$$\begin{cases} \min\limits_{\alpha,\beta} \dfrac{\alpha(1-\alpha)}{1-\alpha+\alpha\beta} \\ \text{s.t.} \quad g_1 = \alpha + (1-\alpha)\beta - \dfrac{V_0}{Ah} \leqslant 0 \\ \qquad \alpha_{\min} \leqslant \alpha \leqslant \alpha_{\max} \\ \qquad \beta \geqslant 0 \end{cases}$$

b) Cho $V_0/(Ah) = 1$ và $\alpha_{\max} = 1$. Giải bài toán cho trường hợp $\alpha_{\min}$ bất kỳ.
c) Cho $V_0/(Ah) = 1{,}2$ và $\alpha_{\min} = 0{,}2$. Giải bài toán với $\alpha_{\max} = 0{,}6$ và $\alpha_{\max} = 0{,}8$.

Bài 2.6: Thiết kế một cột có tiết diện hình ống (Hình 2.16), hai đầu liên kết bản lề, để chịu một tải trọng nén $P = 25$ kN với chi phí thấp nhất. Cột được làm từ vật liệu có ứng suất chảy là 50 MPa, mô-đun đàn hồi là 80 GPa, và khối lượng riêng là 2500 kg/m^3. Chiều dài của cột là 2,5 m. Ứng suất phát sinh trong cột phải nhỏ hơn ứng suất tới hạn mất ổn định cũng như ứng suất chảy. Đường kính trung bình của cột bị giới hạn trong khoảng từ 0,02 m đến 0,14 m, và cột có độ dày thành ống nằm trong khoảng 2 mm đến 8 mm. Chi phí của cột bao gồm chi phí vật liệu và chi phí chế tạo, được tính bằng công thức $5W + 2d$, trong đó W là khối lượng tính bằng kg và d là đường kính trung bình của cột tính bằng cm.



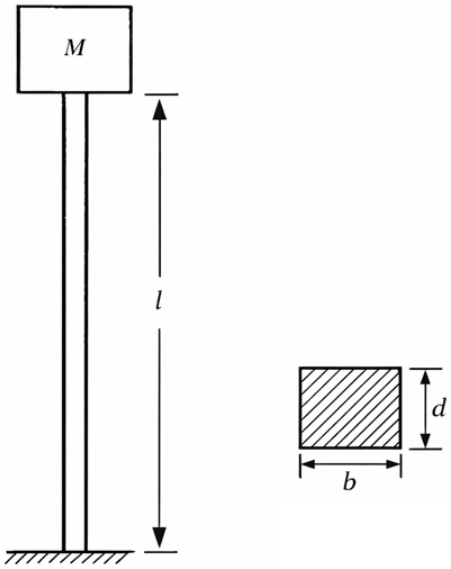
Hình 2.16. Cột tiết diện ống hai đầu liên kết bản lề chịu nén (Bài 2.6).

Bài 2.7: Một cột đồng nhất có tiết diện hình chữ nhật được thiết kế để đỡ một bồn nước có khối lượng M (Hình 2.17). Yêu cầu đặt ra là phải tối thiểu hóa khối lượng của cột để tiết kiệm chi phí, và tối đa hóa tần số dao động riêng theo phương ngang nhằm tránh khả năng xảy ra cộng hưởng do gió. Hãy thiết lập bài toán thiết kế cột để tránh hư hỏng do nén trực tiếp và mất ổn định (buckling). Giả sử ứng suất nén cho phép là σ_{max}.

Bài 2.8: Một dầm chữ nhật hẹp (Hình 2.18) có tiết diện kích thước $b \times h$ được ngàm một đầu, chiều dài $L = 6$ m, chịu lực tập trung ở đầu tự do $p = 45$ kN. Ngoài phá hoại do uốn, dầm còn có thể mất ổn định ngang kèm xoắn; tải trọng tới hạn được cho bởi

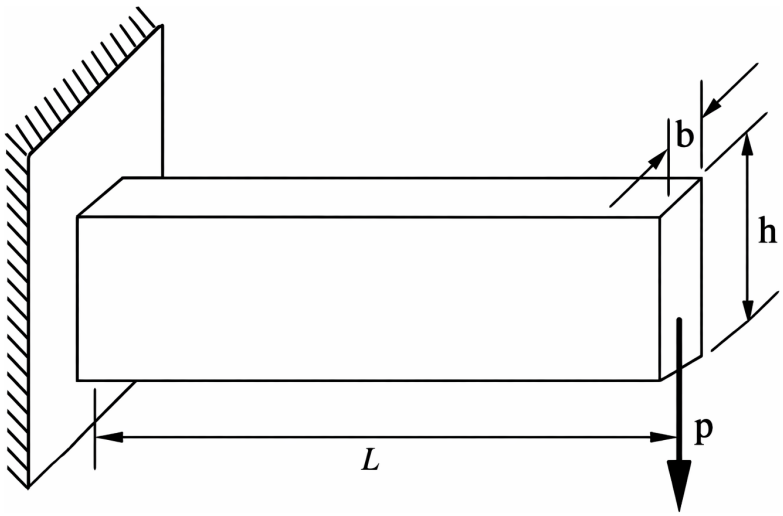
$$p_{cr} = 4{,}013\frac{\sqrt{EI_{\text{least}}c}}{L^2}$$

trong đó E là mô-đun đàn hồi, I_{least} là mô-men quán tính nhỏ nhất của tiết diện, và $c = 0{,}312hb^3G$ là độ cứng xoắn của dầm với G là mô-đun trượt. Yêu cầu thiết kế



Hình 2.17. Cột tiết diện chữ nhật đỡ bồn nước (Bài 2.7).

dầm có khối lượng nhỏ nhất sao cho không bị phá hoại cả về uốn lẫn xoắn. Giả thiết $E = 200$ GPa, $G = 80$ GPa và ứng suất cho phép $[\sigma] = 517$ MPa (kéo và nén).



Hình 2.18. Dầm công-xôn tiết diện chữ nhật hẹp chịu lực tập trung ở đầu tự do (Bài 2.8).

CHƯƠNG 3

CƠ SỞ TOÁN HỌC QUY HOẠCH LỒI

Phương pháp tiếp cận bằng đồ thị ở chương trước dựa trên giả định rằng ta có thể dễ dàng xác định các ràng buộc hoạt động (active constraints) tại điểm nghiệm tối ưu. Tuy nhiên, các bài toán thực tế thường có quy mô lớn và phức tạp, khiến việc xác định trực quan trở nên bất khả thi. Để giải quyết vấn đề đó, chúng ta cần những công cụ mạnh mẽ hơn từ lĩnh vực quy hoạch toán học (mathematical programming).

Chương này trình bày các cơ sở toán học làm nền tảng cho phần lớn các thuật toán tối ưu hóa kết cấu được sử dụng trong cuốn sách. Trước hết, ta phân biệt cực trị địa phương với cực trị toàn cục và làm rõ vì sao tính lồi (convexity) lại đóng vai trò then chốt: đối với bài toán lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục, nhờ đó việc giải bài toán trở nên đáng tin cậy. Tiếp theo, ta xây dựng các tiêu chuẩn kiểm tra tính lồi của tập hợp và hàm số, phát biểu hệ điều kiện tối ưu Karush–Kuhn–Tucker (KKT) cho bài toán có ràng buộc, và cuối cùng giới thiệu lý thuyết đối ngẫu Lagrange (Lagrangian duality) như một công cụ để phân rã và giải các bài toán quy mô lớn. Những khái niệm này sẽ được vận dụng xuyên suốt các chương sau; trình bày chi tiết hơn có thể xem trong [10, Chương 3] và [24].

3.1. Cực trị địa phương và cực trị toàn cục

Xét bài toán tối ưu hóa dưới dạng chuẩn tắc ($\mathbb{P}$):

$$(\mathbb{P}) \quad \begin{cases} \min\limits_{x} \;\; g_0(\mathbf{x}) \\ \text{s.t.} \;\; g_i(\mathbf{x}) \leqslant 0, \quad i = 1, \dots, l \\ \qquad \mathbf{x} \in \mathcal{X}, \end{cases} \tag{3.1}$$

trong đó $g_i(\mathbf{x})$ là các hàm khả vi liên tục và $\mathcal{X}$ là tập hợp các ràng buộc hộp (giới hạn trên và dưới của biến, có thể là vô hạn):

$$\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^n : x_j^{\min} \leqslant x_j \leqslant x_j^{\max}, \quad j = 1, \dots, n\}. \tag{3.2}$$

Lưu ý mọi bài toán cực đại hóa $\max g_0(\mathbf{x})$ đều có thể chuyển về bài toán cực tiểu hóa thông qua phép biến đổi tương đương: $\max g_0(\mathbf{x}) = -\min(-g_0(\mathbf{x}))$. Trong miền thiết kế này, điểm khả dĩ (feasible point) là điểm $\bar{\mathbf{x}}$ nằm trong tập khả dĩ (feasible set), thỏa mãn tất cả các ràng buộc $g_i(\bar{\mathbf{x}}) \leqslant 0$ và $\bar{\mathbf{x}} \in \mathcal{X}$. Cực tiểu toàn cục (global minimum) là điểm khả dĩ $\mathbf{x}^*$ tốt nhất trong toàn bộ không gian tìm kiếm, sao cho $g_0(\mathbf{x}^*) \leqslant g_0(\bar{\mathbf{x}})$ với mọi điểm khả dĩ $\bar{\mathbf{x}}$ khác. Cực tiểu địa phương (local minimum) là điểm khả dĩ $\mathbf{x}^*$ sao cho tồn tại $\varepsilon > 0$ để $g_0(\mathbf{x}^*) \leqslant g_0(\bar{\mathbf{x}})$ với mọi điểm khả dĩ $\bar{\mathbf{x}}$ thỏa mãn $\|\bar{\mathbf{x}} - \mathbf{x}^*\| < \varepsilon$.

Để hiểu rõ hơn bản chất của việc tìm kiếm nghiệm tối ưu, cần phân biệt giữa

bài toán không ràng buộc và bài toán có ràng buộc. Với bài toán không ràng buộc (unconstrained problem), các cực trị thường xuất hiện tại các điểm dừng (stationary points), nơi đạo hàm (gradient) triệt tiêu, tức là $\nabla g_0(\mathbf{x}^*) = \mathbf{0}$. Ngược lại, với bài toán có ràng buộc (constrained problem), cực tiểu địa phương không nhất thiết phải là điểm dừng, bởi nghiệm tối ưu có thể nằm ngay trên biên của miền khả dĩ. Đối với các bài toán tổng quát, cực tiểu địa phương cũng không nhất thiết là cực tiểu toàn cục; đây là thách thức lớn nhất của tối ưu hóa. Tuy nhiên, trong một lớp bài toán đặc biệt quan trọng là bài toán lồi (convex problems), tính chất này được đảm bảo: mọi cực tiểu địa phương đều là cực tiểu toàn cục.

3.2. Tính lồi

3.2.1. Định nghĩa tập lồi và hàm lồi

Một tập hợp $S \subset \mathbb{R}^n$ được gọi là tập lồi (convex set) nếu với mọi $\mathbf{x}_1, \mathbf{x}_2 \in S$ và mọi $\lambda \in (0, 1)$, ta có:

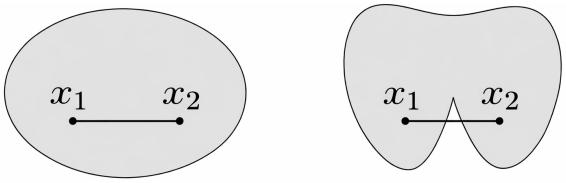
$$\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in S \tag{3.3}$$

Nghĩa là, một tập hợp được gọi là lồi nếu tất cả các điểm nằm trên đoạn thẳng nối hai điểm bất kỳ thuộc tập hợp đó cũng thuộc về tập hợp đó (xem Hình 3.1).

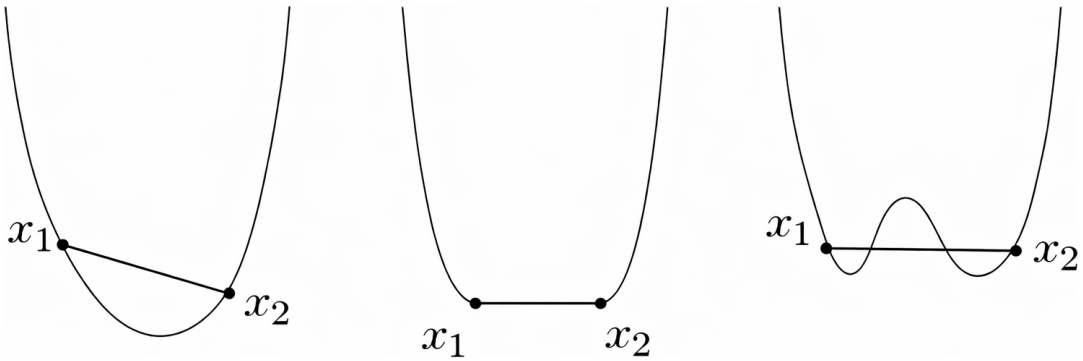
Một hàm số $f : S \to \mathbb{R}$ được gọi là lồi (trên tập lồi $S \subset \mathbb{R}^n$) nếu với mọi $\mathbf{x}_1, \mathbf{x}_2 \in S$ với $\mathbf{x}_1 \neq \mathbf{x}_2$ và mọi $\lambda \in (0, 1)$, ta có:

$$f(\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) \leqslant \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2) \tag{3.4}$$

Hàm số f là lồi chặt (strictly convex) nếu bất đẳng thức chặt ($<$) xảy ra trong biểu thức trên. Hàm số f là lõm (concave) nếu $-f$ là lồi, và lõm chặt (strictly concave) nếu $-f$ là lồi chặt. Các trường hợp này được minh họa trên Hình 3.2.

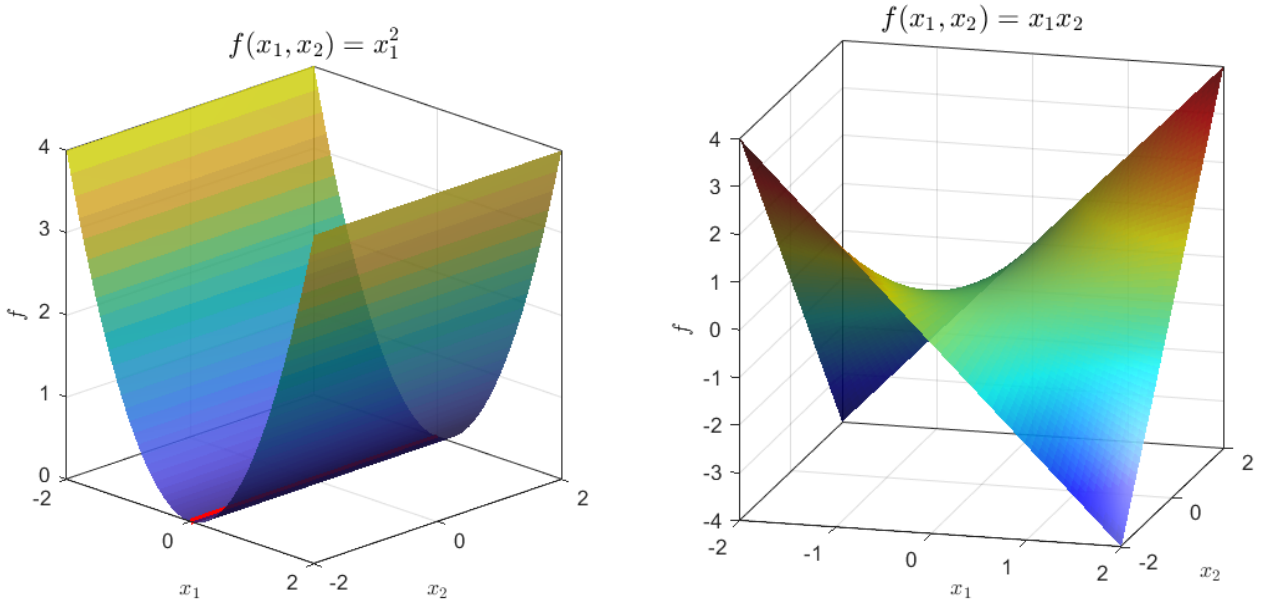


Hình 3.1. Tập lồi (trái) và không lồi (phải).



Hình 3.2. Hàm lồi chặt (trái), lồi (giữa) và không lồi (phải).

Ví dụ 3.1: Kiểm tra tính lồi của các hàm số sau:
$f : \mathbb{R} \to \mathbb{R}, f(x_1) = x_1^2$
$f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1^2$
$f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1 x_2$
Hàm $f : \mathbb{R} \to \mathbb{R}, f(x_1) = x_1^2$ là hàm lồi chặt.
Hàm $f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1^2$ là hàm lồi.
Hàm $f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1 x_2$ là hàm không lồi và cũng không lõm (xem Hình 3.3).



Hình 3.3. Kiểm tra tính lồi của các hàm số trong Ví dụ 3.1.

3.2.2. Tính chất của bài toán tối ưu lồi

Bằng cách áp dụng các định nghĩa về tập lồi và hàm lồi, ta dễ dàng thu được các tính chất sau:

(i) Tập $S = \{\mathbf{x} \in \mathcal{X} : g_i(\mathbf{x}) \leqslant 0, i = 1, \ldots, l\}$ là lồi nếu các hàm $g_i : \mathbb{R}^n \to \mathbb{R}, i = 1, \ldots, l$ là các hàm lồi.

(ii) Cho S là một tập lồi. Nếu $f : S \to \mathbb{R}$ và $g : S \to \mathbb{R}$ là các hàm lồi và $h : S \to \mathbb{R}$ là lồi chặt, thì αf là hàm lồi với $\alpha \geqslant 0$ là một số tùy ý, $f + g$ là hàm lồi và $f + h$ là hàm lồi chặt.

Nếu cả hàm mục tiêu và tập khả dĩ của bài toán $(\mathbb{P})$ đều lồi, bài toán được gọi là bài toán lồi.

3.2.3. Kiểm tra tính lồi của hàm khả vi

Để xác định xem một hàm khả vi liên tục có lồi hay không, chúng ta có thể xem xét gradient (đạo hàm cấp 1) hoặc ma trận Hessian (đạo hàm cấp 2) của nó; đây là cơ sở cho các tiêu chuẩn kiểm tra tính lồi được trình bày trong các định lý tiếp theo.

Định lý 3.1: Cho $f : S \to \mathbb{R}$, trong đó S là tập lồi và f khả vi liên tục. Khi đó f là lồi (hoặc lồi chặt) khi và chỉ khi gradient ∇f là đơn điệu (hoặc đơn điệu chặt).

Một hàm $g : S \to \mathbb{R}^n$ gọi là đơn điệu (monotone) trên S nếu với mọi $\mathbf{x}_1, \mathbf{x}_2 \in S$ với $\mathbf{x}_1 \neq \mathbf{x}_2$ ta có:

$$(\mathbf{x}_2 - \mathbf{x}_1)^T (g(\mathbf{x}_2) - g(\mathbf{x}_1)) \geqslant 0 \tag{3.5}$$

Hàm g là đơn điệu chặt (strictly monotone) trên S nếu bất đẳng thức ở (3.5) là chặt ($>$).

Ví dụ 3.2: Hàm $f : \mathbb{R} \to \mathbb{R}, f(x) = x^4$ là lồi chặt trên $\mathbb{R}$ vì gradient $\nabla f(x) = 4x^3$ là đơn điệu chặt trên $\mathbb{R}$ (bỏ qua hệ số dương 4):

$$\begin{aligned}(x_2 - x_1)(x_2^3 - x_1^3) &= (x_2 - x_1)^2(x_1^2 + x_1x_2 + x_2^2) \\ &= (x_2 - x_1)^2 \left[\left(x_1 + \frac{1}{2}x_2\right)^2 + \frac{3}{4}x_2^2\right] > 0,\ x_1 \neq x_2\end{aligned}$$

Định lý 3.2: Cho $f : S \to \mathbb{R}$, trong đó S là tập lồi mở và f khả vi liên tục hai lần. Khi đó (với mọi $\mathbf{x} \in S$):

(i) f là lồi khi và chỉ khi ma trận Hessian $\nabla^2 f$ là nửa xác định dương (positive semidefinite).

(ii) f là lồi chặt nếu ma trận Hessian $\nabla^2 f$ là xác định dương (positive definite).

trong đó ma trận Hessian $\nabla^2 f(\mathbf{x})$ được định nghĩa bởi:

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_n^2} \end{bmatrix} \tag{3.6}$$

Một ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ là nửa xác định dương nếu $\mathbf{y}^T\mathbf{A}\mathbf{y} \geqslant 0$ với mọi $\mathbf{y} \in \mathbb{R}^n$ và xác định dương nếu $\mathbf{y}^T\mathbf{A}\mathbf{y} > 0$ với mọi $\mathbf{y} \in \mathbb{R}^n$ mà $\mathbf{y} \neq \mathbf{0}$. Tính xác định dương của một ma trận đối xứng có thể được kiểm tra bằng tiêu chuẩn Sylvester như sau:

Định lý 3.3: Một ma trận đối xứng $\mathbf{A} \in \mathbb{R}^{n \times n}$ là xác định dương khi và chỉ khi tất cả các định thức con cấp k ở góc trên bên trái đều dương với mọi $k = 1, \ldots, n$.

Lưu ý: tiêu chuẩn này không mở rộng trực tiếp cho tính nửa xác định dương bằng cách thay "$>$" bởi "$\geqslant$"; khi đó cần xét tất cả các định thức con chính (không chỉ các định thức ở góc trên bên trái), hoặc kiểm tra các trị riêng đều không âm. Bên cạnh tiêu chuẩn Sylvester, tính xác định dương còn có thể được kiểm chứng thông qua các phương pháp khác như phân rã Cholesky hoặc kiểm tra các trị riêng (eigenvalues) của ma trận đều dương. Tuy nhiên, để giữ sự tập trung vào nội dung chính, cuốn sách này sẽ không đi sâu vào chi tiết các thuật toán đại số tuyến tính đó.

Ví dụ 3.3: Xét hàm $f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1^2 + x_2^2$. Khi đó:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix}, \quad \nabla^2 f(x_1, x_2) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

Ma trận Hessian là xác định dương theo tiêu chuẩn Sylvester vì $2 > 0$ và $2 \times 2 - 0 \times 0 > 0$. Do đó, theo Định lý 3.2, f là lồi chặt trên $\mathbb{R}^2$.

3.3. Điều kiện tối ưu KKT của bài toán tối ưu hóa có ràng buộc

Để xác định các điểm cực tiểu địa phương (và cũng là cực tiểu toàn cục đối với bài toán lồi), chúng ta sử dụng hệ điều kiện Karush–Kuhn–Tucker (KKT).

Xét bài toán tối ưu (3.1) ($\mathbb{P}$) có hàm mục tiêu $g_0(\mathbf{x})$ và các ràng buộc bất đẳng thức $g_i(\mathbf{x}) \leqslant 0, i = 1, \ldots, l$. Hàm Lagrange $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^l \to \mathbb{R}$ của bài toán được định nghĩa là:

$$\mathcal{L}(\mathbf{x}, \lambda) = g_0(\mathbf{x}) + \sum_{i=1}^{l} \lambda_i g_i(\mathbf{x}) \tag{3.7}$$

trong đó λ_i là các nhân tử Lagrange tương ứng với ràng buộc thứ i.

Một điểm $\mathbf{x}^* \in \mathbb{R}^n$ được gọi là điểm KKT nếu tồn tại vector nhân tử $\lambda^* \in \mathbb{R}^l$ thỏa mãn hệ điều kiện sau:

1. Điều kiện dừng (stationarity)

Đạo hàm của hàm Lagrange theo các biến thiết kế phải thỏa mãn:

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \lambda^*) = \nabla g_0(\mathbf{x}^*) + \sum_{i=1}^{l} \lambda_i^* \nabla g_i(\mathbf{x}^*) = \mathbf{0} \tag{3.8}$$

2. Điều kiện khả dĩ nguyên thủy (primal feasibility)

Nghiệm phải thỏa mãn mọi ràng buộc của bài toán:

$$g_i(\mathbf{x}^*) \leqslant 0, \quad \forall i = 1, \ldots, l \tag{3.9}$$

3. Điều kiện khả dĩ đối ngẫu (dual feasibility)

Các nhân tử Lagrange phải không âm:

$$\lambda_i^* \geqslant 0, \quad \forall i = 1, \ldots, l \tag{3.10}$$

4. Điều kiện độ lệch bù (complementary slackness)

Nhân tử Lagrange chỉ được phép dương tại các ràng buộc đang hoạt động:

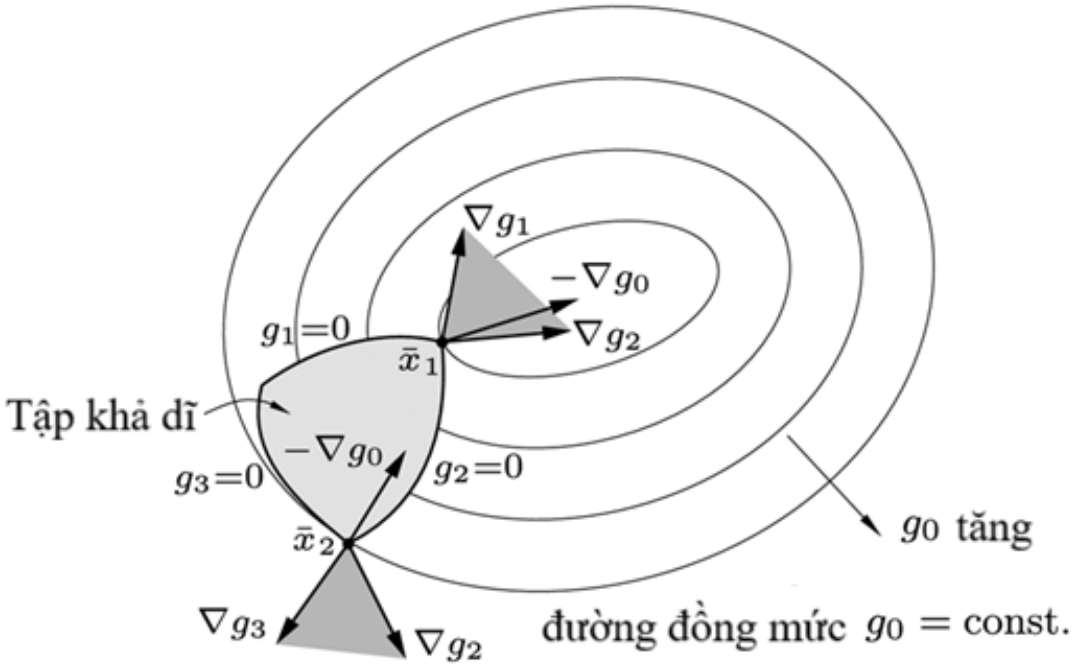
$$\lambda_i^* g_i(\mathbf{x}^*) = 0, \quad \forall i = 1, \ldots, l \tag{3.11}$$

Trong thực tế kỹ thuật, các biến thiết kế thường bị giới hạn bởi cận trên và cận dưới $x_j^{\min} \leqslant x_j \leqslant x_j^{\max}$. Thay vì đưa các giới hạn này vào danh sách $g_i(\mathbf{x})$, ta có thể xử lý chúng trực tiếp trong điều kiện dừng để giảm số lượng nhân tử Lagrange cần tìm. Khi đó, điều kiện $\nabla_{\mathbf{x}} \mathcal{L} = 0$ được thay thế bằng:

$$\begin{aligned}
&\frac{\partial \mathcal{L}}{\partial x_j} \geqslant 0 \text{ nếu } x_j = x_j^{\min} \\
&\frac{\partial \mathcal{L}}{\partial x_j} \leqslant 0 \text{ nếu } x_j = x_j^{\max} \\
&\frac{\partial \mathcal{L}}{\partial x_j} = 0 \text{ nếu } x_j^{\min} < x_j < x_j^{\max}
\end{aligned} \tag{3.12}$$

Với giả thiết các ràng buộc thỏa mãn một điều kiện chính quy (constraint qualification), chẳng hạn các gradient $\nabla g_i(\mathbf{x}^*)$ của các ràng buộc hoạt động độc lập tuyến tính (LICQ), hoặc với bài toán lồi tồn tại điểm $\hat{\mathbf{x}}$ thỏa mãn $g_i(\hat{\mathbf{x}}) < 0$ với mọi i (điều kiện Slater), điều kiện KKT là điều kiện cần: mọi cực tiểu địa phương đều là điểm KKT. Đối với bài toán không lồi, một điểm KKT chưa chắc đã là cực tiểu (nó có thể là điểm yên ngựa hoặc điểm cực đại). Đối với bài toán lồi, KKT là điều kiện đủ: mọi điểm KKT đều là cực tiểu toàn cục.

Minh họa trực quan cho định lý KKT được thể hiện tại Hình 3.4. Theo điều kiện KKT, tại một điểm khả dĩ $\bar{x}$, hướng xuống dốc nhanh nhất của hàm mục tiêu phải nằm trong nón được sinh bởi các vector gradient của các ràng buộc đang hoạt động tại điểm đó.



Hình 3.4. Minh họa hình học của các điều kiện KKT.

Xét điểm $\bar{x}_1$: tại đây, hai ràng buộc g_1 và g_2 đang ở trạng thái hoạt động, tức là $g_1(\bar{x}_1) = 0$ và $g_2(\bar{x}_1) = 0$. Quan sát hình vẽ, ta thấy vector $-\nabla g_0(\bar{x}_1)$ nằm lọt vào giữa góc tạo bởi hai vector $\nabla g_1(\bar{x}_1)$ và $\nabla g_2(\bar{x}_1)$. Điều này tương đương với việc tồn tại các hệ số không âm $\lambda_1 \geqslant 0$ và $\lambda_2 \geqslant 0$ thỏa mãn phương trình cân bằng:

$$-\nabla g_0(\bar{x}_1) = \lambda_1 \nabla g_1(\bar{x}_1) + \lambda_2 \nabla g_2(\bar{x}_1) \tag{3.13}$$

Do đó, $\bar{x}_1$ thỏa mãn các điều kiện KKT và là ứng viên nghiệm tối ưu (là nghiệm tối ưu nếu bài toán lồi).

Xét điểm $\bar{x}_2$: tại vị trí này, các ràng buộc g_2 và g_3 đang hoạt động. Tuy nhiên, vector $-\nabla g_0(\bar{x}_2)$ lại nằm lệch ra ngoài nón tạo bởi $\nabla g_2(\bar{x}_2)$ và $\nabla g_3(\bar{x}_2)$. Nói cách khác, không tồn tại bất kỳ cặp số thực không âm λ_2, λ_3 nào để biểu diễn $-\nabla g_0$ dưới dạng tổ hợp tuyến tính của hai vector gradient ràng buộc:

$$-\nabla g_0(\bar{x}_2) \neq \lambda_2 \nabla g_2(\bar{x}_2) + \lambda_3 \nabla g_3(\bar{x}_2), \quad \forall \lambda_2, \lambda_3 \geqslant 0 \tag{3.14}$$

Vì vậy, $\bar{x}_2$ vi phạm điều kiện KKT, dẫn đến việc điểm này chắc chắn không phải là nghiệm tối ưu.

Ví dụ 3.4: Tìm nghiệm tối ưu của bài toán sau:

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^2} \quad & g_0(\mathbf{x}) = x_1^2 + x_2^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = 1 - x_1 - x_2 \leqslant 0 \end{aligned}$$

Hàm mục tiêu g_0 là lồi (dạng paraboloid) và hàm ràng buộc g_1 là tuyến tính (lồi). Do đó, đây là bài toán tối ưu lồi. Bất kỳ điểm nào thỏa mãn điều kiện KKT đều là nghiệm tối ưu toàn cục.

Hàm Lagrange của bài toán được thiết lập như sau:

$$\mathcal{L}(\mathbf{x}, \lambda) = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$$

Hệ điều kiện KKT để tìm điểm cực trị $(\mathbf{x}^*, \lambda^*)$ bao gồm:
1. Điều kiện dừng:
Đạo hàm của hàm Lagrange theo biến $\mathbf{x}$ phải bằng 0:

$$\nabla_{\mathbf{x}}\mathcal{L} = \begin{bmatrix} 2x_1 - \lambda \\ 2x_2 - \lambda \end{bmatrix} = \mathbf{0} \Rightarrow x_1 = x_2 = \frac{\lambda}{2} \quad (*)$$

2. Điều kiện độ lệch bù:

$$\lambda(1 - x_1 - x_2) = 0$$

Điều kiện này dẫn đến hai trường hợp để biện luận:
Trường hợp 1: Ràng buộc không hoạt động ($\lambda = 0$).
Thay vào $(*)$, ta có $x_1 = x_2 = 0$.
Kiểm tra điều kiện khả dĩ: $g_1(0, 0) = 1 - 0 - 0 = 1 \not\leqslant 0$. (Vi phạm ràng buộc) $\Rightarrow$ Loại.
Trường hợp 2: Ràng buộc hoạt động ($\lambda > 0$).
Khi đó, biểu thức trong ngoặc phải bằng 0:

$$1 - x_1 - x_2 = 0$$

Thay $(*)$ vào phương trình trên:

$$1 - \frac{\lambda}{2} - \frac{\lambda}{2} = 0 \Rightarrow 1 - \lambda = 0 \Rightarrow \lambda = 1$$

Với $\lambda = 1$, ta tính được nghiệm tương ứng:

$$x_1^* = x_2^* = \frac{1}{2}$$

Kiểm tra điều kiện khả dĩ đối ngẫu: $\lambda^* = 1 > 0$ (Thỏa mãn).
Vậy, bộ nghiệm KKT duy nhất là:

$$\mathbf{x}^* = \left(\frac{1}{2}, \frac{1}{2}\right), \quad \lambda^* = 1$$

Vì đây là bài toán lồi, $\mathbf{x}^* = \left(\frac{1}{2}, \frac{1}{2}\right)$ chính là nghiệm tối ưu toàn cục (xem Hình 3.5).

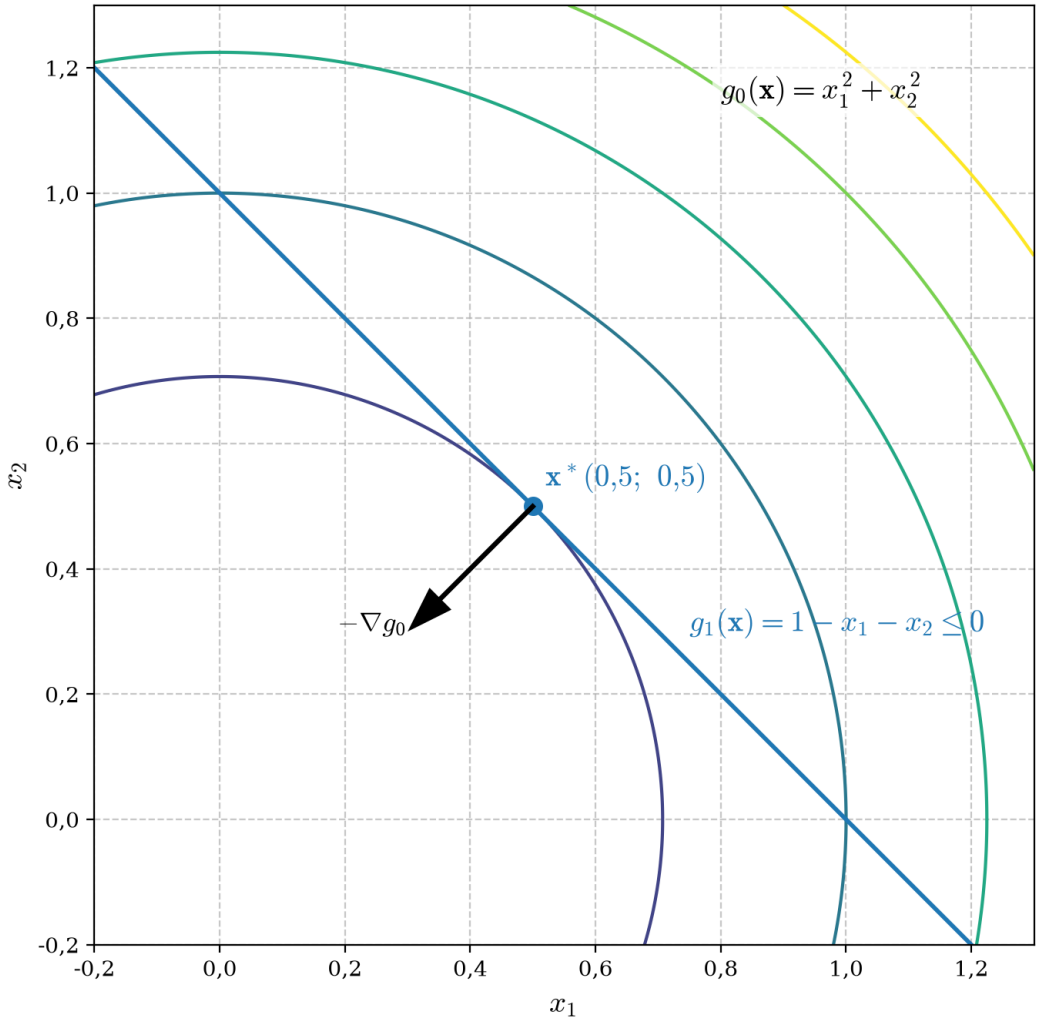
Ví dụ 3.5: Tìm nghiệm tối ưu của bài toán sau:

$$\begin{aligned} \min \quad & f(x_1, x_2) = (x_1 - 5)^2 + (x_2 - 4)^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = x_1 + 2x_2 - 10 \leqslant 0 \\ & g_2(\mathbf{x}) = x_1 - 4 \leqslant 0 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrange

$$\mathcal{L}(\mathbf{x}, \lambda) = (x_1 - 5)^2 + (x_2 - 4)^2 + \lambda_1(x_1 + 2x_2 - 10) + \lambda_2(x_1 - 4)$$

Bước 2: Thiết lập hệ phương trình KKT
Điều kiện dừng:



Hình 3.5. Miền khả dĩ và nghiệm KKT của Ví dụ 3.4.

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 2(x_1 - 5) + \lambda_1 + \lambda_2 = 0 & (1) \\ \frac{\partial \mathcal{L}}{\partial x_2} = 2(x_2 - 4) + 2\lambda_1 = 0 & (2) \end{cases}$$

Điều kiện khả dĩ nguyên thủy:

$$g_1(\mathbf{x}) \leqslant 0,\ g_2(\mathbf{x}) \leqslant 0$$

Điều kiện khả dĩ đối ngẫu:

$$\lambda_1 \geqslant 0,\ \lambda_2 \geqslant 0$$

Điều kiện độ lệch bù:

$$\lambda_1 g_1(\mathbf{x}) = 0,\ \lambda_2 g_2(\mathbf{x}) = 0$$

Bước 3: Giải hệ phương trình
Giả sử cả hai ràng buộc đều hoạt động. Khi đó $g_1 = 0$ và $g_2 = 0$.
Từ $g_2(\mathbf{x}) = x_1 - 4 = 0 \Rightarrow x_1 = 4$.
Thay vào $g_1(\mathbf{x}) = 4 + 2x_2 - 10 = 0 \Rightarrow 2x_2 = 6 \Rightarrow x_2 = 3$.

Vậy nghiệm dự đoán là $\mathbf{x}^* = (4, 3)$. Ta cần tìm λ_1, λ_2 để kiểm tra xem điểm này có thỏa mãn KKT không.

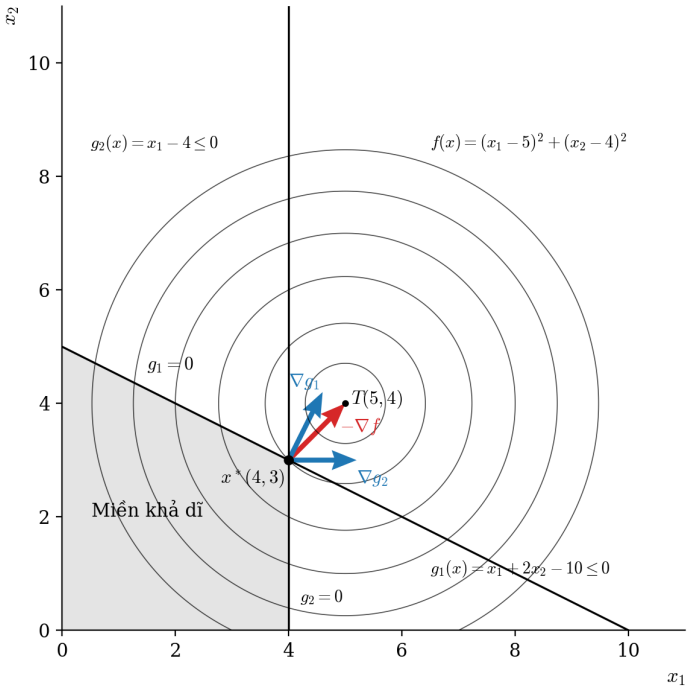
Thay $x_1 = 4, x_2 = 3$ vào phương trình (1) và (2):

Từ (2): $2(3-4) + 2\lambda_1 = 0 \Rightarrow -2 + 2\lambda_1 = 0 \Rightarrow \lambda_1 = 1$.

Từ (1): $2(4-5) + \lambda_1 + \lambda_2 = 0 \Rightarrow -2 + 1 + \lambda_2 = 0 \Rightarrow \lambda_2 = 1$.

Kiểm tra điều kiện: cả $\lambda_1 = 1 > 0$ và $\lambda_2 = 1 > 0$ đều thỏa mãn điều kiện khả dĩ đối ngẫu.

Kết luận: $\mathbf{x}^* = (4, 3)$ là nghiệm tối ưu toàn cục (bài toán lồi: hàm mục tiêu lồi chặt, các ràng buộc tuyến tính, nên điều kiện KKT là đủ). Các ràng buộc hoạt động tại nghiệm được minh họa trên Hình 3.6.



Hình 3.6. Các ràng buộc hoạt động tại nghiệm của Ví dụ 3.5.

Ví dụ 3.6: Tìm nghiệm tối ưu của bài toán sau:

$$\begin{aligned} \min \quad & f(x, y, z) = x^2 + y^2 + z^2 \\ \text{s.t.} \quad & g(x, y, z) = 3 - x - y - z \leqslant 0 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrange

$$\mathcal{L} = x^2 + y^2 + z^2 + \lambda(3 - x - y - z)$$

Bước 2: Thiết lập hệ phương trình KKT

Điều kiện dừng:

$$\nabla\mathcal{L} = 0 \Rightarrow 2x = \lambda,\ 2y = \lambda,\ 2z = \lambda \Rightarrow x = y = z = \frac{\lambda}{2}$$

Điều kiện độ lệch bù:

$$\lambda(3 - x - y - z) = 0$$

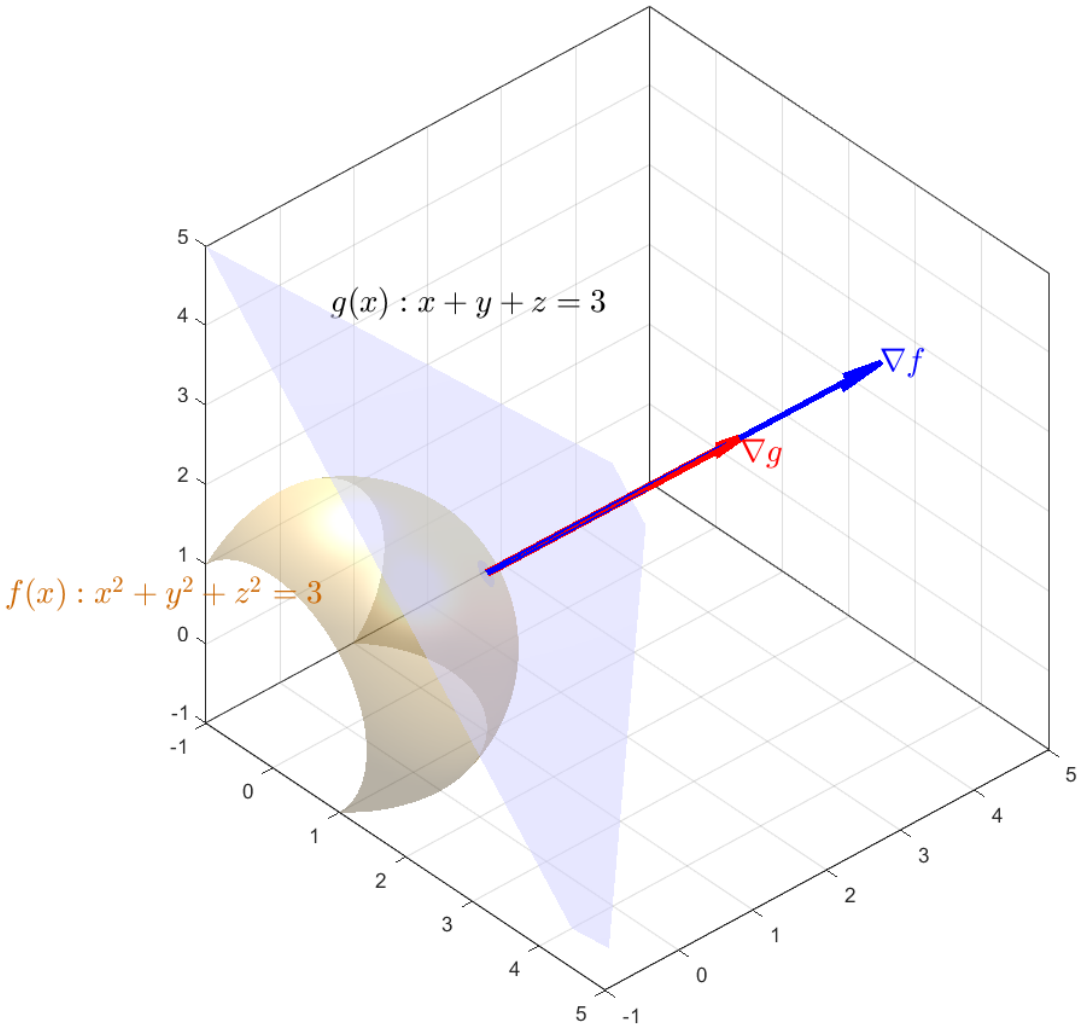
Điều kiện khả dĩ (nguyên thủy và đối ngẫu):

$$\lambda \geqslant 0, \qquad x + y + z \geqslant 3$$

Nếu $\lambda = 0 \Rightarrow x = y = z = 0$ (Loại vì $0 \ngeqslant 3$).
Nếu $\lambda > 0 \Rightarrow x + y + z = 3$. Thay $x = y = z = \frac{\lambda}{2}$ vào:

$$\frac{3\lambda}{2} = 3 \Rightarrow \lambda = 2 \Rightarrow x = y = z = 1$$

Kết luận: nghiệm tối ưu là $(1, 1, 1)$ với giá trị nhỏ nhất $f_{\min} = 3$ (xem Hình 3.7).
Mã nguồn vẽ Hình 3.3, Hình 3.5, Hình 3.6 và Hình 3.7 được cung cấp lần lượt tại Code 3.1, Code 3.2, Code 3.3 và Code 3.4 trên GitHub.



Hình 3.7. Mặt mục tiêu và ràng buộc của Ví dụ 3.6.

3.4. Đối ngẫu Lagrange

Xét một bài toán tối ưu hóa lồi $(\mathbb{P})$ với các biến số $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n$, trong đó tập ràng buộc $\mathcal{X}$ được định nghĩa bởi các giới hạn biên $x_j^{\min} \leqslant x_j \leqslant x_j^{\max}$:

$$(\mathbb{P}) \quad \begin{cases} \min\limits_{\mathbf{x}} \ g_0(\mathbf{x}) \\ \text{s.t.} \quad g_i(\mathbf{x}) \leqslant 0, \quad i = 1, \ldots, l \\ \qquad \mathbf{x} \in \mathcal{X} = \{\mathbf{x} \in \mathbb{R}^n : x_j^{\min} \leqslant x_j \leqslant x_j^{\max}, \quad j = 1, \ldots, n\}, \end{cases} \tag{3.15}$$

trong đó các hàm g_i $(i = 0, \ldots, l)$ là các hàm khả vi liên tục và lồi, g_0 lồi chặt. Xét trường hợp các hàm mục tiêu và hàm ràng buộc là tách được (separable), nghĩa là có thể được biểu diễn dưới dạng tổng của các hàm một biến thành phần:

$$g_i(\mathbf{x}) = \sum_{j=1}^{n} g_{ij}(x_j), \quad i = 0, \ldots, l. \tag{3.16}$$

Hàm Lagrange của $(\mathbb{P})$ trở thành:

$$\begin{aligned} \mathcal{L}(\mathbf{x}, \lambda) &= g_0(\mathbf{x}) + \sum_{i=1}^{l} \lambda_i g_i(\mathbf{x}) \\ &= \sum_{j=1}^{n} g_{0j}(x_j) + \sum_{i=1}^{l} \lambda_i \left(\sum_{j=1}^{n} g_{ij}(x_j) \right) \\ &= \sum_{j=1}^{n} \underbrace{\left(g_{0j}(x_j) + \sum_{i=1}^{l} \lambda_i g_{ij}(x_j) \right)}_{\mathcal{L}_j(x_j, \lambda)}, \end{aligned} \tag{3.17}$$

với $\lambda_i \geqslant 0, \ i = 1, ..., l$.

Chính cấu trúc này làm cho phương pháp đối ngẫu Lagrange trở nên đặc biệt hấp dẫn, cho phép phân rã một bài toán lớn phức tạp thành chuỗi các bài toán con đơn giản hơn. Với mọi $\lambda \geqslant \mathbf{0}$ và mọi $\mathbf{x}$ khả dĩ ta luôn có $\varphi(\lambda) \leqslant g_0(\mathbf{x})$ (đối ngẫu yếu), với φ là hàm đối ngẫu định nghĩa ở (3.19) dưới đây. Khi $(\mathbb{P})$ lồi, g_0 lồi chặt và thỏa mãn điều kiện Slater (tồn tại $\hat{\mathbf{x}} \in \mathcal{X}$ với $g_i(\hat{\mathbf{x}}) < 0$, $i = 1, \ldots, l$), giá trị tối ưu của bài toán gốc và bài toán đối ngẫu bằng nhau (đối ngẫu mạnh) và $\mathbf{x}^*(\lambda^*)$ là nghiệm của $(\mathbb{P})$. Do đó bài toán gốc $(\mathbb{P})$ có thể được giải thông qua bài toán đối ngẫu $(\mathbb{D})$:

$$(\mathbb{D}) \quad \begin{cases} \max\limits_{\lambda} \ \varphi(\lambda) \\ \text{s.t.} \quad \lambda \geqslant \mathbf{0}, \end{cases} \tag{3.18}$$

trong đó hàm mục tiêu đối ngẫu $\varphi(\lambda)$ được định nghĩa là giá trị cực tiểu của hàm Lagrange theo $\mathbf{x}$:

$$\varphi(\lambda) = \min_{\mathbf{x} \in \mathcal{X}} \mathcal{L}(\mathbf{x}, \lambda) \tag{3.19}$$

Việc cực tiểu hóa hàm Lagrange $\mathcal{L}$ trên không gian n chiều $\mathcal{X}$ có thể được quy về việc thực hiện n phép cực tiểu hóa độc lập trên không gian một chiều:

$$\varphi(\lambda) = \min_{\mathbf{x} \in \mathcal{X}} \sum_{j=1}^{n} \mathcal{L}_j(x_j, \lambda) = \sum_{j=1}^{n} \min_{x_j^{\min} \leqslant x_j \leqslant x_j^{\max}} \mathcal{L}_j(x_j, \lambda) \tag{3.20}$$

Với một λ cố định, nghiệm tối ưu x_j^* của từng bài toán con được xác định bằng cách xét đạo hàm riêng của $\mathcal{L}_j$ tại các biên:

$$\begin{aligned} \text{Nếu } \frac{\partial \mathcal{L}_j(x_j^{\min}, \lambda)}{\partial x_j} \geqslant 0 \text{ , thì } x_j^* = x_j^{\min} \\ \text{Nếu } \frac{\partial \mathcal{L}_j(x_j^{\max}, \lambda)}{\partial x_j} \leqslant 0 \text{ , thì } x_j^* = x_j^{\max} \\ \text{Các trường hợp còn lại, } x_j^* \text{ là nghiệm của } \frac{\partial \mathcal{L}_j(x_j, \lambda)}{\partial x_j} = 0 \end{aligned} \tag{3.21}$$

Đạo hàm của hàm mục tiêu đối ngẫu $\varphi(\lambda)$ được tính toán trực tiếp từ các ràng buộc ban đầu:

$$\frac{\partial \varphi(\lambda)}{\partial \lambda_i} = g_i(\mathbf{x}^*(\lambda)), \quad i = 1, \ldots, l \tag{3.22}$$

Ví dụ 3.7: Giải lại Ví dụ 3.4 bằng phương pháp đối ngẫu Lagrange.

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^2} \quad & g_0(\mathbf{x}) = x_1^2 + x_2^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = 1 - x_1 - x_2 \leqslant 0 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrange
Hàm Lagrange $\mathcal{L}(\mathbf{x}, \lambda)$ với nhân tử Lagrange $\lambda \geqslant 0$ là:

$$\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$$

Bước 2: Tìm hàm đối ngẫu $\varphi(\lambda)$
Hàm đối ngẫu là giá trị nhỏ nhất của $\mathcal{L}$ theo biến $\mathbf{x}$: $\varphi(\lambda) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda)$.
Để tìm cực tiểu, ta lấy đạo hàm riêng theo x_1, x_2 và cho bằng 0:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 - \lambda = 0 \Rightarrow x_1 = \frac{\lambda}{2} \\ \frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 - \lambda = 0 \Rightarrow x_2 = \frac{\lambda}{2} \end{cases}$$

Thay x_1, x_2 trở lại vào hàm Lagrange:

$$\varphi(\lambda) = \left(\frac{\lambda}{2}\right)^2 + \left(\frac{\lambda}{2}\right)^2 + \lambda\left(1 - \frac{\lambda}{2} - \frac{\lambda}{2}\right)$$

$$\varphi(\lambda) = \frac{\lambda^2}{2} + \lambda - \lambda^2 = \lambda - \frac{\lambda^2}{2}$$

Bước 3: Giải bài toán đối ngẫu
Bài toán đối ngẫu là tìm $\max_{\lambda \geqslant 0} \varphi(\lambda)$:

$$\max_{\lambda \geqslant 0} \left(\lambda - \frac{\lambda^2}{2}\right)$$

Đạo hàm theo λ:

$$\varphi'(\lambda) = 1 - \lambda$$

Cho $\varphi'(\lambda) = 0 \Rightarrow \lambda^* = 1$ (thỏa mãn điều kiện $\lambda \geqslant 0$).
Bước 4: Tìm nghiệm tối ưu $\mathbf{x}^*$
Thay $\lambda^* = 1$ vào biểu thức x_1, x_2 đã tìm được ở Bước 2:

$$x_1^* = \frac{1}{2}, \quad x_2^* = \frac{1}{2}$$

Kết luận:
Nghiệm tối ưu:

$$\mathbf{x}^* = (0{,}5;\ 0{,}5)$$

Giá trị tối ưu:

$$g_0(\mathbf{x}^*) = 0{,}5^2 + 0{,}5^2 = 0{,}5$$

Ví dụ 3.8: Giải lại Ví dụ 3.5 bằng phương pháp đối ngẫu Lagrange.

$$\begin{aligned} \min \quad & f(x_1, x_2) = (x_1 - 5)^2 + (x_2 - 4)^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = x_1 + 2x_2 - 10 \leqslant 0 \\ & g_2(\mathbf{x}) = x_1 - 4 \leqslant 0 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrange

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = (x_1 - 5)^2 + (x_2 - 4)^2 + \lambda_1(x_1 + 2x_2 - 10) + \lambda_2(x_1 - 4)$$

Bước 2: Tìm biểu thức của $\mathbf{x}$ theo λ
Để tìm hàm đối ngẫu $\varphi(\lambda) = \min_{\mathbf{x}} \mathcal{L}$, ta lấy đạo hàm riêng của $\mathcal{L}$ theo x_1, x_2 và cho bằng 0:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 2(x_1 - 5) + \lambda_1 + \lambda_2 = 0 \Rightarrow x_1 = 5 - \frac{\lambda_1 + \lambda_2}{2} \\ \frac{\partial \mathcal{L}}{\partial x_2} = 2(x_2 - 4) + 2\lambda_1 = 0 \qquad \Rightarrow x_2 = 4 - \lambda_1 \end{cases}$$

Bước 3: Thiết lập hàm đối ngẫu $\varphi(\lambda)$
Thay thế x_1, x_2 vừa tìm được ở trên ngược lại vào hàm $\mathcal{L}$. Sau khi rút gọn đại số, ta được hàm đối ngẫu phụ thuộc hoàn toàn vào λ:

$$\varphi(\lambda_1, \lambda_2) = -1{,}25\lambda_1^2 - 0{,}25\lambda_2^2 - 0{,}5\lambda_1\lambda_2 + 3\lambda_1 + \lambda_2$$

Bước 4: Giải bài toán đối ngẫu
Ta cần tìm cực đại của hàm $\varphi(\lambda_1, \lambda_2)$ với điều kiện $\lambda \geqslant 0$. Lấy đạo hàm riêng theo λ_1, λ_2 và cho bằng 0 để tìm điểm cực trị:

$$\begin{cases} \frac{\partial \varphi}{\partial \lambda_1} = -2{,}5\lambda_1 - 0{,}5\lambda_2 + 3 = 0 \\ \frac{\partial \varphi}{\partial \lambda_2} = -0{,}5\lambda_1 - 0{,}5\lambda_2 + 1 = 0 \end{cases}$$

Suy ra $\lambda_1 = \lambda_2 = 1$. Cả hai giá trị đều thỏa mãn điều kiện $\lambda \geqslant 0$, vậy $\lambda_1^* = 1, \lambda_2^* = 1$.
Bước 5: Tìm nghiệm tối ưu $\mathbf{x}^*$
Thay $\lambda_1^* = 1, \lambda_2^* = 1$ trở lại công thức x ở Bước 2:

$$\begin{cases} x_1^* = 5 - \frac{1+1}{2} = 5 - 1 = 4 \\ x_2^* = 4 - 1 = 3 \end{cases}$$

Kết luận:
Nghiệm tối ưu:

$$\mathbf{x}^* = (4, 3)$$

Giá trị tối ưu:

$$f(\mathbf{x}^*) = (4-5)^2 + (3-4)^2 = 1 + 1 = 2.$$

Ví dụ 3.9: Giải lại Ví dụ 3.6 bằng phương pháp đối ngẫu Lagrange.

Bước 1: Thiết lập hàm Lagrange

Hàm Lagrange $\mathcal{L}(x, y, z, \lambda)$ với nhân tử Lagrange $\lambda \geqslant 0$:

$$\mathcal{L}(x, y, z, \lambda) = (x^2 + y^2 + z^2) + \lambda(3 - x - y - z)$$

Bước 2: Tìm hàm đối ngẫu

Hàm đối ngẫu $\varphi(\lambda)$ là giá trị nhỏ nhất của $\mathcal{L}$ theo các biến x, y, z (coi λ là hằng số). Ta tìm cực tiểu bằng cách đạo hàm và cho bằng 0:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x - \lambda = 0 \Rightarrow x = \frac{\lambda}{2}$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y - \lambda = 0 \Rightarrow y = \frac{\lambda}{2}$$

$$\frac{\partial \mathcal{L}}{\partial z} = 2z - \lambda = 0 \Rightarrow z = \frac{\lambda}{2}$$

Thay các giá trị x, y, z này ngược lại vào hàm $\mathcal{L}$ để tìm $\varphi(\lambda)$:

$$\varphi(\lambda) = \left(\frac{\lambda^2}{4} + \frac{\lambda^2}{4} + \frac{\lambda^2}{4}\right) + \lambda\left(3 - \frac{\lambda}{2} - \frac{\lambda}{2} - \frac{\lambda}{2}\right)$$

$$\varphi(\lambda) = \frac{3\lambda^2}{4} + 3\lambda - \frac{3\lambda^2}{2}$$

$$\varphi(\lambda) = -\frac{3\lambda^2}{4} + 3\lambda$$

Bước 3: Giải bài toán đối ngẫu

Bài toán đối ngẫu là tìm $\max \varphi(\lambda)$ với điều kiện $\lambda \geqslant 0$:

$$\max_{\lambda \geqslant 0}\left(-\frac{3\lambda^2}{4} + 3\lambda\right)$$

Đây là hàm bậc 2 có hệ số $a < 0$ (parabol úp xuống), nên cực đại đạt tại đỉnh, nơi đạo hàm bằng 0:

$$\varphi'(\lambda) = -\frac{6\lambda}{4} + 3 = -\frac{3}{2}\lambda + 3 = 0 \Rightarrow \frac{3}{2}\lambda = 3 \Rightarrow \lambda = 2$$

Kiểm tra điều kiện: $\lambda = 2 > 0$ (thỏa mãn). Vậy nhân tử tối ưu là $\lambda^* = 2$.

Bước 4: Tìm nghiệm bài toán gốc

Thay giá trị $\lambda^* = 2$ trở lại biểu thức của x, y, z:

$$x^* = y^* = z^* = 1$$

Kết luận:
Nghiệm tối ưu:

$$(x^*,\ y^*,\ z^*) = (1,\ 1,\ 1)$$

Giá trị tối ưu:

$$f_{min} = 1^2 + 1^2 + 1^2 = 3$$

3.5. Bài tập

Bài 3.1: Tìm các điểm dừng và xác định điểm cực tiểu địa phương của hàm số sau:

$$f(X_1, X_2) = X_1^3 - X_2X_1 + X_2^2 - X_1 + X_2$$

Bài 3.2: Cho hàm số:

$$f = X_1^4 - 2X_2X_1^2 + X_2^2 + X_1^2 - 2X_1 + 5$$

Tính ∇f (vector gradient) và $\mathbf{H}$ (ma trận Hessian) tại các điểm $\mathbf{P}_1^T = \{0, 0\}$, $\mathbf{P}_2^T = \{2, 2\}$, $\mathbf{P}_3^T = \{1, 1\}$. Kiểm tra xem $\mathbf{H}$ có xác định dương tại mỗi điểm này hay không. Các điều kiện cần và đủ cho một điểm cực tiểu địa phương có được thỏa mãn tại bất kỳ điểm nào trong số này không?

Bài 3.3: Cho bài toán tối ưu hóa có ràng buộc:

$$\begin{aligned} \max\ & Z = 3X_1 + 2X_2 \\ \text{s.t.}\ & 2X_1 + X_2 \leqslant 10 \\ & X_1 + X_2 \leqslant 8 \\ & X_1 \leqslant 4 \\ & X_1, X_2 \geqslant 0 \end{aligned}$$

Kiểm tra điều kiện KKT tại các điểm: $\mathbf{P}_1^T = \{4, 2\}$, $\mathbf{P}_2^T = \{2, 6\}$, và $\mathbf{P}_3^T = \{4, 0\}$. Biểu diễn bằng đồ thị các vector gradient của các hàm ràng buộc và của hàm mục tiêu tại ba điểm nêu trên.

Bài 3.4: Cho bài toán:

$$\begin{aligned} (\min)\ & Z = X_1^2 + 2X_2^2 \\ \text{s.t.}\ & -X_1 - X_2 + 2 \leqslant 0 \end{aligned}$$

Giải bài toán này bằng phương pháp đối ngẫu.

Bài 3.5: Cho bài toán:

$$\begin{aligned} (\min)\ & Z = \frac{5}{X_1} + \frac{2}{X_2} \\ s.t.\ & X_1 + X_2 - 3 \leqslant 0 \\ & 4X_1 + X_2 - 6 \leqslant 0 \\ & X_1 + 3X_2 - 20 \leqslant 0 \\ & X_1, X_2 > 0 \end{aligned}$$

Giải bài toán này bằng phương pháp đối ngẫu.
Các Bài 3.6–3.11 được phỏng theo các bài tập Chương 3 trong [10].

Bài 3.6: Tối thiểu hóa trọng lượng của một giàn hai thanh như Hình 3.8. Chiều dài của các thanh lần lượt là $5l$ và $3l$. Mô-đun đàn hồi là E và khối lượng riêng là ρ cho cả hai thanh, và lực $P > 0$. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1 và A_2. Giàn phải đủ cứng, tức là độ mềm (compliance) phải nhỏ hơn một giá trị cho trước:

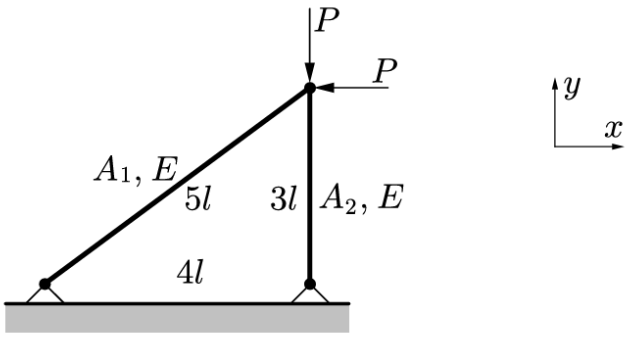
$$-Pu_x - Pu_y \leqslant c_0$$

trong đó (u_x, u_y) là các chuyển vị của nút tự do, và $c_0 > 0$ là một số cho trước.

a) Thiết lập phát biểu toán học bài toán tối ưu.

b) Thay đổi biến số sang các biến không thứ nguyên là $x_i = P/(EA_i)$ với $i = 1, 2$, và giải bài toán tối ưu hóa bằng cách sử dụng các điều kiện KKT.

c) Tương tự như câu b), nhưng giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.



Hình 3.8. Hệ giàn hai thanh chịu lực P với ràng buộc độ mềm (Bài 3.6).

Bài 3.7: Hệ giàn ba thanh trên Hình 3.9 chịu tác dụng của lực $P > 0$. Người ta muốn tối đa hóa độ cứng của giàn bằng cách tối thiểu hóa độ mềm (compliance) Pu_y của nó, trong đó u_y là chuyển vị theo phương y của nút nơi lực P được tác dụng. Thể tích của giàn không được vượt quá giá trị V_0. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1, A_2 và A_3.

a) Thiết lập phát biểu toán học bài toán tối ưu.

b) Giải bài toán tối ưu hóa bằng cách sử dụng các điều kiện KKT.

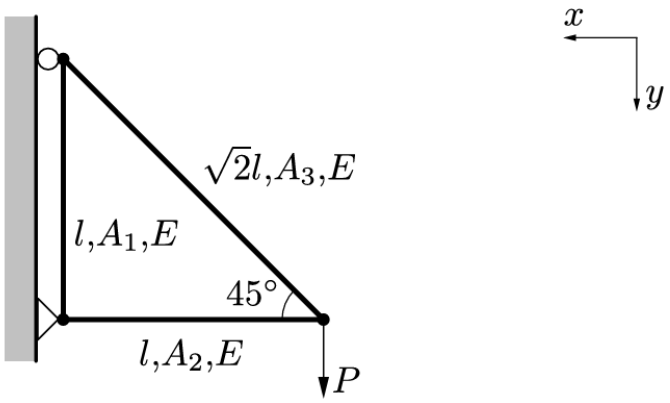
c) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.

Bài 3.8: Hệ giàn hai thanh trên Hình 3.10 chịu tác dụng của lực $P > 0$. Độ mềm $-Pu_y$ cần được tối thiểu hóa, trong đó u_y là chuyển vị theo phương y của nút tự do. Thể tích của giàn không được vượt quá giá trị V_0, và độ lớn của ứng suất trong mỗi thanh (cả kéo và nén) không được phép vượt quá giá trị $(5\alpha Pl)/(6V_0)$, với $\alpha > 0$ là một hằng số không thứ nguyên cho trước. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1 và A_2.

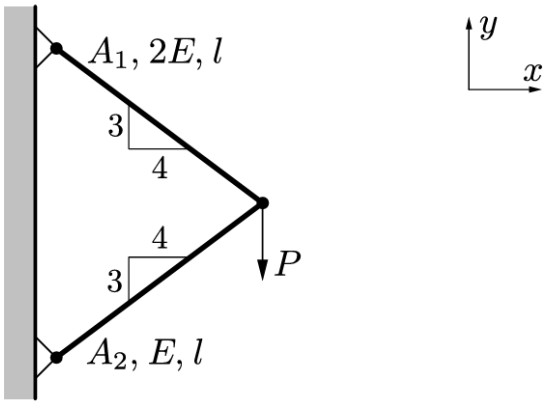
a) Thiết lập phát biểu toán học bài toán tối ưu.

b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange cho tất cả các trường hợp $\alpha > 0$.

Bài 3.9: Độ cứng của hệ giàn ba thanh trên Hình 3.11 cần được tối đa hóa. Cụ thể hơn, người ta muốn tối thiểu hóa chuẩn 1 (1-norm) của vector chuyển vị, tức là:



Hình 3.9. Hệ giàn ba thanh chịu lực P (Bài 3.7).



Hình 3.10. Hệ giàn hai thanh chịu lực P với ràng buộc thể tích và ứng suất (Bài 3.8).

$$|u_{1_x}| + |u_{1_y}| + |u_{2_x}|$$

Giàn chịu tác dụng của hai lực $P > 0$. Thể tích của giàn không được vượt quá giá trị V_0. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1, A_2 và A_3.

a) Thiết lập phát biểu toán học bài toán tối ưu.

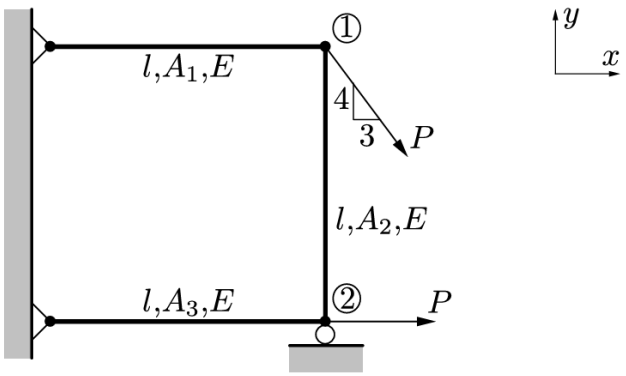
b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.

Bài 3.10: Trọng lượng của hệ giàn ba thanh trên Hình 3.12 cần được tối thiểu hóa với điều kiện giàn phải đủ độ cứng; chuyển vị nút cực đại phải thấp hơn một giá trị định trước:

$$\max(|u_1|, |u_2|, |u_3|) \leqslant u_0$$

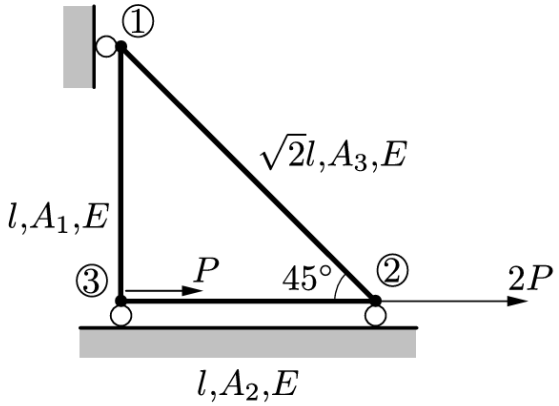
trong đó u_i là chuyển vị của nút i theo phương di động của gối tựa (mỗi nút chỉ có một bậc tự do) và $u_0 > 0$ là một đại lượng vô hướng cho trước. Giàn chịu tác dụng của hai lực đặt vào. Giả thiết $P > 0$. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1, A_2 và A_3.

a) Thiết lập phát biểu toán học bài toán tối ưu.



Hình 3.11. Hệ giàn ba thanh chịu hai lực P (Bài 3.9).

b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.



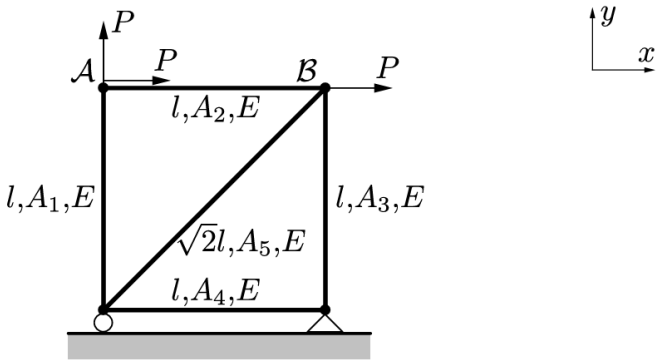
Hình 3.12. Hệ giàn ba thanh chịu hai lực P và $2P$ (Bài 3.10).

Bài 3.11: Thể tích của hệ giàn năm thanh trên Hình 3.13 cần được tối thiểu hóa với điều kiện giàn phải đủ độ cứng. Cụ thể, độ mềm phải thấp hơn giá trị c_0. Tất cả các thanh đều có mô-đun đàn hồi E. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: $A_1, A_2, \ldots, A_5$. Giàn chịu tác dụng của ba lực $P > 0$, sao cho độ mềm có thể được viết dưới dạng:

$$Pu_x^{\mathcal{A}} + Pu_y^{\mathcal{A}} + Pu_x^{\mathcal{B}}$$

trong đó $u_x^{\mathcal{A}}, u_y^{\mathcal{A}}$ là các chuyển vị của nút $\mathcal{A}$ và $u_x^{\mathcal{B}}$ là chuyển vị theo phương x của nút $\mathcal{B}$.

a) Thiết lập phát biểu toán học bài toán tối ưu.
b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.



Hình 3.13. Hệ giàn năm thanh chịu ba lực P (Bài 3.11).

CHƯƠNG 4

CÁC THUẬT TOÁN XẤP XỈ LỒI TƯỜNG MINH TUẦN TỰ

Đối với các bài toán tối ưu hóa kết cấu quy mô lớn, việc viết hàm mục tiêu và các ràng buộc dưới dạng hàm tường minh (explicit functions) của các biến thiết kế là gần như không thể. Giải pháp là tạo ra một chuỗi các bài toán con tường minh (explicit subproblems) để xấp xỉ bài toán gốc và giải chúng theo trình tự. Nội dung chương được trình bày theo [10, Chương 4].

4.1. Quy trình giải tổng quát

Xét công thức lồng ghép (nested formulation) của bài toán tối ưu hóa kết cấu như sau:

$$(\mathbb{SO})_{\text{nf}} \begin{cases} \min\limits_{x} g_0(\mathbf{x}, \mathbf{u}(\mathbf{x})) \\ \text{s.t. } g_i(\mathbf{x}, \mathbf{u}(\mathbf{x})) \leqslant 0, \quad i = 1, \ldots, l \\ \quad \mathbf{x} \in \mathcal{X} \end{cases} \tag{4.1}$$

Bài toán này được giải bằng cách tạo ra và giải một chuỗi các bài toán con xấp xỉ. Quy trình giải lặp được mô tả qua các bước sau:

- Khởi tạo: bắt đầu với thiết kế ban đầu $\mathbf{x}^0$, đặt bộ đếm vòng lặp $k = 0$.
- Phân tích kết cấu: tính vector chuyển vị $\mathbf{u}(\mathbf{x}^k)$ cho thiết kế hiện tại bằng cách thực hiện phân tích phần tử hữu hạn (Finite Element Analysis – FEA), tức là giải phương trình cân bằng $\mathbf{K}(\mathbf{x}^k)\mathbf{u}(\mathbf{x}^k) = \mathbf{F}(\mathbf{x}^k)$.
- Tính toán đạo hàm: tại thiết kế hiện tại $\mathbf{x}^k$, tính toán giá trị hàm mục tiêu $g_0(\mathbf{x}^k)$, các hàm ràng buộc $g_i(\mathbf{x}^k)$, và các gradient (đạo hàm bậc nhất) của chúng $\nabla g_i(\mathbf{x}^k)$ với $i = 0, \ldots, l$.
- Xấp xỉ hóa: thiết lập một bài toán xấp xỉ lồi, tường minh $(\mathbb{SO})^k_{\text{nf}}$ tại điểm $\mathbf{x}^k$ để thay thế cho bài toán gốc $(\mathbb{SO})_{\text{nf}}$.
- Giải bài toán con: giải bài toán tối ưu hóa phi tuyến $(\mathbb{SO})^k_{\text{nf}}$ vừa thiết lập để tìm ra thiết kế mới $\mathbf{x}^{k+1}$.
- Lặp lại: tăng bộ đếm $k = k + 1$ và quay lại bước phân tích kết cấu, trừ khi tiêu chuẩn dừng (hội tụ) đã được thỏa mãn.

4.2. Quy hoạch tuyến tính tuần tự SLP

Trong phép xấp xỉ quy hoạch tuyến tính tuần tự (Sequential Linear Programming – SLP) của bài toán $(\mathbb{SO})_{\text{nf}}$ tại $\mathbf{x}^k$, hàm mục tiêu và tất cả các phương trình ràng buộc đều được tuyến tính hóa tại điểm thiết kế $\mathbf{x}^k$; điều này dẫn đến bài toán con sau đây tại bước lặp k:

$$(\text{SLP}) \begin{cases} \min_{\mathbf{x}} \; g_0(\mathbf{x}^k) + \nabla g_0(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \\ \text{s.t.} \quad g_i(\mathbf{x}^k) + \nabla g_i(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \leqslant 0, \quad i = 1, \dots, l \\ \qquad \mathbf{x} \in \mathcal{X} \\ \qquad -l_j^k \leqslant x_j - x_j^k \leqslant u_j^k, \quad j = 1, \dots, n. \end{cases} \tag{4.2}$$

Ở đây, l_j^k và u_j^k, với $j = 1, \dots, n$, được gọi là các giới hạn dịch chuyển (move limits); chúng được sử dụng vì phép tuyến tính hóa chỉ chính xác khi ở gần thiết kế hiện tại. Các giới hạn dịch chuyển được cập nhật theo một quy tắc do người dùng định nghĩa trong quá trình lặp. Thực tế cho thấy việc lựa chọn các giới hạn dịch chuyển ảnh hưởng đáng kể đến hiệu quả của thuật toán SLP.

Vì hàm mục tiêu và tất cả các ràng buộc trong SLP đều là các hàm affine của $\mathbf{x}$ (tức là chúng có thể được viết dưới dạng $\mathbf{a}^T\mathbf{x} + b$, trong đó $\mathbf{a}$ và b là các hằng số), nên chúng là các hàm lồi. Do đó SLP là một bài toán lồi.

Lời giải Ví dụ 3.5 bằng phương pháp SLP được minh họa trong mã nguồn Code 4.1 trên GitHub.

4.3. Quy hoạch toàn phương tuần tự SQP

Bằng cách thêm một số hạng bậc hai vào khai triển Taylor của hàm mục tiêu trong SLP, ta thu được phép xấp xỉ quy hoạch toàn phương tuần tự (Sequential Quadratic Programming – SQP):

$$(\text{SQP}) \begin{cases} \min_{\mathbf{x}} \; g_0(\mathbf{x}^k) + \nabla g_0(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^k)^T\mathbf{H}(\mathbf{x}^k)(\mathbf{x} - \mathbf{x}^k) \\ \text{s.t.} \quad g_i(\mathbf{x}^k) + \nabla g_i(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \leqslant 0, \quad i = 1, \dots, l \\ \qquad \mathbf{x} \in \mathcal{X}, \end{cases} \tag{4.3}$$

trong đó $\mathbf{H}(\mathbf{x}^k)$ biểu thị một xấp xỉ xác định dương của ma trận Hessian của g_0 tại $\mathbf{x}^k$ (trong các thuật toán SQP chuẩn đây là Hessian của hàm Lagrange, thường được cập nhật bằng công thức tựa Newton BFGS). Điều này sẽ dẫn đến một hàm mục tiêu lồi, do đó SQP là bài toán lồi.

Lời giải Ví dụ 3.5 bằng phương pháp SQP được minh họa trong mã nguồn Code 4.2 trên GitHub.

4.4. Tuyến tính hóa lồi CONLIN

SLP và SQP là các phương pháp tổng quát nhưng không tận dụng được đặc điểm của bài toán kết cấu (ứng suất và chuyển vị thường tỷ lệ nghịch còn trọng lượng tỷ lệ thuận với diện tích). Phương pháp tuyến tính hóa lồi (Convex Linearization – CONLIN) do Fleury và Braibant đề xuất [18] giải quyết vấn đề này bằng cách tuyến tính hóa hỗn hợp: sử dụng biến trực tiếp x_j hoặc biến nghịch đảo $1/x_j$ tùy thuộc vào đặc tính của hàm, trong đó giả thiết các biến thiết kế luôn dương ($x_j > 0$).

Xấp xỉ được xây dựng dựa trên biến trung gian y_j, nhờ đó mỗi thành phần có thể được tuyến tính hóa theo dạng phù hợp với dấu đạo hàm. Đạo hàm riêng theo y_j được tính bằng quy tắc chuỗi:

$$\frac{\partial g_i(\mathbf{x}^k)}{\partial y_j} = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \frac{1}{\left(\frac{dy_j}{dx_j}\right)} \tag{4.4}$$

Từ đó, ta xác định phần đóng góp của từng biến vào tổng xấp xỉ cho hai trường hợp:
Trường hợp $y_j = x_j$ (biến trực tiếp):

$$g_{ij}^{L,k}(\mathbf{x}) = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}(x_j - x_j^k) \tag{4.5}$$

Trường hợp $y_j = 1/x_j$ (biến nghịch đảo):

$$g_{ij}^{R,k}(\mathbf{x}) = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}\frac{1}{\left(-\frac{1}{(x_j^k)^2}\right)}\left(\frac{1}{x_j} - \frac{1}{x_j^k}\right) = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}\frac{x_j^k(x_j - x_j^k)}{x_j} \tag{4.6}$$

Quy tắc chọn biến dựa trên dấu của đạo hàm riêng:
Nếu $\partial g_i/\partial x_j > 0$: dùng $g_{ij}^{L,k}$ (nhóm Ω_+).
Nếu $\partial g_i/\partial x_j \leqslant 0$: dùng $g_{ij}^{R,k}$ (nhóm Ω_-).
Công thức tổng quát tại bước lặp k:

$$g_i^{C,k}(\mathbf{x}) = g_i(\mathbf{x}^k) + \sum_{j\in\Omega_+}\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}(x_j - x_j^k) + \sum_{j\in\Omega_-}\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}\frac{x_j^k}{x_j}(x_j - x_j^k) \tag{4.7}$$

CONLIN luôn tạo ra xấp xỉ thận trọng (conservative) hơn SLP: $g_i^{C,k}(\mathbf{x}) \geqslant g_i^{L,k}(\mathbf{x})$. Tính chất này giúp thu hẹp tập khả dĩ quanh điểm lặp hiện tại, từ đó làm cho quá trình tối ưu hóa an toàn hơn khi các ràng buộc phi tuyến nhạy với biến thiết kế.

Do CONLIN là xấp xỉ lồi và tách biến, phương pháp đối ngẫu Lagrange thường được sử dụng. Để đảm bảo tính lồi chặt và ổn định, một số hạng $\alpha\sum(x_j - x_j^k)^2$ thường được thêm vào hàm mục tiêu.

4.5. Phương pháp tiệm cận động MMA

Phương pháp SLP và CONLIN đều có những hạn chế nhất định. Cụ thể, SLP thường thiếu ổn định và có thể không hội tụ; còn CONLIN đôi khi quá thận trọng, làm chậm tốc độ hội tụ, nhưng trong một số trường hợp khác lại chưa đủ thận trọng.

Để khắc phục, Svanberg (1987) [45] đã đề xuất phương pháp tiệm cận động (Method of Moving Asymptotes – MMA), giới thiệu các tham số linh hoạt gọi là tiệm cận (asymptotes), cho phép điều chỉnh độ cong (curvature) của hàm xấp xỉ để phù hợp với đặc điểm cụ thể của bài toán tại từng bước lặp. Thay vì tuyến tính hóa theo biến x_j hoặc $1/x_j$, MMA sử dụng các biến trung gian tổng quát dựa trên hai tiệm cận: tiệm cận dưới L_j và tiệm cận trên U_j.

Tại bước lặp k, hàm xấp xỉ $g_i^{M,k}(\mathbf{x})$ của hàm gốc $g_i(\mathbf{x})$ được định nghĩa như sau:

$$g_i^{M,k}(\mathbf{x}) = r_i^k + \sum_{j=1}^{n}\left(\frac{p_{ij}^k}{U_j^k - x_j} + \frac{q_{ij}^k}{x_j - L_j^k}\right) \tag{4.8}$$

trong đó các hệ số p_{ij}^k và q_{ij}^k được xác định dựa trên dấu của đạo hàm riêng bậc nhất để đảm bảo tính lồi và sự phù hợp về độ dốc:

$$p_{ij}^k = (U_j^k - x_j^k)^2\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}, \quad q_{ij}^k = 0 \text{ nếu } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} > 0 \tag{4.9}$$

$$p_{ij}^k = 0, \quad q_{ij}^k = -(x_j^k - L_j^k)^2 \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \text{ nếu } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \leqslant 0 \tag{4.10}$$

Số hạng tự do được xác định để đảm bảo giá trị hàm xấp xỉ bằng giá trị hàm gốc tại điểm thiết kế hiện tại $\mathbf{x}^k$:

$$r_i^k = g_i(\mathbf{x}^k) - \sum_{j=1}^{n} \left(\frac{p_{ij}^k}{U_j^k - x_j^k} + \frac{q_{ij}^k}{x_j^k - L_j^k} \right) \tag{4.11}$$

Tại mỗi bước lặp k, ta giải bài toán con tối ưu hóa sau:

$$\begin{cases} \min_{\mathbf{x}} \ g_0^{M,k}(\mathbf{x}) \\ \text{s.t.} \quad g_i^{M,k}(\mathbf{x}) \leqslant 0, \quad i = 1, \ldots, l \\ \qquad \alpha_j^k \leqslant x_j \leqslant \beta_j^k, \quad j = 1, \ldots, n \end{cases} \tag{4.12}$$

trong đó α_j^k và β_j^k là các giới hạn dịch chuyển được thiết lập để đảm bảo biến thiết kế không bao giờ chạm vào các tiệm cận (tránh lỗi chia cho 0). Chúng được chọn như sau:

$$\begin{aligned} \alpha_j^k &= \max(x_j^{\min}, L_j^k + \mu(x_j^k - L_j^k)) \\ \beta_j^k &= \min(x_j^{\max}, U_j^k - \mu(U_j^k - x_j^k)) \end{aligned} \tag{4.13}$$

trong đó $0 < \mu < 1$.

Lưu ý để đảm bảo tính lồi chặt cho hàm mục tiêu, số hạng ổn định dạng $\alpha \sum_{j=1}^{n} \left(x_j - x_j^k\right)^2$ với α là một số dương nhỏ thường được thêm vào hàm $g_0^{M,k}$.

Quy tắc cập nhật tiệm cận: MMA cập nhật vị trí các tiệm cận L_j^k, U_j^k qua từng bước lặp để điều chỉnh độ thận trọng:

Bước khởi tạo ($k = 0, 1$):

$$\begin{aligned} L_j^k &= x_j^k - s_{\text{init}}(x_j^{\max} - x_j^{\min}) \\ U_j^k &= x_j^k + s_{\text{init}}(x_j^{\max} - x_j^{\min}) \end{aligned} \tag{4.14}$$

Bước lặp $k \geqslant 2$: vị trí tiệm cận được điều chỉnh dựa trên xu hướng thay đổi của biến thiết kế $(x_j^k - x_j^{k-1})$ và $(x_j^{k-1} - x_j^{k-2})$.

Trường hợp dao động (oscillation): nếu tích hai bước nhảy trái dấu (biến tăng rồi giảm hoặc ngược lại), ta cần ổn định quá trình bằng cách thu hẹp khoảng cách tiệm cận (tăng độ thận trọng):

$$\begin{aligned} L_j^k &= x_j^k - s_{\text{slower}}(x_j^{k-1} - L_j^{k-1}) \\ U_j^k &= x_j^k + s_{\text{slower}}(U_j^{k-1} - x_j^{k-1}) \end{aligned} \tag{4.15}$$

với $s_{\text{slower}} < 1$.

Trường hợp đơn điệu (monotonic): nếu tích hai bước nhảy cùng dấu (biến tiếp tục tăng hoặc tiếp tục giảm), ta nới rộng khoảng cách tiệm cận để tăng tốc độ hội tụ:

$$\begin{aligned} L_j^k &= x_j^k - s_{\text{faster}}(x_j^{k-1} - L_j^{k-1}) \\ U_j^k &= x_j^k + s_{\text{faster}}(U_j^{k-1} - x_j^{k-1}) \end{aligned} \tag{4.16}$$

với $s_{\text{faster}} > 1$.

Có thể chứng minh MMA là dạng tổng quát bao hàm cả SLP và CONLIN: nếu $L_j^k \to -\infty$ và $U_j^k \to +\infty$ thì MMA trở thành SLP; nếu $L_j^k = 0$ và $U_j^k \to +\infty$ thì MMA trở thành CONLIN.

Ví dụ 4.1: Xét hàm số $g(x) = x + x^2 - \frac{1}{40}x^4$. Chúng ta sẽ xác định các xấp xỉ CONLIN của hàm số này tại hai điểm $\bar{x} = 1$ và $\bar{x} = 6$.

Trước hết, ta xác định đạo hàm của hàm g:

$$g_x(x) = \frac{\partial g(x)}{\partial x} = 1 + 2x - \frac{1}{10}x^3$$

Tại điểm $\bar{x} = 1$:

Ta có $g(\bar{x}) = 1{,}975$ và $g_x(\bar{x}) = 2{,}9 > 0$. Do đạo hàm mang dấu dương, xấp xỉ CONLIN (g^C) sẽ trùng với xấp xỉ tuyến tính (g^L):

$$g^C(x) = g^L(x) = 1{,}975 + 2{,}9(x - 1)$$

Hình 4.1 biểu diễn đồ thị các hàm g, g^C cùng với xấp xỉ nghịch đảo $g^R(x) = 1{,}975 + \frac{2{,}9}{x}(x - 1)$.

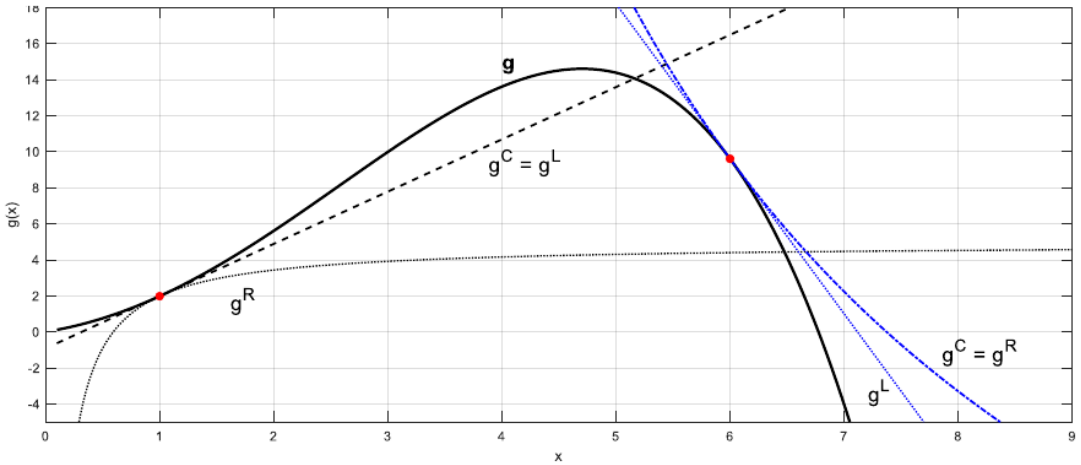
Tại điểm $\bar{x} = 6$:

Ta có $g(\bar{x}) = 9{,}6$ và $g_x(\bar{x}) = -8{,}6 < 0$. Do đạo hàm mang dấu âm, xấp xỉ CONLIN lúc này sẽ chuyển sang dạng xấp xỉ nghịch đảo (g^R):

$$g^C(x) = g^R(x) = 9{,}6 - \frac{51{,}6}{x}(x - 6)$$

Để tiện so sánh, ta cũng tính toán xấp xỉ tuyến tính tại điểm này:

$$g^L(x) = 9{,}6 - 8{,}6(x - 6)$$



Hình 4.1. Các xấp xỉ CONLIN của hàm số g.

Ví dụ 4.2: Làm lại Ví dụ 4.1, xác định xấp xỉ MMA của hàm số này tại điểm $\bar{x} = 1$.

Đạo hàm bậc nhất:

$$g'(x) = 1 + 2x - \frac{1}{10}x^3$$

Tại điểm khảo sát $x^0 = 1$, ta có $g(1) = 1{,}975$ và $g'(1) = 1 + 2 \times 1 - 0{,}1 \times 1^3 = 2{,}9$.

Theo nguyên lý của MMA, dạng của hàm xấp xỉ phụ thuộc vào dấu của đạo hàm bậc nhất tại điểm hiện tại. Vì $g'(1) = 2{,}9 > 0$ (đạo hàm dương), ta áp dụng trường hợp đạo hàm dương trong công thức (4.9). Nghĩa là xấp xỉ MMA sẽ chỉ phụ thuộc vào tiệm cận trên U^0.

Do đó, công thức tổng quát của MMA:

$$g^{M,k}(x) = r^k + \frac{p^k}{U^k - x} + \frac{q^k}{x - L^k}$$

sẽ được rút gọn thành:

$$g^{M,0}(x) = r^0 + \frac{p^0}{U^0 - x}$$

Để hàm xấp xỉ này là một xấp xỉ bậc nhất (first order approximation), nó phải khớp với hàm gốc về cả giá trị và độ dốc tại x^0.

Tính hệ số p^0 (độ cong):

Dựa theo công thức (4.9) cho trường hợp đạo hàm dương:

$$p^0 = (U^0 - x^0)^2 \frac{\partial g(x^0)}{\partial x} = 2{,}9(U^0 - 1)^2$$

Tính hệ số r^0 (số hạng tự do):

Dựa theo công thức (4.11), r^0 được chọn sao cho $g^{M,0}(1) = g(1)$:

$$r^0 = g(x^0) - \frac{p^0}{U^0 - x^0} = 1{,}975 - \frac{p^0}{U^0 - 1}$$

Ví dụ này minh họa cách vị trí của tiệm cận U^0 ảnh hưởng đến hình dạng của hàm xấp xỉ (Hình 4.2):

Khi U^0 gần x^0 (ví dụ $U^0 = 1{,}5$): mẫu số $(U^0 - x)$ biến thiên rất nhanh, tạo ra một hàm lồi có độ cong lớn. Đây là xấp xỉ rất thận trọng, an toàn nhưng bước dịch chuyển nhỏ.

Khi U^0 xa x^0 (ví dụ $U^0 = 20$): độ cong giảm đi, hàm xấp xỉ phẳng hơn.

Khi $U^0 \to +\infty$ (ví dụ $U^0 = 10^4$): số hạng $1/(U - x)$ ứng xử giống hàm tuyến tính. Xấp xỉ MMA lúc này trở về dạng SLP (xấp xỉ tuyến tính), ít thận trọng nhất.

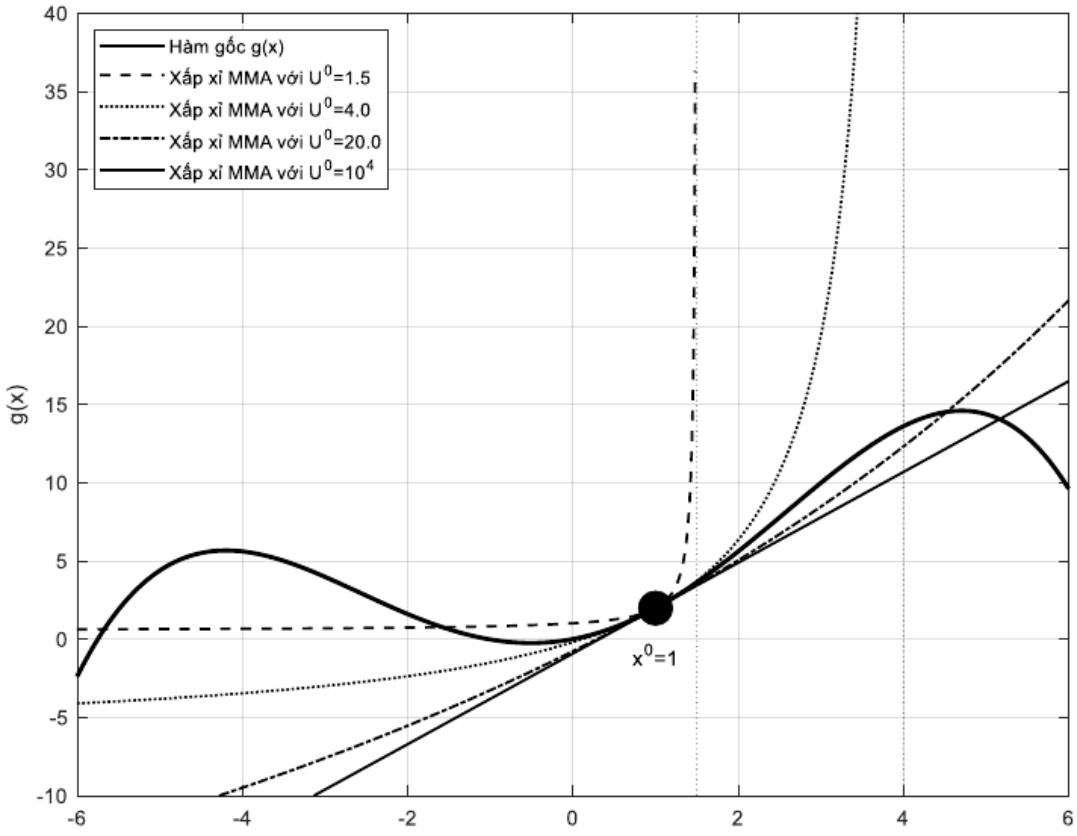
Mã nguồn vẽ Hình 4.1 và Hình 4.2 được cung cấp lần lượt tại Code 4.3 và Code 4.4 trên GitHub.

4.6. Bài tập

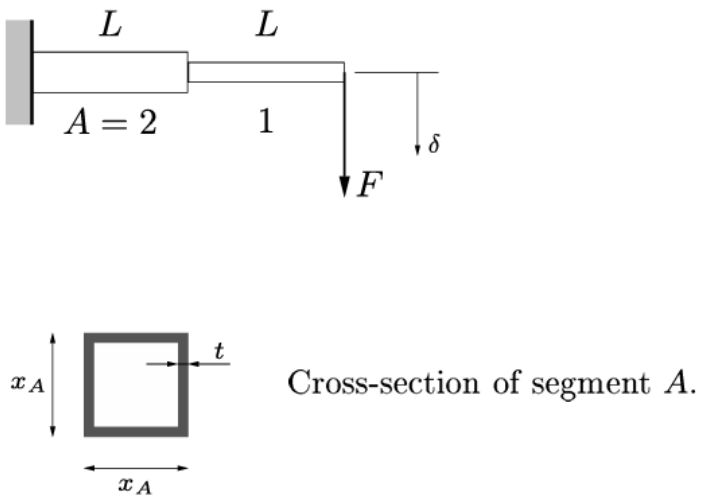
Bài 4.1: Xét bài toán tối thiểu hóa trọng lượng của một dầm công-xôn gồm hai đoạn như Hình 4.3. Bài toán tối ưu có thể được viết dưới dạng như sau:

$$(\mathbb{P})_4 \begin{cases} \min\limits_{x_1,x_2} g_0(x_1,x_2) = x_1 + x_2 \\ \text{s.t.} \begin{cases} g_1(x_1,x_2) = \frac{1}{x_1^3} + \frac{7}{x_2^3} - 1 \leqslant 0 \\ 0{,}1 \leqslant x_1 \leqslant 10, \quad 0{,}1 \leqslant x_2 \leqslant 10 \end{cases} \end{cases}$$

Giải bài toán này bằng CONLIN và MMA.



Hình 4.2. Xấp xỉ MMA của hàm g tại $x^0 = 1$ với các vị trí tiệm cận $U^0 = 1{,}5;\ 4;\ 20$ và 10^4 (Ví dụ 4.2).



Hình 4.3. Dầm công-xôn hai đoạn chịu lực tập trung ở đầu tự do (Bài 4.1).

CHƯƠNG 5

TỐI ƯU ĐỘ CỨNG GIÀN

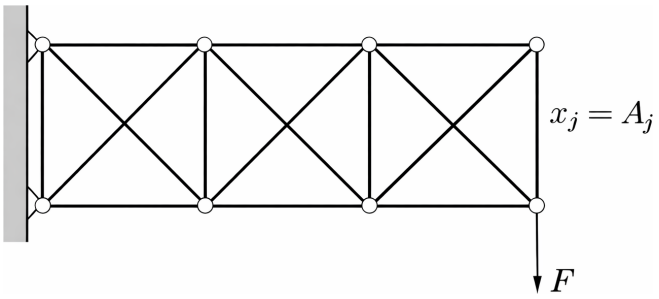
Chương này trình bày bài toán tối ưu độ cứng của hệ giàn với ràng buộc thể tích vật liệu. Sau khi thiết lập bài toán theo công thức lồng ghép và phân tích độ nhạy của độ mềm, chương giới thiệu hai cách giải: phương pháp tiêu chuẩn tối ưu (Optimality Criteria – OC) dựa trên nguyên lý thiết kế đồng ứng suất, và phương pháp tiệm cận động (MMA) đã trình bày ở Chương 4. Ví dụ số kèm mã nguồn giúp người đọc kiểm chứng thuật toán; phần bài tập mở rộng sang bài toán cực tiểu khối lượng của các hệ giàn chuẩn với ràng buộc ứng suất và chuyển vị, kèm dữ liệu mô hình để người đọc tự cài đặt. Phần lý thuyết của chương được trình bày theo [10, Chương 5].

5.1. Thiết lập bài toán tối ưu độ cứng giàn

Mục tiêu của bài toán là tìm vector tiết diện các thanh giàn $\mathbf{x}$ sao cho hệ giàn (Hình 5.1) cứng nhất, hay tương đương là có chuyển vị nhỏ nhất dưới tác dụng của ngoại lực, với một lượng vật liệu cho trước. Đây là dạng bài toán tối ưu độ cứng điển hình, trong đó biến thiết kế ảnh hưởng trực tiếp đến ma trận độ cứng và do đó ảnh hưởng đến đáp ứng chuyển vị của kết cấu.

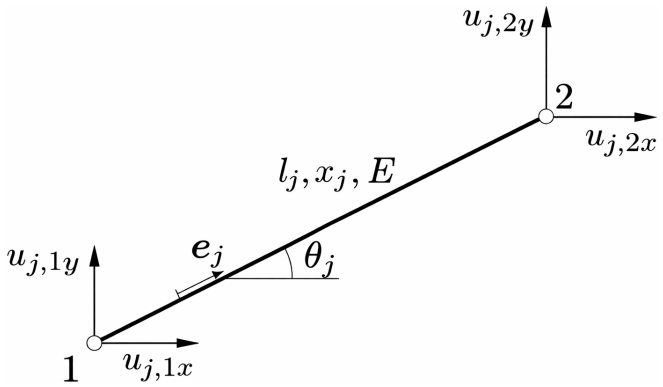
5.1.1. Công thức lồng ghép

Thay vì giải đồng thời cả chuyển vị và thiết kế, ta sử dụng thiết lập lồng ghép (nested formulation). Tại mỗi bước thiết kế, chuyển vị $\mathbf{u}$ được coi là hàm ẩn của biến thiết kế $\mathbf{x}$ thông qua phương trình cân bằng $\mathbf{K}(\mathbf{x})\mathbf{u} = \mathbf{F}$.



Hình 5.1. Hệ giàn cần tối ưu.

Bài toán tối ưu hóa $(\mathbb{P})_{\text{nf}}$ được viết như sau (người đọc tự chứng minh đây là bài toán lồi):



Hình 5.2. Phần tử thanh giàn và góc nghiêng θ_j.

$$(\mathbb{P})_{\text{nf}} \quad \begin{cases} \min\limits_{x} \;\; C(\mathbf{x}) = \mathbf{F}^T\mathbf{u}(\mathbf{x}) \\ \text{s.t.} \;\; \sum\limits_{j=1}^{n} l_j x_j \leqslant V_{\max} \\ \qquad x_j^{\min} \leqslant x_j \leqslant x_j^{\max}, \quad j = 1, \ldots, n \end{cases} \tag{5.1}$$

Hàm mục tiêu $C(\mathbf{x})$ là độ mềm (compliance), giá trị càng nhỏ thì hệ càng cứng. Biến thiết kế x_j là diện tích mặt cắt ngang của thanh thứ j. Ràng buộc là tổng thể tích vật liệu không vượt quá $V_{\max}$ và giới hạn kích thước thanh trong đoạn $[x_{\min}, x_{\max}]$.

5.1.2. Phân tích độ nhạy

Lấy đạo hàm hàm mục tiêu $C(x)$ ở (5.1) theo diện tích một thanh x_j, kết hợp với phương trình cân bằng $\mathbf{K}(x)\mathbf{u}(x) = \mathbf{F}$, ta có (lưu ý $\mathbf{K}(x)$ đối xứng):

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = \mathbf{F}^T\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = (\mathbf{K}(\mathbf{x})\mathbf{u}(\mathbf{x}))^T\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = \mathbf{u}(\mathbf{x})^T\mathbf{K}(\mathbf{x})\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j}, \tag{5.2}$$

Lấy đạo hàm phương trình cân bằng $\mathbf{K}(x)\mathbf{u}(x) = \mathbf{F}$ theo x_j:

$$\frac{\partial \mathbf{K}(\mathbf{x})}{\partial x_j}\mathbf{u}(\mathbf{x}) + \mathbf{K}(\mathbf{x})\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = 0, \tag{5.3}$$

từ đó suy ra:

$$\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = -\mathbf{K}(\mathbf{x})^{-1}\frac{\partial \mathbf{K}(\mathbf{x})}{\partial x_j}\mathbf{u}(\mathbf{x}) = -\mathbf{K}(\mathbf{x})^{-1}\mathbf{K}_j^0\mathbf{u}(\mathbf{x}), \tag{5.4}$$

với $\mathbf{K}_j^0 = \frac{\partial \mathbf{K}(\mathbf{x})}{\partial x_j}$.

Từ (5.2) và (5.4) ta có:

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\mathbf{u}(\mathbf{x})^T\mathbf{K}_j^0\mathbf{u}(\mathbf{x}) \tag{5.5}$$

Vector chuyển vị nút của thanh j (4 thành phần, trong hệ tọa độ tổng thể) $\mathbf{u}_j(\mathbf{x})$ được trích ra từ vector chuyển vị tổng thể $\mathbf{u}(\mathbf{x})$ nhờ ma trận định vị $\mathbf{T}_j$ như sau:

$$\mathbf{u}_j(\mathbf{x}) \;=\; \mathbf{T}_j\mathbf{u}(\mathbf{x}) \tag{5.6}$$

Ma trận độ cứng tổng thể được lắp ghép từ ma trận độ cứng của các thanh:

$$\mathbf{K}(\mathbf{x}) = \sum_{j=1}^{n} \mathbf{K}_j(\mathbf{x}) = \sum_{j=1}^{n} x_j \mathbf{K}_j^0 = \sum_{j=1}^{n} x_j \mathbf{T}_j^T \mathbf{k}_j^0 \mathbf{T}_j \tag{5.7}$$

Biến đổi vế phải của (5.5), với $\mathbf{K}_j^0$ lấy từ (5.7):

$$\mathbf{u}(\mathbf{x})^T \mathbf{K}_j^0 \mathbf{u}(\mathbf{x}) = \mathbf{u}(\mathbf{x})^T \mathbf{T}_j^T \mathbf{k}_j^0 \mathbf{T}_j \mathbf{u}(\mathbf{x}) = (\mathbf{T}_j \mathbf{u}(\mathbf{x}))^T \mathbf{k}_j^0 \mathbf{T}_j \mathbf{u}(\mathbf{x}) = \mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j^0 \mathbf{u}_j(\mathbf{x}), \tag{5.8}$$

Từ (5.5) và (5.8) ta có:

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j^0 \mathbf{u}_j(\mathbf{x}) \tag{5.9}$$

trong đó $\mathbf{k}_j^0$ là ma trận độ cứng đơn vị (ứng với $x_j = 1$) của thanh j trong hệ tọa độ tổng thể:

$$\mathbf{k}_j^0 = \frac{E}{l_j} \begin{bmatrix} c^2 & sc & -c^2 & -sc \\ sc & s^2 & -sc & -s^2 \\ -c^2 & -sc & c^2 & sc \\ -sc & -s^2 & sc & s^2 \end{bmatrix} \tag{5.10}$$

trong đó $s = \sin\theta_j$ và $c = \cos\theta_j$ trong Hình 5.2.

5.2. Tối ưu độ cứng giàn sử dụng phương pháp tiêu chuẩn tối ưu

5.2.1. Thiết kế đồng ứng suất

Nguyên lý thiết kế đồng ứng suất (fully stressed design) là cơ sở vật lý quan trọng nhất của phương pháp tiêu chuẩn tối ưu (OC) trong bài toán tối ưu hóa độ cứng giàn. Nguyên lý này khẳng định rằng: để sử dụng vật liệu hiệu quả nhất, tất cả các thanh có kích thước không chạm cận đều làm việc với cùng một độ lớn ứng suất $|\sigma_j|$ (không nhất thiết bằng ứng suất cho phép). Cơ sở toán học của nguyên lý này được suy dẫn trực tiếp từ điều kiện tối ưu KKT như sau.

Do bài toán tối ưu hóa độ cứng dạng lồng ghép $(\mathbb{P})_{\text{nf}}$ là một bài toán lồi, các điều kiện KKT là điều kiện cần và đủ để tìm nghiệm tối ưu. Ta thiết lập hàm Lagrange để kết hợp hàm mục tiêu và ràng buộc thể tích như sau:

$$\mathcal{L}(\mathbf{x}, \lambda) = C(\mathbf{x}) + \lambda \left(\sum_{j=1}^{n} x_j l_j - V_{\max} \right) \tag{5.11}$$

trong đó λ là nhân tử Lagrange.

Viết lại công thức (5.9) đạo hàm riêng của hàm mục tiêu theo biến thiết kế x_j:

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j^0 \mathbf{u}_j(\mathbf{x}) = -\frac{1}{x_j} \mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j \mathbf{u}_j(\mathbf{x}) = -\frac{2}{x_j} U_j \tag{5.12}$$

trong đó U_j là năng lượng biến dạng của thanh j.

Có thể viết lại U_j dưới dạng:

$$U_j = \frac{1}{2} \int \sigma_j \varepsilon_j dV_j = \frac{1}{2} E \varepsilon_j^2 x_j l_j = \frac{1}{2} \frac{\sigma_j^2}{E} x_j l_j \tag{5.13}$$

trong đó σ_j là ứng suất trong thanh j, E là mô-đun đàn hồi và l_j là chiều dài thanh.

Thế biểu thức (5.13) vào (5.12) ta được:

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\frac{\sigma_j^2}{E} l_j \tag{5.14}$$

Lấy đạo hàm hàm Lagrange (5.11) theo x_j:

$$\frac{\partial \mathcal{L}(\mathbf{x}, \lambda)}{\partial x_j} = \frac{\partial C(\mathbf{x})}{\partial x_j} + \lambda l_j \tag{5.15}$$

Thay (5.14) vào (5.15):

$$\frac{\partial \mathcal{L}(\mathbf{x}, \lambda)}{\partial x_j} = -\frac{\sigma_j^2}{E} l_j + \lambda l_j \tag{5.16}$$

Áp dụng điều kiện KKT (3.12), các điểm KKT (x^*, λ^*) thỏa mãn:

$$\begin{aligned} \sigma_j^2 &\leqslant \lambda^* E \text{ nếu } x_j^* = x_j^{\min} \\ \sigma_j^2 &= \lambda^* E \text{ nếu } x_j^{\min} < x_j^* < x_j^{\max} \\ \sigma_j^2 &\geqslant \lambda^* E \text{ nếu } x_j^* = x_j^{\max} \end{aligned} \tag{5.17}$$

Ta thấy rằng đối với tất cả các thanh có kích thước nằm trong khoảng không chạm cận trên hoặc cận dưới, bình phương ứng suất σ_j^2 là một hằng số. Điều này có nghĩa là độ lớn của ứng suất trong các thanh này là như nhau:

$$|\sigma_j| = \sqrt{\lambda^* E} = \text{const} \tag{5.18}$$

Đây chính là ý nghĩa của thiết kế đồng ứng suất. Điều này đảm bảo vật liệu được phân bố sao cho không có phần tử nào "làm việc quá sức" trong khi phần tử khác lại "nhàn rỗi", từ đó tối đa hóa độ cứng của toàn hệ kết cấu.

5.2.2. Phương pháp tiêu chuẩn tối ưu OC

Đối với bài toán tối ưu hóa độ cứng của hệ giàn chỉ chịu một ràng buộc duy nhất về thể tích vật liệu ($V \leqslant V_{max}$), phương pháp tiêu chuẩn tối ưu (OC) tỏ ra đặc biệt hiệu quả. Phương pháp này cho phép xây dựng các sơ đồ lặp đơn giản nhằm cập nhật biến thiết kế mà không cần giải các bài toán con phức tạp như trong phương pháp MMA. Do bài toán là lồi, điểm KKT mà sơ đồ OC hội tụ tới chính là nghiệm tối ưu toàn cục; tuy nhiên OC là sơ đồ heuristic nên sự hội tụ không được bảo đảm về lý thuyết và thường cần giới hạn dịch chuyển cùng hệ số cản η.

Điều kiện dừng của KKT yêu cầu đạo hàm của hàm Lagrange theo biến thiết kế (5.16) phải bằng 0 tại điểm tối ưu:

$$\frac{\partial \mathcal{L}(\mathbf{x}, \lambda)}{\partial x_j} = \frac{\partial C(\mathbf{x})}{\partial x_j} + \lambda l_j = 0 \quad \Rightarrow \quad -\frac{\partial C(\mathbf{x})}{\partial x_j} = \lambda l_j \tag{5.19}$$

Từ điều kiện tối ưu trên, ta định nghĩa một tỷ số dạng kinh nghiệm (heuristic) B_j đại diện cho tỷ lệ giữa lợi ích (giảm độ mềm) và chi phí (tăng thể tích):

$$B_j = \frac{-\frac{\partial C}{\partial x_j}}{\lambda l_j} \tag{5.20}$$

Sơ đồ lặp cập nhật biến thiết kế như sau:

- Nếu $B_j > 1$: thanh chịu lực lớn, cần tăng diện tích.
- Nếu $B_j < 1$: thanh chịu lực nhỏ, cần giảm diện tích.

Công thức cập nhật biến thiết kế có tính đến hệ số cản η để ổn định hội tụ, thường chọn $\eta = 0{,}5$ như sau:

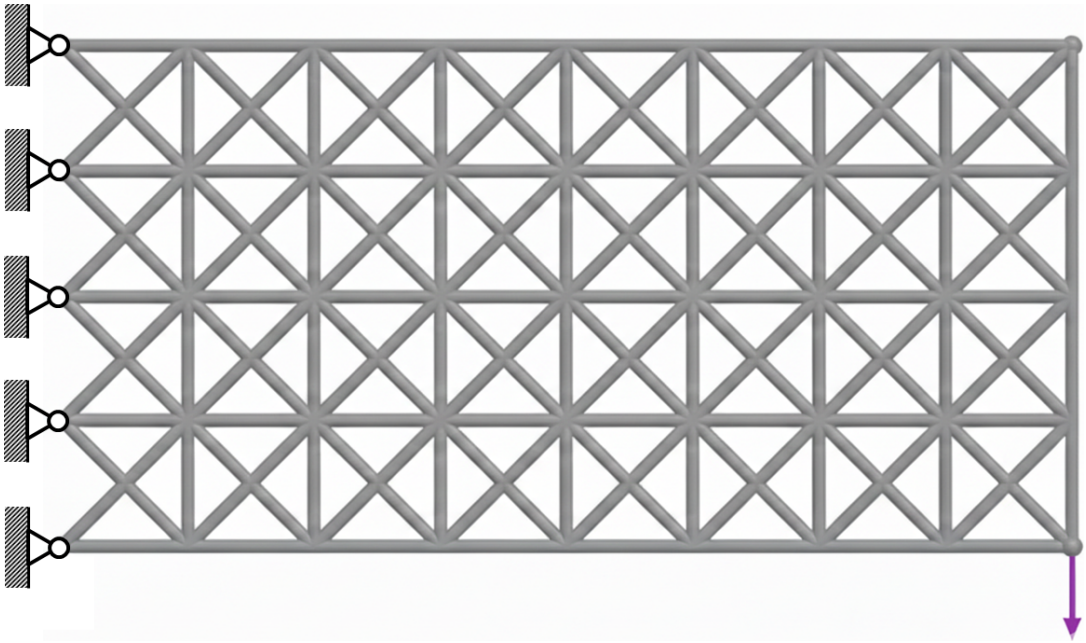
$$x_j^{new} = x_j^{old}(B_j)^{\eta} = x_j^{old}\sqrt{\frac{-\frac{\partial C}{\partial x_j}}{\lambda l_j}} \tag{5.21}$$

Trong thực hành, giá trị cập nhật (5.21) còn bị chặn bởi giới hạn dịch chuyển m (Code 5.2 dùng $m = 0{,}2$) và các cận:

$$x_j^{new} = \max\left\{x_j^{\min}, x_j^{old} - m, \min\left[x_j^{\max}, x_j^{old} + m, x_j^{old}B_j^{\eta}\right]\right\}$$

Nhân tử Lagrange λ trong công thức (5.20) được xác định tại mỗi vòng lặp bằng phương pháp chia đôi (bisection method) sao cho tổng thể tích của kết cấu thỏa mãn chính xác $V_{\max}$.

Ví dụ 5.1: Tối ưu hóa hệ giàn bằng phương pháp tiêu chuẩn tối ưu với các thông số: khoảng cách nút, lực tập trung và mô-đun đàn hồi đều bằng 1 đơn vị (Hình 5.3). Ràng buộc tỷ lệ thể tích là 0,1; biến thiết kế (diện tích thanh) trong đoạn $\left[10^{-9},\ 1\right]$.

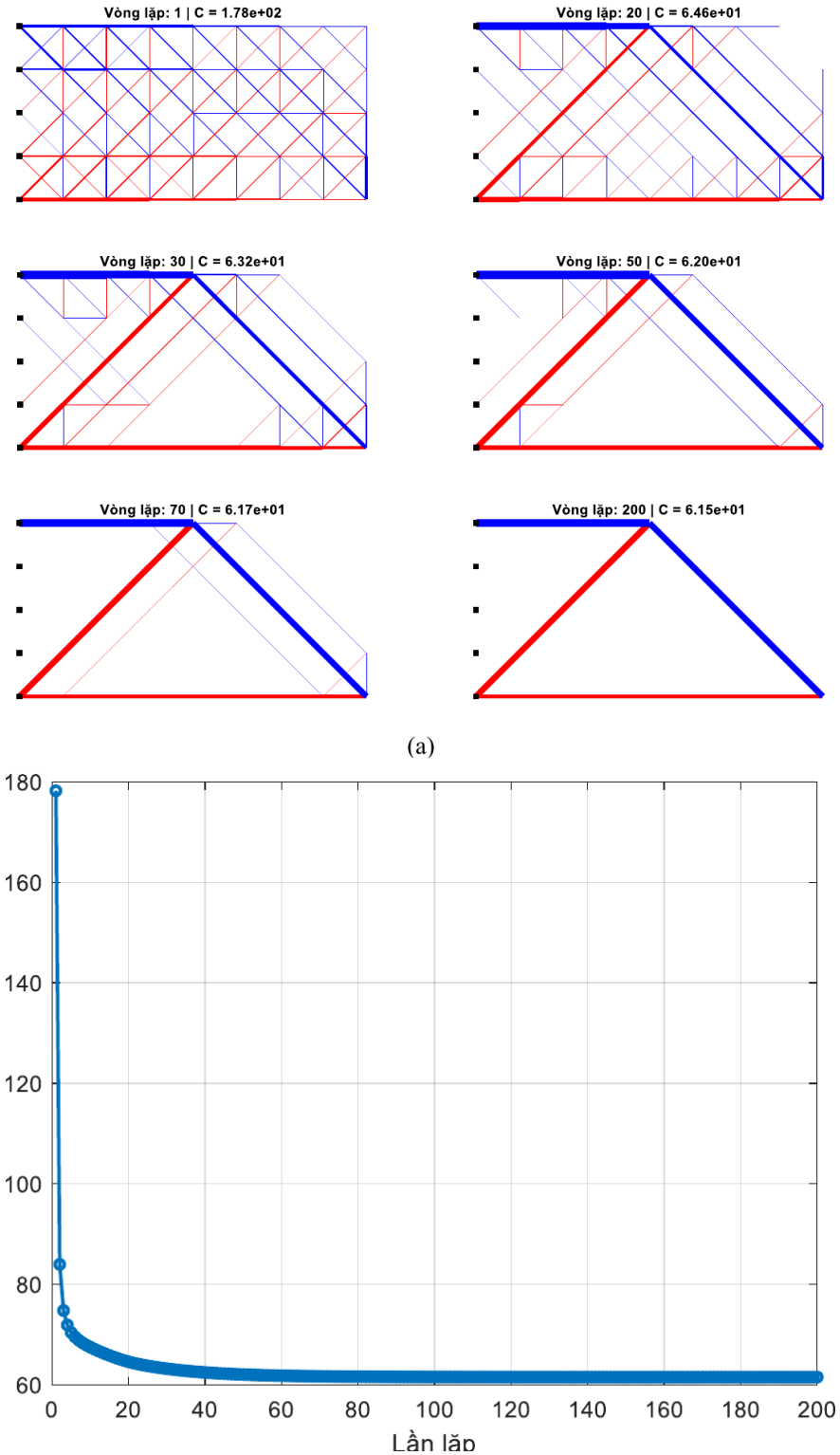


Hình 5.3. Hệ giàn của Ví dụ 5.1.

Phát biểu lại bài toán tối ưu (5.1) với các thông số của Ví dụ 5.1:

$$(\mathbb{P})\begin{cases} \min\limits_{\mathbf{x}} \ C(\mathbf{x}) = \mathbf{F}^T\mathbf{u}(\mathbf{x}) \\ \text{s.t.} \quad \sum\limits_{j=1}^{n} l_j x_j \leqslant 0{,}1\left(\sum\limits_{j=1}^{n} l_j\right) \\ \qquad 10^{-9} \leqslant x_j \leqslant 1, \quad j = 1, \ldots, n \end{cases}$$

Kết quả tối ưu của Ví dụ 5.1 bằng phương pháp OC được thể hiện trên Hình 5.4.



Hình 5.4. (a) Hình thái của hệ giàn sau các bước lặp: độ dày các thanh tỷ lệ thuận với diện tích mặt cắt ngang, màu đỏ biểu thị thanh chịu nén, màu xanh biểu thị thanh chịu kéo; (b) đồ thị độ mềm theo số bước lặp.

Mã nguồn tính Ví dụ 5.1 và vẽ Hình 5.4 được cung cấp tại Code 5.1 trên GitHub. Ngoài ra, Code 5.2 là bản rút gọn giúp người đọc dễ nắm được thuật toán cốt lõi.

5.3. Tối ưu độ cứng giàn sử dụng phương pháp tiệm cận động

Trong lĩnh vực tối ưu hóa kết cấu, phương pháp tiệm cận động được biết đến là một công cụ giải số mạnh mẽ và tổng quát, đặc biệt hiệu quả đối với các bài toán phi tuyến mạnh hoặc có nhiều ràng buộc phức tạp (như ràng buộc ứng suất cục bộ, ràng buộc chuyển vị đa điểm). Tuy nhiên, việc áp dụng MMA đòi hỏi chi phí tính toán lớn để xây dựng và giải các bài toán con tại mỗi bước lặp, đồng thời yêu cầu kỹ thuật cài đặt phức tạp. Quy trình thực hiện như sau:

- Bước 1: khởi tạo bằng cách chọn điểm bắt đầu $\mathbf{x}^0$ (thường chọn phân bố đều vật liệu), chọn các tham số tiệm cận ban đầu và đặt chỉ số vòng lặp $k = 0$.
- Bước 2: phân tích phần tử hữu hạn (FEA). Tại bước lặp k, giải phương trình cân bằng $\mathbf{K}(\mathbf{x}^k)\mathbf{u} = \mathbf{F}$ để tìm vector chuyển vị $\mathbf{u}$.
- Bước 3: tính toán độ nhạy bằng cách tính hàm mục tiêu $C(\mathbf{x}^k)$ và đạo hàm bậc nhất $\frac{\partial C}{\partial x_j}|_{\mathbf{x}^k}$.
- Bước 4: xây dựng bài toán con MMA bằng cách tạo ra các hàm xấp xỉ lồi cho hàm mục tiêu dựa trên đạo hàm vừa tính, sau đó thiết lập bài toán con (subproblem). Bài toán này đơn giản hơn nhiều so với bài toán gốc và có thể giải nhanh bằng phương pháp đối ngẫu (dual method).
- Bước 5: cập nhật biến thiết kế bằng cách giải bài toán con để tìm ra vector thiết kế mới $\mathbf{x}^{k+1}$, rồi kiểm tra điều kiện hội tụ. Nếu đã hội tụ, thuật toán dừng và kết quả là $\mathbf{x}^{k+1}$; nếu chưa hội tụ, cập nhật $k = k + 1$, điều chỉnh các đường tiệm cận (nếu cần) và quay lại bước phân tích phần tử hữu hạn.

Kết quả tối ưu hệ giàn của Ví dụ 5.1 bằng thuật toán MMA được thể hiện trên Hình 5.5. Mã nguồn tạo Hình 5.5 được cung cấp tại Code 5.3 (kèm `mmasub.m`, `subsolv.m` – mã nguồn MMA của Svanberg [46]) trên GitHub.

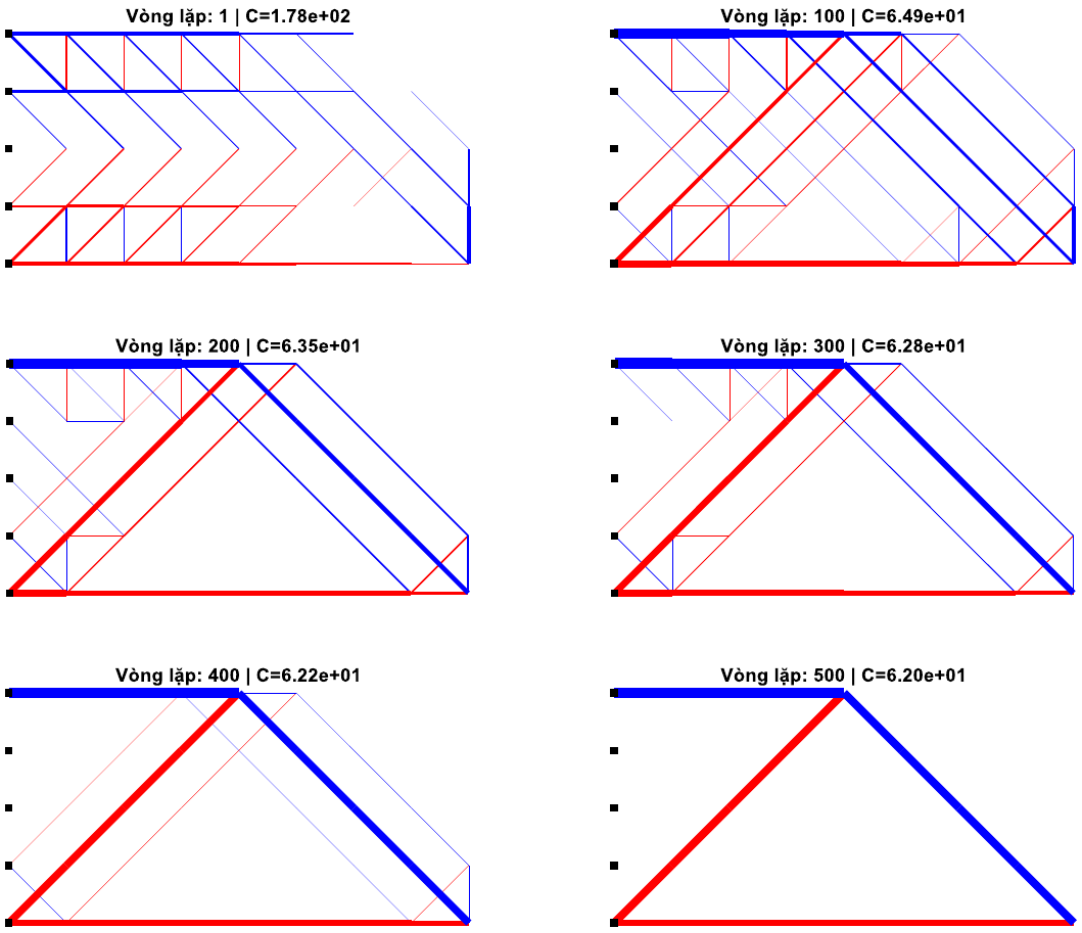
5.4. Bài tập

Các hệ giàn trong Bài 5.1–5.6 là những bài toán kiểm chứng (benchmark) quen thuộc: giàn 10 thanh, 25 thanh và 72 thanh [41], giàn 17 thanh [29], giàn 200 thanh [21] và giàn 18 thanh tối ưu đồng thời kích thước và hình dạng [26]. Số liệu đã được quy đổi sang hệ SI và làm tròn; các tệp dữ liệu mô hình (Code 5.4–5.9) được xây dựng trên cơ sở bộ bài toán giàn của Öztürk và Kahraman [38].

Bài 5.1: Cho kết cấu giàn phẳng 10 thanh như Hình 5.6, với mô-đun đàn hồi $E = 70$ GPa và khối lượng riêng $\rho = 2770$ kg/m^3. Hãy xác định diện tích tiết diện tối ưu của 10 thanh ($1\,\text{cm}^2 \leqslant A_i \leqslant 50\,\text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, đồng thời thỏa mãn ràng buộc ứng suất cho phép $[\sigma] = 175$ MPa và chuyển vị cho phép $[\delta] = 50$ mm. Mã nguồn tham khảo được cung cấp tại Code 5.4 trên GitHub.

Bài 5.2: Cho kết cấu giàn phẳng 17 thanh như Hình 5.7, với mô-đun đàn hồi $E = 210$ GPa và khối lượng riêng $\rho = 7400$ kg/m^3. Hãy xác định diện tích tiết diện ngang tối ưu của các thanh ($1\,\text{cm}^2 \leqslant A_i \leqslant 300\,\text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất $[\sigma] = 350$ MPa và chuyển vị $[\delta] = 50$ mm tại mọi bậc tự do. Mã nguồn tham khảo được cung cấp tại Code 5.5 trên GitHub.

Bài 5.3: Cho kết cấu giàn không gian 25 thanh như Hình 5.8, với mô-đun đàn hồi $E = 70$ GPa và khối lượng riêng $\rho = 2770$ kg/m^3. Hãy xác định diện tích tiết diện tối ưu của 8 nhóm thanh ($0{,}1\,\text{cm}^2 \leqslant A_i \leqslant 22\,\text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn các ràng buộc về ứng suất và chuyển vị $[\delta] = 9$ mm cho cả hai trường hợp



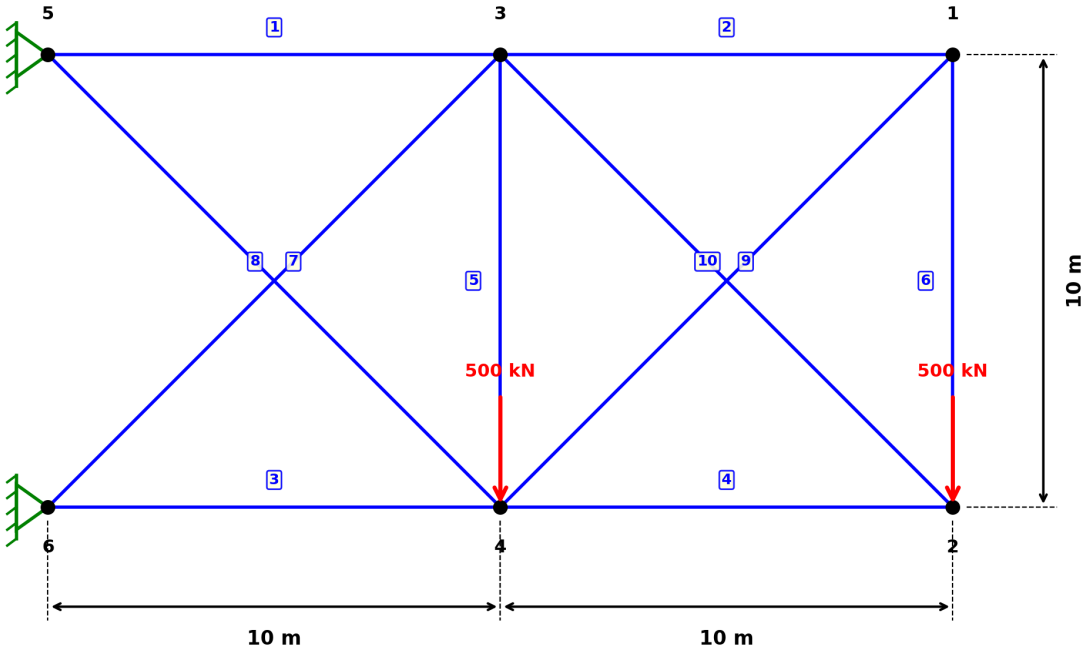
Hình 5.5. Tối ưu hệ giàn Hình 5.3 sau các bước lặp thứ 1, 100, 200, 300, 400 và 500 sử dụng thuật toán MMA.

tải trọng. Mã nguồn tham khảo được cung cấp tại Code 5.6 trên GitHub.

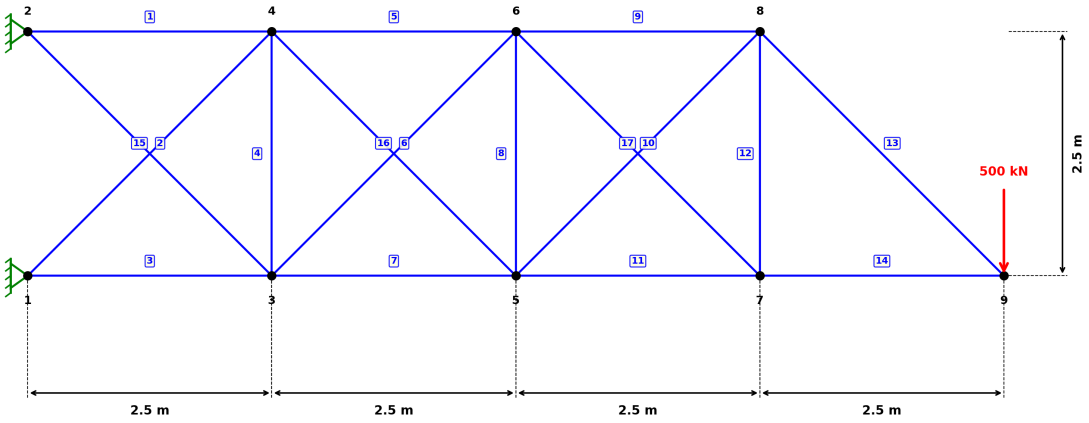
Bài 5.4: Cho kết cấu giàn không gian 72 thanh như Hình 5.9, với mô-đun đàn hồi $E = 70$ GPa và khối lượng riêng $\rho = 2770$ kg/m^3. Hãy xác định diện tích tiết diện tối ưu của 16 nhóm thanh ($1\,\text{cm}^2 \leqslant A_i \leqslant 130\,\text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất $[\sigma] = 175$ MPa và chuyển vị $[\delta] = 6$ mm cho cả hai trường hợp tải trọng. Mã nguồn tham khảo được cung cấp tại Code 5.7 trên GitHub.

Bài 5.5: Cho kết cấu giàn phẳng 200 thanh dạng tháp như Hình 5.10, với mô-đun đàn hồi $E = 210$ GPa và khối lượng riêng $\rho = 7850$ kg/m^3. Hãy xác định diện tích tiết diện tối ưu của 29 nhóm thanh ($1\,\text{cm}^2 \leqslant A_i \leqslant 130\,\text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất $[\sigma] = 70$ MPa cho cả ba trường hợp tải trọng. Mã nguồn tham khảo được cung cấp tại Code 5.8 trên GitHub.

Bài 5.6: Cho kết cấu giàn 18 thanh như Hình 5.11, được chế tạo từ vật liệu có khối lượng riêng $\rho = 2770$ kg/m^3 và mô-đun đàn hồi $E = 69$ GPa. Việc thiết kế tối ưu hóa bao gồm 12 biến độc lập, trong đó các thanh được phân thành 4 nhóm tiết diện biến thiên từ 22 cm^2 đến 116 cm^2, kết hợp cùng biến số về tọa độ ngang và dọc của 4 điểm nút nằm ở biên dưới. Hãy tìm kích thước tiết diện và tọa độ các nút biên tối ưu nhằm cực tiểu hóa tổng khối lượng hệ giàn, đồng thời phải đảm bảo kết cấu thỏa mãn giới hạn ứng suất kéo, nén $|\sigma_i| \leqslant 138$ MPa, không xảy ra hiện tượng mất ổn định Euler ở

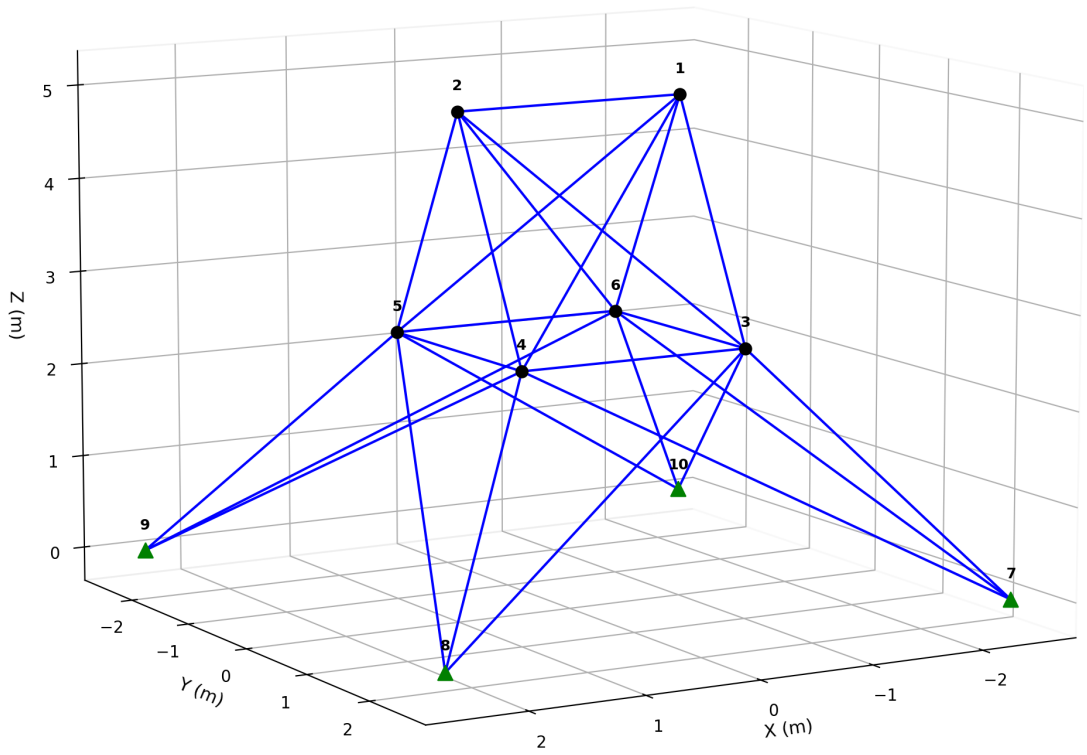


Hình 5.6. Giàn phẳng 10 thanh (Bài 5.1).

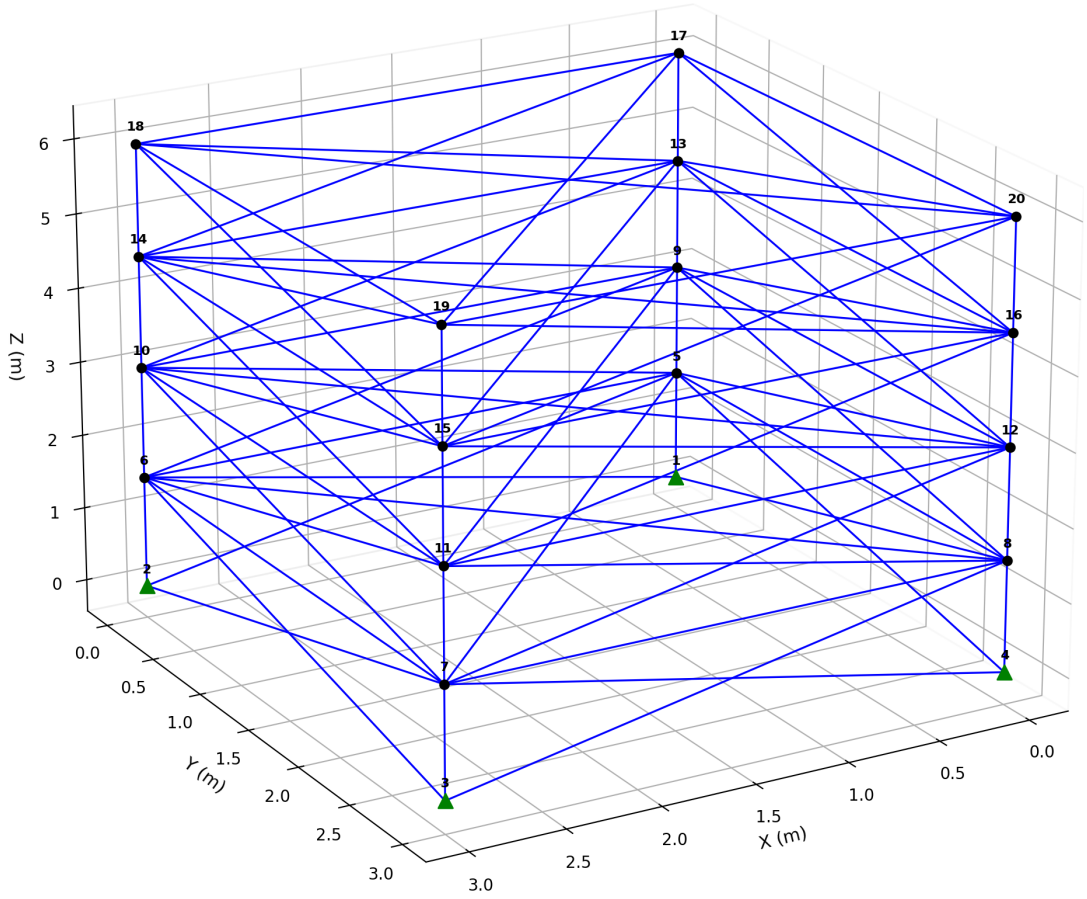


Hình 5.7. Giàn phẳng 17 thanh (Bài 5.2).

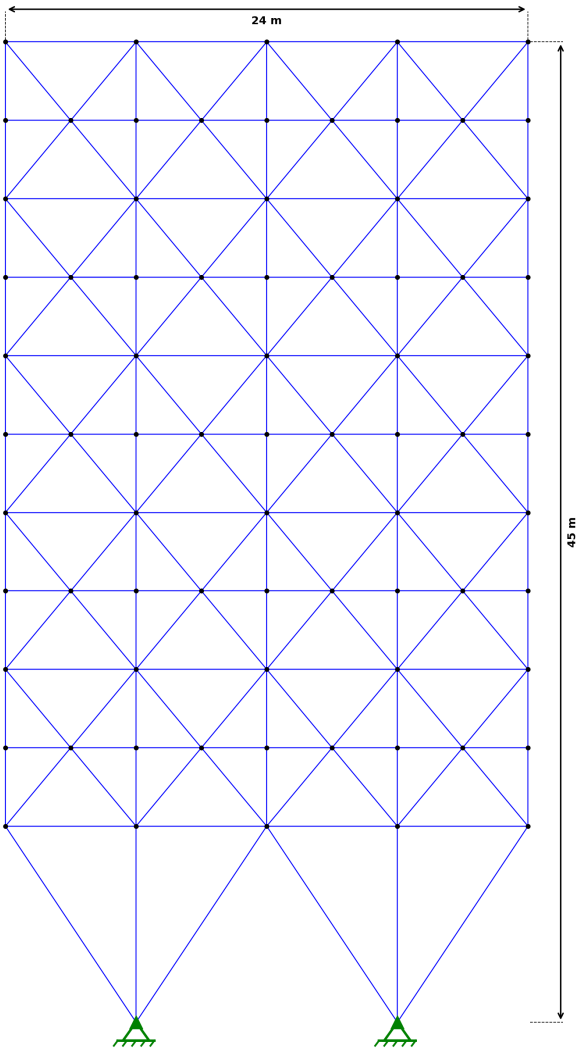
các thanh chịu nén (trị tuyệt đối của ứng suất nén không được vượt quá ứng suất tới hạn $\sigma_{cr} = 4EA_i/L_i^2$ với A_i và L_i lần lượt là diện tích tiết diện và chiều dài của thanh tương ứng), và giới hạn chuyển vị tại mọi nút không vượt quá 25 mm. Mã nguồn tham khảo được cung cấp tại Code 5.9 trên GitHub.



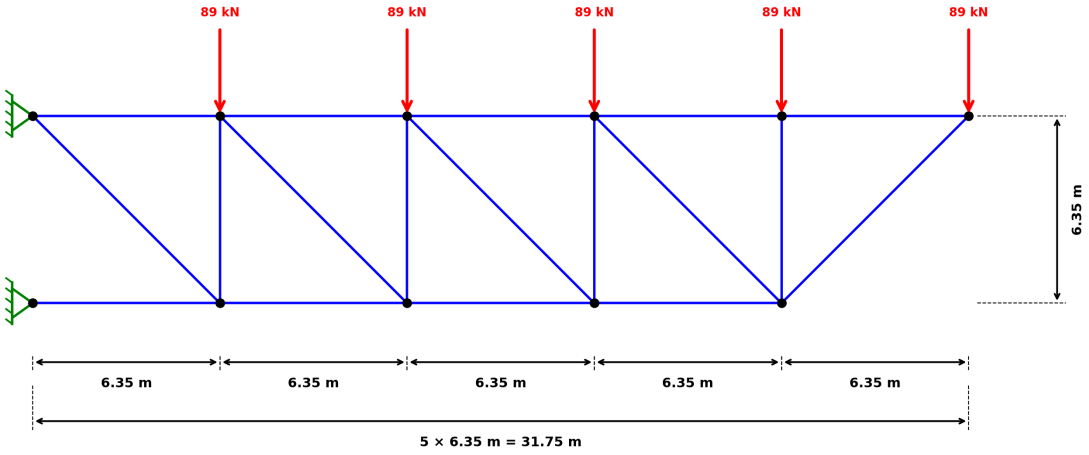
Hình 5.8. Giàn không gian 25 thanh (Bài 5.3).



Hình 5.9. Giàn không gian 72 thanh (Bài 5.4).



Hình 5.10. Giàn phẳng 200 thanh dạng tháp (Bài 5.5).



Hình 5.11. Giàn 18 thanh tối ưu đồng thời kích thước và hình dạng (Bài 5.6).

CHƯƠNG 6

TỐI ƯU CẤU TRÚC

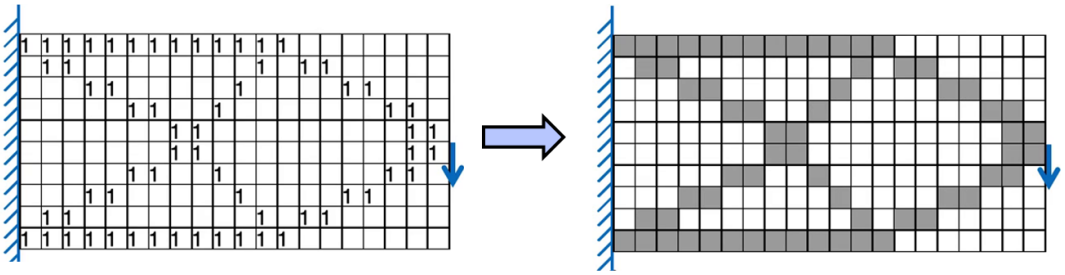
Chương này mở rộng bài toán tối ưu độ cứng sang tối ưu cấu trúc (tối ưu topo) cho kết cấu liên tục, trong đó biến thiết kế là mật độ vật liệu của từng phần tử. Nội dung gồm phát biểu bài toán, phương pháp vật liệu đẳng hướng với sự phạt (Solid Isotropic Material with Penalization – SIMP), kỹ thuật lọc độ nhạy để tránh hiện tượng ô cờ và sơ đồ cập nhật biến thiết kế theo phương pháp tiêu chuẩn tối ưu (OC), minh họa qua bài toán dầm MBB.

6.1. Phát biểu bài toán tối ưu cấu trúc

Bài toán tối ưu cấu trúc được phát biểu như sau:

$$\begin{aligned} \min \quad & C = \mathbf{u}^T \mathbf{K} \mathbf{u} \\ \text{s.t.} \quad & \mathbf{K}\mathbf{u} = \mathbf{F} \\ & \sum_e x_e v_e \leqslant V^* \\ & x_e = \begin{cases} 1 & (\text{vật liệu}) \\ 0 & (\text{lỗ rỗng}) \end{cases}, \forall e \end{aligned} \tag{6.1}$$

trong đó C là độ mềm (compliance), bằng hai lần năng lượng biến dạng (strain energy) của kết cấu; $\mathbf{u}$ là trường chuyển vị (displacement); $\mathbf{K}$ là ma trận độ cứng (stiffness matrix) của kết cấu; $\mathbf{F}$ là vector lực (force vector); v_e là thể tích phần tử e; V^* là thể tích (volume) vật liệu tối đa cho phép của kết cấu; x_e là mật độ (density) của phần tử e; nếu một phần tử có giá trị $x_e = 1$ nghĩa là tại vị trí đó có vật liệu, còn nếu $x_e = 0$ nghĩa là tại vị trí đó là lỗ rỗng. Chẳng hạn như ở Hình 6.1, các phần tử có giá trị $x_e = 1$ sẽ có vật liệu (tô màu xám) và những phần tử có giá trị $x_e = 0$ sẽ không có vật liệu (tô màu trắng).



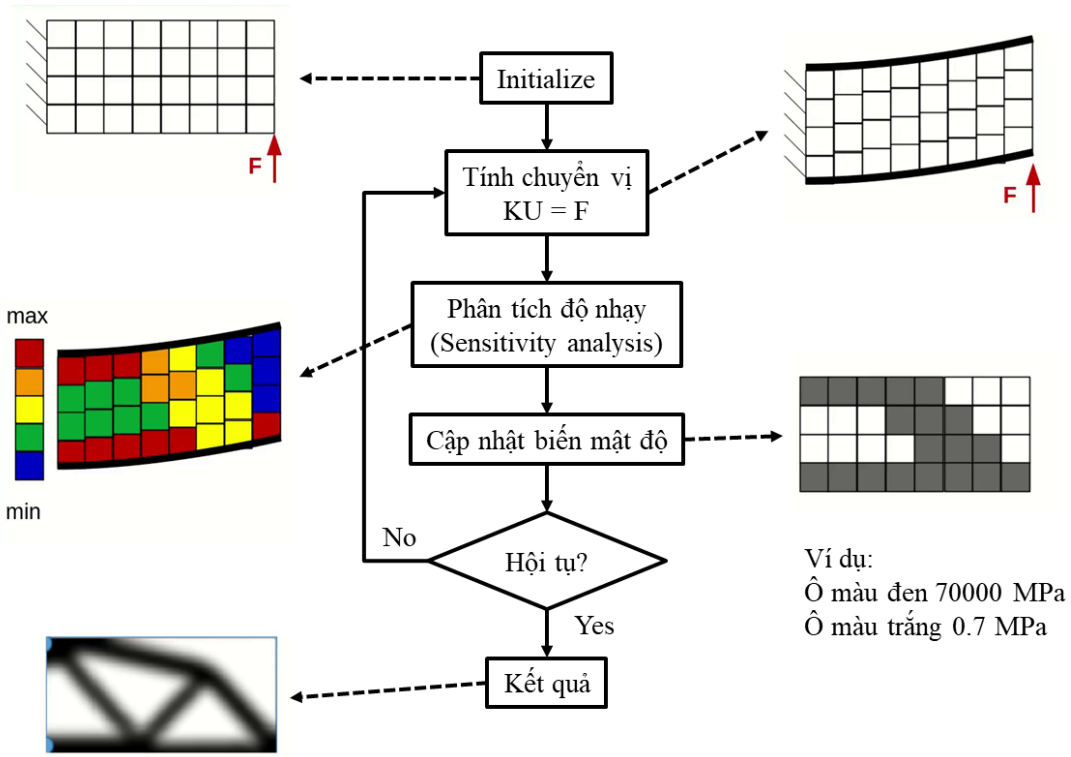
Hình 6.1. Chuyển đổi từ biến mật độ sang kết cấu thực.

Tuy nhiên, bài toán tối ưu cấu trúc với biến mật độ nhị phân rất khó giải. Do đó, trong phần tiếp theo của chương này, ta chỉ xét trường hợp biến mật độ x_e là biến liên tục trong đoạn $[0, 1]$:

$$x_e \in [0, 1] \, , \, \forall e \tag{6.2}$$

Lưu đồ tính toán bài toán tối ưu cấu trúc được thể hiện ở Hình 6.2. Đầu tiên ta giả định một phân bố mật độ nào đó trong miền thiết kế, chẳng hạn gán cho tất cả các phần tử cùng một mật độ bằng tỷ lệ thể tích $f = V^*/V_0$ (V_0 là thể tích miền thiết kế). Tiếp theo, tính trường chuyển vị $\mathbf{u}$ từ phương trình cân bằng tĩnh học $\mathbf{Ku} = \mathbf{F}$. Bước này thông thường sử dụng phương pháp phần tử hữu hạn (Finite Element Method – FEM). Sau đó là bước phân tích độ nhạy (sensitivity analysis) để cập nhật biến thiết kế mật độ dựa vào thuật toán tiêu chuẩn tối ưu.

Lưu ý rằng, để tránh trường hợp ma trận độ cứng bị suy biến, ta không thể gán độ cứng bằng 0 ở các phần tử lỗ rỗng mà phải gán một độ cứng rất nhỏ nào đó: chẳng hạn như với vật liệu nhôm, các phần tử có vật liệu thì mô-đun đàn hồi $E = 70000\,\text{MPa}$, còn lỗ rỗng thì mô-đun đàn hồi $E = 0{,}7\,\text{MPa}$. Sau khi cập nhật biến thiết kế, ta quay lại bước tính toán chuyển vị bằng phương pháp phần tử hữu hạn. Chương trình tính dừng lại khi kết quả tính toán hội tụ, nghĩa là sự thay đổi của biến thiết kế nhỏ hơn một giá trị định trước nào đó.



Hình 6.2. Lưu đồ tính toán bài toán tối ưu cấu trúc.

6.2. Thuật toán tối ưu cấu trúc SIMP

Phương pháp vật liệu đẳng hướng với sự phạt (SIMP) [8, 9] là thuật toán phổ biến nhất trong bài toán tối ưu cấu trúc. Mỗi phần tử e được gán một mật độ x_e liên hệ với mô-đun đàn hồi E_e theo công thức:

$$E_e\left(x_e\right) = x_e^p E_0 \tag{6.3}$$

với $p > 1$ là hệ số phạt (penalization factor), thường lấy $p = 3$, E_0 là mô-đun đàn hồi của vật liệu đặc. Ta phát biểu lại bài toán tối ưu cấu trúc như sau:

$$\begin{aligned} \min_{\mathbf{x}} \;& C\left(\mathbf{x}\right) = \mathbf{u}^T\mathbf{K}\mathbf{u} = \sum_{e=1}^{N} \left(x_e\right)^p \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e \\ \text{s.t.} \;& V\left(\mathbf{x}\right)/V_0 = f \\ & \mathbf{K}\mathbf{u} = \mathbf{F} \\ & \mathbf{0} < \mathbf{x}_{\min} \leqslant \mathbf{x} \leqslant \mathbf{1} \end{aligned} \tag{6.4}$$

trong đó $\mathbf{u}$ và $\mathbf{F}$ lần lượt là vector chuyển vị tổng thể và vector lực tổng thể; $\mathbf{K}$ là ma trận độ cứng tổng thể; $\mathbf{u}_e$ là vector chuyển vị nút của phần tử e; $\mathbf{k}_0$ là ma trận độ cứng phần tử ứng với vật liệu đặc ($x_e = 1$, mô-đun E_0); $\mathbf{x}$ là vector biến mật độ; $\mathbf{x}_{\min}$ là vector biến mật độ tối thiểu (phải lớn hơn 0 để tránh ma trận độ cứng bị suy biến); $N = n_{\text{elx}} n_{\text{ely}}$ là tổng số phần tử trong miền thiết kế, p là hệ số phạt; $V(\mathbf{x})$ và V_0 lần lượt là thể tích với biến mật độ $\mathbf{x}$ và thể tích ban đầu của miền thiết kế; f là tỷ lệ thể tích (volume fraction) cho trước.

Độ nhạy (sensitivity) của hàm mục tiêu được sử dụng để cập nhật biến thiết kế:

$$\frac{\partial C}{\partial x_e} = -p(x_e)^{p-1} \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e \tag{6.5}$$

Để đảm bảo bài toán tối ưu cấu trúc có nghiệm và để tránh sự hình thành của hiện tượng ô cờ (checkerboard patterns) [43], một số hạn chế (restriction) đối với thiết kế tối ưu phải được đưa vào: chẳng hạn sử dụng bộ lọc độ nhạy (sensitivity filter), hoặc bộ lọc mật độ (density filter). Về bản chất đây là các phương pháp xử lý ảnh (image processing) bằng bộ lọc thông thấp (low-pass filter). Các bộ lọc này được thực hiện bằng cách định nghĩa một miền lân cận (neighborhood) bán kính $r_{\min}$ của phần tử e, khi đó độ nhạy hoặc mật độ của phần tử e được lấy bằng trung bình có trọng số của các giá trị tương ứng của tất cả các phần tử trong miền lân cận đó. Với bộ lọc độ nhạy:

$$\frac{\widehat{\partial C}}{\partial x_e} = \frac{1}{x_e \sum\limits_{i=1}^{N} \hat{H}_{ei}} \sum_{i=1}^{N} \hat{H}_{ei} x_i \frac{\partial C}{\partial x_i} \tag{6.6}$$

trong đó trọng số (weight factor) $\hat{H}_{ei}$ được xác định bởi:

$$\hat{H}_{ei} = \max\left(0,\, r_{\min} - \operatorname{dist}\left(e, i\right)\right), \quad e, i = 1, ..., N \tag{6.7}$$

với $\operatorname{dist}(e, i)$ là khoảng cách giữa tâm hai phần tử e và i.

Bước cập nhật biến thiết kế thực chất là một bài toán tìm cực tiểu cho hàm mục tiêu với điều kiện ràng buộc: cho một biến thiết kế $x^{(k)}$, thuật toán cập nhật biến thiết kế tìm ra một biến thiết kế mới $x^{(k+1)}$ làm cho hàm mục tiêu có giá trị nhỏ hơn. Bài báo của Sigmund (2001) [42] sử dụng phương pháp tiêu chuẩn tối ưu (OC) để cập nhật biến thiết kế theo các công thức (6.8) và (6.9) sau:

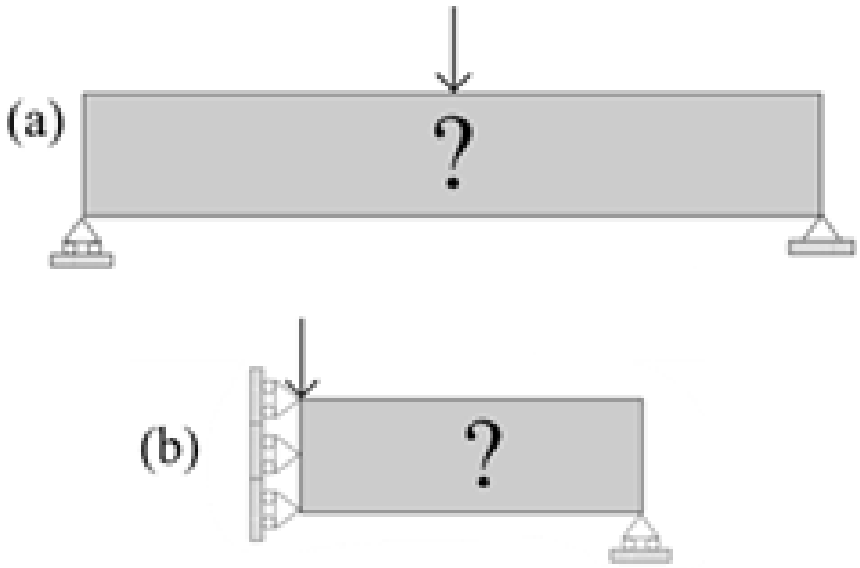
$$x_e^{new} = \begin{cases} \max\left(x_{\min}, x_e - m\right) & \text{nếu } x_e B_e^{\eta} \leqslant \max\left(x_{\min}, x_e - m\right) \\ x_e B_e^{\eta} & \text{nếu } \max\left(x_{\min}, x_e - m\right) < x_e B_e^{\eta} < \min\left(1, x_e + m\right) \\ \min\left(1, x_e + m\right) & \text{nếu } x_e B_e^{\eta} \geqslant \min\left(1, x_e + m\right) \end{cases} \tag{6.8}$$

với m là giới hạn dịch chuyển (move limit), một giá trị dương; $\eta = 0{,}5$ là hệ số cản; và B_e được xác định như sau:

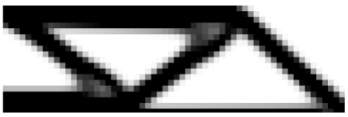
$$B_e = \frac{-\frac{\partial C}{\partial x_e}}{\lambda \frac{\partial V}{\partial x_e}} \tag{6.9}$$

với λ là nhân tử Lagrange (Lagrange multiplier), được xác định bằng phương pháp chia đôi (bisection method).

Ví dụ 6.1: Xét bài toán tối ưu cấu trúc dầm đơn giản (dầm MBB) chịu tải trọng tập trung giữa nhịp như Hình 6.3. Do tính chất đối xứng, chỉ cần xét một nửa dầm, được rời rạc hóa bằng lưới $n_{\text{elx}} \times n_{\text{ely}} = 60 \times 20$ phần tử; các tham số tính toán gồm tỷ lệ thể tích $f = 0{,}5$, hệ số phạt $p = 3$, bán kính lọc $r_{\min} = 1{,}5$ và giới hạn dịch chuyển $m = 0{,}1$. Kết quả tối ưu được thể hiện ở Hình 6.4. Mã nguồn MATLAB (dựa trên chương trình 99 dòng của Sigmund [42]) được cung cấp tại Code 6.1 trên GitHub.



Hình 6.3. Tối ưu hóa cấu trúc dầm MBB: (a) miền thiết kế; (b) nửa miền thiết kế do đối xứng.



Hình 6.4. Kết quả tối ưu hóa cấu trúc dầm MBB.

6.3. Bài tập

Các bài tập sau sử dụng Code 6.1; các đối số của lời gọi `top99(nelx, nely, volfrac, penal, rmin, 1)` tương ứng với các tham số n_{elx}, n_{ely}, f, p và $r_{\min}$ trong Ví dụ 6.1; đối số cuối (`nLoadCase`, số trường hợp tải) luôn giữ bằng 1 vì chương trình chỉ đặt tải và tính độ mềm cho một trường hợp tải. Bài 6.4 và Bài 6.5 phỏng theo các ví dụ mở rộng trong [42].

Bài 6.1: Giải lại Ví dụ 6.1 với hệ số phạt $p = 1$, 2, 3 và 4. So sánh hình dạng cấu trúc tối ưu, giá trị độ mềm (lưu ý độ mềm được tính với mô-đun đã phạt $x_e^p E_0$) và tỷ lệ các phần tử có mật độ trung gian ($0{,}1 < x_e < 0{,}9$). Giải thích vai trò của hệ số phạt trong phương pháp SIMP.

Bài 6.2: Giải Ví dụ 6.1 với bán kính lọc $r_{\min} = 1{,}0$ (tương đương không lọc, vì khi $r_{\min} \leqslant 1$ trọng số của mọi phần tử lân cận bằng 0), 1,5 và 3,0, sau đó với ba lưới 30×10, 60×20 và 120×40 với $r_{\min}$ lần lượt bằng 1,5; 3,0 và 6,0 (tăng $r_{\min}$ theo cùng tỷ lệ với kích thước lưới). Nhận xét về hiện tượng ô cờ và sự phụ thuộc của nghiệm vào lưới phần tử.

Bài 6.3: Với dầm MBB của Ví dụ 6.1, giải bài toán với tỷ lệ thể tích $f = 0{,}3$; 0,4; 0,5 và 0,6. Vẽ đồ thị độ mềm tối ưu theo f và nhận xét về sự đánh đổi giữa lượng vật liệu và độ cứng (liên hệ với khái niệm biên Pareto ở Chương 7).

Bài 6.4: Sửa hàm `FE` trong Code 6.1 để giải bài toán dầm công-xôn: thay hai dòng điều kiện biên của dầm MBB (`F(2,1) = -1` và `fixeddofs = ...`) bằng điều kiện ngàm toàn bộ cạnh trái (`fixeddofs = [1:2*(nely+1)]`) và lực thẳng đứng hướng xuống tại góc dưới bên phải (`F(2*(nelx+1)*(nely+1),1) = -1`). Giải với $n_{\text{elx}} \times n_{\text{ely}} = 32 \times 20$, $f = 0{,}4$, $p = 3$, $r_{\min} = 1{,}2$ và so sánh cấu trúc thu được với các thiết kế trong Hình 1.8.

Bài 6.5: Với điều kiện biên và tải trọng của Bài 6.4 (lưới 45×30, $f = 0{,}5$, $p = 3$, $r_{\min} = 1{,}5$), giả sử có một lỗ tròn bắt buộc tâm ($n_{\text{elx}}/3$, $n_{\text{ely}}/2$), bán kính $n_{\text{ely}}/3$ (tính theo số phần tử). Dùng biến `passive` trong Code 6.1 để gán mật độ nhỏ nhất cho các phần tử nằm trong lỗ: trong hàm `top99`, thay dòng `passive = [];` bằng ma trận `passive` cỡ $n_{\text{ely}} \times n_{\text{elx}}$ có giá trị 1 tại các phần tử có tâm nằm trong lỗ và 0 ở các phần tử còn lại (hàm `OC` sẽ gán mật độ 0,001 cho các phần tử này). Giải bài toán và nhận xét ảnh hưởng của vùng cấm vật liệu đến cấu trúc tối ưu; lưu ý ràng buộc thể tích vẫn tính trên toàn miền nên tỷ lệ vật liệu thực tế bên ngoài lỗ lớn hơn f.

CHƯƠNG 7

TỐI ƯU HÓA ĐA MỤC TIÊU

Chương này giới thiệu bài toán tối ưu hóa kết cấu có nhiều mục tiêu mâu thuẫn nhau: khái niệm chi phối và tối ưu Pareto, các cách tiếp cận cơ bản để xây dựng biên Pareto (phương pháp tổng trọng số, phương pháp ràng buộc epsilon và phương pháp tiến hóa), minh họa qua bài toán giàn hai thanh với hai mục tiêu khối lượng và chuyển vị.

7.1. Giới thiệu

Trong các chương trước, bài toán tối ưu hóa kết cấu được xây dựng dưới dạng một hàm mục tiêu duy nhất. Chẳng hạn, người thiết kế có thể hướng đến việc tối thiểu hóa khối lượng kết cấu, cực đại hóa độ cứng, hoặc giảm chuyển vị tại một vị trí xác định. Tuy nhiên, trong thực tế thiết kế kết cấu, một phương án hiếm khi được đánh giá dựa trên chỉ một tiêu chí. Kỹ sư thường phải đồng thời xem xét nhiều yêu cầu khác nhau, và các yêu cầu này thường có xu hướng mâu thuẫn lẫn nhau, chẳng hạn như kết cấu phải nhẹ nhưng vẫn đủ cứng để hạn chế chuyển vị, chi phí phải thấp nhưng vẫn đảm bảo an toàn và độ bền, hoặc lượng vật liệu sử dụng phải ít nhưng vẫn đáp ứng yêu cầu về ổn định và tuổi thọ khai thác.

Trong những trường hợp như vậy, tối ưu hóa theo một mục tiêu duy nhất thường dẫn đến các thiết kế thiếu cân bằng. Ví dụ, nếu chỉ tập trung tối thiểu hóa khối lượng, nghiệm tối ưu thường nằm sát giới hạn cho phép của các ràng buộc về ứng suất hoặc chuyển vị, khiến kết cấu trở nên nhạy cảm với sai số mô hình hoặc biến động tải trọng. Ngược lại, nếu chỉ cực đại hóa độ cứng, phương án thu được có thể quá nặng và kém hiệu quả về mặt kinh tế.

Vấn đề cốt lõi là không tồn tại một thiết kế đồng thời tốt nhất cho mọi tiêu chí. Việc cải thiện một đặc tính thường phải đánh đổi bằng sự suy giảm của đặc tính khác. Do đó, thay vì tìm kiếm một nghiệm tối ưu duy nhất, kỹ sư cần hiểu rõ mối quan hệ đánh đổi (trade-off) giữa các mục tiêu thiết kế.

Tối ưu hóa đa mục tiêu (Multiobjective Optimization – MOO) được phát triển nhằm giải quyết bài toán này. Mục tiêu của MOO không phải là xác định một phương án tối ưu duy nhất, mà là tìm ra một tập hợp các phương án thiết kế hợp lý, thể hiện những mức cân bằng khác nhau giữa các tiêu chí. Từ tập hợp này, người thiết kế có thể lựa chọn phương án phù hợp nhất dựa trên yêu cầu kỹ thuật, điều kiện kinh tế hoặc các yếu tố thực tế nằm ngoài mô hình tính toán.

Cũng cần lưu ý rằng không phải mọi bài toán có nhiều tiêu chí đều cần được mô hình hóa dưới dạng tối ưu đa mục tiêu. Trong nhiều trường hợp, một số tiêu chí chỉ đóng vai trò như các ràng buộc (ví dụ: chuyển vị không vượt quá giá trị cho phép), hoặc có thể được quy đổi thành một đại lượng tổng hợp như chi phí. Việc sử dụng tối ưu đa mục tiêu chỉ thực sự cần thiết khi không thể xác định trước một tiêu chí tổng

hợp đủ khả năng phản ánh toàn diện chất lượng thiết kế.

Trong phạm vi chương này, tối ưu hóa đa mục tiêu được trình bày ở mức giới thiệu nhằm làm rõ ý nghĩa kỹ thuật và bản chất của các bài toán có nhiều mục tiêu. Trọng tâm không đặt vào các thuật toán chuyên sâu, mà hướng đến việc giúp người đọc hiểu khái niệm đánh đổi giữa các mục tiêu và cách diễn giải kết quả tối ưu trong quá trình ra quyết định thiết kế kết cấu.

7.2. Phát biểu bài toán đa mục tiêu

Trong bài toán tối ưu hóa kết cấu một mục tiêu đã trình bày ở các chương trước, bài toán tổng quát có dạng:

$$\min_{\mathbf{x}} f(\mathbf{x}) \tag{7.1}$$

trong đó $\mathbf{x}$ là vector biến thiết kế và $f(\mathbf{x})$ là hàm mục tiêu dùng để đánh giá chất lượng của phương án thiết kế. Nghiệm tối ưu là phương án cho giá trị nhỏ nhất của hàm mục tiêu trong miền khả dĩ.

Trong tối ưu hóa đa mục tiêu, thay vì chỉ có một đại lượng cần tối ưu, ta đồng thời xem xét nhiều tiêu chí khác nhau. Khi đó, hàm mục tiêu không còn là một đại lượng vô hướng mà trở thành một vector hàm mục tiêu:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix}, \qquad m \geqslant 2 \tag{7.2}$$

trong đó mỗi hàm $f_i(\mathbf{x})$ biểu diễn một tiêu chí thiết kế riêng biệt. Ví dụ trong bài toán kết cấu, các hàm mục tiêu có thể là khối lượng kết cấu, chuyển vị tại nút hoặc chi phí chế tạo... Các ràng buộc của bài toán, bao gồm ràng buộc ứng suất, chuyển vị, ổn định hoặc điều kiện cân bằng, vẫn giữ nguyên như trong bài toán tối ưu một mục tiêu.

Sự khác biệt cơ bản giữa hai loại bài toán nằm ở chỗ: trong bài toán đa mục tiêu, không thể đồng thời cực tiểu tất cả các hàm mục tiêu. Trong phần lớn các hệ kết cấu, các mục tiêu có mối liên hệ phụ thuộc lẫn nhau. Ví dụ, việc tăng diện tích tiết diện để giảm chuyển vị sẽ làm tăng khối lượng; ngược lại, giảm khối lượng thường dẫn đến độ cứng nhỏ hơn và chuyển vị lớn hơn. Do đó, việc cải thiện một mục tiêu thường kéo theo sự suy giảm của mục tiêu khác.

Hiện tượng này được gọi là sự đánh đổi giữa các mục tiêu. Thay vì tồn tại một nghiệm tối ưu duy nhất như trong bài toán một mục tiêu, bài toán đa mục tiêu thường dẫn đến nhiều nghiệm tối ưu tương đương về mặt ý nghĩa thiết kế, mỗi nghiệm thể hiện một mức cân bằng khác nhau giữa các tiêu chí.

Một cách trực quan, có thể xem bài toán đa mục tiêu như quá trình khảo sát toàn bộ không gian thiết kế nhằm trả lời các câu hỏi dạng: cần tăng bao nhiêu khối lượng để giảm một lượng chuyển vị nhất định, hoặc việc cải thiện độ cứng thêm một mức nhỏ có làm chi phí tăng đáng kể hay không. Những thông tin này không thể thu được nếu bài toán được chuyển ngay về dạng một mục tiêu duy nhất từ đầu.

Trong thực tế, việc lựa chọn phương án cuối cùng vẫn luôn tương đương với việc lựa chọn một tiêu chí ưu tiên nào đó. Tuy nhiên, tối ưu hóa đa mục tiêu cho phép quá trình lựa chọn này được thực hiện sau khi đã hiểu rõ mối quan hệ giữa các mục tiêu, thay vì phải giả định trước ngay từ khi xây dựng mô hình toán học. Mục tiếp theo

giới thiệu khái niệm tối ưu Pareto, là nền tảng để xác định và diễn giải các nghiệm tối ưu trong bài toán đa mục tiêu.

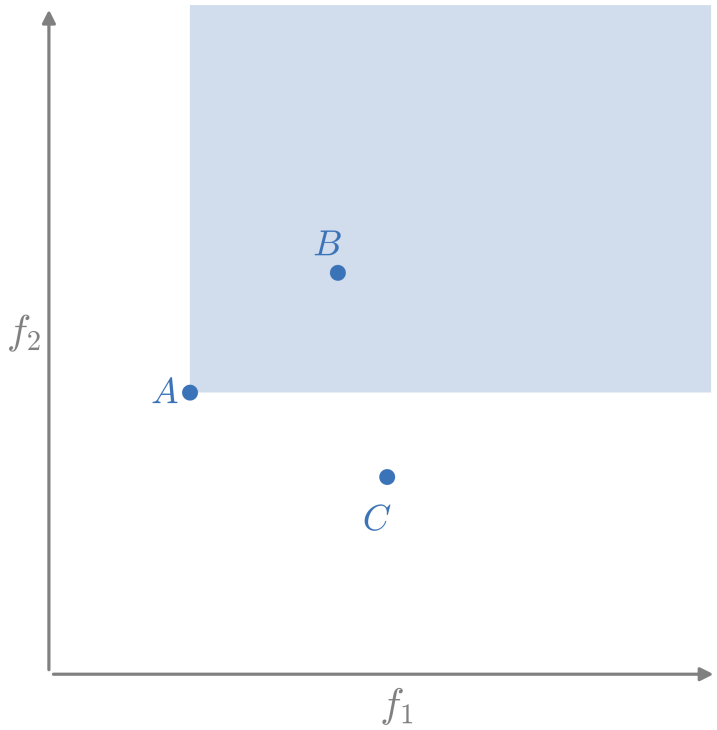
7.3. Khái niệm tối ưu Pareto

Trong bài toán tối ưu hóa một mục tiêu, việc so sánh các phương án thiết kế tương đối đơn giản: phương án nào cho giá trị hàm mục tiêu nhỏ hơn thì được xem là tốt hơn. Khi đó, quá trình tối ưu hướng tới việc tìm ra một nghiệm duy nhất cho giá trị tốt nhất của tiêu chí đã chọn.

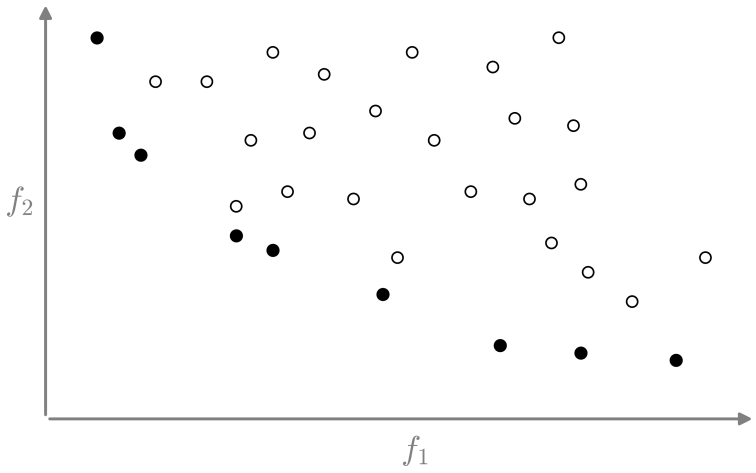
Tuy nhiên, trong bài toán đa mục tiêu, tình huống này thay đổi đáng kể. Một phương án có thể tốt hơn ở một tiêu chí nhưng lại kém hơn ở tiêu chí khác. Ví dụ, một kết cấu nhẹ hơn thường có chuyển vị lớn hơn, trong khi một kết cấu cứng hơn lại có khối lượng lớn hơn. Vì vậy, không còn tồn tại một tiêu chuẩn so sánh đơn giản như trong bài toán một mục tiêu.

Để mô tả mối quan hệ này, tối ưu hóa đa mục tiêu sử dụng khái niệm chi phối (dominance). Xét hai phương án thiết kế A và B. Phương án A được gọi là chi phối (dominate) phương án B nếu A không kém hơn B ở tất cả các mục tiêu và tốt hơn B ít nhất ở một mục tiêu. Trong trường hợp đó, phương án B có thể bị loại bỏ vì tồn tại một phương án khác tốt hơn hoặc tương đương ở mọi khía cạnh.

Ngược lại, một phương án được gọi là không bị chi phối (nondominated) nếu không tồn tại phương án nào khác tốt hơn nó theo nghĩa trên (xem Hình 7.1 và Hình 7.2; xem thêm [35, Chương 9]; mã nguồn vẽ Hình 7.2 tại Code 7.2 trên GitHub). Những nghiệm này đại diện cho các thiết kế mà việc cải thiện thêm một mục tiêu chỉ có thể đạt được bằng cách làm xấu đi ít nhất một mục tiêu khác. Đây chính là bản chất của sự đánh đổi trong thiết kế kỹ thuật.



Hình 7.1. Ba phương án thiết kế A, B và C được biểu diễn theo hai hàm mục tiêu f_1 và f_2. Vùng hình chữ nhật được tô bóng biểu thị các điểm bị phương án thiết kế A chi phối.



Hình 7.2. Biểu đồ thể hiện tất cả các điểm đã được đánh giá trong không gian mục tiêu theo hai hàm mục tiêu f_1 và f_2. Tập các điểm tô đen không bị bất kỳ điểm nào khác chi phối, do đó tạo thành tập Pareto.

Tập hợp tất cả các nghiệm không bị chi phối trong không gian biến thiết kế được gọi là tập Pareto (Pareto set). Khi biểu diễn các nghiệm này trong không gian các hàm mục tiêu, ta thu được biên Pareto (Pareto front). Biên Pareto thể hiện toàn bộ các phương án thiết kế hợp lý mà bài toán có thể đạt được.

Khái niệm Pareto làm thay đổi cách hiểu về tối ưu. Trong bài toán đa mục tiêu, một nghiệm Pareto tối ưu không phải là nghiệm tốt nhất theo mọi tiêu chí, mà là nghiệm mà không thể cải thiện thêm một mục tiêu nào mà không phải đánh đổi ở mục tiêu khác. Do đó, thay vì tồn tại một nghiệm tối ưu duy nhất, bài toán đa mục tiêu thường dẫn đến một tập các nghiệm tối ưu tương đương về mặt ý nghĩa thiết kế.

Đối với kỹ sư kết cấu, biên Pareto là một công cụ giúp hiểu rõ hơn hành vi của hệ kết cấu. Hình dạng của biên Pareto cho biết mức độ đánh đổi giữa các tiêu chí. Ví dụ, nếu một đoạn của biên Pareto có độ dốc lớn, chỉ cần tăng nhẹ khối lượng cũng có thể giảm đáng kể chuyển vị. Ngược lại, ở gần các điểm cực trị, việc cải thiện thêm một tiêu chí thường đòi hỏi sự đánh đổi rất lớn ở tiêu chí còn lại. Những thông tin này giúp người thiết kế nhận biết vùng thiết kế hiệu quả trước khi đưa ra lựa chọn cuối cùng.

Vì vậy, mục tiêu của tối ưu hóa đa mục tiêu không phải là lựa chọn ngay một phương án thiết kế duy nhất, mà là xác định biên Pareto để hỗ trợ quá trình ra quyết định. Việc lựa chọn nghiệm cuối cùng thường phụ thuộc vào các yếu tố ngoài mô hình toán học, chẳng hạn như yêu cầu kinh tế, mức độ an toàn mong muốn hoặc kinh nghiệm của người thiết kế.

Khi bài toán có hai biến thiết kế, một công cụ trực quan thường dùng để khảo sát hàm mục tiêu là đường đẳng trị (contour). Đường đẳng trị của một hàm $f(x_1, x_2)$ là tập hợp các điểm trong mặt phẳng thiết kế thỏa mãn $f(x_1, x_2) = c$, với c là một hằng số. Mỗi đường đẳng trị biểu diễn các phương án thiết kế cho cùng một giá trị hàm mục tiêu. Trong bài toán tối ưu đa mục tiêu, việc vẽ đồng thời các đường đẳng trị của nhiều hàm mục tiêu giúp nhận biết hướng cải thiện của từng mục tiêu và vùng có thể xuất hiện các nghiệm Pareto. Nếu hai họ đường đẳng trị có xu hướng kéo nghiệm về các vùng khác nhau, bài toán thể hiện sự đánh đổi rõ rệt giữa các mục tiêu.

Trong các mục tiếp theo, các khái niệm trên sẽ được minh họa thông qua một ví dụ tối ưu đa mục tiêu của kết cấu giàn đơn giản, qua đó làm rõ ý nghĩa vật lý của biên Pareto trong thiết kế kết cấu.

7.4. Một số cách tiếp cận trong tối ưu hóa đa mục tiêu

Như đã trình bày ở các mục trước, mục tiêu của tối ưu hóa đa mục tiêu không phải là tìm một nghiệm duy nhất, mà là xác định tập các nghiệm Pareto tối ưu thể hiện sự đánh đổi giữa các tiêu chí thiết kế. Trong thực tế, việc xây dựng biên Pareto có thể được thực hiện bằng nhiều phương pháp khác nhau [34]. Tuy nhiên, trong phạm vi cuốn sách này, chỉ những ý tưởng cơ bản nhất sẽ được giới thiệu nhằm giúp người đọc hiểu nguyên lý chung, thay vì đi sâu vào chi tiết thuật toán.

7.4.1. Phương pháp tổng trọng số (weighted sum method)

Cách tiếp cận đơn giản nhất là chuyển bài toán đa mục tiêu về dạng một mục tiêu duy nhất bằng cách gán trọng số cho từng hàm mục tiêu:

$$\min \; F(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x) \tag{7.3}$$

trong đó w_i là các hệ số trọng số thể hiện mức độ ưu tiên tương đối của từng mục tiêu.

Về bản chất, phương pháp này biến bài toán đa mục tiêu thành một chuỗi các bài toán tối ưu một mục tiêu quen thuộc. Bằng cách thay đổi các giá trị trọng số, ta có thể thu được các nghiệm khác nhau trên biên Pareto.

Ưu điểm của phương pháp là đơn giản, dễ áp dụng và có thể sử dụng trực tiếp các thuật toán tối ưu đã trình bày trong các chương trước. Tuy nhiên, phương pháp này tồn tại một số hạn chế quan trọng. Trước hết, việc lựa chọn trọng số phù hợp thường không dễ dàng vì người thiết kế phải xác định trước mức độ ưu tiên giữa các mục tiêu. Ngoài ra, phương pháp tổng trọng số không thể tìm được đầy đủ các nghiệm Pareto trong trường hợp biên Pareto không lồi [12], dẫn đến việc một số phương án thiết kế hợp lý có thể bị bỏ qua. Do những hạn chế này, phương pháp tổng trọng số thường được xem như một cách tiếp cận đơn giản để minh họa ý tưởng đa mục tiêu hơn là một công cụ tổng quát cho mọi bài toán.

7.4.2. Phương pháp ràng buộc epsilon (epsilon-constraint method)

Một cách tiếp cận trực quan hơn là lựa chọn một mục tiêu làm hàm mục tiêu chính, trong khi các mục tiêu còn lại được chuyển thành ràng buộc:

$$\begin{aligned} &\min \;\; f_1(x) \\ &\text{s.t.} \;\; f_2(x) \leqslant \varepsilon_2 \\ &\qquad f_3(x) \leqslant \varepsilon_3, \ldots \end{aligned} \tag{7.4}$$

trong đó ε_i là các giá trị giới hạn được lựa chọn trước.

Bằng cách thay đổi các giá trị ε_i, ta thu được các nghiệm Pareto khác nhau. Cách tiếp cận này phù hợp với tư duy thiết kế kết cấu, vì trong thực tế nhiều yêu cầu kỹ thuật đã tồn tại dưới dạng giới hạn cho phép, ví dụ chuyển vị tối đa hoặc mức ứng suất cho phép.

So với phương pháp tổng trọng số, phương pháp ràng buộc epsilon thường cho phép thu được phân bố nghiệm Pareto đầy đủ hơn và dễ diễn giải hơn về mặt kỹ thuật. Tuy nhiên, việc lựa chọn các giá trị giới hạn phù hợp vẫn cần kinh nghiệm hoặc quá trình thử nghiệm.

7.4.3. Phương pháp tiến hóa đa mục tiêu

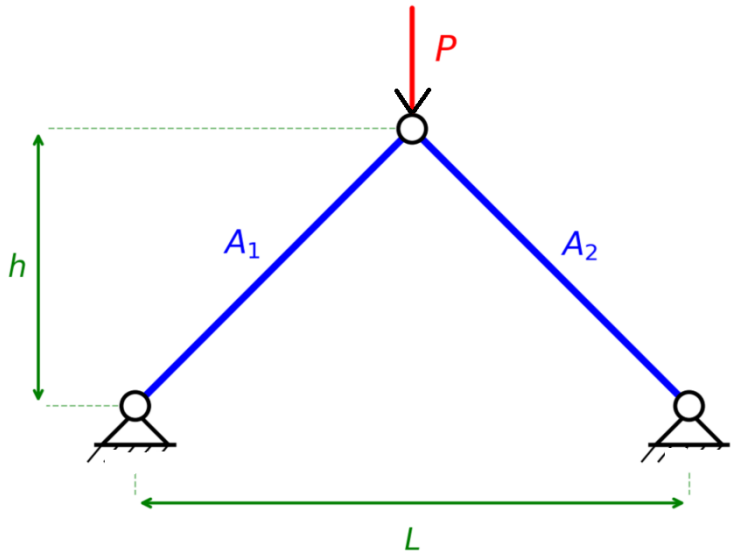
Trong các bài toán kết cấu thực tế với số lượng biến thiết kế lớn hoặc hàm mục tiêu phi tuyến mạnh, việc xây dựng biên Pareto bằng các phương pháp trên có thể trở nên khó khăn. Trong những trường hợp này, các phương pháp tối ưu hóa ngẫu nhiên hoặc tiến hóa, chẳng hạn như thuật toán di truyền đa mục tiêu (multiobjective

genetic algorithm) [13], thường được sử dụng.

Các phương pháp này làm việc với một tập nghiệm đồng thời và có khả năng tạo ra nhiều nghiệm Pareto trong một quá trình tính toán duy nhất. Vì lý do này, các thuật toán tiến hóa thường được sử dụng trong tối ưu hóa đa mục tiêu của kết cấu, đặc biệt trong các bài toán tối ưu rời rạc.

7.5. Ví dụ minh họa

Ví dụ 7.1: Cho hệ giàn phẳng như Hình 7.3 với $L = 2$ m, $h = 1$ m, chịu lực thẳng đứng $P = 50$ kN tại nút đỉnh C (hai thanh dài $L_1 = L_2 = \sqrt{h^2 + (L/2)^2} = \sqrt{2}$ m, nghiêng 45°). Các thanh có diện tích tiết diện lần lượt là A_1 và A_2, với vật liệu có mô-đun đàn hồi $E = 200$ GPa, khối lượng riêng $\rho = 7850$ kg/m^3 và ứng suất cho phép $\sigma_{\text{allow}} = 150$ MPa. Xác định biến thiết kế $\mathbf{x} = [A_1, A_2]^T$ trong khoảng $[200, 4000]$ mm^2 sao cho cực tiểu hóa đồng thời hai mục tiêu: khối lượng tổng cộng của hệ $f_1(\mathbf{x})$ và chuyển vị đứng $f_2(\mathbf{x})$ tại điểm đặt lực, đồng thời phải đảm bảo điều kiện ràng buộc về độ bền sao cho ứng suất thực tế trên mỗi thanh không vượt quá ứng suất cho phép.



Hình 7.3. Hệ giàn phẳng dùng trong ví dụ tối ưu hóa đa mục tiêu.

1. Phát biểu bài toán tối ưu đa mục tiêu

$$\min_{\mathbf{x}} \mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \rho(L_1 A_1 + L_2 A_2) \\ |v_C(\mathbf{x})| \end{bmatrix}$$

với các ràng buộc:

$$|\sigma_i(\mathbf{x})| \leqslant \sigma_{\text{allow}} = 150\,\text{MPa}, \quad i = 1, 2$$

$$200\ \text{mm}^2 \leqslant A_i \leqslant 4000\ \text{mm}^2, \quad i = 1, 2$$

2. Chuẩn hóa và hàm tổng trọng số

Bước chuẩn hóa (Utopia–Nadir): giải riêng các bài toán một mục tiêu để xác định giá trị tốt nhất và xấu nhất tham chiếu của từng mục tiêu. Điểm Utopia gồm các giá trị tốt nhất đạt được khi tối ưu riêng từng mục tiêu, còn điểm Nadir gồm các giá trị

bất lợi tương ứng trên tập nghiệm đang xét. Hai điểm này được dùng để đưa các mục tiêu về cùng thang đo trước khi kết hợp bằng hàm tổng trọng số. Kết quả tối ưu riêng từng mục tiêu được cho trong Bảng 7.1.

Bảng 7.1. Giá trị các mục tiêu khi tối ưu riêng từng mục tiêu

Bài toán	A_1 (mm^2)	A_2 (mm^2)	f_1 (kg)	f_2 (mm)
min f_1 (khối lượng)	235,7	235,7	5,23	1,500
min f_2 (chuyển vị)	4000	4000	88,8	0,088

Do hệ tĩnh định và đối xứng, nội lực hai thanh $N_1 = N_2 = -P/(2\sin 45^\circ)$ không phụ thuộc diện tích (hai thanh chịu nén; ví dụ này bỏ qua điều kiện ổn định), còn $|v_C| = \frac{PL_1}{2E}\left(\frac{1}{A_1} + \frac{1}{A_2}\right)$, nên với cùng khối lượng thì chuyển vị nhỏ nhất khi $A_1 = A_2$. Ràng buộc ứng suất cho cận dưới $A_i \geqslant |N_i|/\sigma_{\text{allow}} = 235{,}7$ mm^2.

Giá trị chuẩn hóa:

$$f_1^{\min} = 5{,}23\,\text{kg}, \quad f_1^{\max} = 88{,}8\,\text{kg}$$

$$f_2^{\min} = 0{,}088\,\text{mm}, \quad f_2^{\max} = 1{,}5\,\text{mm}$$

$$\tilde{f}_i = \frac{f_i - f_i^{\min}}{f_i^{\max} - f_i^{\min}} \in [0, 1]$$

Hàm mục tiêu tổng hợp:

$$F(\mathbf{x}, w) = w\tilde{f}_1(\mathbf{x}) + (1 - w)\tilde{f}_2(\mathbf{x}), \qquad 0 \leqslant w \leqslant 1$$

3. Quét trọng số và kết quả Pareto

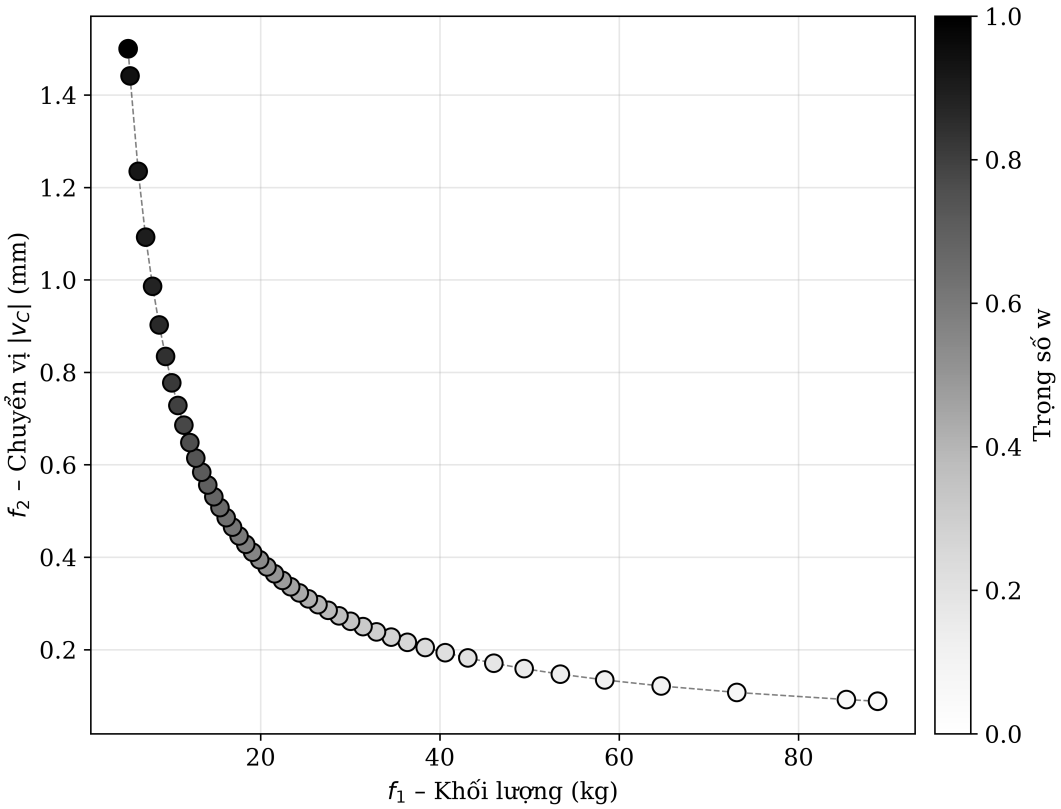
Giải bài toán $\min F(\mathbf{x}, w)$ kèm ràng buộc ứng suất, với w thay đổi từ 0 đến 1, bước 0,1. Kết quả được cho trong Bảng 7.2; Hình 7.4 được vẽ với bước w mịn hơn (0,02; 51 điểm).

Bảng 7.2. Kết quả quét trọng số w

w	A_1 (mm^2)	A_2 (mm^2)	f_1 (kg)	f_2 (mm)
0,0	4000	4000	88,8	0,088
0,1	2913	2913	64,7	0,121
0,2	1942	1942	43,1	0,182
0,3	1483	1483	32,9	0,238
0,4	1189	1189	26,4	0,297
0,5	971	971	21,6	0,364
0,6	793	793	17,6	0,446

w	A_1 (mm^2)	A_2 (mm^2)	f_1 (kg)	f_2 (mm)
0,7	636	636	14,1	0,556
0,8	485	485	10,8	0,728
0,9	324	324	7,2	1,092
1,0	235,7	235,7	5,23	1,500

Biên Pareto trong mặt phẳng (f_1, f_2) có dạng đường cong lồi (hyperbol $f_1 f_2 = 2\rho P L_1^2/E \approx 7{,}85$ kg·mm), xem Hình 7.4; do biên lồi nên phương pháp tổng trọng số tìm được toàn bộ biên. Mã nguồn Python giải ví dụ và vẽ Hình 7.4 được cung cấp tại Code 7.3 trên GitHub.



Hình 7.4. Biên Pareto trong mặt phẳng (f_1, f_2).

4. Nhận xét ý nghĩa kỹ thuật

Vùng w nhỏ (0–0,2) – bão hòa chuyển vị: tăng khối lượng từ 43 đến 89 kg (thêm khoảng 46 kg) chỉ giảm chuyển vị từ 0,18 xuống 0,09 mm (giảm khoảng 0,09 mm). Hiệu quả rất thấp, việc tăng thêm vật liệu không mang lại lợi ích đáng kể.

Vùng w lớn (0,8–1,0) – bão hòa khối lượng: giảm khối lượng từ 10,8 xuống 5,2 kg (tiết kiệm khoảng 5,6 kg) nhưng chuyển vị tăng từ 0,73 lên 1,50 mm (tăng khoảng gấp đôi). Lượng vật liệu tiết kiệm được nhỏ nhưng độ cứng kết cấu giảm nhanh.

Vùng giữa ($w \approx 0{,}3$–$0{,}6$) – đánh đổi hiệu quả: mỗi kg vật liệu bớt/thêm đều tương ứng thay đổi chuyển vị vừa phải. Đây là vùng thiết kế hợp lý nhất.

Đề xuất phương án: chọn điểm "đầu gối" (knee point) ở khoảng $w \approx 0{,}4$–$0{,}5$, tương ứng $A_1 = A_2 \approx 970$–1190 mm^2, cho khối lượng 21,6 đến 26,4 kg và chuyển vị khoảng 0,30–0,36 mm, cân bằng tốt nhất giữa nhẹ và cứng.

7.6. Bài tập

Bài 7.1: Trong không gian thiết kế, hãy vẽ các đường đẳng trị (contour) của hàm mục tiêu cho bài toán không ràng buộc sau đây và vẽ tập tối ưu Pareto:

$$\begin{aligned} \min \ f_1 &= (x_1 - 0{,}75)^2 + (x_2 - 2)^2 \\ f_2 &= (x_1 - 2{,}5)^2 + (x_2 - 1{,}5)^2 \end{aligned}$$

Bài 7.2: Trong không gian thiết kế, hãy biểu diễn bài toán có ràng buộc sau đây và vẽ tập tối ưu Pareto:

$$\begin{aligned} \min \ f_1 &= (x_1 - 3)^2 + (x_2 - 7)^2 \\ f_2 &= (x_1 - 9)^2 + (x_2 - 8)^2 \\ \text{s.t.} \quad g_1 &= 70 - 4x_2 - 8x_1 \leqslant 0 \\ g_2 &= -2{,}5x_2 + 3x_1 \leqslant 0 \\ g_3 &= -6{,}8 + x_1 \leqslant 0 \end{aligned}$$

Bài 7.3: Giải bài toán sau bằng cách sử dụng các điều kiện tối ưu KKT kết hợp với phương pháp tổng trọng số:

$$\begin{aligned} \min \ f_1 &= (x_1 - 3)^2 + (x_2 - 7)^2 \\ f_2 &= (x_1 - 9)^2 + (x_2 - 8)^2 \end{aligned}$$

Hãy viết lời giải cho các biến thiết kế dưới dạng hàm của hai trọng số w_1 và w_2. Sau đó giải lại bài toán bằng phương pháp ràng buộc epsilon, lần lượt chọn f_1 và f_2 làm hàm mục tiêu chính. Đưa ra nhận xét về việc các trọng số và các giá trị epsilon khác nhau ảnh hưởng như thế nào đến lời giải cuối cùng.

Bài 7.4: Giải bài toán sau bằng các điều kiện tối ưu KKT kết hợp với phương pháp tổng trọng số:

$$\begin{aligned} \min \ f_1 &= 20(x_1 - 0{,}75)^2 + (2x_2 - 2)^2 \\ f_2 &= 5(x_1 - 1{,}6)^2 + 2x_2 \\ \text{s.t.} \quad g_1 &= -x_2 \leqslant 0 \end{aligned}$$

Đầu tiên, hãy sử dụng $w_1 = 0{,}1$ và $w_2 = 0{,}9$. Sau đó, giải lại bài toán với $w_1 = 0{,}9$ và $w_2 = 0{,}1$. Tiếp theo, áp dụng phương pháp ràng buộc epsilon bằng cách cực tiểu hóa f_1 với ràng buộc $f_2 \leqslant \varepsilon$ cho một vài giá trị ε khác nhau. Đưa ra nhận xét về trạng thái hoạt động của ràng buộc trong mỗi trường hợp.

Bài 7.5: Thiết lập bài toán sau và giải bằng phương pháp tổng trọng số và phương pháp ràng buộc epsilon.

Xác định chiều cao và bán kính tối ưu của một hình trụ kín để đồng thời tối đa hóa thể tích và tối thiểu hóa diện tích bề mặt. Giả sử hình trụ có độ dày không đáng kể. Chiều cao phải ít nhất là 0,1 m, và bán kính phải ít nhất bằng một nửa chiều cao. Cả chiều cao và bán kính đều không được lớn hơn 2 m.

Sử dụng điểm bắt đầu là $x^{(0)} = (1, 1)$. Với phương pháp tổng trọng số, sử dụng các vector trọng số sau và nhận xét về lời giải: $w = (1;\ 0)$; $w = (0{,}75;\ 0{,}25)$; $w = (0{,}5;\ 0{,}5)$; $w = (0{,}25;\ 0{,}75)$; $w = (0;\ 1)$. Với phương pháp ràng buộc epsilon, hãy cực đại hóa thể tích trong khi khống chế diện tích bề mặt bởi một vài giới hạn cho trước, sau đó đổi vai trò hai mục tiêu và so sánh kết quả.

Bài 7.6: Một hệ giàn phẳng hai thanh có hai biến thiết kế là diện tích tiết diện A_1 và A_2. Khối lượng của hệ được viết dưới dạng $f_1(\mathbf{x}) = \rho(L_1A_1 + L_2A_2)$, còn chuyển vị tại nút chịu tải được ký hiệu là $f_2(\mathbf{x}) = |v_C(\mathbf{x})|$. Các biến thiết kế thỏa mãn $A_{\min} \leqslant A_i \leqslant A_{\max}$, $i = 1, 2$, và ứng suất trong mỗi thanh không được vượt quá σ_{allow}. Hãy thiết lập bài toán tối ưu đa mục tiêu nhằm đồng thời giảm khối lượng và giảm chuyển vị. Sau đó chuyển bài toán về dạng một mục tiêu bằng phương pháp tổng trọng số và phương pháp ràng buộc epsilon. Giải thích ý nghĩa kỹ thuật của từng nghiệm thu được trên biên Pareto.

Bài 7.7: Xét một dầm công-xôn có tiết diện chữ nhật với chiều rộng b cố định và chiều cao h là biến thiết kế. Hai mục tiêu thiết kế là cực tiểu hóa khối lượng dầm và cực tiểu hóa chuyển vị đầu tự do. Dầm phải thỏa mãn ràng buộc ứng suất uốn cho phép và giới hạn hình học $h_{\min} \leqslant h \leqslant h_{\max}$. Hãy phát biểu bài toán tối ưu đa mục tiêu, sau đó áp dụng phương pháp ràng buộc epsilon bằng cách cực tiểu hóa khối lượng với ràng buộc chuyển vị không vượt quá ε. Thay đổi ε và nhận xét sự đánh đổi giữa khối lượng và độ cứng của dầm.

CHƯƠNG 8

THUẬT TOÁN TỐI ƯU METAHEURISTIC

Chương này trình bày nguyên lý chung của các thuật toán tối ưu metaheuristic, sau đó đi sâu vào thuật toán di truyền và thuật toán luyện kim mô phỏng. Hai ví dụ tối ưu dầm công-xôn và khung thép phẳng được giải bằng thuật toán luyện kim mô phỏng; phần bài tập gồm các hệ giàn chuẩn với ràng buộc ứng suất, mất ổn định và tần số dao động riêng.

8.1. Giới thiệu

Thuật ngữ metaheuristic được kết hợp từ meta (vượt ra ngoài/tổng quát) và heuristic (tìm kiếm/khám phá). Về bản chất, đây là một khung thuật toán cung cấp các quy tắc mang tính xác suất để hướng dẫn quá trình tìm kiếm thoát khỏi các cực trị địa phương (local optima).

Khác với các phương pháp giải tích truyền thống vốn dựa vào đạo hàm để xác định hướng đi, metaheuristic vận hành dựa trên cơ chế thử và sai cùng tính ngẫu nhiên. Do đó, chúng không đảm bảo sẽ tìm ra lời giải chính xác tuyệt đối hoặc tối ưu toán học như các phương pháp truyền thống. Tuy nhiên, đối với các bài toán kỹ thuật phức tạp, nơi hàm mục tiêu có thể không liên tục, có nhiều bẫy cực trị địa phương hoặc không thể tính được đạo hàm, metaheuristic là công cụ hiệu quả để tìm ra lời giải gần tối ưu (near-optimal) trong một khoảng thời gian tính toán chấp nhận được.

Dựa trên nguồn cảm hứng và cơ chế tìm kiếm, các thuật toán metaheuristic thường được phân thành các nhóm chính sau:

(1) Dựa trên tiến hóa sinh học (Biological Evolution Based): Nhóm này mô phỏng các nguyên lý của học thuyết Darwin về sự tiến hóa và di truyền, tiêu biểu là thuật toán di truyền (Genetic Algorithm – GA) [22, 25] mô phỏng quá trình chọn lọc, lai ghép, đột biến; thuật toán tiến hóa vi sai (Differential Evolution – DE) [44] tập trung vào sự khác biệt giữa các cá thể.

(2) Dựa trên trí tuệ bầy đàn (Swarm Intelligence Based): Mô phỏng hành vi tập thể và tương tác xã hội để tự tổ chức tìm kiếm, như tối ưu hóa bầy đàn phần tử (Particle Swarm Optimization – PSO) [28] dựa trên cách đàn chim di chuyển; thuật toán bầy kiến (Ant Colony Optimization – ACO) [15] tìm đường qua dấu vết pheromone.

(3) Dựa trên quá trình vật lý (Physics Process Inspired): Nhóm này dựa trên các quy luật vật lý và nhiệt động lực học, điển hình là thuật toán luyện kim mô phỏng (Simulated Annealing – SA) [30] mô phỏng quá trình nung nóng và làm nguội vật liệu; thuật toán tối ưu hóa va chạm các hạt (Colliding Bodies Optimization – CBO) [27] dựa trên định luật bảo toàn động lượng.

(4) Dựa trên quá trình hóa học hoặc xã hội (Chemical or Social Process Based): Mô

phỏng các phản ứng hóa học hoặc tương tác xã hội, chẳng hạn thuật toán tìm kiếm sự hòa hợp (Harmony Search – HS) [20] mô phỏng việc tinh chỉnh nhạc cụ; tối ưu hóa dựa trên việc dạy và học (Teaching–Learning-Based Optimization – TLBO) [39] mô phỏng sự truyền đạt kiến thức trong lớp học.

Trong khuôn khổ chương này, hai thuật toán đại diện được xem xét là thuật toán di truyền (GA) và thuật toán luyện kim mô phỏng (SA). Hai phương pháp này đại diện cho hai hướng tiếp cận khác nhau trong tối ưu hóa: GA mô phỏng quá trình tiến hóa sinh học và thực hiện tìm kiếm dựa trên quần thể, cho phép khảo sát đồng thời nhiều vùng của không gian thiết kế; trong khi đó SA dựa trên nguyên lý nhiệt động lực học và thực hiện quá trình tìm kiếm dựa trên một lời giải duy nhất, với cơ chế dịch chuyển cục bộ để thoát khỏi cực trị địa phương (local optima). Đây là các phương pháp nền tảng, có độ ổn định cao và đã được chứng minh hiệu quả trong nhiều bài toán tối ưu hóa kết cấu thực tế.

Sự khác biệt cốt lõi giữa các thuật toán metaheuristic và phương pháp tối ưu hóa truyền thống dựa trên đạo hàm nằm ở cơ chế tìm kiếm và yêu cầu về hàm mục tiêu. Các phương pháp truyền thống có tính xác định (deterministic), hội tụ nhanh và đạt độ chính xác cao đối với các hàm mục tiêu trơn và lồi. Tuy nhiên, nhược điểm lớn nhất là phụ thuộc vào điểm xuất phát, dễ bị mắc kẹt tại cực trị địa phương nên khó tìm được lời giải tối ưu toàn cục cho các bài toán phi tuyến phức tạp.

Ngược lại, metaheuristic vận hành theo cơ chế ngẫu nhiên (stochastic), cho phép thăm dò không gian thiết kế rộng rãi mà không yêu cầu thông tin đạo hàm. Nhờ các cơ chế đặc thù như đột biến trong GA hay tiêu chuẩn Metropolis trong SA, metaheuristic có khả năng thoát khỏi các bẫy cực trị địa phương để tìm đến vùng chứa cực trị toàn cục. Dù vậy, phương pháp này đòi hỏi chi phí tính toán lớn do phải đánh giá hàm mục tiêu nhiều lần, tốc độ hội tụ chậm ở giai đoạn cuối và cần tinh chỉnh kỹ lưỡng các thông số điều khiển (hyperparameters).

8.2. Thuật toán di truyền

Mục này trình bày nguyên lý cơ bản và quy trình vận hành của thuật toán di truyền (GA).

8.2.1. Nguyên lý cơ bản

Bản chất của GA là quá trình sàng lọc thông tin di truyền tốt qua nhiều thế hệ:

Cá thể (individual/chromosome): Trong tối ưu hóa kết cấu, một cá thể đại diện cho một phương án thiết kế hoàn chỉnh. Nó được mã hóa thành một chuỗi (thường là nhị phân hoặc số thực) chứa các gen – chính là các thông số như kích thước tiết diện hay tọa độ nút.

Quần thể (population): Tập hợp các phương án thiết kế hiện có. Kích thước quần thể là một tham số quan trọng: nếu quá nhỏ sẽ làm mất đi sự đa dạng di truyền, nếu quá lớn sẽ gây tốn kém chi phí tính toán.

Độ thích nghi (fitness): Đây là thước đo chất lượng của một cá thể. Trong bài toán tối ưu hóa kết cấu thép, độ thích nghi thường được thiết kế tỷ lệ nghịch với hàm mục tiêu (trọng lượng) và có kèm theo mức phạt nếu vi phạm các ràng buộc về ứng suất hay độ võng.

8.2.2. Quy trình vận hành

Thuật toán tiến hóa qua các bước lặp đi lặp lại để không ngừng cải thiện chất lượng quần thể (lưu đồ trên Hình 8.1):

1. Khởi tạo và đánh giá: Một quần thể ban đầu được tạo ra ngẫu nhiên để đảm bảo tính bao quát. Mỗi cá thể sau đó được đưa vào mô hình phân tích (như mô hình phần

tử hữu hạn) để tính toán độ thích nghi.

2. Chọn lọc (selection): Mục đích là chọn ra những bố mẹ tốt để truyền lại gen cho thế hệ sau. Phương pháp phổ biến nhất là vòng quay may mắn (Roulette Wheel Selection), trong đó xác suất một cá thể được chọn tỷ lệ thuận với độ thích nghi của nó:

$$P(x_i) = \frac{F(x_i)}{\sum_{j=1}^{N} F(x_j)} \tag{8.1}$$

trong đó $F(x_i) \geqslant 0$ là độ thích nghi (với bài toán cực tiểu có thể lấy $F = 1/(1 + f_p)$ hoặc $F = f_{p,\max} - f_p$, f_p là hàm mục tiêu đã cộng phạt) và N là kích thước quần thể.

3. Lai ghép (crossover): Đây là toán tử quan trọng nhất để tạo ra các giải pháp mới bằng cách kết hợp ưu điểm của hai bố mẹ.

Lai ghép đơn điểm: Cắt hai chuỗi gen tại một vị trí ngẫu nhiên và tráo đổi các phần đuôi cho nhau.

Lai ghép đa điểm/đồng nhất: Cho phép trao đổi thông tin di truyền linh hoạt hơn giữa các cá thể con.

4. Đột biến (mutation): Với một xác suất rất nhỏ, các gen trong cá thể con sẽ bị thay đổi ngẫu nhiên (ví dụ: đổi 0 thành 1 trong mã nhị phân). Toán tử này đóng vai trò sống còn trong việc duy trì sự đa dạng, giúp quần thể không bị hội tụ quá sớm vào một cực trị địa phương và có thể "nhảy" sang những vùng không gian mới tiềm năng hơn.

8.3. Thuật toán luyện kim mô phỏng

Mục này trình bày nguyên lý cơ bản và quy trình vận hành của thuật toán luyện kim mô phỏng (SA).

8.3.1. Nguyên lý cơ bản

Trong kỹ thuật luyện kim, để đạt được một cấu trúc tinh thể bền vững và ít khuyết tật nhất (trạng thái năng lượng thấp nhất), vật liệu được nung nóng đến nhiệt độ rất cao, nơi các nguyên tử có thể di chuyển hỗn loạn ở mức năng lượng lớn. Sau đó, vật liệu được làm nguội một cách chậm rãi và có kiểm soát, cho phép các nguyên tử dần dần sắp xếp vào các vị trí ổn định nhất.

Trong tối ưu hóa kết cấu, quá trình này được ánh xạ như sau:

Trạng thái vật chất: Tương ứng với một phương án thiết kế.

Năng lượng: Tương ứng với giá trị hàm mục tiêu (chẳng hạn trọng lượng kết cấu). Trạng thái năng lượng thấp nhất chính là lời giải tối ưu.

Nhiệt độ (T): Một tham số điều khiển mức độ chấp nhận các thay đổi ngẫu nhiên.

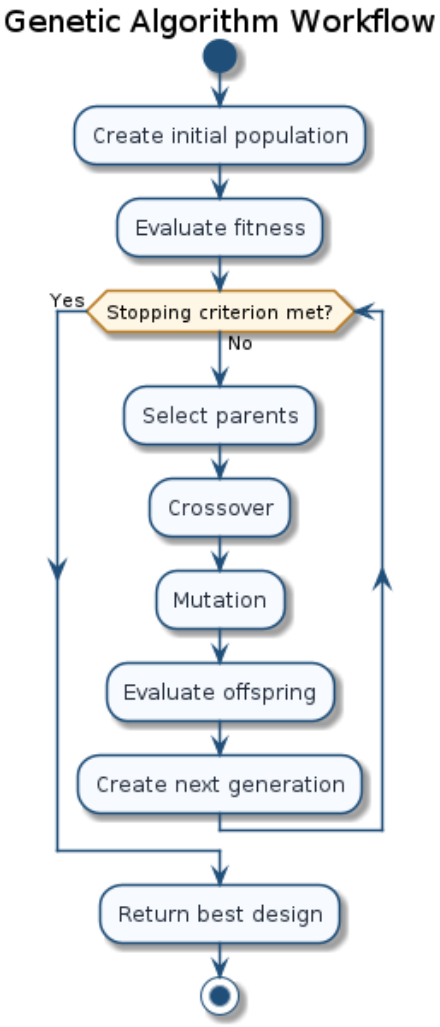
8.3.2. Quy trình vận hành

Chìa khóa giúp SA thoát khỏi các bẫy cực trị địa phương là khả năng chấp nhận các lời giải "tệ hơn" một cách tạm thời. Cơ chế này dựa trên tiêu chuẩn Metropolis [36] như sau:

Nếu lời giải mới không kém hơn lời giải hiện tại ($\Delta E \leqslant 0$), thuật toán sẽ chấp nhận nó ngay lập tức.

Nếu lời giải mới tệ hơn ($\Delta E > 0$), nó vẫn có thể được chấp nhận với xác suất P được tính theo công thức:

$$P(\Delta E) = \exp\left(-\frac{\Delta E}{T}\right) \tag{8.2}$$



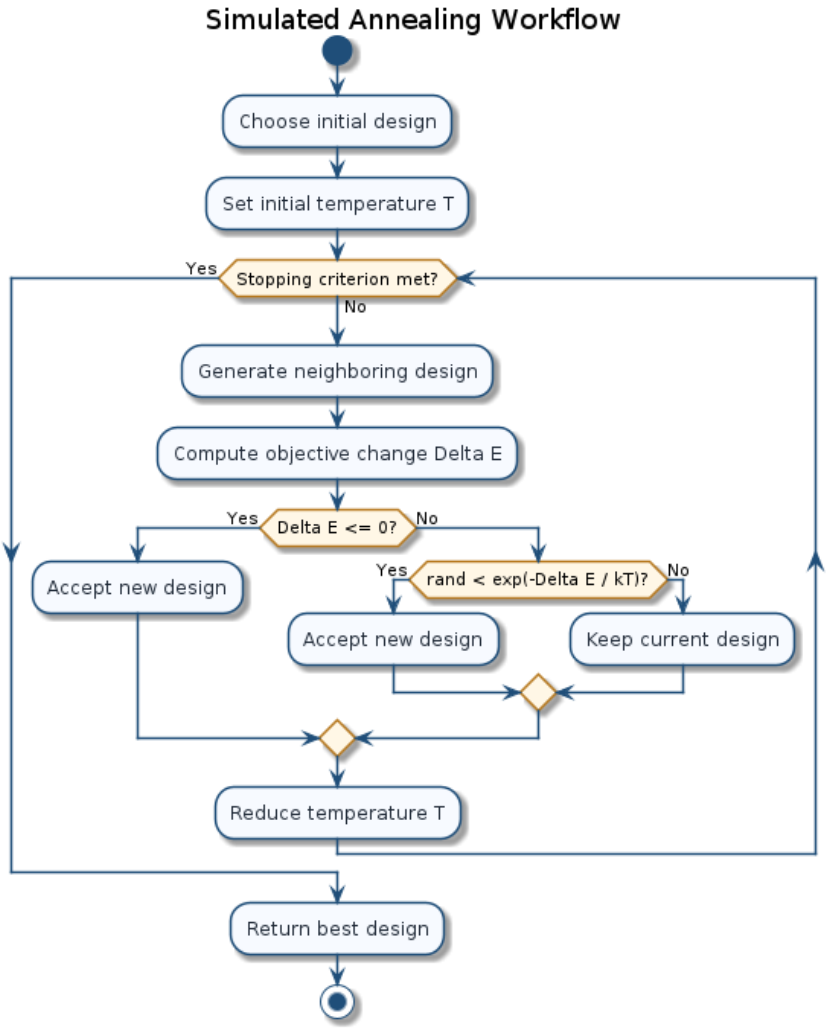
Hình 8.1. Lưu đồ thuật toán di truyền.

trong đó ΔE là độ chênh lệch giá trị hàm mục tiêu giữa lời giải mới và lời giải hiện tại, T là nhiệt độ hiện tại (trong vật lý, mẫu số là $k_B T$ với k_B là hằng số Boltzmann; trong tối ưu hóa hằng số này được gộp vào T).

Ở giai đoạn đầu, khi nhiệt độ còn cao, xác suất chấp nhận các lời giải tệ hơn là rất lớn, giúp thuật toán tự do khám phá (exploration) toàn bộ không gian thiết kế. Khi nhiệt độ giảm dần, xác suất này thấp đi, thuật toán trở nên khắt khe hơn và tập trung vào việc khai thác (exploitation) vùng lời giải tiềm năng để hội tụ về lời giải tốt (gần tối ưu toàn cục). Lưu đồ thuật toán luyện kim mô phỏng được thể hiện trên Hình 8.2.

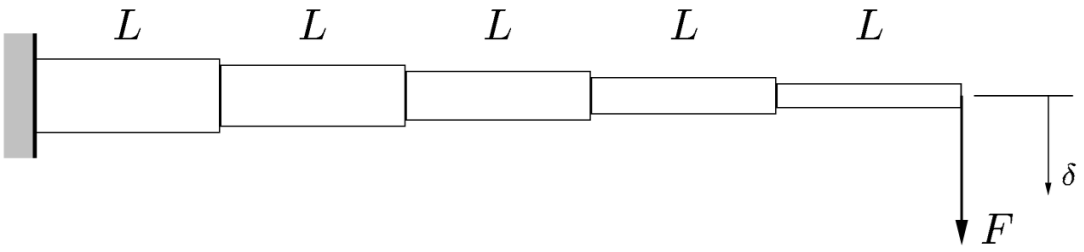
8.4. Ví dụ minh họa

Ví dụ 8.1: Bài toán đặt ra mục tiêu tối ưu hóa khối lượng của một dầm công-xôn bằng thép S270 ($E = 210$ GPa, $\sigma_{yield} = 270$ MPa) có tổng chiều dài $5L = 3$ m, chịu tải trọng tập trung $F = 500$ kN tại đầu tự do (Hình 8.3). Kết cấu dầm được chia thành 5 phân đoạn bằng nhau, mỗi đoạn dài $L = 0{,}6$ m với mặt cắt ngang là ống tròn rỗng, được xác định bởi 10 biến thiết kế tương ứng với bán kính trong r_{in} và bán kính ngoài r_{out} của từng đoạn. Quá trình tối ưu hóa bằng thuật toán luyện kim mô phỏng phải đảm bảo các điều kiện biên và ràng buộc kỹ thuật, bao gồm: độ võng tại đầu dầm



Hình 8.2. Lưu đồ thuật toán luyện kim mô phỏng.

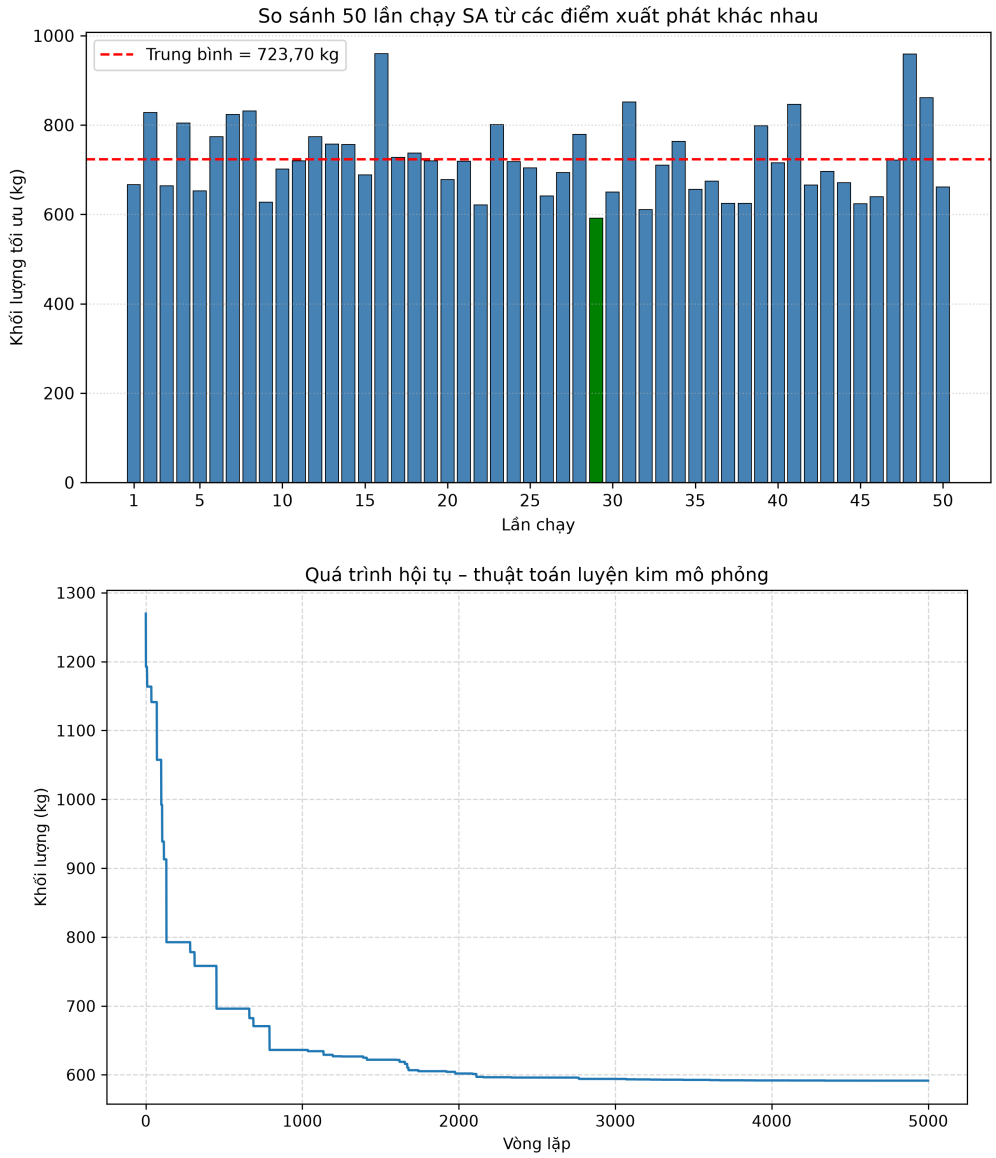
không vượt quá 2 cm, ứng suất lớn nhất tại mọi vị trí không vượt quá giới hạn chảy của vật liệu, và kích thước hình học phải thỏa mãn tính lắp ghép (khả năng chồng lấn để hàn nối giữa các đoạn). Ngoài ra, các biến số thiết kế cần tuân thủ giới hạn thực tế với bán kính ngoài từ 1 đến 30 cm và chiều dày thành ống nằm trong khoảng 0,1 đến 5 cm để đảm bảo tính khả thi trong chế tạo.



Hình 8.3. Dầm công-xôn năm đoạn tiết diện ống tròn rỗng (Ví dụ 8.1).

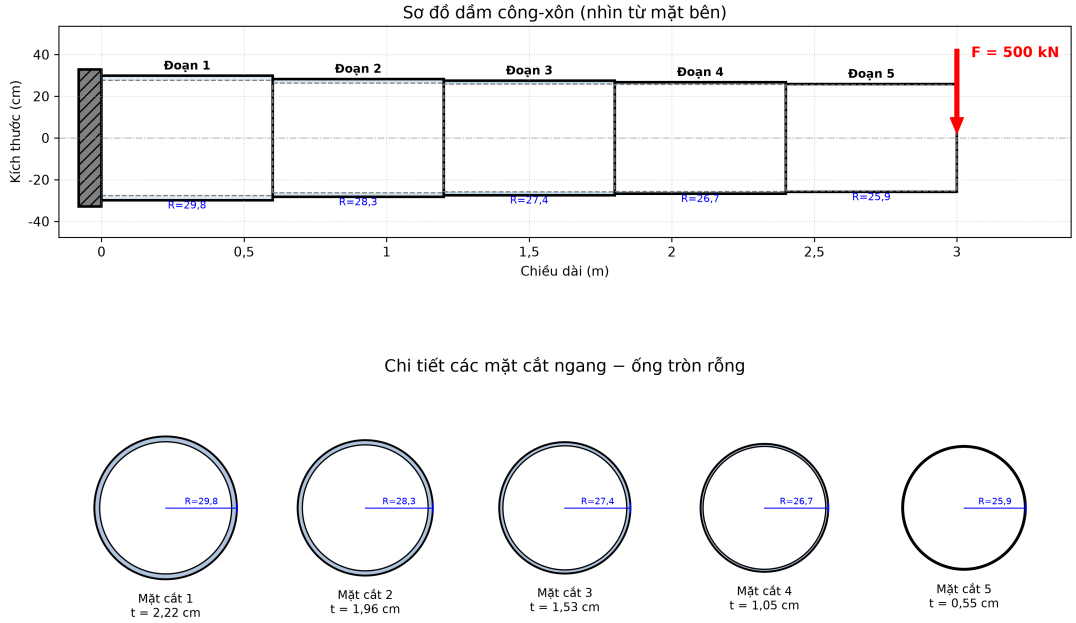
Kết quả tối ưu bằng thuật toán luyện kim mô phỏng được trình bày trong Hình 8.4 và Hình 8.5; mã nguồn tại Code 8.1 trên GitHub. Thiết kế tốt nhất thu được có

khối lượng 591,57 kg, chuyển vị đầu dầm 1,75 cm và ứng suất lớn nhất 269,98 MPa, thỏa mãn các ràng buộc về độ võng (2 cm) và giới hạn chảy (270 MPa); ràng buộc ứng suất là ràng buộc hoạt động. Code 8.1 chạy thuật toán 50 lần độc lập từ các điểm xuất phát ngẫu nhiên; kết quả trên là lần chạy tốt nhất, còn khối lượng thu được ở các lần chạy dao động từ 591,57 đến 959,65 kg (trung bình 723,70 kg, hệ số biến thiên 11,7%), cho thấy kết quả của thuật toán ngẫu nhiên phụ thuộc đáng kể vào điểm xuất phát. Để so sánh, giải cùng bài toán bằng phương pháp quy hoạch toàn phương tuần tự (SQP) cho khối lượng khoảng 551,6 kg (bán kính ngoài của cả năm đoạn đạt cận trên 30 cm), tức nghiệm của thuật toán luyện kim mô phỏng còn cao hơn nghiệm tối ưu khoảng 7%.



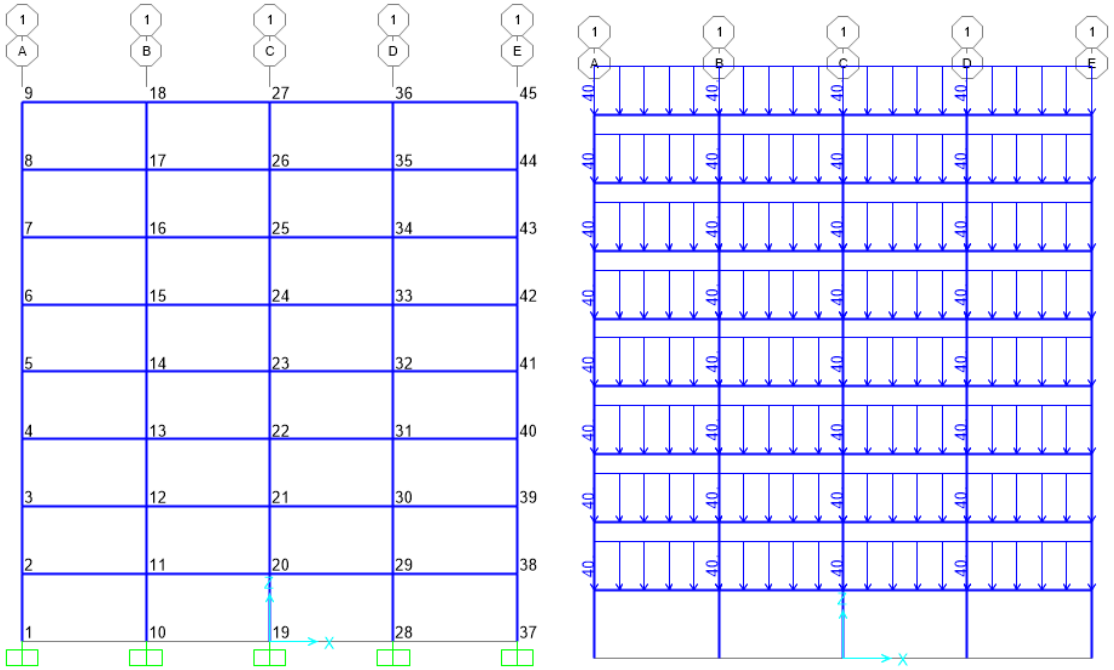
Hình 8.4. Kết quả tối ưu dầm công-xôn bằng thuật toán luyện kim mô phỏng: (a) khối lượng tốt nhất của 50 lần chạy độc lập (cột màu xanh lá là lần chạy tốt nhất); (b) quá trình hội tụ của lần chạy tốt nhất.

Ví dụ 8.2: Tối ưu tiết diện cho khung thép phẳng 4 nhịp, 8 tầng như Hình 8.6, với chiều dài mỗi nhịp 5 m và chiều cao tầng 3 m. Mỗi tầng chịu tải phân bố đều 40 kN/m



Hình 8.5. Chi tiết tiết diện tối ưu của dầm công-xôn (Ví dụ 8.1).

trên mỗi nhịp. Thép A36 (Grade 36) được sử dụng cho toàn bộ kết cấu.



(a) Mô hình khung thép (b) Tải trọng

Hình 8.6. Mô hình khung thép và tải trọng.

Việc phân tích phần tử hữu hạn và kiểm tra thiết kế được thực hiện trong SAP2000 thông qua giao diện lập trình OAPI [11] và công cụ Steel Design. Thuật toán luyện kim mô phỏng được sử dụng trong quá trình tối ưu, và việc thiết kế được thực hiện

theo phương pháp LRFD của tiêu chuẩn AISC 360-16 [6].

Các cột và dầm trong kết cấu được nhóm lại sao cho tiết diện thay đổi sau mỗi hai tầng. Do đó, tổng cộng có 4 nhóm cột và 4 nhóm dầm. Tiết diện của mỗi nhóm là một biến thiết kế được chọn từ danh sách tiết diện cho trước.

Danh sách tiết diện sử dụng trong quá trình tối ưu gồm các tiết diện chữ W được chọn từ catalog AISC. Danh sách này bao gồm các tiết diện có kích thước từ W12 đến W33. Việc lựa chọn tiết diện phù hợp cho từng nhóm sẽ được thực hiện từ danh sách này, và tiết diện tối ưu sẽ được xác định bởi thuật toán tối ưu.

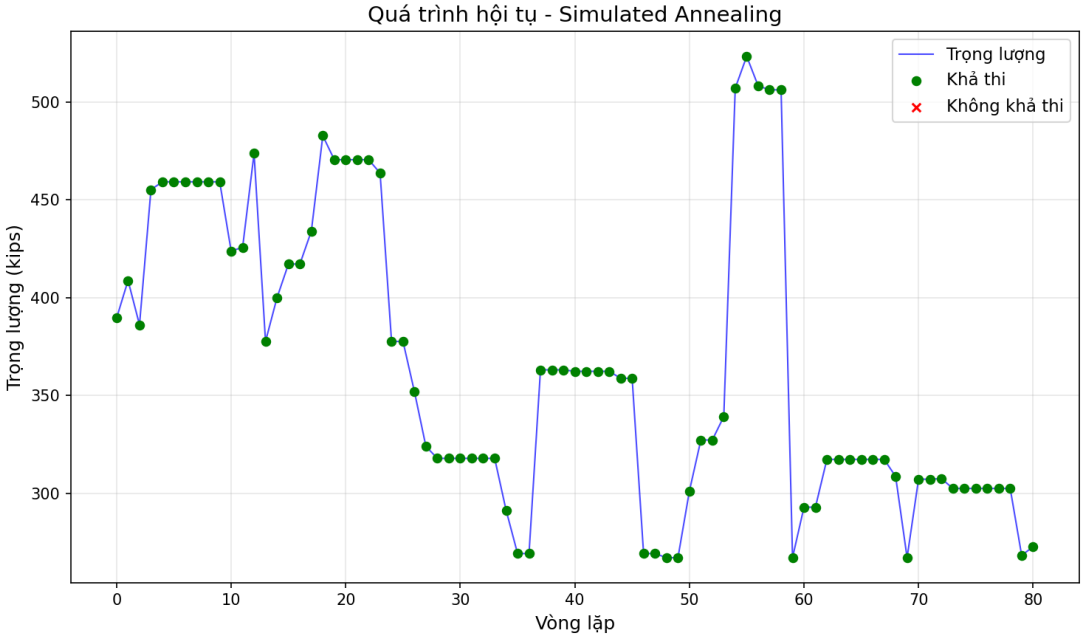
Hàm mục tiêu của bài toán tối ưu là tối thiểu hóa tổng khối lượng của kết cấu:

$$f(x) = \sum_{i=1}^{n} \rho_i A_i L_i \tag{8.3}$$

trong đó ρ_i là khối lượng riêng của vật liệu, A_i là diện tích tiết diện và L_i là chiều dài phần tử.

Để tính khối lượng kết cấu, một trường hợp tải có tên là `weight` được tạo trong mô hình. Do trường hợp tải này không chứa các lực khác, tổng phản lực gối theo phương trục z sẽ bằng trọng lượng của kết cấu (chia cho g để được khối lượng).

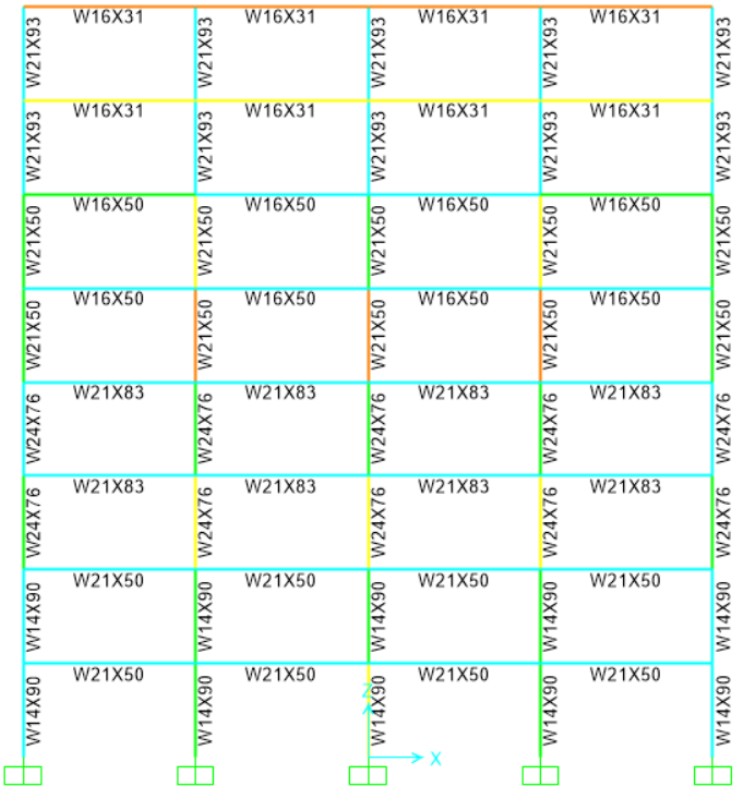
Các ràng buộc thiết kế được kiểm tra bằng công cụ Steel Design của SAP2000 trong quá trình tối ưu. Phương thức `VerifyPassed()` trả về số lượng phần tử khung không thỏa mãn yêu cầu tiêu chuẩn thiết kế. Bên trong công cụ này, toàn bộ các kiểm tra, từ chiều dài tính toán ổn định đến các phương trình tương tác, đều được thực hiện.



Hình 8.7. Quá trình tối ưu hóa của thuật toán luyện kim mô phỏng.

Quá trình hội tụ và khung tối ưu được thể hiện ở Hình 8.7 và Hình 8.8; mã nguồn tại Code 8.2 trên GitHub. Như đã đề cập, thuật toán luyện kim mô phỏng là một phương pháp ngẫu nhiên. Vì vậy, việc thu được các kết quả khác nhau ở mỗi lần chạy là điều bình thường. Do đó, trong các nghiên cứu, thuật toán thường được chạy nhiều lần độc lập để đánh giá tính ổn định và hiệu năng tổng thể.

Trong thực tế, nhiều nghiên cứu tối ưu kết cấu không sử dụng OAPI mà thay vào đó tự lập trình kiểm tra ràng buộc thiết kế. Cách tiếp cận này giúp tăng đáng kể tốc



Hình 8.8. Tiết diện tối ưu của khung thép (Ví dụ 8.2).

độ tính toán, bằng cách áp dụng trực tiếp các phương trình tương tác sau để kiểm tra điều kiện bền của thanh (Tiêu chuẩn AISC 360-16 [6], các phương trình H1-1a và H1-1b):

$$\begin{aligned} &\frac{P_u}{\phi_c P_n} \geq 0{,}2; \quad c_1 = \frac{P_u}{\phi_c P_n} + \frac{8}{9}\left(\frac{M_{ux}}{\phi_b M_{nx}} + \frac{M_{uy}}{\phi_b M_{ny}}\right) \\ &\frac{P_u}{\phi_c P_n} < 0{,}2; \quad c_2 = \frac{P_u}{2\phi_c P_n} + \left(\frac{M_{ux}}{\phi_b M_{nx}} + \frac{M_{uy}}{\phi_b M_{ny}}\right) \\ &\forall i: \ c_i \leq 1{,}0 \end{aligned} \tag{8.4}$$

8.5. Bài tập

Bài 8.1: Cho kết cấu giàn không gian 120 thanh dạng vòm như Hình 8.9, với mô-đun đàn hồi $E = 210$ GPa, khối lượng riêng $\rho = 7971{,}8$ kg/m^3 ($0{,}288$ lb/in^3) và giới hạn chảy $f_y = 400$ MPa. Hãy xác định diện tích tiết diện tối ưu của 7 nhóm thanh ($5\,\text{cm}^2 \leqslant A_i \leqslant 130\,\text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất kéo $\sigma \leqslant 0{,}6 f_y$, ràng buộc mất ổn định theo tiêu chuẩn AISC cho các thanh chịu nén, và chuyển vị $[\delta] = 5$ mm tại mọi bậc tự do. Mã nguồn tham khảo được cung cấp tại Code 8.3 trên GitHub.

Kiểm tra ràng buộc ứng suất và mất ổn định theo tiêu chuẩn AISC (ASD 1989) [5]; số liệu bài toán và công thức bán kính quán tính dưới đây lấy theo bộ bài toán của Öztürk và Kahraman [38]:

Ứng suất kéo cho phép được xác định là $\sigma_{allow}^{tension} = 0{,}6 f_y$. Đối với các thanh

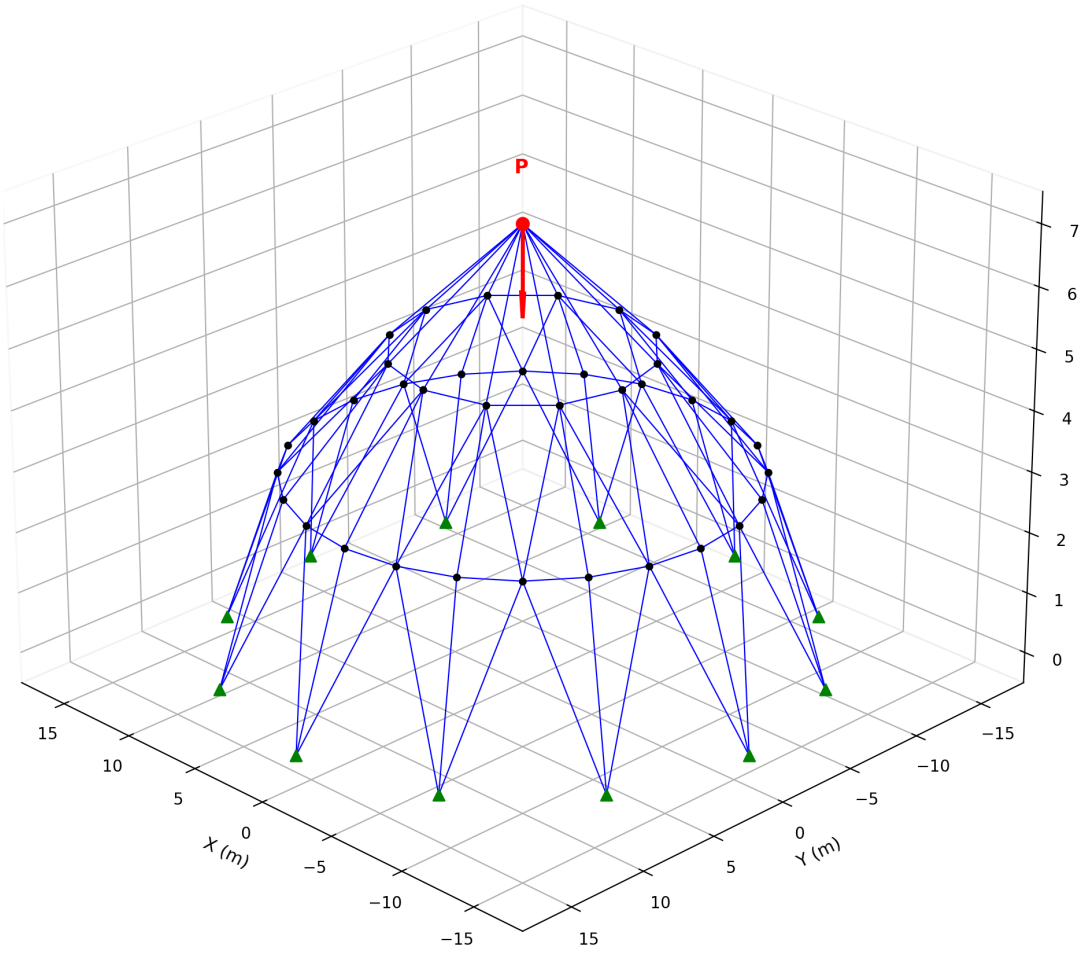
chịu nén, ứng suất nén cho phép σ_{allow}^{comp} phụ thuộc vào độ mảnh $\lambda_i = \frac{kL_i}{r_i}$, trong đó L_i là chiều dài thanh, $k = 1$ là hệ số chiều dài hiệu dụng, r_i là bán kính quán tính, được xấp xỉ theo diện tích mặt cắt ngang A_i bằng công thức thực nghiệm: $r_i = 0{,}4993\, A_i^{0{,}6777}$ (với A_i tính bằng in^2, r_i tính bằng in), tương đương trong hệ SI: $r_i = 0{,}0254 \times 0{,}4993 \times \left(A_i/(6{,}4516 \times 10^{-4})\right)^{0{,}6777}$ (m, với A_i tính bằng m^2).

Tùy thuộc vào giá trị độ mảnh tới hạn $C_c = \sqrt{\frac{2\pi^2 E}{f_y}}$, ứng suất nén cho phép được tính theo hai trường hợp:

Nếu $\lambda_i < C_c$ (mất ổn định phi đàn hồi): $\sigma_{allow}^{comp} = \dfrac{\left[1 - \frac{\lambda_i^2}{2C_c^2}\right] f_y}{\frac{5}{3} + \frac{3\lambda_i}{8C_c} - \frac{\lambda_i^3}{8C_c^3}}$.

Nếu $\lambda_i \geqslant C_c$ (mất ổn định đàn hồi Euler): $\sigma_{allow}^{comp} = \frac{12\pi^2 E}{23\lambda_i^2}$.

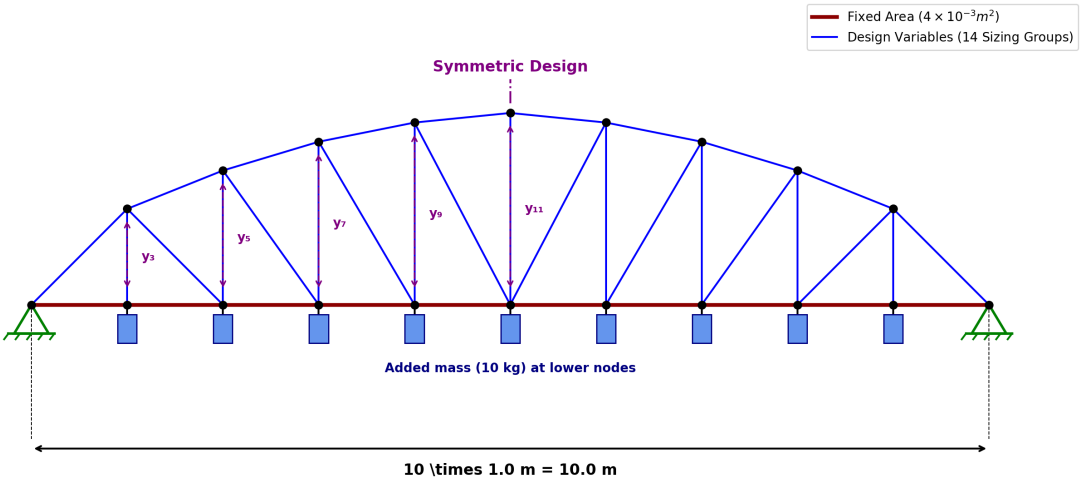
Yêu cầu tối ưu là đảm bảo ứng suất thực tế trong các thanh không vượt quá giới hạn trên $\left(\frac{|\sigma_i|}{\sigma_{allow}} \leqslant 1\right)$.



Hình 8.9. Giàn không gian 120 thanh dạng vòm (Bài 8.1).

Bài 8.2: Cho kết cấu giàn 37 thanh như Hình 8.10 với mô-đun đàn hồi $E = 2{,}1 \times 10^{11}$ Pa, khối lượng riêng $\rho = 7800$ kg/m^3, và các khối lượng tập trung 10 kg được gắn thêm tại các nút tự do của cánh dưới. Các thanh cánh dưới (10 thanh) có tiết diện cố định $A = 4 \times 10^{-3}$ m^2. Hãy xác định đồng thời hình dạng (tọa độ y của 5 nút cánh trên,

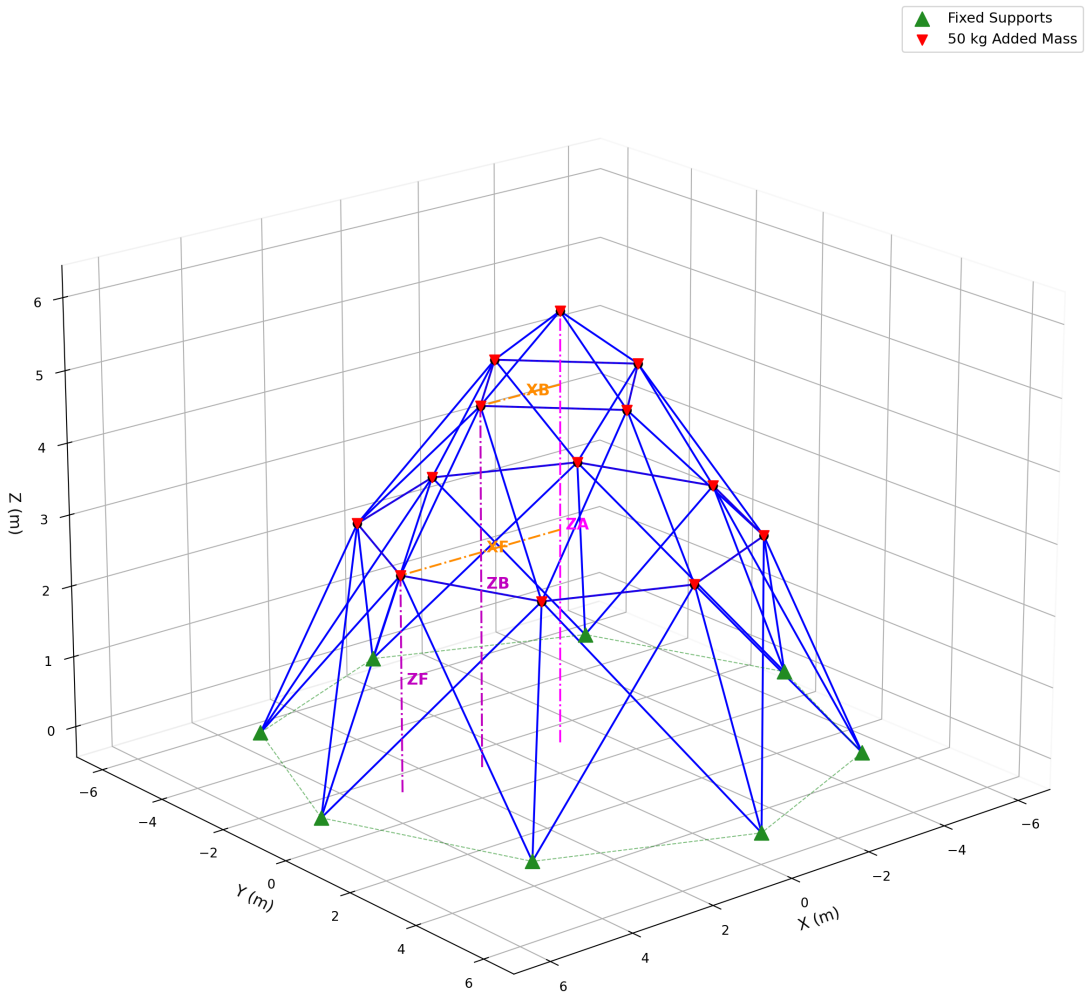
biến thiên đối xứng trong khoảng 0,5 m $\leqslant y_i \leqslant$ 5,0 m) và kích thước tiết diện tối ưu của 14 nhóm thanh còn lại (10^{-4} m^2 $\leqslant A_i \leqslant 100 \times 10^{-4}$ m^2) sao cho khối lượng kết cấu là nhỏ nhất. Quá trình thiết kế phải đảm bảo hệ kết cấu thỏa mãn ràng buộc đối với ba tần số dao động riêng đầu tiên: $f_1 \geqslant 20$ Hz, $f_2 \geqslant 40$ Hz và $f_3 \geqslant 60$ Hz (bài toán của Wang và cộng sự [48]). Mã nguồn tham khảo được cung cấp tại Code 8.4 trên GitHub.



Hình 8.10. Giàn 37 thanh với ràng buộc tần số dao động riêng (Bài 8.2).

Bài 8.3: Cho kết cấu giàn không gian 52 thanh dạng vòm như Hình 8.11, với mô-đun đàn hồi $E = 2{,}1 \times 10^{11}$ Pa, khối lượng riêng $\rho = 7800$ kg/m^3, và khối lượng tập trung 50 kg tại 13 nút tự do của vòm (khối lượng tác dụng theo cả ba phương). Hãy xác định đồng thời hình dạng (5 biến tọa độ không gian: Z_A; X_B, Z_B, X_F, Z_F của 2 vòng trung gian) và tiết diện tối ưu của 8 nhóm thanh ($10^{-4} \leqslant A_i \leqslant 10^{-3}$ m^2) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn đồng thời hai ràng buộc tần số dao động riêng: $f_1 \leqslant 15{,}916$ Hz và $f_2 \geqslant 28{,}648$ Hz (bài toán của Lin và cộng sự [33]). Mã nguồn tham khảo được cung cấp tại Code 8.5 trên GitHub.

Bài 8.4: Đặt 4 khối lượng 500 kg tại các nút tự do của hệ giàn trong Bài 5.1. Giải bài toán tối ưu với ràng buộc tần số dao động riêng: $f_1 \geqslant 7$ Hz, $f_2 \geqslant 15$ Hz, $f_3 \geqslant 20$ Hz (bài toán của Grandhi và Venkayya [23]).



Hình 8.11. Giàn không gian 52 thanh dạng vòm (Bài 8.3).

CHƯƠNG 9

ỨNG DỤNG TỐI ƯU KẾT CẤU TRONG CÔNG TRÌNH CẦU

Các chương trước đã xây dựng lần lượt những công cụ của tối ưu kết cấu: cách phát biểu bài toán, điều kiện tối ưu KKT, các thuật toán xấp xỉ lồi như SQP, tối ưu đa mục tiêu và các thuật toán tiến hóa. Chương này vận dụng các công cụ đó vào ba bài toán thực tế của công trình cầu, trong đó mỗi bài toán gắn với một nguồn số liệu đã công bố hoặc một tiêu chuẩn thiết kế hiện hành. Mỗi ví dụ được trình bày theo một quy trình trọn vẹn: mô tả kỹ thuật, lập mô hình toán học, kiểm chứng mô hình tính, giải bằng nhiều phương pháp và diễn giải kết quả cho người thiết kế. Mục đích không chỉ là tìm ra một con số tối ưu mà còn là cho thấy những quyết định mô hình hóa ảnh hưởng thế nào tới nghiệm và người kỹ sư cần đọc kết quả tối ưu ra sao.

9.1. Bài toán tối ưu trong thiết kế và khai thác công trình cầu

9.1.1. Đặc điểm của bài toán tối ưu công trình cầu

So với các ví dụ mang tính học thuật ở những chương trước, bài toán tối ưu công trình cầu có bốn đặc điểm nổi bật. Thứ nhất, ràng buộc của bài toán được lấy từ tiêu chuẩn thiết kế như TCVN 11823:2017 [4], EN 1990 [16] và EN 1991-2 [17]. Mỗi tiêu chuẩn quy định nhiều trạng thái giới hạn (cường độ, sử dụng, mỏi) và nhiều giai đoạn làm việc (thi công, khai thác), nên số ràng buộc lớn; nhiều ràng buộc chỉ tính được qua một chuỗi thủ tục kiểm toán chứ không có dạng công thức tường minh. Thứ hai, tải trọng chủ yếu là hoạt tải di động (HL-93, đoàn tàu), nên mỗi hàm ràng buộc thực chất là giá trị bao của nhiều vị trí xếp tải, nhiều đoàn tàu và nhiều vận tốc. Các hàm bao như vậy thường không trơn và cần được xử lý cẩn thận khi dùng các phương pháp dựa trên đạo hàm.

Thứ ba, cầu đường sắt chịu tác dụng có tính chu kỳ của các trục bánh xe, nên bài toán thường mang tính động lực học, với hiện tượng cộng hưởng khiến phản ứng nhạy với tần số riêng của kết cấu. Thứ tư, với cầu đang khai thác, mô hình tính toán chỉ đáng tin cậy khi được hiệu chỉnh theo dữ liệu đo; khi đó bài toán tối ưu là một bài toán ngược, trong đó ẩn số là các tham số của mô hình chứ không phải kích thước thiết kế.

Từ các đặc điểm trên, có thể phân biệt ba nhóm bài toán tối ưu thường gặp. Nhóm thứ nhất là thiết kế cấu kiện, tức tối ưu kích thước tiết diện theo tiêu chuẩn. Nhóm thứ hai là thiết kế giải pháp giảm dao động, trong đó biến thiết kế là tham số của thiết bị như bộ giảm chấn khối lượng điều chỉnh (Tuned Mass Damper – TMD) hay giảm chấn nhớt. Nhóm thứ ba là nhận dạng và hiệu chỉnh mô hình từ dữ liệu quan trắc. Ba ví dụ trong chương lần lượt thuộc ba nhóm này.

9.1.2. Các ví dụ trong chương và phương pháp áp dụng

Bảng 9.1 tóm tắt ba ví dụ của chương. Ví dụ 9.1 dựng lại bài toán tối ưu dầm thép liên hợp của Hùng và cộng sự [1], vốn đã có lời giải công bố để đối chiếu. Ví dụ 9.2 là bài toán đặc thù của cầu đường sắt cao tốc: thiết kế TMD để đưa gia tốc mặt cầu về trong giới hạn cho phép của EN 1990 khi chịu các đoàn tàu của mô hình tải trọng cao tốc (High Speed Load Model – HSLM) loại A (HSLM-A). Ví dụ 9.3 dùng dữ liệu đo dao động trên cầu giàn thép đường sắt Nam Ô (Đà Nẵng) do Tran-Ngoc và cộng sự [47] công bố để hiệu chỉnh mô hình phần tử hữu hạn. Ba ví dụ được sắp xếp theo mức độ phức tạp của mô hình kết cấu: từ một tiết diện dầm, qua một mô hình động lực học một nhịp, đến một mô hình khung không gian.

Bảng 9.1. Các ví dụ của chương và phương pháp áp dụng

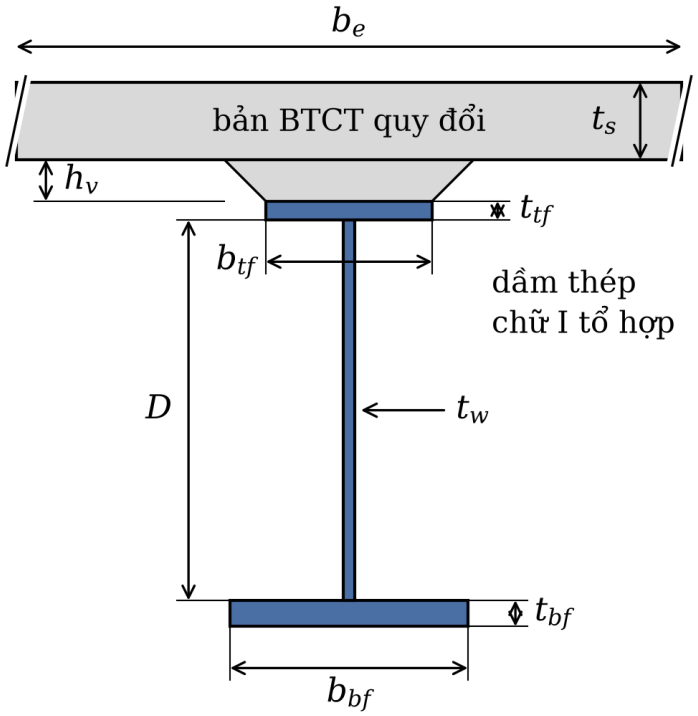
	Ví dụ 9.1	**Ví dụ 9.2**	**Ví dụ 9.3**
bài toán	dầm thép I liên hợp nhịp 33 m	TMD cho cầu đường sắt cao tốc	hiệu chỉnh mô hình cầu Nam Ô
biến thiết kế	6 kích thước tiết diện	tỷ số tần số, tỷ số cản (và khối lượng) TMD	4 tham số mô hình
mục tiêu	khối lượng thép	gia tốc lớn nhất mặt cầu	sai số tần số riêng
ràng buộc	TCVN 11823:2017	cận của biến	cận của tham số
phương pháp	SQP, DE, KKT, ràng buộc epsilon	Den Hartog, đồ thị, SQP, GA, Pareto	bình phương tối thiểu, GA, độ nhạy
chương liên quan	3, 4, 7, 8	2, 4, 7, 8	4, 5, 8

9.2. Tối ưu dầm thép chữ I liên hợp theo TCVN 11823:2017

9.2.1. Mô tả bài toán và số liệu

Ví dụ 9.1: Xét một dầm trong của cầu dầm thép liên hợp bản bê tông cốt thép (BTCT) nhịp giản đơn dài 33 m (nhịp tính toán $L = 32$ m), mặt cắt ngang gồm sáu dầm chủ cách nhau 2300 mm, theo ví dụ thứ nhất của Hùng và cộng sự [1]. Dầm thép là tiết diện chữ I tổ hợp từ ba bản thép, thép có $F_y = 345$ MPa và $E = 2 \times 10^5$ MPa; bản bê tông quy đổi có bề rộng $b_e = 2300$ mm, chiều dày $t_s = 241{,}6$ mm, cường độ chịu nén $f'_c = 30$ MPa, tỷ số mô-đun đàn hồi $n = 8$ (Hình 9.1). Tĩnh tải tác dụng lên một dầm gồm bản mặt cầu và vút 13,35 N/mm, dầm ngang và sườn tăng cường 1,75 N/mm, neo chịu cắt và mối nối 0,2 N/mm, lớp phủ mặt đường 3,62 N/mm. Hoạt tải là HL-93 (xe tải hoặc xe hai trục cộng tải trọng làn), hệ số xung kích 33%, hệ số phân bố cho mô-men và lực cắt lần lượt là 0,598 và 0,793. Các tổ hợp tải trọng được xét là Cường độ I (CĐ1: $1{,}25\,DC + 1{,}5\,DW + 1{,}75\,LL$), Sử dụng II (SD2: $1{,}0\,DC + 1{,}0\,DW + 1{,}3\,LL$) và Mỏi, trong đó DC, DW, LL lần lượt là tĩnh tải kết cấu, tĩnh tải lớp phủ và hoạt tải (kể cả lực xung kích). Uốn và cắt theo CĐ1 được kiểm tra tại năm mặt cắt từ gối đến giữa nhịp; SD2, độ võng và mỏi được kiểm tra tại giữa nhịp.

Bài báo gốc cung cấp hai mốc so sánh có giá trị cho người học. Thiết kế theo kinh nghiệm có bản cánh trên 300×16 mm, bản cánh dưới 500×22 mm, bản bụng



Hình 9.1. Tiết diện dầm liên hợp và sáu biến thiết kế (sơ đồ không theo tỷ lệ).

14 × 1125 mm (bản cánh ghi bề rộng × chiều dày, bản bụng ghi chiều dày × chiều cao), khối lượng 8173 kg. Nghiệm tối ưu công bố, tìm bằng thuật toán tiến hóa vi sai (DE), có khối lượng 5399 kg, tức chỉ bằng 66% thiết kế kinh nghiệm.

Để dựng lại bài toán, ví dụ này cần bổ sung một số giả thiết mà bài báo không nêu chi tiết: bản bụng có sườn tăng cường đứng với khoảng cách $d_0 = D$ ở khoang trong và $d_0 = 0{,}5D$ ở khoang đầu dầm; chi tiết mỏi kiểm tra là đường hàn sườn đứng vào bản cánh dưới (chi tiết loại C′, ngưỡng mỏi biên độ không đổi $(\Delta F)_{TH} = 82{,}7$ MPa); khi thi công, bản cánh trên được giằng bởi dầm ngang cách nhau 4 m; kiểm tra giai đoạn thi công chỉ xét 1,25 lần tĩnh tải giai đoạn 1 (kể cả bê tông ướt), chưa kể hoạt tải thi công (người, máy hoàn thiện, với hệ số 1,5 theo [4]) và trình tự đổ bê tông. Các giả thiết này được nêu rõ để người đọc có thể thay đổi và quan sát ảnh hưởng của chúng.

9.2.2. Phát biểu bài toán tối ưu

Biến thiết kế là sáu kích thước của dầm thép $\mathbf{x} = [b_{tf}, t_{tf}, b_{bf}, t_{bf}, t_w, D]$ (mm), với cận dưới và cận trên như bài báo: bề rộng bản cánh trong khoảng 50–600 mm, chiều dày bản cánh 5–100 mm, chiều dày bản bụng 5–30 mm và chiều cao bản bụng 500–3000 mm. Hàm mục tiêu là khối lượng dầm thép dài 33 m. Bài toán được viết dưới dạng chuẩn của Chương 3:

$$\begin{aligned} &\min_{\mathbf{x}} \; m(\mathbf{x}) = \rho\, L_d\, (b_{tf} t_{tf} + D t_w + b_{bf} t_{bf}) \\ &\text{s.t.} \quad g_i(\mathbf{x}) \leqslant 0, \;\; i = 1, \ldots, l; \qquad \mathbf{x}^{\min} \leqslant \mathbf{x} \leqslant \mathbf{x}^{\max} \end{aligned} \tag{9.1}$$

trong đó $\rho = 7850$ kg/m^3 là khối lượng riêng của thép, $L_d = 33$ m là chiều dài dầm, g_i là các ràng buộc đã chuẩn hóa và l là số ràng buộc (16 hoặc 19, tùy mô hình). Mỗi ràng

buộc được chuẩn hóa về dạng tỷ số không thứ nguyên, ví dụ $g = M_u/(\phi_f M_n) - 1 \leqslant 0$ cho điều kiện chịu uốn, nhờ đó các ràng buộc có cùng thang đo và giá trị g cho biết ngay mức dư (khi âm) hay mức vi phạm (khi dương).

Các ràng buộc gồm năm nhóm. Nhóm cấu tạo gồm $D/t_w \leqslant 150$, $b_f/(2t_f) \leqslant 12$, $b_f \geqslant D/6$, $t_f \geqslant 1{,}1t_w$ và $0{,}1 \leqslant I_{yc}/I_{yt} \leqslant 10$. Nhóm cường độ gồm điều kiện chịu uốn của tiết diện liên hợp theo mô-men dẻo M_p (tiết diện đặc chắc, có xét điều kiện dẻo dai $D_p \leqslant 0{,}42D_t$, với D_p là khoảng cách từ mặt trên bản bê tông tới trục trung hòa dẻo và D_t là chiều cao toàn tiết diện liên hợp) và điều kiện chịu cắt của bản bụng có sườn tăng cường (kể sức kháng sau oằn do trường kéo ở khoang trong). Nhóm sử dụng gồm ứng suất ở bản cánh trên và bản cánh dưới do tổ hợp SD2 gây ra không vượt quá $0{,}95R_hF_y$ (R_h là hệ số lai, bằng 1 với tiết diện đồng nhất) và độ võng do hoạt tải không vượt $L/800$ (tính với hệ số phân bố mô-men như bài báo, thiên về an toàn so với cách xếp tải mọi làn mà tiêu chuẩn khuyến nghị). Nhóm mỏi giới hạn biên độ ứng suất ở bản cánh dưới do xe tải mỏi. Nhóm thi công yêu cầu dầm thép, khi một mình chịu bê tông ướt và các tĩnh tải giai đoạn 1, không bị chảy, không mất ổn định cục bộ hay xoắn ngang của bản cánh nén và không oằn bản bụng.

Một điểm quan trọng là ứng suất trong dầm phụ thuộc vào trình tự thi công. Khi thi công không có chống đỡ, tĩnh tải giai đoạn 1 do riêng dầm thép chịu, lớp phủ do tiết diện liên hợp dài hạn ($3n$) chịu, còn hoạt tải do tiết diện liên hợp ngắn hạn (n) chịu. Để thấy rõ ảnh hưởng của giả thiết này, ví dụ xét hai mô hình. Mô hình A bám sát giả thiết của bài báo: mọi tải trọng do tiết diện liên hợp chịu, không kể trọng lượng bản thân dầm thép, không kiểm tra giai đoạn thi công và tính mỏi với hệ số tải trọng 0,75 (hệ số này đi cùng mô-men mỏi trong bảng kiểm chứng của bài báo; ghép nó với ngưỡng $(\Delta F)_{TH}$ đầy đủ là kém an toàn hơn kiểm toán mỏi của tiêu chuẩn, nhưng ràng buộc mỏi của mô hình A còn dư khoảng 45% nên đơn giản hóa này không ảnh hưởng tới nghiệm). Mô hình B đầy đủ theo trình tự thi công không chống đỡ: kể trọng lượng bản thân, kiểm tra giai đoạn thi công và tính mỏi với hệ số tải trọng 1,5 (tổ hợp Mỏi I) [4].

9.2.3. Kiểm chứng mô hình tính toán

Trước khi tối ưu, chương trình tính phải được kiểm chứng, vì một thuật toán tối ưu sẽ khai thác triệt để mọi sai sót của mô hình. Bài báo gốc cung cấp các bảng kiểm chứng nội lực và mô-men dẻo của thiết kế kinh nghiệm, nên ở đây ta dùng chúng để đối chiếu (Bảng 9.2); riêng mô-men mỏi được lấy từ bảng kiểm chứng của dầm 50 m (Bài 9.1), vì bài báo chỉ công bố giá trị này cho dầm đó. Mô-men CĐ1 tại các mặt cắt chênh lệch không quá 0,35%. Lực cắt tại gối sai khác lớn hơn (0,9%), nhiều khả năng do cách xếp tải trọng làn khi tính lực cắt; vì lực cắt không quyết định nghiệm (xem Mục 9.2.5) nên sai khác này được chấp nhận.

Bảng 9.2. Kiểm chứng chương trình tính với số liệu của Hùng và cộng sự [1]

Đại lượng	Bài báo	Chương trình	Sai khác
M_u tại 1/8 nhịp (kN·m)	3340,05	3342,19	+0,06%
M_u tại 3/8 nhịp (kN·m)	7026,28	7050,52	+0,34%
M_u tại giữa nhịp (kN·m)	7471,95	7468,90	−0,04%

Đại lượng	Bài báo	Chương trình	Sai khác
V_u tại gối (kN)	1135,67	1145,87	+0,90%
M_p, trục trung hòa dẻo trong bản bê tông, $t_w = 12$ mm (kN·m)	9645,23	9645,23	0,00%
M_p, trục trung hòa dẻo trong bản cánh trên, $t_w = 25$ mm (kN·m)	13 102,03	13 102,03	0,00%
M_p, trục trung hòa dẻo trong bản bụng, $t_w = 32$ mm (kN·m)	14 639,94	14 639,93	0,00%
Mô-men mỏi giữa nhịp, dầm 50 m (kN·m)	1403,07	1403,03	0,00%

Việc kiểm chứng mô-men dẻo cho một kết quả đáng chú ý. Bài báo kiểm chứng M_p cho ba trường hợp vị trí trục trung hòa dẻo (trong bản bê tông, trong bản cánh trên và trong bản bụng, bằng cách thay đổi t_w). Cả ba giá trị chỉ được tái lập chính xác khi giữa mặt dưới bản quy đổi và mặt trên bản cánh có một khoảng vút quy đổi $h_v = 98{,}4$ mm; giá trị này được xác định từ một trường hợp và khớp đúng cho cả hai trường hợp còn lại, nên có thể coi là một thông số hình học của bài toán gốc. Quá trình này minh họa một thói quen nên có khi dùng số liệu công bố: dựng lại các bảng kiểm chứng trước, và khi chưa khớp thì tìm thông số bị thiếu thay vì bỏ qua sai khác.

9.2.4. Lời giải bằng SQP và thuật toán tiến hóa

Bài toán (9.1) có sáu biến và 16 (mô hình A) hoặc 19 (mô hình B) ràng buộc. Các ràng buộc chỉ liên tục từng khúc và không khả vi tại một số điểm, vì chúng chứa các phép lấy giá trị lớn nhất theo mặt cắt và các nhánh công thức của tiêu chuẩn (đặc chắc hay không đặc chắc, công thức sức kháng cắt theo độ mảnh); tại ranh giới đặc chắc – không đặc chắc, ràng buộc uốn thậm chí có thể nhảy bậc. Ta giải bài toán bằng hai cách. Cách thứ nhất là SQP (Chương 4) với đạo hàm sai phân hữu hạn, xuất phát từ thiết kế kinh nghiệm và từ 30 điểm ngẫu nhiên trong miền thiết kế.

Cách thứ hai là thuật toán tiến hóa vi sai DE [44], một thuật toán tiến hóa gần với GA ở Chương 8: mỗi cá thể đột biến được tạo bằng cách cộng vào một cá thể hiệu của hai cá thể khác nhân với hệ số F, sau đó được lai ghép với cá thể hiện tại và chọn lọc một-một. Các tham số được lấy đúng như bài báo: quần thể 25 cá thể, 500 thế hệ, $F = 0{,}7$, hệ số lai ghép $CR = 0{,}6$, chạy độc lập 20 lần. Trong DE, ràng buộc được xử lý bằng hàm phạt:

$$m_p(\mathbf{x}) = m(\mathbf{x})\Big(1 + \alpha \sum_i \max\{g_i(\mathbf{x}), 0\}\Big) \tag{9.2}$$

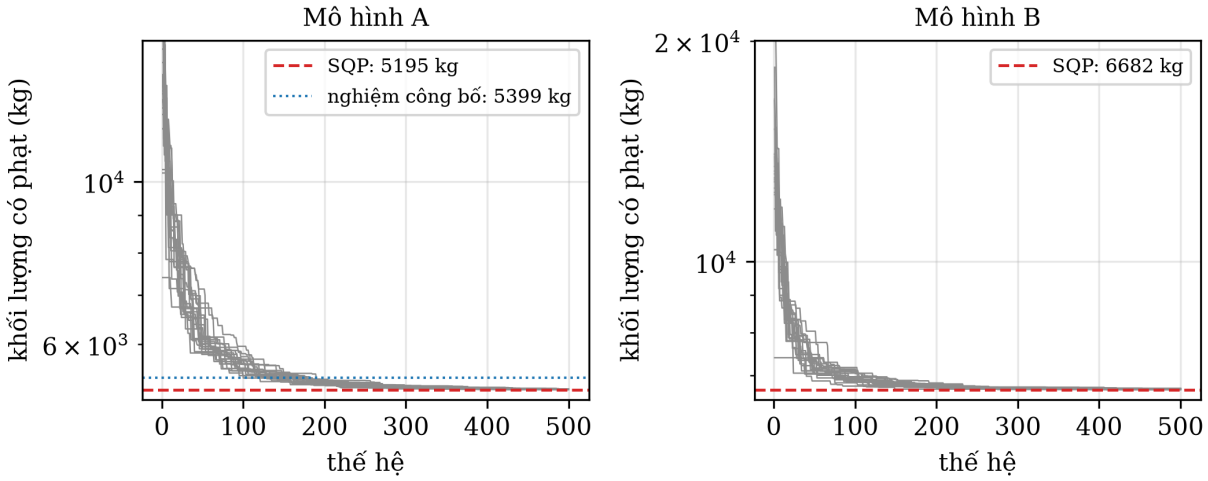
với $\alpha = 10^4$, giống như trong bài báo gốc.

Bảng 9.3 và Hình 9.2 trình bày kết quả. Ở đây một lượt đánh giá hàm là một lần kiểm toán toàn bộ các ràng buộc của một phương án. Với mô hình A, SQP hội tụ tới nghiệm khả dĩ từ cả 31 điểm xuất phát, nghiệm tốt nhất nặng 5195 kg, mỗi lần giải cần trung bình khoảng 140 lượt đánh giá hàm (kể cả các lượt tính đạo hàm sai phân); DE cho khối lượng từ 5201 đến 5212 kg qua 20 lần chạy, mỗi lần cần 12 525 lượt đánh giá hàm. Nghiệm của mô hình A nhẹ hơn nghiệm công bố khoảng 3,8%; khi kiểm tra theo mô hình A, nghiệm công bố thỏa mãn mọi ràng buộc trừ ứng suất bản cánh dưới ở trạng thái sử dụng (vượt 5,5%). Hai kết quả khá sát nhau; phần chênh lệch nhiều

khả năng do những chi tiết của mô hình kiểm toán mà bài báo không công bố, nên có thể coi bộ giải đã được kiểm chứng ở mức hợp lý. Với mô hình B, SQP cho nghiệm 6682 kg, nhẹ hơn thiết kế kinh nghiệm 18%, còn DE cho 6688–6717 kg.

Bảng 9.3. Kết quả tối ưu dầm thép nhịp 33 m (kích thước tính bằng mm)

Phương án	**Bản cánh trên**	**Bản cánh dưới**	**Bản bụng**	m **(kg)**
Thiết kế kinh nghiệm [1]	300×16	500×22	14×1125	8173
Nghiệm công bố (DE) [1]	$228{,}6 \times 10{,}0$	$240{,}6 \times 25{,}2$	$9{,}12 \times 1368{,}6$	5399
Mô hình A, SQP	$191{,}4 \times 8{,}4$	$191{,}4 \times 50{,}4$	$7{,}66 \times 1148{,}6$	5195
Mô hình A, DE (tốt nhất / trung bình)	20 lần chạy, độ lệch chuẩn 3,9 kg			5201 / 5206
Mô hình B, SQP	$245{,}7 \times 17{,}0$	$413{,}9 \times 17{,}2$	$9{,}83 \times 1474{,}0$	6682
Mô hình B, DE (tốt nhất / trung bình)	20 lần chạy, độ lệch chuẩn 8,4 kg			6688 / 6704
Mô hình B, kích thước rời rạc	250×16	420×18	10×1430	6699



Hình 9.2. Hội tụ của 20 lần chạy DE so với nghiệm SQP.

So sánh hai phương pháp rút ra một nhận xét thực hành. Khi các hàm ràng buộc liên tục từng khúc và chỉ không trơn ở một số điểm, SQP với nhiều điểm xuất phát vừa cần ít lượt đánh giá hàm hơn hàng chục lần (31 lần giải cần tổng cộng khoảng 4400 lượt, so với khoảng 250 000 lượt của 20 lần chạy DE; trên máy tính của nhóm tác giả là khoảng 1 giây so với khoảng 1 phút) vừa cho nghiệm tốt hơn DE một chút, vì DE dừng ở lân cận nghiệm sau số thế hệ cho trước. Ưu thế của thuật toán tiến hóa bộc lộ khi biến thiết kế là rời rạc. Trong thực tế, bản thép được chọn theo danh mục chiều dày (8, 10, 12, 14, 16, 18, 20, 22, 25 mm, …) và bề rộng làm tròn tới 10 mm; DE

với biến rời rạc (chỉ số trong danh mục kích thước; 60 cá thể, 800 thế hệ, 10 lần chạy; chiều cao bản bụng cũng làm tròn tới 10 mm) tìm được thiết kế 250×16 / 420×18 / 10×1430 mm nặng 6699 kg, chỉ nặng hơn nghiệm liên tục 0,3%.

9.2.5. Phân tích nghiệm tối ưu

Giá trị của một nghiệm tối ưu nằm ở chỗ nó cho biết điều gì quyết định thiết kế. Tại nghiệm của mô hình B có năm ràng buộc hoạt động: $D/t_w = 150$, $b_{bf}/(2t_{bf}) = 12$, $b_{tf} = D/6$, ứng suất bản cánh dưới ở trạng thái sử dụng SD2 và điều kiện oằn bản bụng khi thi công. Các điều kiện cường độ còn dư nhiều (uốn dư 24%, cắt dư 56%), điều kiện mỏi dư khoảng 7%. Như vậy, ít nhất trong ví dụ này, đối với dầm thép liên hợp nhịp giản đơn thi công không chống đỡ, chính trạng thái sử dụng và giai đoạn thi công chứ không phải trạng thái cường độ mới quyết định kích thước dầm.

Ngược lại, trong mô hình A các ràng buộc hoạt động là $D/t_w \leqslant 150$, hai điều kiện bề rộng tối thiểu $b_{tf} \geqslant D/6$ và $b_{bf} \geqslant D/6$, điều kiện $t_{tf} \geqslant 1{,}1t_w$, ứng suất SD2 và độ võng; nghiệm có bản cánh dưới hẹp và dày (191×50 mm) vì không còn điều kiện thi công khống chế. Khi kiểm tra nghiệm công bố theo mô hình B, điều kiện oằn bản bụng khi thi công bị vượt 71%, bản cánh nén khi thi công vượt 44%, ứng suất SD2 vượt 29% và biên độ ứng suất mỏi vượt 15%. Điều đó không có nghĩa là bài báo sai, mà cho thấy nghiệm tối ưu phụ thuộc trực tiếp vào giả thiết về trình tự thi công và các trạng thái giới hạn được kiểm tra.

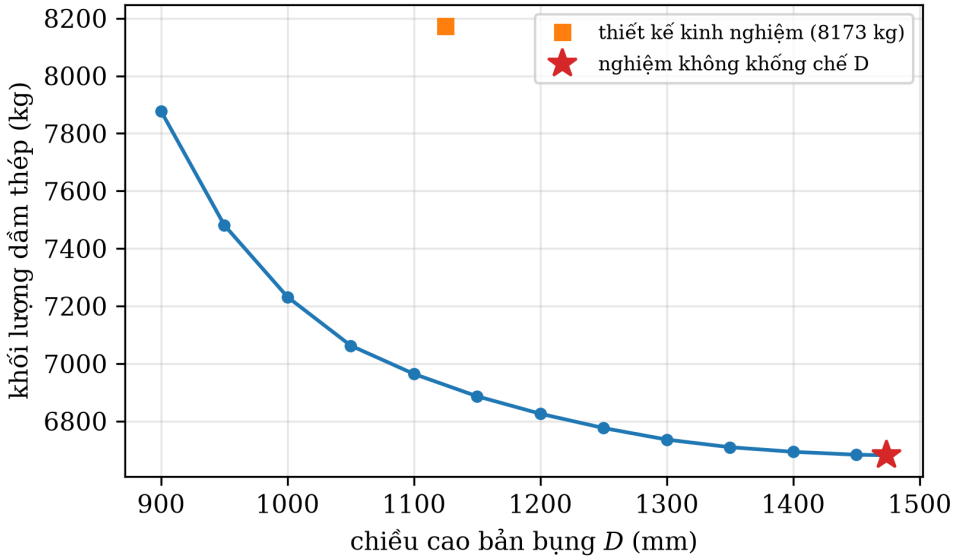
Để kiểm tra nghiệm theo lý thuyết của Chương 3, ta tính nhân tử Lagrange bằng phương pháp bình phương tối thiểu không âm, với gradient tính bằng sai phân hữu hạn, từ điều kiện dừng $\nabla m(\mathbf{x}^*) + \sum_{i \in \mathcal{A}} \lambda_i \nabla g_i(\mathbf{x}^*) = \mathbf{0}$, $\lambda_i \geqslant 0$, trên tập ràng buộc hoạt động $\mathcal{A}$. Chuẩn phần dư của điều kiện dừng (3.8) chỉ khoảng $1{,}3 \times 10^{-7}$ kg/mm, trong khi chuẩn gradient của hàm mục tiêu khoảng 400 kg/mm, tức điều kiện KKT được thỏa mãn với độ chính xác cao. Vì các ràng buộc đã chuẩn hóa, nhân tử λ_i có ý nghĩa trực tiếp: nếu ràng buộc i được thay bằng $g_i \leqslant \varepsilon_i$ thì $\lambda_i = -\partial m^*/\partial \varepsilon_i$, nên nới lỏng ràng buộc thêm 1% ($\varepsilon_i = 0{,}01$) sẽ giảm khối lượng khoảng $0{,}01\lambda_i$ kg (Bảng 9.4). Giải lại bài toán với ràng buộc SD2 được nới 1% cho mức giảm 36,4 kg, khớp với dự báo 36,7 kg. Đây là xấp xỉ bậc nhất, chỉ đúng khi mức nới nhỏ; khi thay đổi lớn, tập ràng buộc hoạt động sẽ thay đổi. Ràng buộc ứng suất SD2 có nhân tử lớn nhất, nên dùng thép cường độ cao hơn cho bản cánh dưới là hướng có lợi nhất; tuy nhiên lợi ích này có giới hạn: điều kiện mỏi (không phụ thuộc F_y) chỉ còn dư khoảng 7% và sẽ sớm trở thành hoạt động, còn tiết diện lai phải xét hệ số $R_h < 1$.

Bảng 9.4. Ràng buộc hoạt động và nhân tử Lagrange (mô hình B)

Ràng buộc hoạt động	λ_i **(kg)**	**Nới lỏng 1% giảm được (kg)**
Ứng suất bản cánh dưới, SD2	3668,6	36,7
Oằn bản bụng khi thi công	998,4	10,0
$D/t_w \leqslant 150$	110,1	1,1
$b_{tf} \geqslant D/6$	46,8	0,5
$b_{bf}/(2t_{bf}) \leqslant 12$	16,3	0,2

Trong thực tế, chiều cao kết cấu cầu thường bị khống chế bởi tĩnh không dưới cầu hoặc cao độ đường đầu cầu. Khi đó câu hỏi thiết kế trở thành: nếu chiều cao bản bụng

bị giới hạn thì khối lượng tăng bao nhiêu? Câu hỏi này được trả lời bằng phương pháp ràng buộc epsilon của Chương 7: cực tiểu khối lượng với ràng buộc bổ sung $D \leqslant D_{\max}$ cho một dãy giá trị $D_{\max}$. Hình 9.3 cho thấy đường đánh đổi (biên Pareto) giữa hai mục tiêu "nhẹ" và "thấp". Khi $D_{\max}$ giảm từ 1474 mm (nghiệm không khống chế) xuống 1200 mm, khối lượng chỉ tăng 2,2%; nhưng khi $D_{\max} = 900$ mm, khối lượng tăng 18%. Ràng buộc độ võng trở thành hoạt động từ $D_{\max} \approx 1000$ mm, và ở 900 mm chiều dày bản cánh dưới đã chạm cận trên 100 mm (bản cánh khoảng 165×100 mm), tức đoạn cuối của đường cong còn bị giới hạn bởi miền biến chứ không chỉ bởi tiêu chuẩn. Những tiết diện ở đầu thấp của đường đánh đổi như vậy khó chế tạo và, với bản thép dày, cường độ chảy thường phải giảm theo chiều dày; đoạn này chỉ nên đọc như xu hướng. Đoạn dốc của đường đánh đổi ($D_{\max}$ dưới khoảng 1100 mm) là vùng mà mỗi milimét chiều cao giảm được phải trả bằng nhiều thép.



Hình 9.3. Đường đánh đổi khối lượng – chiều cao bản bụng (mô hình B), xây dựng bằng phương pháp ràng buộc epsilon.

Khi dùng các nghiệm trên, cần nhớ một số đơn giản hóa của mô hình kiểm toán. Thứ nhất, hoạt tải thi công chưa được kể, trong khi oằn bản bụng khi thi công là ràng buộc hoạt động có nhân tử lớn thứ hai, nên nghiệm của mô hình B thiên về không an toàn đối với ràng buộc này. Thứ hai, cận dưới 5 mm lấy theo bài báo nhỏ hơn chiều dày tối thiểu 8 mm của thép kết cấu theo [4]; nghiệm mô hình A có $t_w = 7{,}66$ mm nên chưa thỏa mãn yêu cầu cấu tạo này (nghiệm mô hình B và nghiệm rời rạc thì thỏa mãn). Thứ ba, công thức sức kháng cắt có trường kéo ở khoang trong chỉ được dùng khi $2Dt_w/(b_{fc}t_{fc} + b_{ft}t_{ft}) \leqslant 2{,}5$; tỷ số này bằng 2,56 ở nghiệm SQP mô hình B và 2,99 ở nghiệm công bố, nên đúng ra phải dùng công thức giảm của tiêu chuẩn (sức kháng cắt danh định nhỏ hơn khoảng 20%), nhưng vì ràng buộc cắt còn dư 56% nên nghiệm không thay đổi. Cuối cùng, bản cánh trên hẹp (khoảng 250 mm) cần được kiểm tra ổn định khi vận chuyển và cẩu lắp; một khuyến nghị thực hành thường dùng là $b_{fc} \geqslant L/85$ theo chiều dài đoạn dầm vận chuyển. Mã nguồn của ví dụ được cung cấp tại Code 9.1 trên GitHub (bản in ở phụ lục).

9.3. Tối ưu bộ giảm chấn khối lượng cho cầu đường sắt cao tốc

9.3.1. Cộng hưởng của cầu khi đoàn tàu chạy qua và yêu cầu của tiêu chuẩn

Một đoàn tàu gồm các toa có chiều dài bằng nhau tạo ra một chuỗi lực trục lặp lại với khoảng cách D. Khi đoàn tàu chạy với vận tốc v, các trục đi qua một điểm trên cầu với tần số v/D, và cầu bị kích thích cộng hưởng khi tần số này (hoặc một bội số của nó) trùng với tần số riêng f_1 của cầu. Vận tốc cộng hưởng là:

$$v_i = \frac{f_1 D}{i}, \qquad i = 1, 2, 3, \ldots \tag{9.3}$$

Với các đoàn tàu cao tốc dài 400 m, hàng chục trục liên tiếp tích lũy năng lượng dao động, nên phản ứng động của cầu, đặc biệt là gia tốc mặt cầu, có thể lớn hơn nhiều so với khi không có cộng hưởng. Gia tốc lớn làm mất ổn định lớp balát và giảm an toàn chạy tàu, vì vậy EN 1990 Phụ lục A2 [16] giới hạn gia tốc mặt cầu ở 3,5 m/s^2 đối với đường có balát.

Để kiểm tra yêu cầu này cho các tuyến liên vận quốc tế, EN 1991-2 [17] quy định mười đoàn tàu chuẩn HSLM-A1 đến A10, khác nhau ở số toa trung gian N, chiều dài toa D (18–27 m), khoảng cách trục của giá chuyển hướng d và lực trục P (Bảng 9.5). Trong mục này D và d là các ký hiệu của EN 1991-2, khác với ký hiệu ở Ví dụ 9.1. Phân tích động phải xét dãy vận tốc từ 40 m/s (144 km/h) đến 1,2 lần vận tốc thiết kế, với tỷ số cản lấy bằng giá trị cận dưới mà tiêu chuẩn cho phép (1% đối với cầu bê tông dự ứng lực nhịp từ 20 m).

Khi một cầu hiện hữu không thỏa mãn giới hạn gia tốc, ví dụ khi nâng vận tốc khai thác, một giải pháp gia cường là gắn thêm thiết bị giảm chấn. TMD là một hệ khối lượng – lò xo – cản được điều chỉnh tần số gần tần số của dạng dao động cần giảm; việc thiết kế TMD cho cầu đường sắt cao tốc đã được nghiên cứu nhiều, ví dụ trong [32, 49], bên cạnh giải pháp giảm chấn nhớt [37].

Bảng 9.5. Các đoàn tàu HSLM-A theo EN 1991-2 [17]

Tàu	N	D (m)	d (m)	P (kN)	**Tàu**	N	D (m)	d (m)	P (kN)
A1	18	18	2,0	170	A6	13	23	2,0	180
A2	17	19	3,5	200	A7	13	24	2,0	190
A3	16	20	2,0	180	A8	12	25	2,5	190
A4	15	21	3,0	190	A9	11	26	2,0	210
A5	14	22	2,0	170	A10	11	27	2,0	210

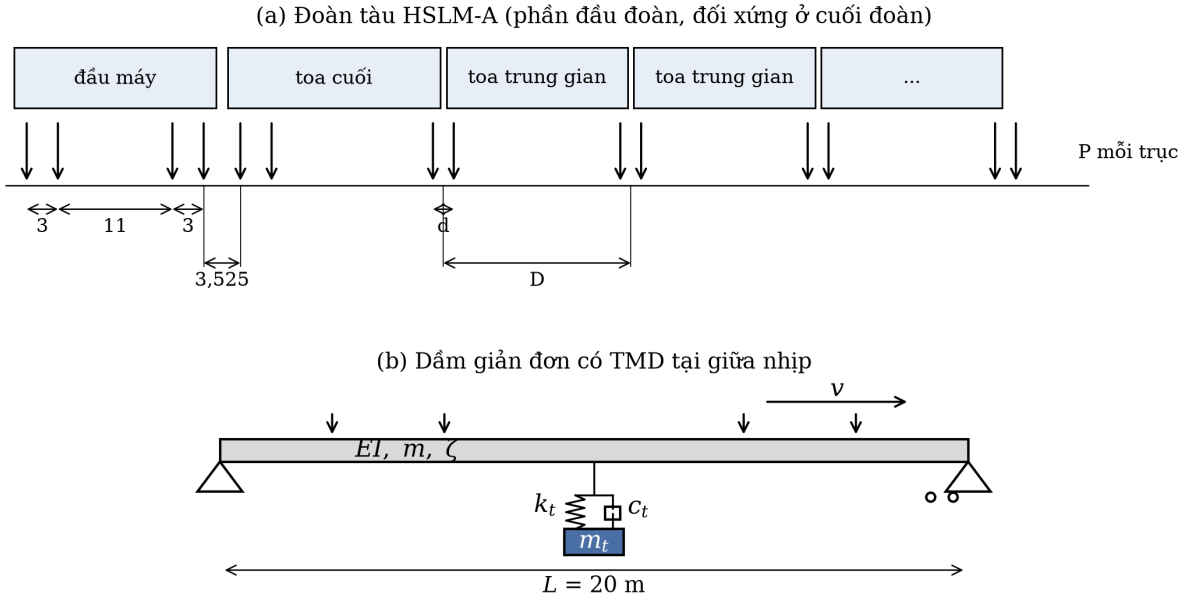
9.3.2. Mô hình hệ cầu – bộ giảm chấn chịu đoàn tàu HSLM-A

Ví dụ 9.2: Xét một cầu đường sắt cao tốc bằng bê tông dự ứng lực, dầm giản đơn nhịp $L = 20$ m, khối lượng phân bố $m = 15$ t/m (kể cả balát, tà vẹt và ray), tần số riêng thứ nhất $f_1 = 6$ Hz và tỷ số cản $\zeta = 1\%$, trên tuyến có vận tốc thiết kế 300 km/h. Đây là một cầu giả định do các tác giả lập với các tham số nằm trong phạm vi thường gặp, không phải một công trình cụ thể. Yêu cầu: thiết kế một TMD đặt tại giữa nhịp để gia tốc mặt cầu không vượt 3,5 m/s^2 với cả mười đoàn tàu HSLM-A trong dải vận tốc 150–360 km/h. Để gọn, cận dưới của dải được lấy là 150 km/h thay cho 144 km/h theo EN 1991-2; phép kiểm tra bổ sung tại 144; 145 và 147,5 km/h (vùng cộng hưởng bậc cao, trong đó có cộng hưởng bậc ba v_3 của đoàn A3) cho gia tốc lớn nhất 2,83 m/s^2 khi chưa có TMD và không quá 1,94 m/s^2 với mọi phương án TMD tối ưu dưới đây, nên đoạn vận tốc này không quyết định.

Cầu được mô hình hóa là dầm Euler–Bernoulli giản đơn, phản ứng được khai triển theo các dạng dao động $\phi_n(x) = \sin(n\pi x/L)$ với khối lượng tổng quát $M = mL/2$ và tần số $\omega_n = n^2\omega_1$. Ba dạng đầu được xét, vì EN 1990 yêu cầu kể các dạng có tần số đến giá trị lớn nhất trong ba giá trị 30 Hz, $1{,}5f_1$ và f_3. Tải trọng là các lực trục di động với vận tốc không đổi (bỏ qua tương tác tàu – cầu), đặt theo hình học đoàn tàu ở Hình 9.4a. Lực tổng quát thứ n là:

$$F_n(t) = \sum_k P_k \, \phi_n(vt - a_k) \, H_k(t) \tag{9.4}$$

trong đó P_k là lực trục thứ k, a_k là khoảng cách từ trục k tới trục đầu và $H_k(t) = 1$ khi trục k ở trên cầu, bằng 0 khi ngược lại.



Hình 9.4. (a) Hình học đoàn tàu HSLM-A theo EN 1991-2 (kích thước tính bằng m); (b) mô hình dầm giản đơn có TMD tại giữa nhịp.

TMD có khối lượng $m_t = \mu M$ (μ là tỷ số khối lượng), tần số $f_t = \gamma f_1$ và tỷ số cản ζ_t, nối với dầm tại giữa nhịp $x_t = L/2$ ($\phi_1 = 1$, $\phi_2 = 0$, $\phi_3 = -1$) (Hình 9.4b). Gọi q_n là tọa độ tổng quát và z là chuyển vị của khối lượng TMD, hệ phương trình chuyển động là:

$$\begin{aligned} M\left(\ddot{q}_n + 2\zeta\omega_n\dot{q}_n + \omega_n^2 q_n\right) + \phi_n(x_t)\, R_t &= F_n(t), \qquad n = 1, 2, 3, \\ m_t \ddot{z} &= R_t, \\ R_t &= k_t\,(w_t - z) + c_t\,(\dot{w}_t - \dot{z}) \end{aligned} \tag{9.5}$$

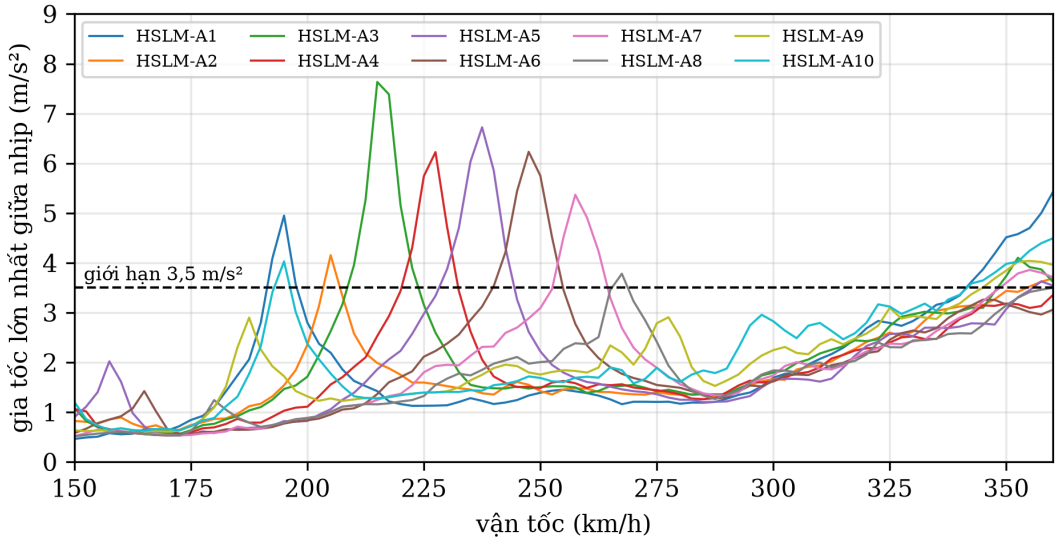
trong đó R_t là lực tương tác giữa dầm và TMD, $w_t = \sum_j \phi_j(x_t) q_j$ là chuyển vị của dầm tại điểm gắn TMD, $k_t = m_t(2\pi f_t)^2$ và $c_t = 2\zeta_t m_t(2\pi f_t)$. Gia tốc giữa nhịp là $a(t) = \sum_n \phi_n(L/2)\ddot{q}_n(t)$.

Bài toán tối ưu cần đánh giá hàm mục tiêu hàng nghìn lần, mỗi lần với mười đoàn tàu và 85 vận tốc (bước 2,5 km/h), nên cách giải phương trình (9.5) quyết định chi phí tính toán của bài toán tối ưu. Vì hệ là tuyến tính, phản ứng được tính trong miền tần số: biến đổi Fourier nhanh (Fast Fourier Transform – FFT) của các lực tổng quát

$F_n(t)$ được tính trước một lần cho tất cả các cặp (đoàn tàu, vận tốc); với mỗi bộ tham số TMD chỉ cần lập và giải hệ phương trình 4×4 (ma trận độ cứng động) tại mỗi tần số rồi biến đổi Fourier ngược. Một lượt đánh giá hàm (850 trường hợp) mất khoảng 0,1–0,3 giây trên máy tính cá nhân. Lời giải miền tần số được kiểm chứng bằng tích phân trực tiếp theo thời gian cho bốn trường hợp, trong đó có một trường hợp có TMD; sai khác lớn nhất là 3,3%.

9.3.3. Phản ứng của cầu khi chưa có bộ giảm chấn

Hình 9.5 biểu diễn gia tốc lớn nhất tại giữa nhịp theo vận tốc khi chưa có TMD. Các đỉnh nhọn lớn nhất tương ứng với cộng hưởng bậc hai $v_2 = f_1 D/2$ (các đỉnh nhỏ hơn ở 155–195 km/h là cộng hưởng bậc ba v_3): với $f_1 = 6$ Hz, đoàn A3 ($D = 20$ m) cộng hưởng ở vận tốc $v_2 = 216$ km/h (đỉnh tính được trên lưới vận tốc nằm tại 215 km/h) và cho gia tốc lớn nhất 7,63 m/s^2, tức hơn hai lần giới hạn. Các đoàn tàu có toa dài hơn cộng hưởng ở vận tốc cao hơn, theo đúng công thức (9.3). Ở cuối dải vận tốc, gia tốc tăng dần vì đoàn A1 tiến gần vận tốc cộng hưởng bậc nhất $f_1 D = 389$ km/h, nằm ngay ngoài dải tính toán. Chi tiết này sẽ đóng vai trò quan trọng khi thiết kế TMD.



Hình 9.5. Gia tốc lớn nhất giữa nhịp theo vận tốc của mười đoàn tàu HSLM-A khi chưa có TMD.

9.3.4. Tối ưu tham số bộ giảm chấn

Với tỷ số khối lượng μ cho trước, bài toán thiết kế TMD là một bài toán min–max:

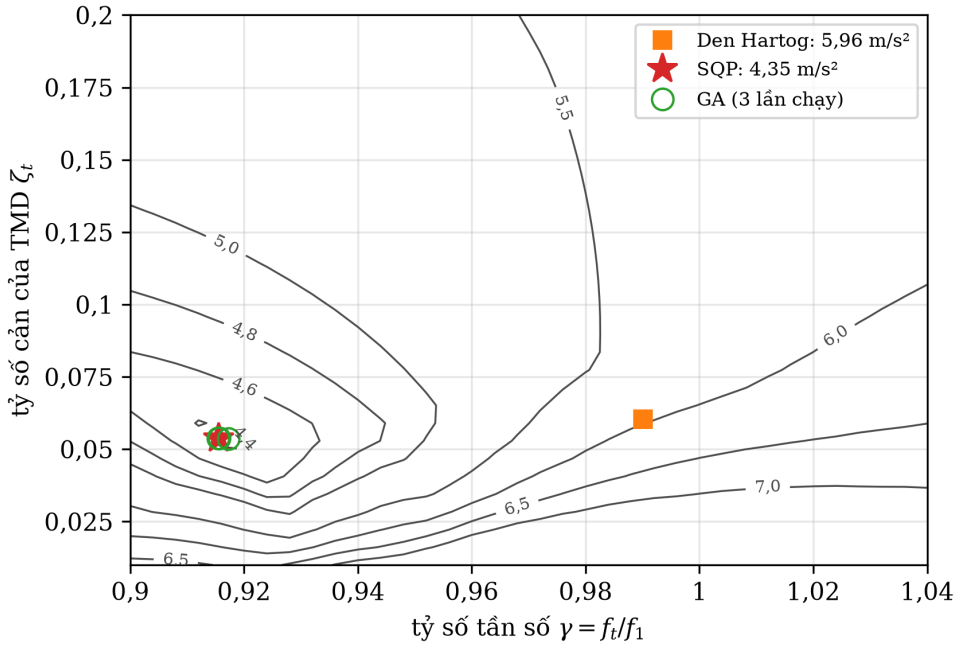
$$\min_{\gamma,\,\zeta_t} \; a_{\max}(\gamma, \zeta_t) = \max_{c \in \mathcal{C}} \; \max_{t} \, |a_c(t; \gamma, \zeta_t)| \tag{9.6}$$

trong đó $\mathcal{C}$ là tập 850 trường hợp (đoàn tàu, vận tốc) và $a_c(t)$ là gia tốc giữa nhịp trong trường hợp c. Điểm xuất phát tự nhiên là công thức kinh điển của Den Hartog [14] cho TMD gắn vào hệ chính một bậc tự do không cản chịu lực điều hòa:

$$\gamma_{\mathrm{DH}} = \frac{1}{1+\mu}, \qquad \zeta_{t,\mathrm{DH}} = \sqrt{\frac{3\mu}{8(1+\mu)^3}} \tag{9.7}$$

Với $\mu = 1\%$ (TMD nặng 1,5 t), công thức cho $\gamma_{\mathrm{DH}} = 0{,}990$, $\zeta_{t,\mathrm{DH}} = 0{,}060$ và gia tốc lớn nhất giảm từ 7,63 xuống 5,96 m/s^2 – vẫn vượt giới hạn.

Vì bài toán (9.6) chỉ có hai biến, ta có thể dùng phương pháp đồ thị của Chương 2: tính $a_{\max}$ trên một lưới 36×32 điểm và vẽ các đường đẳng trị (Hình 9.6). Đồ thị đường đẳng trị cho thấy nghiệm tối ưu nằm xa điểm Den Hartog, tại tỷ số tần số khoảng 0,92, tức TMD được điều chỉnh thấp hơn tần số riêng của cầu khoảng 8%. Đồ thị cũng cho thấy hàm mục tiêu có "thung lũng" hẹp và các đường đẳng trị gãy khúc, đặc trưng của hàm max lấy trên nhiều trường hợp.



Hình 9.6. Đường đẳng trị của gia tốc lớn nhất (m/s^2) theo γ và ζ_t với $\mu = 1\%$, cùng nghiệm Den Hartog, SQP và GA.

Hàm $a_{\max}$ không khả vi tại những điểm mà trường hợp quyết định thay đổi, do đó SQP không thích hợp để áp dụng trực tiếp. Cách xử lý chuẩn là dạng biến phụ (epigraph): đưa thêm biến s và giải bài toán tương đương, trong đó các hàm ràng buộc trơn hơn nhiều vì sự chuyển đổi trường hợp quyết định không còn nằm trong hàm mục tiêu:

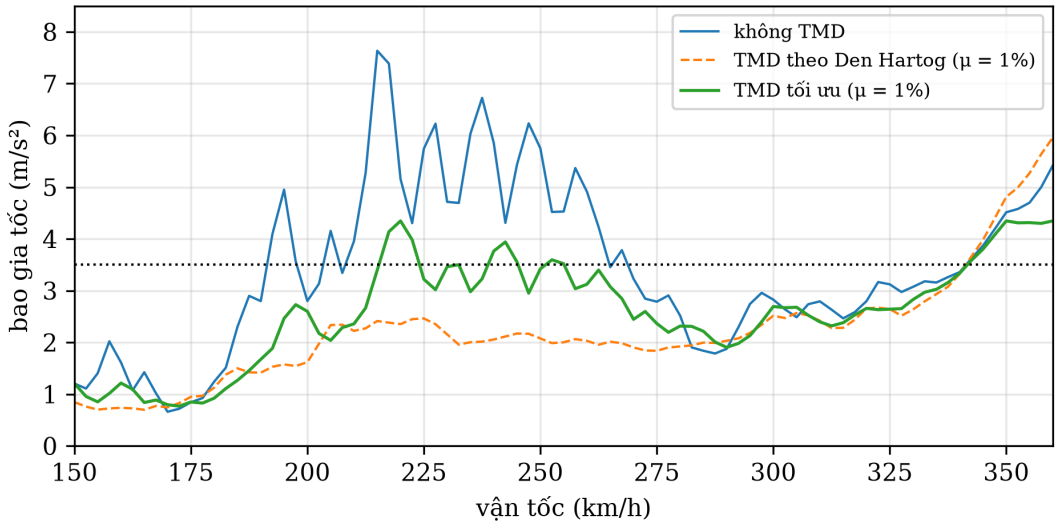
$$\min_{\gamma,\,\zeta_t,\,s} \; s \qquad \text{s.t.} \qquad \max_{t} \left|a_c(t;\gamma,\zeta_t)\right| \leqslant s, \quad c \in \mathcal{C} \tag{9.8}$$

trong đó mỗi trường hợp $c \in \mathcal{C}$ tạo thành một ràng buộc riêng (tổng cộng 850 ràng buộc). Ở ví dụ này một lượt đánh giá hàm là một lần tính phản ứng của cả 850 trường hợp. Xuất phát từ nghiệm Den Hartog, SQP hội tụ sau 8 vòng lặp (45 lượt đánh giá hàm, kể cả các lượt tính đạo hàm sai phân) tới $\gamma = 0{,}9155$, $\zeta_t = 0{,}054$ với $a_{\max} = 4{,}35$ m/s^2. GA mã hóa số thực (cùng ý tưởng với GA ở Chương 8, nhưng dùng chọn lọc đấu cặp, lai ghép số học, đột biến Gauss và giữ cá thể tốt nhất; 24 cá thể, 40 thế hệ) chạy ba lần cho 4,349; 4,349 và 4,364 m/s^2. Kết quả này, cùng với đồ thị đường đẳng trị, củng cố nhận định rằng nghiệm SQP là cực tiểu toàn cục trong miền xét, nhưng mỗi lần chạy GA cần 944 lượt đánh giá hàm (Bảng 9.6).

Bảng 9.6. Tham số TMD và gia tốc lớn nhất với $\mu = 1\%$ ($m_t = 1{,}5$ t)

Phương án	γ	ζ_t	$a_{\max}$ (m/s^2)	**Số lượt đánh giá hàm**
Không TMD	–	–	7,63	1
Công thức Den Hartog	0,990	0,060	5,96	1
Lưới đồ thị 36×32	–	–	4,35	1152
SQP dạng biến phụ	0,9155	0,054	4,35	45
GA (tốt nhất trong 3 lần)	0,9155	0,054	4,35	944 / lần

Tại sao nghiệm tối ưu lệch xa công thức Den Hartog? Hình 9.7 so sánh đường bao gia tốc (lớn nhất trên mười đoàn tàu) theo vận tốc. TMD theo Den Hartog được điều chỉnh gần tần số riêng của cầu (5,94 Hz) nên giảm rất tốt các đỉnh cộng hưởng bậc hai trong khoảng 200–275 km/h. Tuy nhiên, TMD tách đỉnh cộng hưởng của cầu thành hai tần số của hệ ghép, khoảng 5,68 và 6,28 Hz, và tần số dưới nằm sát tần số kích thích $v/D = 100/18 = 5{,}56$ Hz của đoàn A1 ở 360 km/h, nên phản ứng tại đây tăng từ 5,42 lên 5,96 m/s^2 so với khi không có TMD. Nghiệm min–max chấp nhận phản ứng lớn hơn ở giữa dải vận tốc để hạ đỉnh lớn nhất ở cuối dải: TMD được điều chỉnh về 5,49 Hz, gần tần số kích thích bất lợi, và tại nghiệm các đỉnh của đường bao tại 220 km/h (đoàn A3) và tại 350–360 km/h (đoàn A1) có cùng giá trị 4,35 m/s^2 – đặc điểm điển hình của nghiệm bài toán min–max. Điều này cho thấy: công thức tối ưu cho tải điều hòa không còn tối ưu khi tải là một tập hợp nhiều trường hợp, và trường hợp quyết định có thể thay đổi khi thiết kế thay đổi.

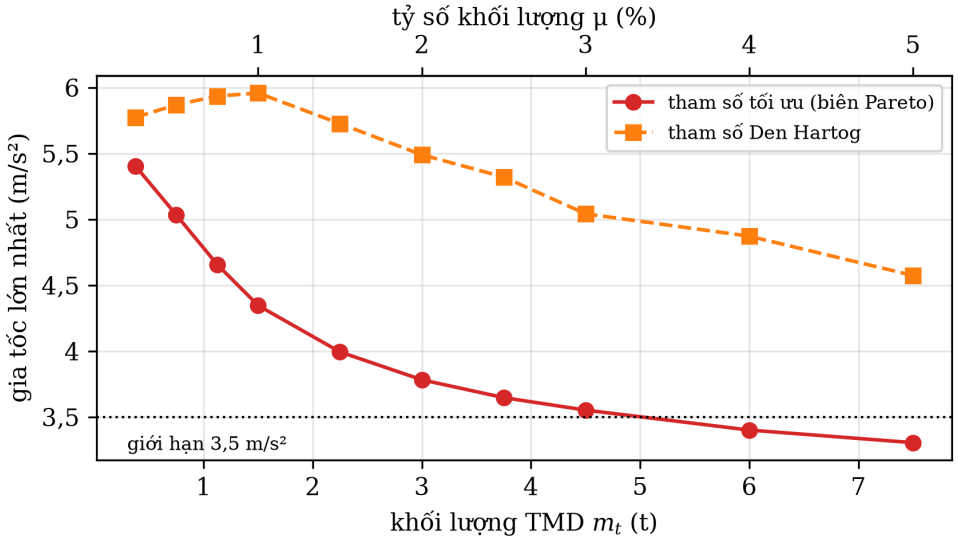


Hình 9.7. Đường bao gia tốc theo vận tốc khi không có TMD, với TMD theo Den Hartog và với TMD tối ưu ($\mu = 1\%$).

9.3.5. Đánh đổi giữa khối lượng bộ giảm chấn và gia tốc mặt cầu

Với $\mu = 1\%$, gia tốc vẫn vượt giới hạn, nên câu hỏi tiếp theo là cần TMD nặng bao nhiêu. Đây là bài toán hai mục tiêu: cực tiểu khối lượng TMD và cực tiểu gia tốc lớn

nhất. Giải bài toán (9.8) cho một dãy giá trị cố định của μ từ 0,25% đến 5%, mỗi lần khởi đầu từ điểm tốt hơn giữa nghiệm của giá trị trước và điểm Den Hartog, ta thu được biên Pareto ở Hình 9.8. Vì $a^*_{\max}$ giảm đơn điệu theo μ, việc cố định μ tương đương ràng buộc $\mu \leqslant \varepsilon$ của phương pháp ràng buộc epsilon (Chương 7), với khối lượng TMD đóng vai trò ràng buộc. Giới hạn 3,5 m/s^2 đạt được khi $\mu \approx 3{,}3$–$3{,}4\%$ (nội suy giữa hai điểm tính $\mu = 3\%$ và 4%), tức TMD nặng khoảng 5 t. Ở mức khối lượng này TMD được điều chỉnh thấp hơn f_1 tới khoảng 15% ($\gamma \approx 0{,}85$, $\zeta_t \approx 0{,}10$ tại $\mu = 3\%$), và trường hợp quyết định vẫn là đoàn A1 ở cận trên của dải vận tốc; vì vậy nghiệm phụ thuộc trực tiếp vào việc chọn vận tốc lớn nhất cần kiểm tra. Nếu dùng tham số Den Hartog, ngay cả TMD nặng 7,5 t ($\mu = 5\%$) vẫn cho gia tốc 4,57 m/s^2.



Hình 9.8. Biên Pareto giữa khối lượng TMD và gia tốc lớn nhất, so với tham số Den Hartog.

Hình dạng biên Pareto cung cấp thông tin thiết kế quan trọng. Lợi ích của mỗi tấn TMD giảm dần: tăng m_t từ 0,4 lên 1,5 t giảm gia tốc 1,06 m/s^2, nhưng tăng từ 4,5 lên 7,5 t chỉ giảm thêm 0,25 m/s^2. Một phần nguyên nhân là gia tốc do dạng thứ ba (54 Hz) gây ra, mà TMD điều chỉnh theo dạng thứ nhất hầu như không tác động. Tại trường hợp quyết định, phần đóng góp của dạng thứ ba ở thời điểm đạt đỉnh dao động từ khoảng 0,2 đến 1,0 m/s^2 tùy giá trị μ (khoảng 0,35 m/s^2 khi trường hợp quyết định là đoàn A1 ở 360 km/h, như với $\mu = 1\%$, 3% và 4%); ở vận tốc cao, khi trường hợp quyết định là đoàn A1 ở 350 km/h ($\mu = 2\%$) hoặc đoàn A3 ở 352,5–357,5 km/h ($\mu = 2{,}5\%$ và 5%), phần này xấp xỉ 0,7–1,0 m/s^2 và giá trị lớn nhất theo thời gian của nó vượt 1 m/s^2. Trường hợp quyết định cũng chuyển qua lại giữa đoàn A1 ở cuối dải vận tốc và đoàn A3 (ở 220 km/h hoặc ở 352,5–357,5 km/h) khi μ thay đổi.

Người thiết kế cũng cần nhớ giới hạn của mô hình. Mô hình lực di động bỏ qua tương tác tàu – cầu và thường cho kết quả thiên về an toàn ở vùng cộng hưởng; EN 1991-2 cho phép kể đến tương tác này bằng tỷ số cản bổ sung, khoảng $\Delta\zeta \approx 0{,}3\%$ với nhịp 20 m, nhưng ví dụ này không dùng hệ số đó. Nghiệm tối ưu, do được điều chỉnh sát một trường hợp bất lợi, có thể nhạy với sai lệch tần số thực của cầu, nên cần kiểm tra độ bền vững của nghiệm khi f_1 thay đổi vài phần trăm (Bài 9.3). Một nguồn lệch tần số quan trọng chính là khối lượng đoàn tàu: với HSLM-A1, khối lượng phân bố của toa khoảng $2P/(gD) \approx 1{,}9$ t/m, tức chừng 13% khối lượng cầu, nên tần số của hệ

cầu – tàu khi đoàn tàu ở trên cầu có thể thấp hơn f_1 tới khoảng 6%, điều mà mô hình lực di động không thể hiện được. Mã nguồn của ví dụ được cung cấp tại Code 9.2 trên GitHub (bản in ở phụ lục).

9.4. Hiệu chỉnh mô hình cầu giàn thép đường sắt Nam Ô từ dữ liệu đo

9.4.1. Cầu Nam Ô và dữ liệu đo dao động

Hiệu chỉnh mô hình là một bước quan trọng trong quan trắc và đánh giá cầu đang khai thác. Mô hình phần tử hữu hạn lập từ hồ sơ thiết kế luôn chứa những đại lượng không chắc chắn, như độ cứng thực của liên kết, khối lượng của các bộ phận không kết cấu hay điều kiện gối; khi đã được hiệu chỉnh cho khớp với dữ liệu đo, mô hình có thể dùng để đánh giá khả năng chịu lực, dự báo phản ứng do tải trọng mới hoặc làm mốc tham chiếu để phát hiện hư hỏng [19]. Về bản chất, đây là một bài toán tối ưu: tìm các tham số mô hình để sai khác giữa đặc trưng dao động tính toán và đo được là nhỏ nhất.

Ví dụ 9.3: Cầu Nam Ô nằm trên tuyến đường sắt Thống Nhất (Hà Nội – Thành phố Hồ Chí Minh) tại thành phố Đà Nẵng, gồm bốn nhịp giàn thép giản đơn dài 75 m, được xây dựng năm 2011 trong dự án nâng cấp an toàn các cầu trên tuyến. Nhịp thứ nhất có hai giàn chủ cách nhau khoảng 5 m (đọc từ hình vẽ của bài báo), biên trên dạng đa giác, cao 13 m ở giữa nhịp, chia thành tám khoang; ray đặt trực tiếp trên dầm dọc. Tran-Ngoc và cộng sự [47] đã đo dao động môi trường của nhịp này bằng mười gia tốc kế qua tám lần đo và nhận dạng được các tần số riêng bằng phương pháp nhận dạng không gian con ngẫu nhiên (Stochastic Subspace Identification – SSI). Năm dạng đầu được dùng để hiệu chỉnh mô hình (Bảng 9.7).

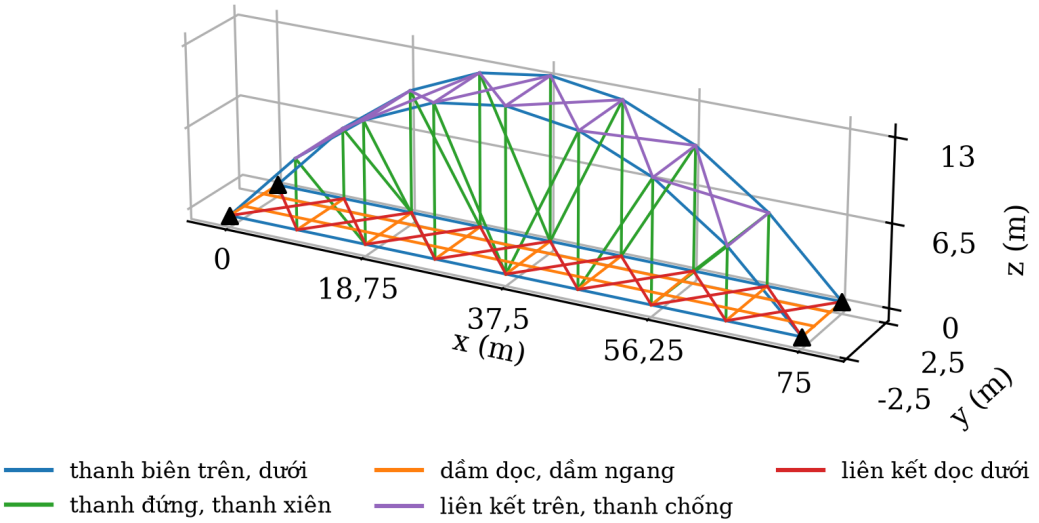
Bảng 9.7. Tần số riêng đo được của nhịp thứ nhất cầu Nam Ô [47] và của mô hình ban đầu

Dạng	Mô tả	Đo (Hz)	Tỷ số cản	Ban đầu (Hz)	Sai số
1	ngang thứ nhất	1,45	0,82%	0,99	−31,9%
2	xoắn thứ nhất	3,11	0,19%	2,16	−30,4%
3	ngang thứ hai	3,28	0,27%	2,85	−13,0%
4	uốn đứng thứ nhất	4,62	2,54%	3,91	−15,3%
5	xoắn thứ hai	6,05	0,28%	3,92	−35,2%

9.4.2. Mô hình phần tử hữu hạn ban đầu

Mô hình phần tử hữu hạn được dựng lại từ số liệu của bài báo dưới dạng khung không gian gồm 136 phần tử dầm Euler–Bernoulli với sáu bậc tự do mỗi nút (Hình 9.9). Hình học lấy theo các hình vẽ của bài báo: tám khoang dài 9,375 m, các nút biên trên nằm trên đường parabol có đỉnh cao 13 m, thanh đứng tại mọi nút và thanh xiên dốc về phía giữa nhịp. Hệ mặt cầu gồm dầm ngang tại mỗi nút biên dưới và hai dầm dọc; hệ liên kết gồm liên kết dọc dưới và liên kết dọc trên dạng chữ X cùng các thanh chống ngang tại nút biên trên. Chín nhóm tiết diện lấy theo bảng đặc trưng hình học của bài báo; thép có $E = 200$ GPa và khối lượng riêng 7850 kg/m^3; khối lượng ray và bộ phận không kết cấu $m_{ns} = 247{,}2$ kg/m được đặt trên mỗi biên dưới như trong bài báo. Cũng như trong bài báo, đầu thanh đứng và thanh xiên nối với nút giàn qua lò

xo xoay k_θ để mô tả liên kết nửa cứng; gối được mô tả bằng lò xo theo phương đứng k_v, gối di động có lò xo dọc k_h. Những đại lượng mà bài báo không cung cấp, như hằng số xoắn của tiết diện và vị trí dầm dọc, được giả định ở mức hợp lý; vì vậy mô hình ở đây là một mô hình độc lập, không trùng với mô hình của các tác giả (mô hình gốc trong [47] gồm 137 nút và 227 phần tử dầm Timoshenko, lập bằng StaBil).



Hình 9.9. Mô hình phần tử hữu hạn không gian của nhịp thứ nhất cầu Nam Ô (tam giác: vị trí gối).

Với các giá trị ban đầu lấy theo bài báo ($k_\theta = 8\times10^5$ N·m/rad, $k_v = 1{,}3\times10^{10}$ N/m, $k_h = 1{,}5\times10^7$ N/m), mô hình cho các tần số thấp hơn giá trị đo từ 13 đến 35% (Bảng 9.7). Cần lưu ý rằng với cùng bộ tham số này, mô hình nửa cứng của Tran-Ngoc và cộng sự cho các tần số 1,47; 3,14; 3,32; 4,80 và 6,96 Hz, tức sai khác dưới 4% với bốn dạng đầu; các tần số của mô hình dựng lại ở đây thậm chí gần với mô hình liên kết khớp của bài báo (1,18; 2,76; 3,11; 3,79; 3,94 Hz) hơn. Sai lệch lớn vì vậy chủ yếu phản ánh những gì chưa biết khi dựng lại mô hình (hằng số xoắn, vị trí dầm dọc, chi tiết liên kết, cách quy đổi k_θ), chứ không chứng tỏ công trình thực cứng hơn hồ sơ thiết kế. Ví dụ này vì vậy nên được đọc như một bài tập về quy trình hiệu chỉnh – chọn tham số theo độ nhạy, đọc các tín hiệu cảnh báo từ cận của tham số – trên một mô hình có sai số mô hình hóa đáng kể. Đây cũng là tình huống thường gặp trong thực tế: hiệu chỉnh sẽ bù cả sai số mô hình hóa, và người kỹ sư phải phân biệt hai nguồn sai số này.

Một khó khăn của mô hình giàn không gian là có nhiều dạng dao động cục bộ của các thanh đứng dài xen giữa các dạng tổng thể. Để ghép đúng dạng tính toán với dạng đo khi không có số liệu dạng dao động, mỗi dạng được phân loại theo động năng của các nút biên: dạng có phần động năng ở các nút biên nhỏ hơn một nửa được coi là dạng cục bộ, các dạng còn lại được xếp thành dạng ngang, uốn đứng hoặc xoắn theo chuyển vị của hai giàn.

9.4.3. Hiệu chỉnh mô hình như một bài toán tối ưu

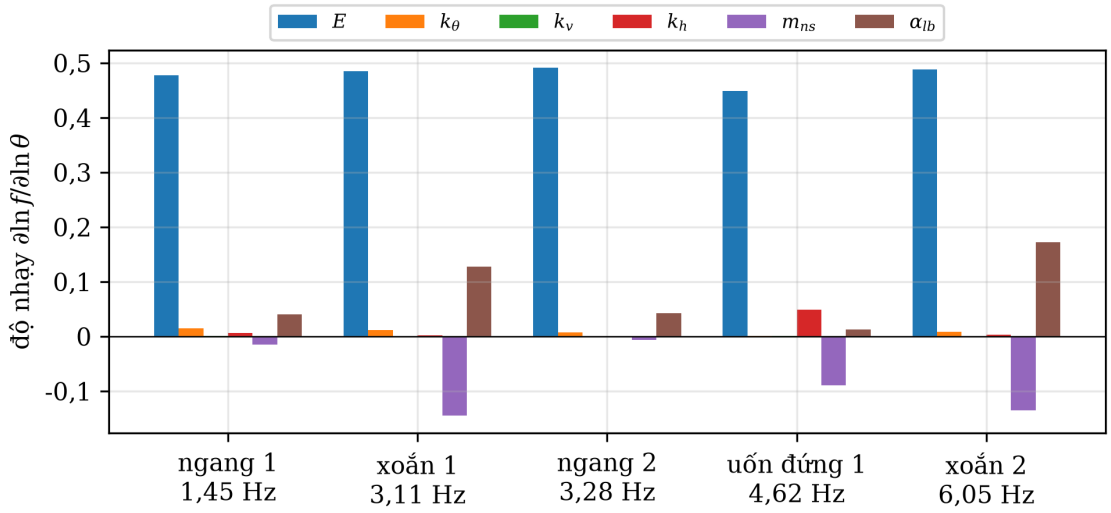
Gọi $\boldsymbol{\theta}$ là vector tham số cần hiệu chỉnh, $f_k(\boldsymbol{\theta})$ là tần số tính toán và $\tilde{f}_k$ là tần số đo của dạng k. Bài toán hiệu chỉnh được phát biểu dưới dạng bình phương tối thiểu có

cận:

$$\min_{\boldsymbol{\theta}} \; J(\boldsymbol{\theta}) = \sum_{k=1}^{5} w_k \left(\frac{f_k(\boldsymbol{\theta}) - \tilde{f}_k}{\tilde{f}_k} \right)^2 \qquad \text{s.t.} \qquad \boldsymbol{\theta}^{\min} \leqslant \boldsymbol{\theta} \leqslant \boldsymbol{\theta}^{\max} \tag{9.9}$$

với các trọng số $w_k = 1$. Bài báo gốc dùng thêm số hạng $1 - \mathrm{MAC}$, với MAC (Modal Assurance Criterion) là chỉ số tương quan dạng dao động; ở đây số liệu dạng dao động đo không được công bố dưới dạng số nên chỉ dùng tần số.

Bước quan trọng nhất của hiệu chỉnh là chọn tham số. Một tham số chỉ nên được hiệu chỉnh nếu nó vừa không chắc chắn về mặt vật lý vừa có ảnh hưởng đáng kể tới các đại lượng đo. Ảnh hưởng đó được lượng hóa bằng độ nhạy tương đối $S_{k,j} = \partial \ln f_k / \partial \ln \theta_j$, tính bằng sai phân hữu hạn. Hình 9.10 cho độ nhạy của năm tần số tại một mô hình tham chiếu có liên kết gần cứng ($k_\theta = 8 \times 10^7$ N·m/rad), khi đó các dạng tổng thể được phân biệt rõ. Mô-đun đàn hồi E ảnh hưởng đều tới mọi dạng (độ nhạy gần 0,5 như lý thuyết $f \propto \sqrt{E}$); khối lượng không kết cấu m_{ns} ảnh hưởng mạnh tới các dạng xoắn và uốn đứng; hệ số nhân α_{lb} cho độ cứng (và diện tích) của các thanh liên kết dọc dưới – đại diện cho đóng góp của hệ mặt cầu vào độ cứng trong mặt phẳng đáy giàn – ảnh hưởng chủ yếu tới các dạng xoắn; trong khi các lò xo gối k_v, k_h gần như không ảnh hưởng vì gối đã rất cứng.



Hình 9.10. Độ nhạy tương đối của năm tần số riêng theo tham số mô hình, tại mô hình tham chiếu $k_\theta = 8 \times 10^7$ N·m/rad (sai phân trung tâm $\pm 5\%$, riêng k_θ là $\pm 10\%$).

Từ phân tích độ nhạy, ta so sánh hai bộ tham số. Bộ 1 gồm các tham số của bài báo: $E \in [190; 220]$ GPa, k_v, k_h (cận như bài báo) và k_θ. Riêng cận của k_θ được mở rộng so với bài báo (vốn chỉ cho $k_\theta \in [7; 9] \times 10^5$ N·m/rad) thành 10^5–10^9 N·m/rad theo thang logarit, vì với mô hình dựng lại, giá trị 8×10^5 N·m/rad gần như tương đương liên kết khớp; đây là lựa chọn của ví dụ này, không phải của bài báo. Bộ 2 chọn theo độ nhạy, thay hai lò xo gối bằng $m_{ns} \in [100; 350]$ kg/m và $\alpha_{lb} \in [0{,}5; 3]$. Bài toán (9.9) được giải bằng hai cách: phương pháp bình phương tối thiểu có cận kiểu Gauss–Newton với vùng tin cậy (thuật toán vùng tin cậy phản xạ) và GA mã hóa số thực như ở Ví dụ 9.2 (20 cá thể, 25 thế hệ). Phương pháp thứ nhất có cùng ý tưởng xấp xỉ bậc hai như SQP ở Chương 4, nhưng ma trận Hessian được xấp xỉ bởi

$\nabla^2 J \approx 2\mathbf{J}_r^T \mathbf{J}_r$ (bỏ qua số hạng chứa đạo hàm bậc hai của phần dư), với $\mathbf{J}_r$ là ma trận Jacobi của vector phần dư. Trong bài báo gốc, các tác giả dùng GA và tối ưu hóa bầy đàn phần tử (PSO) [28].

9.4.4. Kết quả hiệu chỉnh và thảo luận

Bảng 9.8, Bảng 9.9, Hình 9.11 và Hình 9.12 trình bày kết quả. Ở ví dụ này một lượt đánh giá hàm là một lần phân tích dao động riêng. Với bộ tham số 1, hàm mục tiêu giảm từ 0,359 xuống 0,033 nhưng tất cả tham số dừng ở cận trên ($E = 220$ GPa, liên kết gần như cứng, lò xo gối lớn nhất) và sai số dạng ngang thứ nhất, xoắn thứ nhất vẫn còn 10–14%. Khi mọi tham số đều dừng ở cận, đó là dấu hiệu các tham số được chọn không đủ để giải thích sai lệch, tức mô hình có sai số về cấu trúc chứ không chỉ sai số tham số. Với bộ tham số 2, hàm mục tiêu giảm tới 0,0093, bốn trong năm tần số khớp với sai số dưới 5%. Tuy vậy, ngay cả với bộ 2, E và k_θ vẫn dừng ở cận trên; thực chất chỉ m_{ns} và α_{lb} được xác định từ số liệu đo, nên mô hình sau hiệu chỉnh vẫn mang dấu hiệu sai số cấu trúc đã nêu. GA và phương pháp bình phương tối thiểu cho cùng giá trị hàm mục tiêu ($J \approx 0{,}0093$) với các tham số gần nhau (m_{ns} từ 116 đến 118 kg/m, α_{lb} từ 1,64 đến 1,69) – một biểu hiện của "thung lũng" phẳng sẽ bàn ở cuối mục; phương pháp bình phương tối thiểu chỉ cần khoảng 120 lượt đánh giá hàm (kể cả các lần tính ma trận Jacobi bằng sai phân hữu hạn), so với 495 lượt của GA.

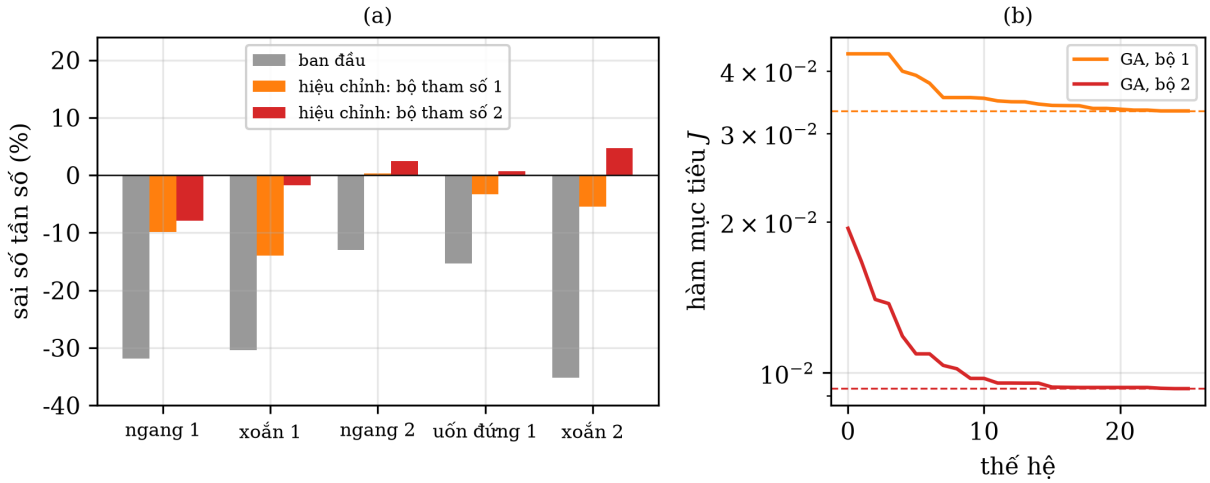
Bảng 9.8. Tần số riêng (Hz) trước và sau hiệu chỉnh mô hình cầu Nam Ô

Phương án	Ngang 1	Xoắn 1	Ngang 2	Uốn đứng 1	Xoắn 2	J
Đo [47]	1,45	3,11	3,28	4,62	6,05	–
Ban đầu	0,99	2,16	2,85	3,91	3,92	0,359
Bộ 1	1,31	2,68	3,29	4,47	5,72	0,033
Bộ 2	1,34	3,06	3,36	4,65	6,33	0,0093
Sai số bộ 2	−7,9%	−1,7%	+2,4%	+0,7%	+4,7%	–

Bảng 9.9. Giá trị tham số trước và sau hiệu chỉnh

Tham số	Ban đầu	Bộ 1	Bộ 2	Bộ 2 (GA)
E (GPa)	200	220	219,9	220,0
k_θ (N·m/rad)	8×10^5	10^9	10^9	10^9
k_v (10^{10} N/m)	1,3	2,0	1,3	1,3
k_h (10^7 N/m)	1,5	2,0	1,5	1,5
m_{ns} (kg/m)	247,2	247,2	117,7	115,6
α_{lb}	1,00	1,00	1,69	1,64
J	0,359	0,033	0,0093	0,0093

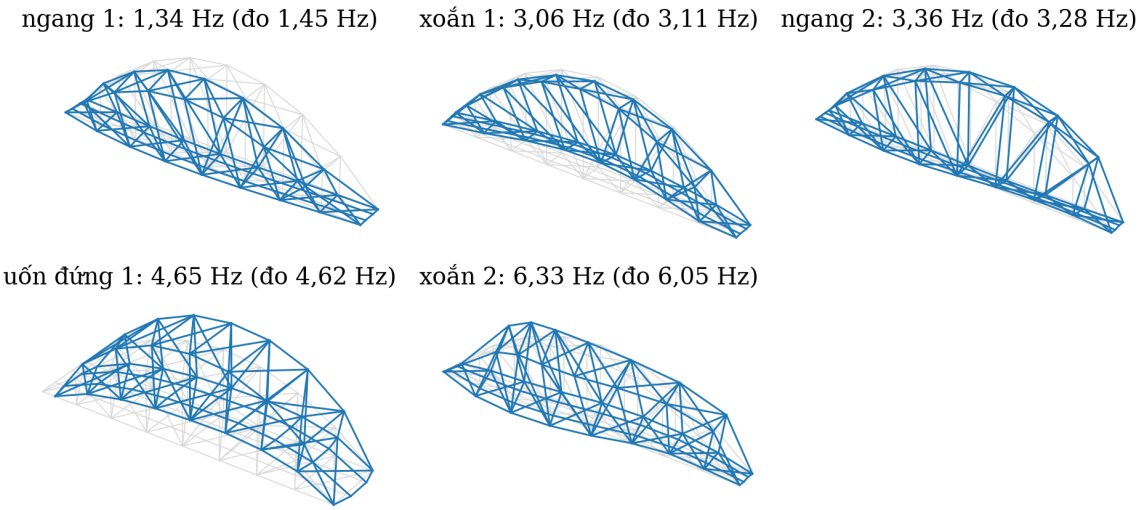
Giá trị các tham số sau hiệu chỉnh (Bảng 9.9) cần được diễn giải từ góc độ kỹ thuật, và ở đây phải đặc biệt thận trọng. Mô-đun đàn hồi đạt cận trên 220 GPa và liên kết



Hình 9.11. (a) Sai số tần số trước và sau hiệu chỉnh; (b) quá trình hội tụ của GA với hai bộ tham số, so với nghiệm bình phương tối thiểu (nét đứt).

thanh bụng trở thành gần như cứng, trong khi trong bài báo gốc k_θ sau hiệu chỉnh vẫn ở mức nửa cứng (7,6–7,8 $\times 10^5$ N·m/rad) và E gần như không đổi. Điều đó cho thấy mô hình dựng lại còn thiếu một phần độ cứng, nên E và k_θ ở đây là các tham số bù sai số mô hình hóa, không phải đặc trưng vật liệu hay liên kết thực của cầu. Khối lượng không kết cấu giảm còn 118 kg/m trên mỗi biên dưới, khoảng một nửa giá trị ban đầu. Kết quả này cũng không nên đọc như một phát hiện về công trình: tổng 236 kg/m là thấp so với khối lượng ray, tà vẹt và ray hộ bánh của một mặt cầu trần; hơn nữa, khối lượng này được đặt trên các biên dưới (cách tim cầu 2,5 m) trong khi ray thực tế nằm trên dầm dọc gần tim cầu, nên mô-men quán tính khối đối với trục cầu bị phóng đại và các dạng xoắn bị hạ thấp – đúng là các dạng nhạy nhất với m_{ns} (Hình 9.10). Việc m_{ns} giảm một nửa vì vậy nhiều khả năng là sự bù cho cách đặt khối lượng và phần độ cứng còn thiếu của mô hình; chỉ hồ sơ công trình mới trả lời được câu hỏi này. Tương tự, hệ số $\alpha_{lb} = 1{,}69$ cho thấy mô hình cần thêm độ cứng trong mặt phẳng liên kết dọc dưới – tức độ cứng xoắn của hệ giàn – mà ở công trình thực có thể đến từ bản mặt cầu, dầm dọc và ray cùng tham gia chịu lực. Dạng ngang thứ nhất vẫn còn sai số 7,9%, nghĩa là nhiều khả năng còn một nguồn độ cứng ngang chưa được mô hình hóa.

Ví dụ này cho thấy ba đặc điểm cần lưu ý của bài toán ngược. Thứ nhất, tính không duy nhất: với năm tần số và bốn tham số, nhiều tổ hợp tham số có thể cho cùng một mức sai khác, nhất là khi các tham số có ảnh hưởng tương tự nhau (cả E và m_{ns} đều thay đổi tỷ số độ cứng trên khối lượng); số liệu dạng dao động (MAC) và các đại lượng đo khác như chuyển vị hay biến dạng giúp giảm tính không duy nhất này. Thứ hai, chọn tham số theo độ nhạy quan trọng hơn chọn thuật toán: cả hai thuật toán đều tìm được cực tiểu, nhưng chỉ bộ tham số phù hợp mới cho mô hình tốt. Thứ ba, tham số dừng ở cận là một tín hiệu cảnh báo về cấu trúc của mô hình, cần được người kỹ sư phân tích thay vì nới rộng cận cho đến khi khớp số liệu; một mô hình khớp số đo chưa chắc có các tham số đúng, vì tham số hiệu chỉnh có thể chỉ đang bù cho sai số mô hình hóa. Mã nguồn của ví dụ được cung cấp tại Code 9.3 trên GitHub (bản in ở phụ lục).



Hình 9.12. Năm dạng dao động của mô hình sau hiệu chỉnh (bộ tham số 2) so với tần số đo.

9.5. Nhận xét chung

Ba ví dụ của chương, tuy khác nhau về loại bài toán, cùng dẫn tới một số bài học chung cho việc áp dụng tối ưu kết cấu trong thực tế. Bài học đầu tiên là phải kiểm chứng mô hình tính trước khi tối ưu: ở Ví dụ 9.1, việc dựng lại các bảng kiểm chứng của bài báo đã giúp phát hiện thông số vút bị thiếu; ở Ví dụ 9.2, lời giải miền tần số được đối chiếu với tích phân trực tiếp. Bài học thứ hai là diễn giải nghiệm thông qua các ràng buộc hoạt động và nhân tử Lagrange (đối với bài toán thiết kế) hoặc qua độ nhạy (đối với bài toán hiệu chỉnh): chúng cho người kỹ sư biết điều gì thực sự quyết định kết quả và nên đầu tư vào đâu nếu muốn cải thiện.

Bài học thứ ba liên quan tới lựa chọn thuật toán. Khi bài toán trơn hoặc có thể đưa về dạng trơn hơn (như dạng biến phụ của bài toán min–max), các phương pháp dựa trên gradient như SQP và bình phương tối thiểu cần ít lượt đánh giá hàm hơn thuật toán tiến hóa từ vài lần đến hàng chục lần (khoảng 90 lần ở Ví dụ 9.1, 20 lần ở Ví dụ 9.2, nhưng chỉ khoảng 4 lần ở Ví dụ 9.3, nơi mỗi ma trận Jacobi sai phân tốn thêm bốn lượt đánh giá) và cho nghiệm chính xác hơn. Thuật toán tiến hóa phát huy ưu thế khi biến rời rạc, khi hàm mục tiêu có nhiều cực trị địa phương, hoặc khi cần kiểm tra độc lập xem nghiệm tìm được có phải là nghiệm toàn cục hay không. Bài học thứ tư là giá trị của biên Pareto: ở cả Ví dụ 9.1 và Ví dụ 9.2, đường đánh đổi cho biết nhiều điều hơn một nghiệm đơn lẻ, đặc biệt là vùng mà lợi ích biên giảm nhanh.

Bài học thứ năm, rút ra từ Ví dụ 9.3, là việc chọn tham số quan trọng không kém việc chọn thuật toán: phân tích độ nhạy giúp loại bỏ các tham số ít ảnh hưởng, còn các tham số dừng ở cận là tín hiệu cho thấy mô hình còn sai số về cấu trúc. Khi mô hình được dựng lại với nhiều giả thiết, các tham số hiệu chỉnh có thể chỉ bù cho sai số mô hình hóa, nên không được đọc chúng như đặc trưng của công trình thực nếu chưa đối chiếu với hồ sơ. Cuối cùng, mọi kết quả tối ưu chỉ đúng trong phạm vi giả thiết của mô hình đã tạo ra nó; trách nhiệm của người kỹ sư là hiểu, nêu rõ và kiểm tra những giả thiết đó.

9.6. Bài tập

Bài 9.1: Áp dụng mô hình B của Ví dụ 9.1 cho ví dụ thứ hai của Hùng và cộng sự [1]: dầm biên của cầu nhịp 50 m (nhịp tính toán 49 m), bản quy đổi $b_e = 2000$ mm, $t_s = 245{,}2$ mm, hệ số phân bố mô-men và lực cắt cùng bằng 0,60, tĩnh tải bản mặt cầu và vút 11,77 N/mm, dầm ngang và sườn tăng cường 0,94 N/mm, neo chịu cắt 0,2 N/mm, lan can 8,4 N/mm, lớp phủ 3,375 N/mm. Các số liệu khác (vật liệu, hoạt tải, giả thiết về sườn tăng cường và giằng khi thi công) lấy như Ví dụ 9.1. So sánh nghiệm với thiết kế kinh nghiệm (bản cánh trên 400×25, bản cánh dưới 500×30, bản bụng 14×2000 mm; 20 802,5 kg) và nghiệm công bố (15 826 kg); chỉ ra các ràng buộc hoạt động.

Bài 9.2: Với mô hình B của Ví dụ 9.1, thay hàm mục tiêu khối lượng bằng chi phí $C = c_s m + c_w L_w$, trong đó L_w là tổng chiều dài đường hàn bụng – cánh và sườn tăng cường, c_s là đơn giá thép (theo kg) và c_w là đơn giá hàn (theo m). Khảo sát sự thay đổi của nghiệm tối ưu khi tỷ số c_w/c_s thay đổi. Liên hệ với nhận xét về giá thành gia công trong [1].

Bài 9.3: Trong Ví dụ 9.2, giả sử tần số riêng thực của cầu sai khác $\pm 5\%$ so với giá trị thiết kế. Giữ nguyên tần số f_t (tính bằng Hz) và tỷ số cản ζ_t của TMD tối ưu ($\mu = 1\%$), đánh giá gia tốc lớn nhất của mặt cầu trong hai trường hợp này. Thiết lập và giải bài toán tối ưu bền vững: cực tiểu giá trị gia tốc lớn nhất lấy trên cả ba giá trị tần số riêng $(0{,}95 f_1;\ f_1;\ 1{,}05 f_1)$, rồi so sánh với nghiệm ban đầu.

Bài 9.4: Mở rộng Ví dụ 9.2 cho hệ hai TMD đặt tại giữa nhịp, có tổng tỷ số khối lượng $\mu = 2\%$ chia đều cho hai TMD; tần số và tỷ số cản của từng TMD là biến thiết kế (bốn biến). Giải bằng SQP dạng biến phụ và bằng GA; so sánh với một TMD có cùng tổng khối lượng và nhận xét về hiệu quả của hệ nhiều TMD [32].

Bài 9.5: Trong Ví dụ 9.3, thêm hệ số nhân α_{ub} cho mô-men quán tính I_z (độ cứng uốn ngang) của các thanh biên trên làm tham số thứ năm. Giải lại bài toán hiệu chỉnh và nhận xét về sai số dạng ngang thứ nhất. Sau đó lấy trọng số w_k tỷ lệ nghịch với tỷ số cản đo được của từng dạng và so sánh kết quả (trong Code 9.3, phần dư được nhân trực tiếp với hệ số `W`, nên cần đặt `W` $= \sqrt{w_k}$). Vì sao không nên để số tham số vượt quá số tần số đo?

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

[1] Trương Việt Hùng, Hà Mạnh Hùng, Đinh Văn Thuật, Hoàng Văn Phúc. Thiết kế tối ưu dầm thép tổ hợp chữ I trong kết cấu cầu liên hợp nhịp đơn giản theo TCVN 11823:2017. *Tạp chí Khoa học Công nghệ Xây dựng (NUCE)*, 15(3V), 55–68, 2021.

[2] Đặng Xuân Phương. *Thiết kế tối ưu trong kỹ thuật và cơ khí với sự hỗ trợ của máy tính*. NXB Khoa học và Kỹ thuật, Hà Nội, 2017.

[3] Lê Hồng Quân. *Các phương pháp tối ưu thiết kế*. NXB Xây dựng, Hà Nội, 2022.

[4] Bộ Khoa học và Công nghệ. *TCVN 11823:2017 Thiết kế cầu đường bộ*. Hà Nội, 2017.

Tài liệu tiếng nước ngoài

[5] American Institute of Steel Construction (AISC). *Manual of Steel Construction – Allowable Stress Design*, 9th ed. AISC, Chicago, 1989.

[6] American Institute of Steel Construction (AISC). *Specification for Structural Steel Buildings (ANSI/AISC 360-16)*. AISC, Chicago, 2016.

[7] Arora J.S. *Introduction to Optimum Design*, 2nd ed. Elsevier Academic Press, San Diego, 2004.

[8] Bendsøe M.P. Optimal shape design as a material distribution problem. *Structural Optimization*, 1(4), 193–202, 1989. DOI: 10.1007/BF01650949

[9] Bendsøe M.P., Sigmund O. *Topology Optimization: Theory, Methods, and Applications*. Springer, Berlin, 2003. DOI: 10.1007/978-3-662-05086-6

[10] Christensen P.W., Klarbring A. *An Introduction to Structural Optimization*. Solid Mechanics and Its Applications, vol. 153. Springer, Dordrecht, 2008. DOI: 10.1007/978-1-4020-8666-3

[11] Computers and Structures, Inc. (CSI). *SAP2000 v24 – CSI Open Application Programming Interface (OAPI) Documentation*. CSI, Walnut Creek, California.

[12] Das I., Dennis J.E. A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems. *Structural Optimization*, 14(1), 63–69, 1997. DOI: 10.1007/BF01197559

[13] Deb K., Pratap A., Agarwal S., Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197, 2002. DOI: 10.1109/4235.996017

[14] Den Hartog J.P. *Mechanical Vibrations*, 4th ed. McGraw-Hill, New York, 1956.

[15] Dorigo M., Maniezzo V., Colorni A. Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1), 29–41, 1996. DOI: 10.1109/3477.484436

[16] European Committee for Standardization (CEN). *EN 1990:2002/A1:2005 Eurocode – Basis of structural design, Annex A2: Application for bridges*. CEN, Brussels, 2005.

[17] European Committee for Standardization (CEN). *EN 1991-2:2003 Eurocode 1: Actions on structures – Part 2: Traffic loads on bridges*. CEN, Brussels, 2003.

[18] Fleury C., Braibant V. Structural optimization: a new dual method using mixed variables. *International Journal for Numerical Methods in Engineering*, 23(3), 409–428, 1986. DOI: 10.1002/nme.1620230307

[19] Friswell M.I., Mottershead J.E. *Finite Element Model Updating in Structural Dynamics*. Kluwer Academic Publishers, Dordrecht, 1995.

[20] Geem Z.W., Kim J.H., Loganathan G.V. A new heuristic optimization algorithm: harmony search. *Simulation*, 76(2), 60–68, 2001. DOI: 10.1177/003754970107600201

[21] Ghasemi M.R., Hinton E., Wood R.D. Optimization of trusses using genetic algorithms for discrete and continuous variables. *Engineering Computations*, 16(3), 272–303, 1999. DOI: 10.1108/02644409910266403

[22] Goldberg D.E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, Massachusetts, 1989.

[23] Grandhi R.V., Venkayya V.B. Structural optimization with frequency constraints. *AIAA Journal*, 26(7), 858–866, 1988. DOI: 10.2514/3.9979

[24] Haftka R.T., Gürdal Z. *Elements of Structural Optimization*, 3rd ed. Solid Mechanics and Its Applications, vol. 11. Kluwer Academic Publishers, Dordrecht, 1992. DOI: 10.1007/978-94-011-2550-5

[25] Holland J.H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, 1975.

[26] Imai K., Schmit L.A. Configuration optimization of trusses. *Journal of the Structural Division (ASCE)*, 107(5), 745–756, 1981. DOI: 10.1061/JSDEAG.0005702

[27] Kaveh A., Mahdavi V.R. Colliding bodies optimization: a novel meta-heuristic method. *Computers & Structures*, 139, 18–27, 2014. DOI: 10.1016/j.compstruc.2014.04.005

[28] Kennedy J., Eberhart R. Particle swarm optimization. *Proceedings of ICNN'95 – International Conference on Neural Networks*, vol. 4, 1942–1948. IEEE, Perth, 1995. DOI: 10.1109/ICNN.1995.488968

[29] Khot N.S., Berke L. Structural optimization using optimality criteria methods. Trong: Atrek E., Gallagher R.H., Ragsdell K.M., Zienkiewicz O.C. (chủ biên), *New Directions in Optimum Structural Design*. John Wiley & Sons, New York, 1984.

[30] Kirkpatrick S., Gelatt C.D., Vecchi M.P. Optimization by simulated annealing. *Science*, 220(4598), 671–680, 1983. DOI: 10.1126/science.220.4598.671

[31] Kirsch U. *Structural Optimization: Fundamentals and Applications*. Springer, Berlin, 1993. DOI: 10.1007/978-3-642-84845-2

[32] Lin C.C., Wang J.F., Chen B.L. Train-induced vibration control of high-speed railway bridges equipped with multiple tuned mass dampers. *Journal of Bridge Engineering*, 10(4), 398–414, 2005. DOI: 10.1061/(ASCE)1084-0702(2005)10:4(398)

[33] Lin J.H., Che W.Y., Yu Y.S. Structural optimization on geometrical configuration and element sizing with statical and dynamical constraints. *Computers & Structures*, 15(5), 507–515, 1982. DOI: 10.1016/0045-7949(82)90002-5

[34] Marler R.T., Arora J.S. Survey of multi-objective optimization methods for engineering. *Structural and Multidisciplinary Optimization*, 26(6), 369–395, 2004. DOI: 10.1007/s00158-003-0368-6

[35] Martins J.R.R.A., Ning A. *Engineering Design Optimization*. Cambridge University Press, Cambridge, 2021. DOI: 10.1017/9781108980647

[36] Metropolis N., Rosenbluth A.W., Rosenbluth M.N., Teller A.H., Teller E. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6), 1087–1092, 1953. DOI: 10.1063/1.1699114

[37] Museros P., Martínez-Rodrigo M.D. Vibration control of simply supported beams under moving loads using fluid viscous dampers. *Journal of Sound and Vibration*, 300(1–2), 292–315, 2007. DOI: 10.1016/j.jsv.2006.08.007

[38] Öztürk H.T., Kahraman H.T. Meta-heuristic search algorithms in truss optimization: research on stability and complexity analyses. *Applied Soft Computing*, 145, 110573, 2023. DOI: 10.1016/j.asoc.2023.110573

[39] Rao R.V., Savsani V.J., Vakharia D.P. Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems. *Computer-Aided Design*, 43(3), 303–315, 2011. DOI: 10.1016/j.cad.2010.12.015

[40] Rao S.S. *Engineering Optimization: Theory and Practice*, 5th ed. John Wiley & Sons, Hoboken, 2019. DOI: 10.1002/9781119454816

[41] Schmit L.A., Farshi B. Some approximation concepts for structural synthesis. *AIAA Journal*, 12(5), 692–699, 1974. DOI: 10.2514/3.49321

[42] Sigmund O. A 99 line topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization*, 21(2), 120–127, 2001. DOI: 10.1007/s001580050176

[43] Sigmund O., Petersson J. Numerical instabilities in topology optimization: a survey on procedures dealing with checkerboards, mesh-dependencies and local minima. *Structural Optimization*, 16(1), 68–75, 1998. DOI: 10.1007/BF01214002

[44] Storn R., Price K. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359, 1997. DOI: 10.1023/A:1008202821328

[45] Svanberg K. The method of moving asymptotes – a new method for structural optimization. *International Journal for Numerical Methods in Engineering*, 24(2), 359–373, 1987. DOI: 10.1002/nme.1620240207

[46] Svanberg K. *MMA and GCMMA – two methods for nonlinear optimization*. Ghi chú kỹ thuật, Optimization and Systems Theory, KTH, Stockholm, 2007.

[47] Tran-Ngoc H., Khatir S., De Roeck G., Bui-Tien T., Nguyen-Ngoc L., Abdel Wahab M. Model updating for Nam O bridge using particle swarm optimization algorithm and genetic algorithm. *Sensors*, 18(12), 4131, 2018. DOI: 10.3390/s18124131

[48] Wang D., Zhang W.H., Jiang J.S. Truss optimization on shape and sizing with frequency constraints. *AIAA Journal*, 42(3), 622–630, 2004. DOI: 10.2514/1.1711

[49] Wang J.F., Lin C.C., Chen B.L. Vibration suppression for high-speed railway bridges using tuned mass dampers. *International Journal of Solids and Structures*, 40(2), 465–491, 2003. DOI: 10.1016/S0020-7683(02)00589-9

PHỤ LỤC CODE

Phụ lục này in lại mã nguồn MATLAB và Python được nhắc đến trong các chương, sắp xếp theo thứ tự chương. Số hiệu mỗi mục (Code N.k) trùng với tên thư mục `examples/Code N.k` trong kho mã https://github.com/chungntu/structural-optimization-book-code; tại đó có đầy đủ các tệp dữ liệu, tệp mô hình và hình kết quả. Các tệp dữ liệu giàn (Code 5.6–5.9, 8.3–8.5) chỉ được in phần đầu và phần cuối, có giữ nguyên số dòng gốc; phần tọa độ nút và liên kết thanh được lược bỏ. Các tệp dữ liệu này được chuyển thể từ bộ bài toán giàn chuẩn của Öztürk và Kahraman [38] và được phân phối theo giấy phép BSD của bộ dữ liệu gốc (Copyright (c) 2024, Hamdi Tolga Kahraman); toàn văn giấy phép có trong tệp `LICENSE_Kahraman_BSD.txt` ở các thư mục tương ứng. Mã được in nguyên trạng nên số thập phân trong mã và trong kết quả in ra màn hình dùng dấu chấm theo cú pháp của ngôn ngữ lập trình. Các chương trình Python dùng các thư viện `numpy`, `scipy` và `matplotlib`; Code 4.1 và Code 4.2 cần Optimization Toolbox của MATLAB; Code 8.2 cần SAP2000 và thư viện `pywin32`.

Chương 1. Nhập môn tối ưu kết cấu

Code 1.1: Vẽ Hình 1.1 (Python)

```python
import os
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np

def draw_beam_diagram():
    fig, ax = plt.subplots(figsize=(12, 4))

    # 1. Vẽ thân dầm (hình chữ nhật nét đứt, bo góc)
    # pad=0.1 mở rộng viền ra ngoài -> đáy thực tế ở y = 1.1 - 0.1 = 1.0
    beam = patches.FancyBboxPatch((1, 1.1), 10, 1.8,
                                  boxstyle="round,pad=0.1",
                                  linestyle='--',
                                  edgecolor='black',
                                  facecolor='none',
                                  linewidth=1.5)
    ax.add_patch(beam)

    # 2. Vẽ gối đỡ (hình tam giác và gạch chéo)
    # Đỉnh tam giác ở y=1.0 -> vừa chạm đáy dầm (visual bottom = 1.0)
    # Gối bên trái
    left_support = plt.Polygon([[1.0, 0.5], [1.4, 1.0], [1.8, 0.5]],
                               closed=True, fill=False, lw=1.5)
    ax.add_patch(left_support)
    ax.plot([0.8, 2.0], [0.5, 0.5], color='black', lw=1.5)
    for i in range(5):
        ax.plot([0.85 + i*0.25, 1.05 + i*0.25], [0.35, 0.5], color='black', lw=1)

    # Gối bên phải
    right_support = plt.Polygon([[10.2, 0.5], [10.6, 1.0], [11.0, 0.5]],
                                closed=True, fill=False, lw=1.5)
    ax.add_patch(right_support)
    ax.plot([10.0, 11.2], [0.5, 0.5], color='black', lw=1.5)
    for i in range(5):
        ax.plot([10.05 + i*0.25, 10.25 + i*0.25], [0.35, 0.5], color='black', lw=1)
```

```

    # 3. Vẽ các mũi tên lực (F và các mũi tên phụ)
    ax.annotate('', xy=(6, 3.1), xytext=(6, 4.5),
                arrowprops=dict(facecolor='black', arrowstyle='->', lw=2))
    ax.text(6, 4.6, '$F$', fontsize=30, ha='center', fontweight='bold', fontfamily='Times New Roman')

    ax.annotate('', xy=(5, 3.1), xytext=(5, 3.6),
                arrowprops=dict(facecolor='black', arrowstyle='->', lw=1.5))
    ax.annotate('', xy=(7, 3.1), xytext=(7, 3.6),
                arrowprops=dict(facecolor='black', arrowstyle='->', lw=1.5))

    # 4. Vẽ 2 lỗ rỗng dạng đường cong mềm phức tạp (không phải ellipse)
    theta = np.linspace(0, 2*np.pi, 200)

    # Lỗ bên trái - blob bất đối xứng
    r_left = 0.55 + 0.12*np.sin(3*theta) + 0.08*np.cos(2*theta) + 0.06*np.sin(4*theta - 1)
    cx_l, cy_l = 3.5, 2.05
    ax.plot(cx_l + r_left*1.2*np.cos(theta),
            cy_l + r_left*0.7*np.sin(theta),
            linestyle='--', color='black', lw=1)

    # Lỗ bên phải - blob khác
    r_right = 0.55 + 0.10*np.cos(3*theta + 0.5) + 0.09*np.sin(2*theta) + 0.05*np.cos(5*theta)
    cx_r, cy_r = 8.5, 2.05
    ax.plot(cx_r + r_right*1.2*np.cos(theta),
            cy_r + r_right*0.7*np.sin(theta),
            linestyle='--', color='black', lw=1)

    # 5. Dấu hỏi chấm ở giữa
    ax.text(6, 1.9, '?', fontsize=60, ha='center', va='center', fontfamily='Times New Roman')

    # Thiết lập khung nhìn
    ax.set_xlim(0, 12)
    ax.set_ylim(0, 5)
    ax.set_aspect('equal')
    ax.axis('off')

    plt.tight_layout()
    script_dir = os.path.dirname(os.path.abspath(__file__))  # lưu hình cạnh file mã
    plt.savefig(os.path.join(script_dir, 'figure_1_1.png'), dpi=300,
                bbox_inches='tight', facecolor='white')
    plt.show()

if __name__ == "__main__":
    draw_beam_diagram()
```

Chương 2. Tối ưu hóa kết cấu bằng phương pháp đồ thị

Code 2.1: Vẽ Hình 2.2

```
clc; clear; close all;

% --- 1. Thông số ban đầu ---
A1_opt = 0.25; % Tọa độ điểm tối ưu
A2_opt = 0.25;
L_axis = 1.2;  % Giới hạn trục

% --- 2. Khởi tạo đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 650, 550]);
hold on; axis off; axis equal;
axis([-0.1 L_axis+0.1 -0.15 L_axis+0.1]);
set(gca, 'FontSize', 12);

% --- 3. Vẽ vùng khả thi (Màu xám) ---
% Tạo biên cong và đa giác vùng khả thi
t = linspace(0, pi/2, 50);
r_curve = 0.95;
x_curve = A1_opt + r_curve * cos(t);
y_curve = (A2_opt + r_curve * sin(t)) .* (1 - 0.15*t);
fill([A1_opt, 1.2, x_curve, A1_opt, A1_opt], ...
     [A2_opt, A2_opt, y_curve, 1.2, A2_opt], ...
     [0.85 0.85 0.85], 'EdgeColor', 'none');

% --- 4. Vẽ biên ràng buộc ---
```

```
plot([A1_opt, A1_opt], [A2_opt, 1.2], 'k-', 'LineWidth', 2); % Biên dọc
plot([A1_opt, 1.2], [A2_opt, A2_opt], 'k-', 'LineWidth', 2); % Biên ngang

% --- 5. Vẽ hàm mục tiêu & Gradient ---
delta = 0.25; len_line = 0.35;
cx = A1_opt + delta; cy = A2_opt + delta;
% Đường đồng mức (dốc -1)
plot([cx - len_line, cx + len_line], [cy + len_line, cy - len_line], 'k-', 'LineWidth', 1.5);
% Vector gradient (vuông góc)
quiver(cx, cy, -0.15, -0.15, 'k', 'LineWidth', 1.2, 'MaxHeadSize', 0.8, 'AutoScale', 'off');
text(cx + 0.15, cy + 0.05, '$A_1 + A_2 = \mathrm{const.}$', 'Interpreter', 'latex', 'FontSize', 14);

% --- 6. Vẽ trục tọa độ ---
plot([0 L_axis], [0 0], 'k-', 'LineWidth', 1); % Trục hoành
text(L_axis, -0.05, '$A_1$', 'Interpreter', 'latex', 'FontSize', 14);
plot([0 0], [0 L_axis], 'k-', 'LineWidth', 1); % Trục tung
text(-0.08, L_axis, '$A_2$', 'Interpreter', 'latex', 'FontSize', 14);
% Mũi tên trục
plot(L_axis, 0, 'k>', 'MarkerFaceColor', 'k', 'MarkerSize', 5);
plot(0, L_axis, 'k^', 'MarkerFaceColor', 'k', 'MarkerSize', 5);

% --- 7. Điểm tối ưu ---
plot(A1_opt, A2_opt, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 5);
text(A1_opt + 0.02, A2_opt + 0.04, 'tối ưu', 'FontSize', 12);

% --- 8. Chú thích trục & Công thức ---
% Giống và ghi chú A1*
plot([A1_opt A1_opt], [0 -0.03], 'k-');
plot([A1_opt A1_opt], [0 A2_opt], 'k:', 'LineWidth', 0.5);
text(A1_opt - 0.1, -0.15, '$A_1^* = \frac{F \cos \alpha}{\sigma_0}$', ...
     'Interpreter', 'latex', 'FontSize', 14);

% Giống và ghi chú A2*
plot([0 -0.03], [A2_opt A2_opt], 'k-');
plot([0 A1_opt], [A2_opt A2_opt], 'k:', 'LineWidth', 0.5);
text(-0.45, A2_opt, '$A_2^* = \frac{F \sin \alpha}{\sigma_0}$', 'Interpreter', 'latex', 'FontSize', 14);

hold off;saveas(gcf, 'figure_2_2.png');
```

Code 2.2: Vẽ Hình 2.5

```
rt3 = sqrt(3);
% --- 1. Thiết lập thông số & Lưới ---
x1_int = rt3/4; x2_int = 1/rt3;          % Giao điểm trục
x1_opt = rt3/7; x2_opt = rt3/7;          % Điểm tối ưu
[X1, X2] = meshgrid(linspace(1e-3, 0.9, 800), linspace(1e-3, 0.9, 800));
F = 4./(3*X1) + 1./X2;                   % Hàm mục tiêu

% --- 2. Vẽ đồ thị ---
figure('Color', 'w'); hold on; box off; pbaspect([1 1 1]);

% Miền chấp nhận (Tam giác xám)
fill([0, 0, x1_int], [0, x2_int, 0], [0.85 0.85 0.85], 'EdgeColor', 'none');

% Đường ràng buộc
x_line = linspace(0, x1_int, 200);
plot(x_line, x2_int - (4/3)*x_line, 'k', 'LineWidth', 2);

% Đường đồng mức & Nhãn
[C, h] = contour(X1, X2, F, [7, 9.43, 14], 'k', 'LineWidth', 1);
clabel(C, h, 'FontSize', 10, 'LabelSpacing', 300);

% Đánh dấu điểm tối ưu
plot(x1_opt, x2_opt, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x1_opt + 0.02, x2_opt + 0.02, 'opt.', 'FontSize', 10);

% --- 3. Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 10);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 10);
axis([0 0.9 0 0.9]);

% Cài đặt Ticks & Font LaTeX
set(gca, 'XTick', x1_int, 'XTickLabel', {'$\frac{\sqrt{3}}{4}$'}, ...
         'YTick', x2_int, 'YTickLabel', {'$\frac{1}{\sqrt{3}}$'}, ...
         'TickLabelInterpreter', 'latex', 'FontSize', 10);
```

```
hold off;
```

Code 2.3: Vẽ Hình 2.8 (trường hợp a)

```
% --- 1. Thiet lap du lieu ---
x1_left = linspace(0.001, 0.499, 200);
x1_right = linspace(0.501, 1.0, 200);
x1_all = linspace(0, 1, 400);

% Cac ham rang buoc & Tinh toan
sigma1 = @(x) (2.*x.*(x-1))./(1-2.*x); y_sigma1 = sigma1(x1_right);
sigma2 = @(x) (1/sqrt(2))-x;           y_sigma2 = sigma2(x1_all);
sigma3 = @(x) (2.*x.^2)./(1-2.*x);     y_sigma3 = sigma3(x1_left);

% --- 2. Ve do thi ---
figure; set(gcf, 'Position', [100, 100, 600, 525]); hold on; axis([0 1 0 1]); box on;

% Ve cac duong rang buoc (Net dam + Net dut + Text)
plot(x1_right, y_sigma1, 'k-', 'LineWidth', 2); plot(x1_right-0.01, y_sigma1, 'k:', 'LineWidth', 1);
text(0.75, 0.95, '$\sigma_1=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 12);

plot(x1_all, y_sigma2, 'k-', 'LineWidth', 2); plot(x1_all-0.01, y_sigma2-0.01, 'k:', 'LineWidth', 1);
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 12);

plot(x1_left, y_sigma3, 'k-', 'LineWidth', 2); plot(x1_left-0.005, y_sigma3+0.01, 'k:', 'LineWidth', 1);
text(0.4, 0.95, '$\sigma_3=-\sigma_0$', 'Interpreter', 'latex', 'FontSize', 12);

% Ve Diem toi uu A
x_A = (2 + sqrt(2))/4; y_A = sqrt(2)/4;
plot(x_A, y_A, 'k+', 'MarkerSize', 10, 'LineWidth', 2);
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 14);

% Ve cac duong dong muc
C_opt = 3*x_A + y_A;
plot(x1_all, C_opt - 3*x1_all, 'k-', 'LineWidth', 0.8);
for os = [-1.5, -0.8, 0.8, 1.5], plot(x1_all, (C_opt+os) - 3*x1_all, 'k-', 'LineWidth', 0.5); end
text(0.5, 0.5, '$3x_1 + x_2 = \mathrm{const}$', 'Interpreter', 'latex', ...
     'FontSize', 12, 'BackgroundColor', 'w');

% Nhan truc & Thiet lap cuoi cung
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
set(gca, 'TickDir', 'in', 'Layer', 'top'); hold off;
```

Code 2.4: Vẽ Hình 2.9 (trường hợp b)

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính các điểm đặc biệt ---
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
y_A = sigma2_func(x_A);
x_B = 0.5;
y_B = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

% Độ lệch nét đứt (tạo bóng)
off = 0.015;

```

```
% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
% 1. Đường bóng
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2);
text(0.09, 0.92, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
% 1. Đường bóng
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2);
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
% 1. Đường bóng
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2);
text(0.37, 0.55, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Chi tiết khác ---
% Điểm A
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

% Điểm B
plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.01, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

% Đường đồng mức
contours = [1.1, 1.6, 2.1, 2.6];
for i = 1:length(contours)
    C = contours(i);
    plot(x1_all, C - 2*x1_all, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn phương trình
text(0.69, 0.75, '$2x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% Trục và Định dạng
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in');
set(gca, 'XTick', 0:0.1:1.1);
set(gca, 'YTick', 0:0.1:1.1);
set(gca, 'LineWidth', 1);
hold off;
```

Code 2.5: Vẽ Hình 2.10 (trường hợp c)

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính các điểm đặc biệt ---
```

```
% Điểm A: Giao của Sigma 1 & 2
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
y_A = sigma2_func(x_A);

% Điểm B: Giao Sigma 1 với trục hoành
x_B = 0.5;
y_B = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

% Độ lệch nét đứt (tạo bóng)
off = 0.015;

% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
% 1. Đường bóng
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2);
text(0.08, 0.92, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
% 1. Đường bóng
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2);
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
% 1. Đường bóng
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2);
text(0.33, 0.95, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Các chi tiết khác ---
% Điểm A
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

% Điểm B
plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.02, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

% --- Đường đồng mức (Contours) ---
C_opt = sqrt(2); % Giá trị tối ưu
contours = [C_opt, 1.8, 2.2, 2.6];

for i = 1:length(contours)
    C = contours(i);
    y_contour = C - 2*sqrt(2)*x1_all; % Phương trình contour
    plot(x1_all, y_contour, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn hàm mục tiêu
text(0.65, 0.8, '$2^{1.5}x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% Trục và Định dạng
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in');
set(gca, 'XTick', 0:0.1:1.1);
set(gca, 'YTick', 0:0.1:1.1);
set(gca, 'LineWidth', 1);
```

```
91
92 hold off;
```

Code 2.6: Vẽ Hình 2.11 (trường hợp d)

```
1  % --- Khởi tạo ---
2  clc; clear; close all;
3
4  % --- 1. Thiết lập dữ liệu ---
5  % Tạo lưới điểm cho từng vùng ràng buộc
6  x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
7  x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
8  x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2
9
10 % --- 2. Các hàm ràng buộc ---
11 sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
12 sigma2_func = @(x) (1 / sqrt(2)) - x;
13 sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);
14
15 % --- 3. Tính điểm đặc biệt ---
16 % Điểm A: Giao của Sigma 1 & 2
17 k = 1 / sqrt(2);
18 x_A = k / (4*k - 1);
19 y_A = sigma2_func(x_A);
20
21 % Điểm B: Giao Sigma 1 với trục hoành
22 x_B = 0.5;
23 y_B = 0;
24
25 % --- 4. Vẽ đồ thị ---
26 figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
27 hold on; axis([0 1 0 1]); box on;
28
29 % Độ lệch tạo bóng (shadow)
30 off = 0.015;
31
32 % --- Vẽ Sigma 3 (Trái) ---
33 y_s3 = sigma3_func(x1_left);
34 mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
35 x3_plot = x1_left(mask_s3);
36 y3_plot = y_s3(mask_s3);
37 plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
38 plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2); % Chính
39 text(0.09, 0.92, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);
40
41 % --- Vẽ Sigma 2 (Thẳng) ---
42 y_s2 = sigma2_func(x1_all);
43 plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
44 plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2); % Chính
45 text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);
46
47 % --- Vẽ Sigma 1 (Phải) ---
48 y_s1 = sigma1_func(x1_right);
49 mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
50 x1_plot = x1_right(mask_s1);
51 y1_plot = y_s1(mask_s1);
52 plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
53 plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2); % Chính
54 text(0.34, 0.65, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);
55
56 % --- Các điểm A, B ---
57 plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
58 text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');
59
60 plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
61 text(x_B + 0.01, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');
62
63 % --- Đường đồng mức (Hàm mục tiêu: f = 4x1 + x2) ---
64 C_opt = 4*x_A + y_A;        % Giá trị tối ưu tại A
65 contours = [C_opt, 2.5, 3.1, 3.7];
66
67 for i = 1:length(contours)
68     C = contours(i);
69     y_contour = C - 4*x1_all; % Phương trình: x2 = C - 4*x1
70     plot(x1_all, y_contour, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
```

```
end

% Nhãn hàm mục tiêu
text(0.79, 0.65, '$4x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in', 'LineWidth', 1);
set(gca, 'XTick', 0:0.1:1.1, 'YTick', 0:0.1:1.1);

hold off;
```

Code 2.7: Vẽ Hình 2.12 (trường hợp e)

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng ràng buộc
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính toán điểm đặc biệt ---
% Điểm A: Giao của Sigma 1 & Sigma 2
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
y_A = sigma2_func(x_A);

% Điểm B: Giao Sigma 1 với trục hoành
x_B = 0.5;
y_B = 0;

% Điểm C: Giao Sigma 2 với trục hoành (x = 1/sqrt(2))
x_C = 1 / sqrt(2);
y_C = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

% Độ lệch tạo bóng (shadow)
off = 0.015;

% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2); % Chính
text(0.09, 0.89, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2); % Chính
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2); % Chính
text(0.34, 0.79, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Đánh dấu các điểm A, B, C ---
```

```
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_A + 0.01, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.01, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

plot(x_C, y_C, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_C + 0.01, 0.02, 'C', 'FontSize', 12, 'FontWeight', 'bold');

% --- Đường đồng mức (Hàm mục tiêu: f = x1 + x2) ---
% Hệ số góc -1, song song với Sigma 2
contours = [0.9, 1.1, 1.3, 1.5];
for i = 1:length(contours)
    C = contours(i);
    % Phương trình: x2 = C - x1
    plot(x1_all, C - x1_all, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn hàm mục tiêu
text(0.7, 0.82, '$x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in', 'LineWidth', 1);
set(gca, 'XTick', 0:0.1:1.1, 'YTick', 0:0.1:1.1);

hold off;
```

Code 2.8: Vẽ Hình 2.13

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập lưới dữ liệu ---
x1_vals = linspace(0.01, 1.6, 500);
x2_vals = linspace(0, 5.5, 500);
[X1, X2] = meshgrid(x1_vals, x2_vals);

% --- 2. Tính toán ràng buộc (Constraint) ---
% Phương trình: (4x1^2 + 200x2^2 + 40x1x2) / (x1^2 * (2x1 + 11x2)^2) = 1
Numer = 4.*X1.^2 + 200.*X2.^2 + 40.*X1.*X2;
Denom = X1.^2 .* (2.*X1 + 11.*X2).^2;
Constraint = Numer ./ Denom;

% Trích xuất tọa độ đường cong từ dữ liệu contour
C_data = contourc(x1_vals, x2_vals, Constraint, [1 1]);

% Xử lý dữ liệu để vẽ liên tục
idx = 1;
x_curve = [];
y_curve = [];
while idx < size(C_data, 2)
    len = C_data(2, idx);
    x_curve = [x_curve, C_data(1, idx+1 : idx+len), NaN];
    y_curve = [y_curve, C_data(2, idx+1 : idx+len), NaN];
    idx = idx + len + 1;
end

% --- 3. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 650, 550]);
hold on; axis([0 1.5 0 5]); box on;

% --- A. Vẽ hàm mục tiêu (11x1 + x2 = C) ---
C_values = [2, 5, 8, 11, 14, 17];

for i = 1:length(C_values)
    C = C_values(i);
    y_line = C - 11 * x1_vals; % Phương trình đường thẳng

    % Chỉ vẽ phần nằm trong giới hạn trục
    mask = y_line >= 0 & y_line <= 5.5;
    if any(mask)
        plot(x1_vals(mask), y_line(mask), '-', 'Color', [0.5 0.5 0.5], 'LineWidth', 0.8);
    end
```

```
end

% Nhãn hàm mục tiêu
text(0.55, 4.3, '$11x_1+x_2=\mathrm{const.}$', 'Interpreter', 'latex', 'FontSize', 11);

% --- B. Vẽ đường cong ràng buộc ---
off = 0.008; % Độ lệch tạo bóng

% 1. Đường bóng (Nét đứt)
plot(x_curve - off, y_curve, ':', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);

% 2. Đường chính (Nét liền đậm)
plot(x_curve, y_curve, 'k-', 'LineWidth', 1.5);

% Nhãn ràng buộc
text(1.28, 3.5, '$\mathbf{u}^T\mathbf{u}=\delta_0^2$', 'Interpreter', 'latex', ...
     'FontSize', 12, 'FontWeight', 'bold');

% --- 4. Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);

% Tùy chỉnh Tick marks
set(gca, 'TickDir', 'in');
set(gca, 'XTick', 0:0.5:1.5);
set(gca, 'YTick', 0:0.5:5);
set(gca, 'LineWidth', 1, 'FontSize', 10);

hold off;
```

Code 2.9: Vẽ Hình 2.14

```
clear; clc; close all;

% Thiết lập cửa sổ hiển thị chính
figure('Color', 'w', 'Units', 'pixels', 'Position', [50, 50, 1000, 500]);

%% ========================================================================
% HÌNH CON BÊN TRÁI: CẬN CẢNH TRUNG BÌNH
% ========================================================================

subplot(1, 2, 1); hold on; box on; axis([0.95 1.1 0 0.2]);

% 1. Khởi tạo lưới dữ liệu và tính toán hàm ràng buộc
x1_vals = linspace(0.94, 1.11, 400);
x2_vals = linspace(0, 0.21, 400);
[X1, X2] = meshgrid(x1_vals, x2_vals);

% Công thức ràng buộc: (4x1^2 + 200x2^2 + 40x1x2) / (x1^2 * (2x1 + 11x2)^2)
Numer = 4.*X1.^2 + 200.*X2.^2 + 40.*X1.*X2;
Denom = X1.^2 .* (2.*X1 + 11.*X2).^2;
Constraint = Numer ./ Denom;

% Trích xuất đường đẳng mức (Level Set) tại giá trị bằng 1
C_data = contourc(x1_vals, x2_vals, Constraint, [1 1]);
idx = 1; x_curve = []; y_curve = [];
while idx < size(C_data, 2)
    len = C_data(2, idx);
    x_curve = [x_curve, C_data(1, idx+1 : idx+len), NaN];
    y_curve = [y_curve, C_data(2, idx+1 : idx+len), NaN];
    idx = idx + len + 1;
end

% 2. Vẽ các đường mức hàm mục tiêu (11x1 + x2 = C)
% Phân bổ các đường thẳng dựa trên giao điểm trục hoành
x_intercepts = 0.96 : 0.04 : 1.12;
C_vals_1 = 11 * x_intercepts;

for C = C_vals_1
    y_line = C - 11 * x1_vals;
    mask = y_line >= -0.01 & y_line <= 0.21; % Giới hạn vẽ trong khung hình
    if any(mask)
        plot(x1_vals(mask), y_line(mask), '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.8);
    end
end
```

```
44
45 % 3. Hiển thị đường ràng buộc (kèm hiệu ứng bóng đổ)
46 off1 = 0.002;
47 plot(x_curve - off1, y_curve, ':', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);
48 plot(x_curve, y_curve, 'k-', 'LineWidth', 1.5);
49
50 % Định dạng trục và nhãn
51 xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
52 ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
53 set(gca, 'TickDir', 'in', 'LineWidth', 1, 'FontSize', 12, ...
54     'XTick', 0.95:0.05:1.1, 'YTick', 0:0.02:0.2);
55
56 %% ========================================================================
57 % HÌNH CON BÊN PHẢI: CẬN CẢNH CHI TIẾT
58 % ========================================================================
59
60 subplot(1, 2, 2); hold on; box on; axis([0.995 0.997 0.005 0.025]);
61
62 % 1. Khởi tạo lưới dữ liệu siêu mịn cho khu vực tối ưu
63 x1_vals_2 = linspace(0.994, 0.998, 500);
64 x2_vals_2 = linspace(0.004, 0.026, 500);
65 [X1_2, X2_2] = meshgrid(x1_vals_2, x2_vals_2);
66
67 Numer_2 = 4.*X1_2.^2 + 200.*X2_2.^2 + 40.*X1_2.*X2_2;
68 Denom_2 = X1_2.^2 .* (2.*X1_2 + 11.*X2_2).^2;
69 Constraint_2 = Numer_2 ./ Denom_2;
70
71 % Trích xuất đường cong ràng buộc mịn
72 C_data_2 = contourc(x1_vals_2, x2_vals_2, Constraint_2, [1 1]);
73 idx = 1; x_curve_2 = []; y_curve_2 = [];
74 while idx < size(C_data_2, 2)
75     len = C_data_2(2, idx);
76     x_curve_2 = [x_curve_2, C_data_2(1, idx+1 : idx+len), NaN];
77     y_curve_2 = [y_curve_2, C_data_2(2, idx+1 : idx+len), NaN];
78     idx = idx + len + 1;
79 end
80
81 % 2. Vẽ hệ thống hàm mục tiêu song song dày đặc
82 x_targets = 0.995 : 0.0004 : 0.998;
83 C_vals_2 = 11 * x_targets + 0.01545; % Căn chỉnh tại độ cao y = 0.015
84
85 for C = C_vals_2
86     y_line = C - 11 * x1_vals_2;
87     mask = y_line >= 0 & y_line <= 0.03;
88     if any(mask)
89         plot(x1_vals_2(mask), y_line(mask), '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.8);
90     end
91 end
92
93 % 3. Hiển thị đường ràng buộc mịn
94 off2 = 0.00005;
95
96 plot(x_curve_2 - off2, y_curve_2, ':', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);
97 plot(x_curve_2, y_curve_2, 'k-', 'LineWidth', 1.5);
98
99 % 4. Đánh dấu điểm tối ưu A
100 x_A = 0.995259; y_A = 0.0170132;
101 plot(x_A, y_A, 'k+', 'MarkerSize', 9, 'LineWidth', 1.5);
102 text(x_A + 0.0001, y_A + 0.001, 'A', 'FontSize', 12, 'FontWeight', 'bold');
103
104 % Định dạng trục và nhãn chi tiết
105 xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
106 ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
107 set(gca, 'TickDir', 'in', 'LineWidth', 1, 'FontSize', 12);
108 xticks([0.995 0.9955 0.996 0.9965 0.997]);
109 yticks([0.005 0.01 0.015 0.02 0.025]);
110 xtickformat('%.4f');
111
112 hold off;
```

Chương 3. Cơ sở toán học quy hoạch lồi

Code 3.1: Vẽ Hình 3.3 (Ví dụ 3.1)

```
%% TRỰC QUAN HÓA TÍNH LỒI - LÕM
clc; clear; close all;

% Dữ liệu lưới cho bề mặt 3D
[X, Y] = meshgrid(linspace(-2, 2, 60));

%% ĐỒ THỊ (a): HÀM LỒI CHẶT f(x) = x^2 (Parabol)
figure('Color', 'w', 'Position', [50, 400, 500, 400]);
x1 = linspace(-2, 2, 100);
y1 = x1.^2;
plot(x1, y1, 'b-', 'LineWidth', 2.5);
grid on; box on;
title('\bf $$f(x_1) = x_1^2$$', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$$x_1$$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$$f$$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'FontSize', 10);

%% ĐỒ THỊ (b): HÀM LỒI (KHÔNG CHẶT)
% f(x1, x2) = x1^2 (Máng xối)
figure('Color', 'w', 'Position', [560, 400, 500, 400]);
Z1 = X.^2;
surf(X, Y, Z1, 'EdgeColor', 'none', 'FaceAlpha', 0.85);
hold on;
colormap('parula');
% Đường chỉ thị đáy phẳng (trục dọc)
plot3([0 0], [-2 2], [0 0], 'r-', 'LineWidth', 3);
view(45, 20);
axis square; grid on; box on;
camlight; lighting gouraud;
title('\bf $$f(x_1, x_2) = x_1^2$$', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$$x_1$$', 'Interpreter', 'latex');
ylabel('$$x_2$$', 'Interpreter', 'latex');
zlabel('$$f$$', 'Interpreter', 'latex');
rotate3d on;

%% ĐỒ THỊ (c): HÀM KHÔNG LỒI - KHÔNG LÕM f(x1, x2) = x1 * x2 (Yên ngựa)
figure('Color', 'w', 'Position', [1070, 400, 500, 400]);
Z2 = X .* Y;
surf(X, Y, Z2, 'EdgeColor', 'none', 'FaceAlpha', 0.9);
hold on;
colormap('jet');
view(15, 15);
axis square; grid on; box on;
camlight; lighting gouraud;
title('\bf $$f(x_1, x_2) = x_1 x_2$$', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$$x_1$$', 'Interpreter', 'latex');
ylabel('$$x_2$$', 'Interpreter', 'latex');
zlabel('$$f$$', 'Interpreter', 'latex');
rotate3d on;
```

Code 3.2: Vẽ Hình 3.5 (Ví dụ 3.4; Python)

```
import numpy as np
import os
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# --- Cấu hình Matplotlib ---
# Thiết lập font LaTeX (Computer Modern)
plt.rcParams['mathtext.fontset'] = 'cm'
plt.rcParams['font.family'] = 'serif'

# --- 1. Chuẩn bị dữ liệu ---
x = np.linspace(-0.2, 1.3, 400)
y = np.linspace(-0.2, 1.3, 400)
X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2  # Hàm mục tiêu

# --- 2. Khởi tạo hình vẽ ---
fig, ax = plt.subplots(figsize=(7, 7))

# --- 3. Vẽ đồ thị ---
# Đường đồng mức (Contours)
```

```
contours = ax.contour(X, Y, Z, levels=np.arange(0, 2.6, 0.5), cmap='viridis')

# Đường ràng buộc (Constraint): x1 + x2 = 1
x_line = np.linspace(-0.2, 1.2, 100)
y_line = 1 - x_line
line_color = '#1f77b4'
ax.plot(x_line, y_line, color=line_color, linewidth=2)

# Điểm tối ưu (Optimal point)
ax.plot(0.5, 0.5, 'o', color=line_color, markersize=8, zorder=5)

# Vector Gradient (Hướng giảm)
ax.arrow(0.5, 0.5, -0.15, -0.15,
         head_width=0.04, head_length=0.06,
         fc='black', ec='black', linewidth=2, zorder=5)

# --- 4. Thêm nhãn chú thích (Annotations) ---

# Nhãn hàm mục tiêu
ax.text(0.8, 1.15, r'$g_0(\mathbf{x}) = x_1^2 + x_2^2$',
        fontsize=14, color='black',
        bbox=dict(facecolor='white', edgecolor='none', alpha=0.7))

# Nhãn ràng buộc g1(x)
ax.text(0.75, 0.3, r'$g_1(\mathbf{x}) = 1 - x_1 - x_2 \leq 0$',
        fontsize=14, color=line_color, ha='left')

# Nhãn điểm tối ưu
ax.text(0.52, 0.52, r'$\mathbf{x}^* (0{,}5;\ 0{,}5)$',
        fontsize=14, color=line_color,
        ha='left', va='bottom', zorder=10)

# Nhãn Gradient
ax.text(0.3, 0.3, r'$-\nabla g_0$', fontsize=13, ha='right')

# --- 5. Hoàn thiện hiển thị ---
# Thiết lập trục và lưới
ax.set_xlabel('$x_1$', fontsize=14)
ax.set_ylabel('$x_2$', fontsize=14)
ax.set_xlim([-0.2, 1.3])
ax.set_ylim([-0.2, 1.3])
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_aspect('equal')

# Nhãn trục dùng dấu phẩy thập phân theo quy ước của sách
vn_ticks = FuncFormatter(lambda v, pos: f"{v:.1f}".replace('.', ','))
ax.xaxis.set_major_formatter(vn_ticks)
ax.yaxis.set_major_formatter(vn_ticks)

plt.tight_layout()
script_dir = os.path.dirname(os.path.abspath(__file__))  # lưu hình cạnh file mã
plt.savefig(os.path.join(script_dir, 'Figure_1.png'), dpi=200, bbox_inches='tight')
plt.show()
```

Code 3.3: Vẽ Hình 3.6 (Ví dụ 3.5; Python)

```
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Cấu hình hệ thống hiển thị ---
plt.rcParams['font.family'] = 'serif'
plt.rcParams['mathtext.fontset'] = 'cm'
plt.rcParams['font.size'] = 11

# --- 2. Khởi tạo dữ liệu ---
x = np.linspace(0, 11, 400)
y = np.linspace(0, 11, 400)
X, Y = np.meshgrid(x, y)
Z = (X - 5)**2 + (Y - 4)**2  # Hàm mục tiêu f(x)

# --- 3. Thiết lập khung hình ---
# Sử dụng kích thước 6x6 để đảm bảo khung hình vuông
fig, ax = plt.subplots(figsize=(6, 6), dpi=200)

# --- 4. Vẽ các thành phần hình học ---
```

```
# a) Miền khả dĩ (Feasible Region)
x_fill = np.linspace(0, 4, 100)
y_fill = (10 - x_fill)/2
ax.fill_between(x_fill, 0, y_fill, color='#d9d9d9', alpha=0.7)
ax.text(1.5, 2, 'Miền khả dĩ', fontsize=11, ha='center', color='black')

# b) Đường biên ràng buộc (Boundaries)
ax.plot(x, (10 - x)/2, 'k-', linewidth=1.2)                  # Biên g1
ax.axvline(x=4, color='k', linewidth=1.2, ymin=0, ymax=1)    # Biên g2

# c) Đường đồng mức (Contour Lines)
levels = [0.5, 2, 5, 9, 14, 20]
contours = ax.contour(X, Y, Z, levels=levels, colors='black', linewidths=0.6, alpha=0.7)

# d) Các điểm đặc biệt
# Điểm tối ưu x*
ax.plot(4, 3, 'ko', markersize=5)
ax.text(2.8, 2.6, r'$x^*(4,3)$', fontsize=11, fontweight='bold')

# Tâm hàm mục tiêu T
ax.plot(5, 4, 'ko', markersize=3)
ax.text(5.1, 4.0, r'$T(5,4)$', fontsize=11)

# --- 5. Biểu diễn Vector Gradient ---
x0, y0 = 4, 3
arrow_width = 0.008
head_width = 4

# Vector -grad(f)
ax.quiver(x0, y0, 1, 1, angles='xy', scale_units='xy', scale=1,
          color='#d62728', width=arrow_width, headwidth=head_width, label=r'$-\nabla f$')
ax.text(4.9, 3.5, r'$-\nabla f$', color='#d62728', fontsize=11)

# Vector grad(g1)
s1 = 0.6
ax.quiver(x0, y0, 1*s1, 2*s1, angles='xy', scale_units='xy', scale=1,
          color='#1f77b4', width=arrow_width, headwidth=head_width)
ax.text(4.0, 4.3, r'$\nabla g_1$', color='#1f77b4', fontsize=11)

# Vector grad(g2)
s2 = 1.2
ax.quiver(x0, y0, 1*s2, 0*s2, angles='xy', scale_units='xy', scale=1,
          color='#1f77b4', width=arrow_width, headwidth=head_width)
ax.text(5.2, 2.6, r'$\nabla g_2$', color='#1f77b4', fontsize=11)

# --- 6. Hoàn thiện bố cục và chú thích ---
# Thiết lập giới hạn trục (vuông vức 0-11)
ax.set_xlim(0, 11)
ax.set_ylim(0, 11)

# Tên trục
ax.set_xlabel(r'$x_1$', fontsize=12, loc='right')
ax.set_ylabel(r'$x_2$', fontsize=12, loc='top')

# Nhãn phương trình
ax.text(1.5, 4.6, r'$g_1=0$', fontsize=11, rotation=0)
ax.text(6.5, 1.0, r'$g_1(x) = x_1 + 2x_2 - 10 \leq 0$', fontsize=10)
ax.text(4.2, 0.5, r'$g_2=0$', fontsize=10)
ax.text(0.5, 8.5, r'$g_2(x) = x_1 - 4 \leq 0$', fontsize=10)
ax.text(6.5, 8.5, r'$f(x) = (x_1 - 5)^2 + (x_2 - 4)^2$', fontsize=10)

# Cấu hình hiển thị trục (Center spines)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_position('zero')
ax.spines['bottom'].set_position('zero')

# Thiết lập tỷ lệ 1:1 (Aspect Ratio)
ax.set_aspect('equal')

plt.tight_layout()
plt.show()
```

Code 3.4: Vẽ Hình 3.7 (Ví dụ 3.6)

```
% SCRIPT: KKT_Visualization_Final.m - vẽ Hình 3.7 (Ví dụ 3.6)
% MÔ TẢ:
%   Minh họa trực quan điều kiện Karush-Kuhn-Tucker (KKT) trong không gian 3D.
%   Biểu diễn điểm tiếp xúc giữa mặt cầu mục tiêu f(x) và mặt phẳng ràng buộc g(x).
%   Hiển thị tính đồng phương của các vector gradient tại điểm tối ưu.
%
%% 1. Khởi tạo môi trường và Cấu hình đồ họa
clear; clc; close all;

figure('Color', 'w', 'Position', [100, 100, 900, 800]);
hold on;
grid on; box on; axis equal;

%% 2. Định nghĩa Thông số Bài toán
% Điểm tối ưu cục bộ (Local Optimum / Stationary Point)
Opt = [1, 1, 1];

% Bán kính mặt cầu tại điểm tối ưu
R_opt = sqrt(3);

%% 3. Trực quan hóa Mặt phẳng Ràng buộc (Constraint Plane)
% Tạo lưới tọa độ cho mặt phẳng
[X_plane, Y_plane] = meshgrid(linspace(-1, 4.5, 50));

% Phương trình mặt phẳng: z = 3 - x - y
Z_plane = 3 - X_plane - Y_plane;

% Vẽ mặt phẳng ràng buộc
surf(X_plane, Y_plane, Z_plane, 'FaceColor', [0.7 0.7 0.9], ...
     'FaceAlpha', 0.5, 'EdgeColor', 'none');

% Ghi chú phương trình ràng buộc g(x)
text(-2.9, 2.4, 1.5, '$g(x): x + y + z = 3$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', 'k', 'FontWeight', 'bold');

%% 4. Trực quan hóa Hàm mục tiêu (Objective Function)
% Tạo dữ liệu mặt cầu
[Xs, Ys, Zs] = sphere(80);

% Vẽ mặt cầu mục tiêu
surf(Xs*R_opt, Ys*R_opt, Zs*R_opt, 'FaceColor', [1 0.8 0.4], ...
     'FaceAlpha', 0.4, 'EdgeColor', 'none');

% Ghi chú phương trình hàm mục tiêu f(x)
text(-1.16, -3.58, 1.82, '$f(x): x^2 + y^2 + z^2 = 3$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', [0.8 0.4 0], 'FontWeight', 'bold');

%% 5. Hiển thị Vector Gradient tại Điểm tối ưu
% Thiết lập hệ số tỉ lệ (Scaling factors) để hiển thị vector rõ ràng hơn
scale_g = 1.8;  % Hệ số tỉ lệ cho vector gradient ràng buộc
scale_f = 2.8;  % Hệ số tỉ lệ cho vector gradient hàm mục tiêu

% --- Vector Gradient của Ràng buộc (Nabla g) ---
% Vẽ vector pháp tuyến mặt phẳng (Màu đỏ)
quiver3(Opt(1), Opt(2), Opt(3), scale_g, scale_g, scale_g, ...
    'r', 'LineWidth', 4, 'MaxHeadSize', 0.5, 'AutoScale', 'off');

% Nhãn cho vector Nabla g
text(Opt(1)+scale_g, Opt(2)+scale_g, Opt(3)+scale_g - 0.2, '$\nabla g$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', 'r', 'FontWeight', 'bold');

% --- Vector Gradient của Hàm mục tiêu (Nabla f) ---
% Vẽ vector pháp tuyến mặt cầu (Màu xanh dương)
quiver3(Opt(1), Opt(2), Opt(3), scale_f, scale_f, scale_f, ...
    'b', 'LineWidth', 3, 'MaxHeadSize', 0.5, 'AutoScale', 'off');

% Nhãn cho vector Nabla f
text(Opt(1)+scale_f, Opt(2)+scale_f, Opt(3)+scale_f, '$\nabla f$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', 'b', 'FontWeight', 'bold');

%% 6. Cấu hình Ánh sáng và Góc nhìn (Camera & Lighting)
% Thiết lập ánh sáng Gouraud để tăng chiều sâu 3D
camlight left; camlight right;
```

```
lighting gouraud;

% Đặt góc nhìn cố định
view(48, 37);

% Thiết lập giới hạn không gian hiển thị
xlim([-1 5]); ylim([-1 5]); zlim([-1 5]);

hold off;
```

Chương 4. Các thuật toán xấp xỉ lồi tường minh tuần tự

Code 4.1: Giải Ví dụ 3.5 bằng quy hoạch tuyến tính tuần tự (SLP)

```
% TỐI ƯU HÓA BẰNG QUY HOẠCH TUYẾN TÍNH TUẦN TỰ (SLP)
% lưu ý cần Optimization Toolbox
%     min   f(x) = (x1 - 5)^2 + (x2 - 4)^2
%     s.t.  g1(x): x1 + 2*x2 - 10 <= 0
%           g2(x): x1 - 4 <= 0
clc; clear;
% 1. Khởi tạo thông số ban đầu
x = [0; 0];           % Điểm khởi tạo x0
tol = 1e-6;           % Sai số cho phép để dừng thuật toán
max_iter = 50;        % Số vòng lặp tối đa
move_limit = 0.5;     % Giới hạn dịch chuyển
% 2. Vòng lặp chính SLP (Sequential Linear Programming)
for k = 1:max_iter
    % Bước A: Tính toán giá trị tại điểm lặp hiện tại và đạo hàm
    f_val = (x(1)-5)^2 + (x(2)-4)^2;
    grad_f = [2*(x(1)-5); 2*(x(2)-4)];
    % Tính giá trị các hàm ràng buộc và ma trận đạo hàm các ràng buộc
    g_val = [x(1) + 2*x(2) - 10; x(1) - 4];
    grad_g = [1, 2; 1, 0];
    % Bước B: Thiết lập và giải bài toán con quy hoạch tuyến tính tuần tự
    lb = [-move_limit; -move_limit]; % Giới hạn dưới của d
    ub = [move_limit; move_limit];   % Giới hạn trên của d
    opts = optimoptions('linprog', 'Display', 'off');
    [d, ~, exitflag] = linprog(grad_f, grad_g, -g_val, [], [], lb, ub, opts);
    % Bước C: Cập nhật nghiệm và kiểm tra điều kiện hội tụ
    x = x + d;
    if norm(d) < tol
        break;
    end
end
% 3. Hiển thị kết quả tối ưu
fprintf('Nghiệm tối ưu x*: [%.4f; %.4f]\n', x);
fprintf('Giá trị f(x*):    %.4f\n', (x(1)-5)^2 + (x(2)-4)^2);
```

Code 4.2: Giải Ví dụ 3.5 bằng quy hoạch toàn phương tuần tự (SQP)

```
% TỐI ƯU HÓA BẰNG QUY HOẠCH TOÀN PHƯƠNG TUẦN TỰ (SQP)
% lưu ý cần Optimization Toolbox
%     min   f(x) = (x1 - 5)^2 + (x2 - 4)^2
%     s.t.  g1(x): x1 + 2*x2 - 10 <= 0
%           g2(x): x1 - 4 <= 0
clc; clear;
% 1. Khởi tạo thông số ban đầu
x = [0; 0];           % Điểm khởi tạo x0
tol = 1e-6;           % Sai số cho phép để dừng thuật toán
max_iter = 50;        % Số vòng lặp tối đa
H = eye(2);           % Xấp xỉ Hessian cố định (ma trận đơn vị); hàm mục tiêu bậc hai nên vẫn hội tụ
% 2. Vòng lặp chính SQP (Sequential Quadratic Programming)
for k = 1:max_iter
    % Bước A: Tính toán giá trị tại điểm lặp hiện tại và đạo hàm
    f_val = (x(1)-5)^2 + (x(2)-4)^2;
    grad_f = [2*(x(1)-5);
              2*(x(2)-4)];
    % Tính giá trị các hàm ràng buộc và ma trận đạo hàm các ràng buộc
    g_val = [x(1) + 2*x(2) - 10; x(1) - 4];
    grad_g = [1, 2; 1, 0];
    % Bước B: Thiết lập và giải bài toán con quy hoạch toàn phương
    opts = optimoptions('quadprog', 'Display', 'off');
```

```
    [d, ~, exitflag] = quadprog(H, grad_f, grad_g, -g_val, [],[],[],[],[], opts);
    % Bước C: Cập nhật nghiệm và kiểm tra điều kiện hội tụ
    x = x + d;
    if norm(d) < tol
        break;
    end
end
% 3. Hiển thị kết quả tối ưu
fprintf('Nghiệm tối ưu x*:         [%0.4f; %0.4f]\n', x);
fprintf('Giá trị hàm mục tiêu f*:  %0.4f\n', (x(1)-5)^2 + (x(2)-4)^2);
```

Code 4.3: Vẽ Hình 4.1 (xấp xỉ CONLIN, Ví dụ 4.1)

```
% ========================================================================
% Xấp xỉ CONLIN
% ========================================================================

clc; clear; close all;

% 1. ĐỊNH NGHĨA HÀM SỐ VÀ ĐẠO HÀM
g = @(x) x + x.^2 - (1/40).*x.^4;
dg = @(x) 1 + 2.*x - (1/10).*x.^3;

% Tạo miền giá trị x để vẽ
x = linspace(0.1, 9, 500);
y_true = g(x);

% 2. TÍNH TOÁN TẠI X1 = 1
x1 = 1;
g1 = g(x1);
dg1 = dg(x1);
gL1 = g1 + dg1 .* (x - x1);               % Tuyến tính
gR1 = g1 + dg1 .* (x1 ./ x) .* (x - x1);  % Nghịch đảo

% 3. TÍNH TOÁN TẠI X2 = 6
x2 = 6;
g2 = g(x2);
dg2 = dg(x2);
gL2 = g2 + dg2 .* (x - x2);               % Tuyến tính
gR2 = g2 + dg2 .* (x2 ./ x) .* (x - x2);  % Nghịch đảo

% 4. VẼ ĐỒ THỊ
figure('Color', 'w', 'Position', [100, 100, 800, 500]);
hold on; grid on; box on;

% --- Vẽ các đường ---
plot(x, y_true, 'k-', 'LineWidth', 2);     % Hàm gốc
plot(x, gL1, 'k--', 'LineWidth', 1.5);     % CONLIN tại x=1 (Tuyến tính)
plot(x, gR1, 'k:', 'LineWidth', 1.2);      % So sánh tại x=1
plot(x, gR2, 'b-.', 'LineWidth', 1.5);     % CONLIN tại x=6 (Nghịch đảo)
plot(x, gL2, 'b:', 'LineWidth', 1.2);      % So sánh tại x=6

% --- Đánh dấu điểm ---
plot(x1, g1, 'ro', 'MarkerFaceColor', 'r');
plot(x2, g2, 'ro', 'MarkerFaceColor', 'r');

% 5. TRANG TRÍ & CHÚ THÍCH (TEXT NGANG)
ylim([-5 18]);
xlim([0 9]);
xlabel('x');
ylabel('g(x)');

% Chú thích trực tiếp (Text nằm ngang mặc định)
text(4, 15, 'g', 'FontSize', 16, 'FontWeight', 'bold');

% Khu vực x = 1
text(2.6, 8.5, 'g^C = g^L', 'FontSize', 16);
text(2.6, 4.5, 'g^R', 'FontSize', 16);

% Khu vực x = 6
text(7.2, 5, 'g^C = g^R', 'FontSize', 16);
text(6.8, -2, 'g^L', 'FontSize', 16);

hold off;
```

Code 4.4: Vẽ Hình 4.2 (xấp xỉ MMA, Ví dụ 4.2)

```
% ========================================================================
% Phương pháp MMA
% ========================================================================

clc; clear; close all;

% 1. ĐỊNH NGHĨA HÀM SỐ VÀ ĐẠO HÀM
g = @(x) x + x.^2 - (1/40).*x.^4;
dg = @(x) 1 + 2.*x - (1/10).*x.^3;

% Điểm khảo sát x0 = 1
x0 = 1;
g0 = g(x0);       % g(1) = 1.975
dg0 = dg(x0);     % g'(1) = 2.9

% Miền giá trị x để vẽ
x = linspace(-6, 6, 2000); % Tăng điểm ảnh để vẽ mượt hơn
y_true = g(x);

% 2. CÁC GIÁ TRỊ TIỆM CẬN TRÊN (U)
U_values = [1.5, 4, 20, 10000];
line_styles = {'--', ':', '-.', '-'};
line_colors = {'k', 'k', 'k', 'k'};

% 3. VẼ ĐỒ THỊ
figure('Color', 'w', 'Position', [100, 100, 800, 600]);
hold on; grid on; box on;

% Vẽ hàm gốc g(x)
plot(x, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Hàm gốc g(x)');

% Vòng lặp tính và vẽ MMA
for i = 1:length(U_values)
    U = U_values(i);

    % Tính toán hệ số (đạo hàm dương -> dùng U)
    p = (U - x0)^2 * dg0;
    r = g0 - p / (U - x0);
    g_mma = r + p ./ (U - x);

    % --- Ngắt đường cong tại tiệm cận đứng ---
    % Với U = 1.5
    if U == 1.5
        g_mma(x >= 1.5) = NaN;
    end

    % Với U = 4
    if U == 4
        g_mma(abs(x - U) < 0.05) = NaN; % Chỉ ngắt tại điểm gián đoạn
    end

    % Giới hạn hiển thị trục y để hình không bị biến dạng
    g_mma(g_mma > 50 | g_mma < -50) = NaN;

    % Thiết lập Label
    if U == 10000
        legend_label = sprintf('Xấp xỉ MMA với U^0=10^4');
    else
        legend_label = sprintf('Xấp xỉ MMA với U^0=%.1f', U);
    end

    % Vẽ đường cong
    plot(x, g_mma, line_styles{i}, 'Color', line_colors{i}, ...
         'LineWidth', 1.2, 'DisplayName', legend_label);

    % Vẽ đường tiệm cận đứng minh họa cho U=1.5 và U=4
    if U <= 4
         xline(U, 'k:', 'LineWidth', 0.5, 'HandleVisibility', 'off');
    end
end

% 4. TRANG TRÍ
xlim([-6 6]);
```

```
ylim([-10 40]);
xlabel('x');
ylabel('g(x)');

% Đánh dấu điểm x0
plot(x0, g0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 16, 'HandleVisibility', 'off');
text(x0, g0-3, 'x^0=1', 'HorizontalAlignment', 'center');

legend('Location', 'northwest');
hold off;
```

Chương 5. Tối ưu độ cứng giàn

Code 5.1: Giải Ví dụ 5.1 bằng phương pháp OC và vẽ Hình 5.4

```
% ------------------------------------------------------------
% TỐI ƯU HÓA CẤU TRÚC GIÀN
% Phương pháp: Optimality Criteria (OC)
% ------------------------------------------------------------
clear; clc; close all;

%% 1. CẤU HÌNH & TẠO LƯỚI
E        = 1.0;
Vol_frac = 0.10;
xmin     = 1e-9;
xmax     = 1.0;

% Thông số lưới (8x4)
nelx = 8;
nely = 4;
dx   = 1;
dy   = 1;

% Tạo tọa độ nút
[X, Y] = meshgrid(0:dx:nelx*dx, 0:dy:nely*dy);
nodes  = [X(:), Y(:)];
num_nodes = size(nodes, 1);

% Tạo phần tử thanh (Ground Structure)
bars = [];
node_map = reshape(1:num_nodes, nely+1, nelx+1);

for i = 1:nelx
    for j = 1:nely
        nBL = node_map(j, i);   nBR = node_map(j, i+1);
        nTL = node_map(j+1, i); nTR = node_map(j+1, i+1);
        % Kết nối: Ngang, Dọc, Chéo (X)
        bars = [bars; nBL nBR; nTL nTR; nBL nTL; nBR nTR; nBL nTR; nTL nBR];
    end
end

% Loại bỏ các thanh trùng lặp và sắp xếp
bars = unique(sort(bars, 2), 'rows');
num_bars = size(bars, 1);

% Tính toán đặc tính hình học (Chiều dài, Cos, Sin)
d = nodes(bars(:,2), :) - nodes(bars(:,1), :);
L = sqrt(sum(d.^2, 2));
C = d(:,1) ./ L;
S = d(:,2) ./ L;

% Hàm tính ma trận độ cứng phần tử
GetKe = @(Area, len, c, s) (E * Area / len) * ...
    [ c^2  c*s -c^2 -c*s;
      c*s  s^2 -c*s -s^2;
     -c^2 -c*s  c^2  c*s;
     -c*s -s^2  c*s  s^2];

% --- Tối ưu: Tính trước Ke đơn vị (Area=1) cho tất cả các thanh ---
Ke0_store = cell(num_bars, 1);
for i = 1:num_bars
    Ke0_store{i} = GetKe(1, L(i), C(i), S(i));
end
```

```

%% 2. ĐIỀU KIỆN BIÊN & TẢI TRỌNG
total_dof = num_nodes * 2;
tol = 1e-6; % Sai số cho phép khi tìm tọa độ

% Ngàm cứng các nút tại biên trái (x = 0)
fixed_nodes = find(abs(nodes(:,1)) < tol);
fixed_dofs  = [2*fixed_nodes-1; 2*fixed_nodes];
free_dofs   = setdiff(1:total_dof, fixed_dofs(:));

% Tải trọng tại góc dưới bên phải
load_node = find(abs(nodes(:,1) - nelx*dx) < tol & abs(nodes(:,2)) < tol);
F = zeros(total_dof, 1);
F(2*load_node) = -1;

%% 3. THIẾT LẬP HIỂN THỊ
max_iter = 200;
% Các mốc vòng lặp muốn hiển thị
loop_snapshots = [1, 20, 30, 50, 70, max_iter];

figure('Name', 'Qua trinh Toi uu hoa', 'Color', 'w', 'Position', [100 100 1000 700]);

% Khởi tạo biến thiết kế (Diện tích mặt cắt)
x = Vol_frac * xmax * ones(num_bars, 1);

%% 4. VÒNG LẶP TỐI ƯU HÓA
change = 1;
iter = 0;
move = 0.3; % Giới hạn bước nhảy (Move limit)
C_hist = []; % Lịch sử độ mềm

fprintf('%-10s %-15s %-15s\n', 'Iter', 'Compliance', 'Change');

while (change > 1e-6) && (iter < max_iter)
    iter = iter + 1;
    x_old = x;

    % --- PHÂN TÍCH FEM ---
    K = sparse(total_dof, total_dof);
    for i = 1:num_bars
        % Ke = Ke0 * x(i) (Vì vật liệu tuyến tính)
        Ke = Ke0_store{i} * x(i);

        idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
        K(idx, idx) = K(idx, idx) + Ke;
    end

    % Giải hệ phương trình cân bằng: K*u = F
    u = zeros(total_dof, 1);
    u(free_dofs) = K(free_dofs, free_dofs) \ F(free_dofs);

    % Tính độ mềm (Compliance)
    compliance = F' * u;
    C_hist = [C_hist; compliance];

    % --- TÍNH ĐỘ NHẠY (SENSITIVITY) ---
    dc = zeros(num_bars, 1);
    for i = 1:num_bars
        idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
        ue = u(idx);
        % dc = -ue^T * Ke0 * ue
        dc(i) = -ue' * Ke0_store{i} * ue;
    end

    % --- CẬP NHẬT BIẾN THIẾT KẾ (OPTIMALITY CRITERIA) ---
    l1 = 0; l2 = 1e9;
    V_limit = Vol_frac * sum(xmax * L);

    % Phương pháp chia đôi (Bisection) để tìm nhân tử Lagrange
    while (l2 - l1)/(l1 + l2) > 1e-6
        l_mid = 0.5 * (l2 + l1);
        B = -dc ./ (l_mid * L);

        % Cập nhật x với move limit và giới hạn biên
        x_new = max(xmin, max(x - move, min(xmax, min(x + move, x .* sqrt(B)))));
```

```
134
135         if sum(x_new .* L) > V_limit
136             l1 = l_mid;
137         else
138             l2 = l_mid;
139         end
140     end
141
142     change = max(abs(x_new - x));
143     x = x_new;
144
145     % In tiến độ ra màn hình
146     fprintf('%-10d %-15.4e %-15.4e\n', iter, compliance, change);
147
148     % --- VẼ HÌNH SNAPSHOT ---
149     % Vẽ nếu iter nằm trong danh sách HOẶC là vòng lặp cuối cùng (hội tụ)
150     is_snapshot = ismember(iter, loop_snapshots);
151     is_final    = (change <= 1e-6) || (iter == max_iter);
152
153     if is_snapshot || is_final
154         % Xác định vị trí subplot
155         if is_snapshot
156             idx_in_list = find(loop_snapshots == iter, 1);
157         else
158             % Nếu hội tụ sớm, vẽ vào ô cuối cùng
159             idx_in_list = length(loop_snapshots);
160         end
161
162         if idx_in_list <= 6 % Chỉ vẽ nếu còn chỗ trong lưới 3x2
163             subplot(3, 2, idx_in_list);
164             cla; hold on; axis equal; axis off;
165             title(sprintf('Iter: %d | C = %.2e', iter, compliance));
166
167             % Sắp xếp để vẽ thanh nhỏ trước, thanh lớn sau
168             [~, sort_ids] = sort(x);
169
170             for k = 1:num_bars
171                 bar_id = sort_ids(k);
172                 val = x(bar_id);
173
174                 % Chỉ vẽ các thanh có diện tích đáng kể
175                 if val > 0.01
176                     n1 = nodes(bars(bar_id,1), :);
177                     n2 = nodes(bars(bar_id,2), :);
178
179                     % Tính biến dạng dọc trục để tô màu
180                     dof_idx = [2*bars(bar_id,1)-1, 2*bars(bar_id,1), ...
181                                2*bars(bar_id,2)-1, 2*bars(bar_id,2)];
182                     u_local = u(dof_idx);
183
184                     axial_def = (u_local(3) - u_local(1)) * C(bar_id) + ...
185                                 (u_local(4) - u_local(2)) * S(bar_id);
186
187                     if axial_def > 1e-12
188                         col = 'b'; % Xanh: Kéo
189                     elseif axial_def < -1e-12
190                         col = 'r'; % Đỏ: Nén
191                     else
192                         col = [0.2 0.2 0.2]; % Xám: Không chịu lực
193                     end
194
195                     % Độ dày nét vẽ tỷ lệ với diện tích mặt cắt
196                     width = 6 * val / xmax;
197                     plot([n1(1) n2(1)], [n1(2) n2(2)], '-', 'Color', col, 'LineWidth', width);
198                 end
199             end
200             % Vẽ các nút cố định
201             plot(nodes(fixed_nodes,1), nodes(fixed_nodes,2), 'ks', ...
202                 'MarkerFaceColor', 'k', 'MarkerSize', 5);
203             drawnow;
204         end
205     end
206 end
207
208 fprintf('Hoan thanh! Vong lap: %d, Do mem cuoi cung: %.4e\n', iter, compliance);
```

```
209
210 %% 5. ĐỒ THỊ HỘI TỤ
211 figure('Name', 'Lich su hoi tu', 'Color', 'w');
212 plot(1:iter, C_hist, '-o', 'Color', 'b', 'LineWidth', 1.5, 'MarkerSize', 4, 'MarkerFaceColor', 'b');
213 grid on;
214 xlabel('Vòng lặp (Iterations)');
215 ylabel('Độ mềm (Compliance)');
216 title(['Hội tụ tại vòng lặp ', num2str(iter)]);
```

Code 5.2: Bản rút gọn của Code 5.1 (tệp `source_code_154.m` trong thư mục Code 5.1)

```
1  % TỐI ƯU HÓA CẤU TRÚC GIÀN
2  clear; clc; close all;
3
4  %% 1. THIẾT LẬP & TẠO LƯỚI
5  nelx = 8; nely = 4; Vol = 0.1; E = 1; % Thông số cơ bản
6  [X,Y] = meshgrid(0:nelx, 0:nely); nodes = [X(:) Y(:)];
7  num_nodes = size(nodes,1);
8  bars = []; map = reshape(1:num_nodes, nely+1, nelx+1);
9
10 % Tạo kết nối (Ground Structure)
11 for i=1:nelx, for j=1:nely
12     n = [map(j,i) map(j,i+1); map(j+1,i) map(j+1,i+1)];
13     bars = [bars; n(1,1) n(1,2); n(2,1) n(2,2); n(1,1) n(2,1); ...
14             n(1,2) n(2,2); n(1,1) n(2,2); n(2,1) n(1,2)];
15 end; end
16 bars = unique(sort(bars,2),'rows'); nb = size(bars,1);
17
18 % Tính toán hình học
19 d = nodes(bars(:,2),:) - nodes(bars(:,1),:);
20 L = sqrt(sum(d.^2,2)); C = d(:,1)./L; S = d(:,2)./L;
21
22 %% 2. FEM & TẢI TRỌNG
23 dofs = 2*num_nodes;
24 fixed = find(nodes(:,1)==0);
25 free = setdiff(1:dofs, [2*fixed-1; 2*fixed]);
26 load_n = find(nodes(:,1)==nelx & nodes(:,2)==0);
27 F = sparse(2*load_n, 1, -1, dofs, 1);
28
29 % Hàm độ cứng phần tử
30 KeFunc = @(a,l,c,s) (E*a/l)*[c^2 c*s -c^2 -c*s; c*s s^2 -c*s -s^2; ...
31                              -c^2 -c*s c^2 c*s; -c*s -s^2 c*s s^2];
32
33 %% 3. VÒNG LẶP TỐI ƯU (OPTIMALITY CRITERIA)
34 x = Vol*ones(nb,1); loop = 0; change = 1;
35 while change > 1e-6 && loop < 200
36     loop = loop + 1; xold = x;
37
38     % Phân tích phần tử hữu hạn
39     K = sparse(dofs, dofs);
40     for i=1:nb
41         idx = [2*bars(i,1)-1:2*bars(i,1), 2*bars(i,2)-1:2*bars(i,2)];
42         K(idx,idx) = K(idx,idx) + KeFunc(x(i), L(i), C(i), S(i));
43     end
44     U = zeros(dofs,1); U(free) = K(free,free) \ F(free);
45
46     % Tính độ nhạy
47     dc = zeros(nb,1);
48     for i=1:nb
49         idx = [2*bars(i,1)-1:2*bars(i,1), 2*bars(i,2)-1:2*bars(i,2)];
50         dc(i) = -U(idx)' * KeFunc(1, L(i), C(i), S(i)) * U(idx);
51     end
52
53     % Cập nhật biến thiết kế
54     l1=0; l2=1e9; move=0.2;
55     while (l2-l1)/(l1+l2) > 1e-6
56         lm = 0.5*(l1+l2); xnew = max(1e-9, max(x-move, min(1, min(x+move, x.*sqrt(-dc./(lm*L))))));
57         if sum(xnew.*L) > Vol*sum(L), l1=lm; else, l2=lm; end
58     end
59     change = max(abs(xnew-x)); x = xnew;
60     fprintf('Iter: %3d | Obj: %.4f | Vol: %.3f\n', loop, F'*U, sum(x.*L)/sum(L));
61 end
62
63 %% 4. HIỂN THỊ KẾT QUẢ
```

```
figure('Color','w'); axis equal off; hold on;
title(['Compliance: ' num2str(F'*U)]);
for i=1:nb
    if x(i) > 0.01 % Chỉ vẽ thanh có kích thước đáng kể
        plot(nodes(bars(i,:),1), nodes(bars(i,:),2), 'k', 'LineWidth', 5*x(i));
    end
end
plot(nodes(fixed,1), nodes(fixed,2), 'k^', 'MarkerFaceColor','k'); % Vẽ gối
```

Code 5.3: Giải Ví dụ 5.1 bằng thuật toán MMA và vẽ Hình 5.5

```
% ---------------------------------------------------------------
% TỐI ƯU HÓA CẤU TRÚC GIÀN DÙNG THUẬT TOÁN MMA
% ---------------------------------------------------------------

clear; clc; close all;

%% 1. CẤU HÌNH & TẠO LƯỚI
E = 1e7;   % Lưu ý: thang đo E, F khác Ví dụ 5.1 (E = 1, F = 1); hình dạng tối ưu
           % không đổi vì độ mềm được chuẩn hóa bởi C0, nhưng giá trị C in trên hình khác.
Vol_frac = 0.10;

xmin = 1e-3;
xmax = 1.0;

% Thông số lưới (8x4)
nelx = 8; nely = 4; dx = 100; dy = 100;

[X, Y] = meshgrid(0:dx:nelx*dx, 0:dy:nely*dy);
nodes = [X(:), Y(:)];
num_nodes = size(nodes, 1);

bars = [];
node_map = reshape(1:num_nodes, nely+1, nelx+1);
for i = 1:nelx
    for j = 1:nely
        nBL = node_map(j, i);   nBR = node_map(j, i+1);
        nTL = node_map(j+1, i); nTR = node_map(j+1, i+1);
        bars = [bars; nBL nBR; nTL nTR; nBL nTL; nBR nTR; nBL nTR; nTL nBR];
    end
end
bars = unique(sort(bars, 2), 'rows');
num_bars = size(bars, 1);

dxy = nodes(bars(:,2), :) - nodes(bars(:,1), :);
L = sqrt(sum(dxy.^2, 2));
C = dxy(:,1) ./ L;
S = dxy(:,2) ./ L;

GetKe = @(Area, len, c, s) (E * Area / len) * ...
    [ c^2  c*s -c^2 -c*s;  c*s  s^2 -c*s -s^2;
     -c^2 -c*s  c^2  c*s; -c*s -s^2  c*s  s^2];

%% 2. ĐIỀU KIỆN BIÊN & TẢI TRỌNG
total_dof = num_nodes * 2;

fixed_nodes = find(nodes(:,1) < 1e-6);          % biên trái ngàm (fix x,y)
fixed_dofs  = [2*fixed_nodes-1; 2*fixed_nodes];
free_dofs   = setdiff(1:total_dof, fixed_dofs(:));

load_node = find(nodes(:,1)==nelx*dx & nodes(:,2)==0);
F = zeros(total_dof, 1);
F(2*load_node) = -10000;

%% 3. THIẾT LẬP HIỂN THỊ SNAPSHOTS
max_iter = 500;

loop_snapshots = [1, 100, 200, 300, 400, 500];
figure('Name', 'Toi uu hoa gian (MMA) - Snapshots', ...
       'Color', 'w', 'Position', [50 50 1000 800]);

%% 4. THIẾT LẬP MMA
n = num_bars;   % số biến thiết kế
m = 1;          % 1 ràng buộc thể tích
```

```
64
65 x     = Vol_frac * xmax * ones(n,1);
66 xold1 = x;
67 xold2 = x;
68
69 low = xmin*ones(n,1);
70 upp = xmax*ones(n,1);
71
72 % Tham số MMA
73 a0 = 1;
74 a  = zeros(m,1);
75
76 % Hệ số phạt c lớn để ràng buộc thể tích được thỏa mãn chặt chẽ
77 c_mma = 1e5*ones(m,1);
78 d_mma = zeros(m,1);
79
80 V_limit = Vol_frac * sum(xmax * L);
81
82 % Lưu lịch sử
83 C_hist = nan(max_iter,1);
84 g_hist = nan(max_iter,1);
85
86 fprintf('Dang chay toi uu hoa (MMA - siet chat)...\n');
87
88 %% 5. VÒNG LẶP TỐI ƯU HÓA (MMA)
89 C0 = [];   % scale compliance
90
91 for iter = 1:max_iter
92     % --- PHÂN TÍCH FEM ---
93     K = sparse(total_dof, total_dof);
94     for i = 1:n
95         Ke = GetKe(x(i), L(i), C(i), S(i));
96         idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
97         K(idx, idx) = K(idx, idx) + Ke;
98     end
99
100    u = zeros(total_dof, 1);
101    u(free_dofs) = K(free_dofs, free_dofs) \ F(free_dofs);
102
103    compliance = F' * u;
104
105    if iter == 1
106        C0 = max(compliance, 1e-12);
107    end
108
109    C_hist(iter) = compliance;
110
111    % --- ĐỘ NHẠY MỤC TIÊU: dC/dx ---
112    dc = zeros(n,1);
113    for i = 1:n
114        Ke0 = GetKe(1, L(i), C(i), S(i));   % tuyến tính theo Area
115        idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
116        ue = u(idx);
117        dc(i) = -ue' * Ke0 * ue;
118    end
119
120    % --- RÀNG BUỘC THỂ TÍCH: g(x) <= 0 ---
121    g = sum(x.*L)/V_limit - 1;
122    g_hist(iter) = g;
123
124    dgdx = (L / V_limit)';   % 1 x n
125
126    % --- SCALE MỤC TIÊU cho MMA ---
127    f0val = compliance / C0;
128    df0dx = dc / C0;
129
130    fval = g;                % m x 1
131    dfdx = dgdx;             % m x n
132
133    % --- Giới hạn dịch chuyển 0.05 áp qua tiệm cận low/upp trước khi gọi MMA ---
134    % (biến thể của MMA chuẩn ở Chương 4: tiệm cận đặt cố định quanh x)
135    move = 0.05;
136    range = move*(xmax - xmin);
137    low = max(xmin, x - range);
138    upp = min(xmax, x + range);
```

```
139
140     % --- GỌI MMA SUBPROBLEM ---
141     [xmma,~,~,~,~,~,~,~,~,low,upp] = mmasub( ...
142         m,n,iter,x,xmin*ones(n,1),xmax*ones(n,1),xold1,xold2, ...
143         f0val,df0dx,fval,dfdx,low,upp,a0,a,c_mma,d_mma);
144
145     % --- Co x về miền khả dĩ nếu tổng thể tích vượt giới hạn ---
146     if sum(xmma.*L) > V_limit
147         xmma = xmma * (V_limit / sum(xmma.*L));
148         xmma = min(xmax, max(xmin, xmma));
149     end
150
151     % --- CẬP NHẬT ---
152     xold2 = xold1;
153     xold1 = x;
154     x = xmma;
155
156     change = max(abs(x - xold1));
157
158     % --- VẼ SNAPSHOT ---
159     if ismember(iter, loop_snapshots)
160         idx_in_list = find(loop_snapshots == iter);
161         subplot(3,2,idx_in_list);
162         hold on; axis equal; axis off;
163         title(sprintf('Vòng lặp: %d | C=%.2e', iter, compliance));
164
165         [~, sort_ids] = sort(x);
166         for k = 1:n
167             bar_id = sort_ids(k);
168             val = x(bar_id);
169             if val > 0.01
170                 n1 = nodes(bars(bar_id,1), :);
171                 n2 = nodes(bars(bar_id,2), :);
172
173                 dof_idx = [2*bars(bar_id,1)-1, 2*bars(bar_id,1), 2*bars(bar_id,2)-1, 2*bars(bar_id,2)];
174                 u_local = u(dof_idx);
175                 axial_def = (u_local(3) - u_local(1)) * C(bar_id) + ...
176                             (u_local(4) - u_local(2)) * S(bar_id);
177
178                 if axial_def > 1e-12
179                     col = 'b'; % kéo
180                 elseif axial_def < -1e-12
181                     col = 'r'; % nén
182                 else
183                     col = 'k';
184                 end
185
186                 width = 6 * val / xmax;
187                 plot([n1(1) n2(1)], [n1(2) n2(2)], '-', 'Color', col, 'LineWidth', width);
188             end
189         end
190         plot(nodes(fixed_nodes,1), nodes(fixed_nodes,2), 'ks', 'MarkerFaceColor', 'k', 'MarkerSize', 4);
191         drawnow;
192     end
193
194     fprintf('iter=%3d | C=%.3e | g=%.3e | change=%.3e\n', iter, compliance, g, change);
195
196     % --- TIÊU CHÍ DỪNG: vừa change nhỏ vừa ràng buộc gần 0 ---
197     if (change < 1e-6) && (abs(g) < 1e-6)
198         fprintf('Hoi tu! iter=%d | C=%.3e | g=%.3e\n', iter, compliance, g);
199         break;
200     end
201 end
202
203 iter_end = find(~isnan(C_hist), 1, 'last');
204 fprintf('Hoan thanh! Vong lap cuoi: %d, C: %.3e, g: %.3e\n', ...
205     iter_end, C_hist(iter_end), g_hist(iter_end));
```

Hai hàm `mmasub.m` và `subsolv.m` mà Code 5.3 gọi tới thuộc mã nguồn MMA của K. Svanberg (phiên bản 2007) [46] và không được in lại ở đây. Người đọc có thể lấy hai hàm này từ trang web của tác giả hoặc từ thư mục Code 5.3 trong kho mã của sách.

Code 5.4: Dữ liệu giàn phẳng 10 thanh (Bài 5.1)

```
function Truss_Data=TrussData_10bar
% ------------------------------------------------------------------------
% Tệp dữ liệu được lập theo định dạng của
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (Pa)
E=7e10;  % 70 GPa

% Node coordinates (m)
% L = 10 m
dnkoor=10*[2 1 0;2 0 0;1 1 0;1 0 0;0 1 0;0 0 0];

% Bar connections between nodes
% Bar 1: 5-3   Bar 2: 3-1   Bar 3: 6-4   Bar 4: 4-2
% Bar 5: 4-3   Bar 6: 2-1   Bar 7: 5-4   Bar 8: 6-3
% Bar 9: 3-2   Bar 10: 4-1
eldn=[5 3;3 1;6 4;4 2;4 3;2 1;5 4;6 3;3 2;4 1];

% Support conditions (1=fixed, 0=free) [x y z]
% Nodes 5 and 6 are fixed supports
MesKos=[0 0 1;0 0 1;0 0 1;0 0 1;1 1 1;1 1 1];

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Downward loads at nodes 2 and 4
yuk=zeros(size(dnkoor));
yuk(2,:)=[0 -500000 0];  % 500 kN
yuk(4,:)=[0 -500000 0];  % 500 kN

% Allowable Stress (Pa)
maxGER=1.75e8;  % 175 MPa

% Allowed Displacement (m)
maxYER=0.05;  % 50 mm

% Khối lượng riêng (kg/m^3)
GS=2770;

% Design variable bounds (m^2)
LB=ones(1,10)*1e-4;   % 1 cm^2
UB=ones(1,10)*50e-4;   % 50 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk',yuk,'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
    'LB',LB,'UB',UB);
```

Code 5.5: Dữ liệu giàn phẳng 17 thanh (Bài 5.2)

```
function Truss_Data=TrussData_17bar
% ------------------------------------------------------------------------
% Nguồn: chuyển thể (quy đổi đơn vị, bổ sung chú thích và dữ liệu) từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (Pa)
E=2.1e11;  % 210 GPa (steel)

% Node coordinates (m)
% Unit spacing = 2.5 m
dnkoor=2.5*[0 0 0; 0 1 0; 1 0 0; 1 1 0; 2 0 0; 2 1 0; 3 0 0; 3 1 0; 4 0 0];

% Bar connections between nodes
```

```
eldn=[2 4;2 3;1 3;3 4; 4 6;4 5;3 5;5 6;6 8;6 7;5 7;7 8;8 9;7 9;1 4;3 6;5 8];

% Support conditions (1=fixed, 0=free) [x y z]
% Nodes 1 and 2 are fixed supports
MesKos=[1 1 1;1 1 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1];

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
yuk=zeros(size(dnkoor));
yuk(9,:)=[0 -500000 0];  % 500 kN

% Allowable Stress (Pa)
maxGER=3.5e8;  % 350 MPa

% Allowed Displacement (m)
maxYER=0.05;  % 50 mm

% Khối lượng riêng (kg/m^3)
GS=7400;  % steel

% Design variable bounds (m^2)
LB=ones(1,17)*1e-4;     % 1 cm^2
UB=ones(1,17)*300e-4;  % 300 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk',yuk,'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
    'LB',LB,'UB',UB);
```

Code 5.6: Dữ liệu giàn không gian 25 thanh (Bài 5.3)

```
function Truss_Data=TrussData_25bar
% ------------------------------------------------------------------------
% Nguồn: chuyển thể (quy đổi đơn vị, bổ sung chú thích và dữ liệu) từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (Pa)
E=7e10;  % 70 GPa (aluminum)
```

... (dòng 16–55: tọa độ nút và liên kết thanh; xem tệp đầy đủ trên GitHub)

```
% Bar grouping (8 groups, 8 design variables)
% Group 1: bar 1          (top cross)
% Group 2: bars 2-5       (top-to-mid diagonal)
% Group 3: bars 6-9       (top-to-mid diagonal)
% Group 4: bars 10-11     (mid horizontal)
% Group 5: bars 12-13     (mid horizontal)
% Group 6: bars 14-17     (mid-to-base diagonal)
% Group 7: bars 18-21     (mid-to-base diagonal)
% Group 8: bars 22-25     (mid-to-base vertical)
groups={[1],[2:5],[6:9],[10:11],[12:13],[14:17],[18:21],[22:25]};

% Support conditions (1=fixed, 0=free) [x y z]
% Nodes 7-10 are fixed supports
MesKos=[0 0 0;0 0 0;0 0 0;0 0 0;0 0 0;0 0 0;1 1 1;1 1 1;1 1 1;1 1 1];

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Loading #1
yuk1=zeros(size(dnkoor));
yuk1(1,:)=[0      90000 -25000];   % 90 kN, -25 kN
yuk1(2,:)=[0     -90000 -25000];   % -90 kN, -25 kN

% Loading #2
yuk2=zeros(size(dnkoor));
yuk2(1,:)=[5000   45000 -25000];   % 5 kN, 45 kN, -25 kN
yuk2(2,:)=[0      45000 -25000];   % 45 kN, -25 kN
yuk2(3,:)=[2500   0       0];      % 2.5 kN
yuk2(6,:)=[2500   0       0];      % 2.5 kN
```

```
% Allowable Stress (Pa) - per group (recalculated for SI coordinates)
maxBAS=[220; 78; 121; 220; 220; 48; 49; 82]*1e6;  % MPa (compression)
maxCEK=275e6;  % 275 MPa (tension)

% Allowed Displacement (m)
maxYER=0.009;  % 9 mm

% Khối lượng riêng (kg/m^3)
GS=2770;  % aluminum

% Design variable bounds (m^2)
LB=ones(1,8)*0.1e-4;    % 0.1 cm^2
UB=ones(1,8)*22e-4;     % 22 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk1',yuk1,'yuk2',yuk2,...
    'maxBAS',maxBAS','maxCEK',maxCEK,'maxYER',maxYER,'GS',GS,...
```

Code 5.7: Dữ liệu giàn không gian 72 thanh (Bài 5.4)

```
function Truss_Data=TrussData_72bar
% ------------------------------------------------------------------------
% Nguồn: chuyển thể (quy đổi đơn vị, bổ sung chú thích và dữ liệu) từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (Pa)
E=7e10;  % 70 GPa (aluminum)
```

... (dòng 16–113: tọa độ nút và liên kết thanh; xem tệp đầy đủ trên GitHub)

```
% Bar grouping (16 groups, 16 design variables)
% Group 1:  bars 1-4     (story 1 vertical)
% Group 2:  bars 5-12    (story 1 diagonal)
% Group 3:  bars 13-16   (story 1 horizontal)
% Group 4:  bars 17-18   (story 1 cross)
% Group 5:  bars 19-22   (story 2 vertical)
% Group 6:  bars 23-30   (story 2 diagonal)
% Group 7:  bars 31-34   (story 2 horizontal)
% Group 8:  bars 35-36   (story 2 cross)
% Group 9:  bars 37-40   (story 3 vertical)
% Group 10: bars 41-48   (story 3 diagonal)
% Group 11: bars 49-52   (story 3 horizontal)
% Group 12: bars 53-54   (story 3 cross)
% Group 13: bars 55-58   (story 4 vertical)
% Group 14: bars 59-66   (story 4 diagonal)
% Group 15: bars 67-70   (story 4 horizontal)
% Group 16: bars 71-72   (story 4 cross)
groups={[1:4],[5:12],[13:16],[17:18],...
        [19:22],[23:30],[31:34],[35:36],...
        [37:40],[41:48],[49:52],[53:54],...
        [55:58],[59:66],[67:70],[71:72]};
```

... (dòng 136–158: điều kiện biên; xem tệp đầy đủ trên GitHub)

```
% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Loading #1
yuk1=zeros(size(dnkoor));
yuk1(17,:)=[25000 25000 -25000];  % 25 kN each direction

% Loading #2
yuk2=zeros(size(dnkoor));
yuk2(17,:)=[0 0 -25000];  % 25 kN downward
yuk2(18,:)=[0 0 -25000];
yuk2(19,:)=[0 0 -25000];
yuk2(20,:)=[0 0 -25000];

% Allowable Stress (Pa)
```

```
maxGER=1.75e8;  % 175 MPa

% Allowed Displacement (m)
maxYER=0.006;  % 6 mm

% Khối lượng riêng (kg/m^3)
GS=2770;  % aluminum

% Design variable bounds (m^2)
LB=ones(1,16)*1e-4;     % 1 cm^2
UB=ones(1,16)*130e-4;  % 130 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk1',yuk1,'yuk2',yuk2,...
    'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
    'groups',{groups},'LB',LB,'UB',UB);
```

Code 5.8: Dữ liệu giàn phẳng 200 thanh (Bài 5.5)

```
function Truss_Data=TrussData_200bar
% ------------------------------------------------------------------------
% Nguồn: chuyển thể (quy đổi đơn vị, bổ sung chú thích và dữ liệu) từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (Pa)
E=2.1e11;  % 210 GPa (steel)
```

... (dòng 16–339: tọa độ nút, liên kết thanh và 29 nhóm thanh; xem tệp đầy đủ trên GitHub)

```
% Support conditions (1=fixed, 0=free) [x y z]
% All nodes: z fixed (planar truss), nodes 76-77 fully fixed
MesKos = zeros(77,3);
MesKos(:,3) = 1;            % z fixed for all (planar)
MesKos(76,:) = [1 1 1];     % support
MesKos(77,:) = [1 1 1];     % support

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Loading #1: horizontal loads at left-edge nodes
yuk1=zeros(size(dnkoor));
left_nodes = [1, 6, 15, 20, 29, 34, 43, 48, 57, 62, 71];
for i = 1:length(left_nodes)
    yuk1(left_nodes(i),:) = [5000 0 0];  % 5 kN horizontal
end

% Loading #2: vertical loads at most nodes
yuk2=zeros(size(dnkoor));
loaded_nodes = [1,2,3,4,5,6,8,10,12,14,15,16,17,18,19,...
    20,22,24,26,28,29,30,31,32,33,...
    34,36,38,40,42,43,44,45,46,47,...
    48,50,52,54,56,57,58,59,60,61,...
    62,64,66,68,70,71,72,73,74,75];
for i = 1:length(loaded_nodes)
    yuk2(loaded_nodes(i),:) = [0 -45000 0];  % 45 kN downward
end

% Loading #3: combined
yuk3 = yuk1 + yuk2;

% Allowable Stress (Pa)
maxGER=7e7;  % 70 MPa

% Khối lượng riêng (kg/m^3)
GS=7850;  % steel

% Design variable bounds (m^2)
LB=ones(1,29)*1e-4;     % 1 cm^2
```

```
UB=ones(1,29)*130e-4;  % 130 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk1',yuk1,'yuk2',yuk2,'yuk3',yuk3,...
    'maxGER',maxGER,'GS',GS,...
    'groups',{groups},'LB',LB,'UB',UB);
```

Code 5.9: Dữ liệu giàn 18 thanh (Bài 5.6)

```
function Truss_Data=TrussData_18bar
% ------------------------------------------------------------------------
% Nguồn: chuyển thể (quy đổi đơn vị, bổ sung chú thích và dữ liệu) từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (Pa)
E=6.9e10; % 69 GPa (~1e7 psi)
```

. . . (dòng 16–49: liên kết thanh và điều kiện biên; xem tệp đầy đủ trên GitHub)

```
% Loading     ----- > Load(dn_no,:)=[x y z] (N)
yuk=zeros(size(MesKos));
yuk(1,:)=[0 -89000 0]; % 89 kN (~20000 lbs)
yuk(2,:)=[0 -89000 0];
yuk(4,:)=[0 -89000 0];
yuk(6,:)=[0 -89000 0];
yuk(8,:)=[0 -89000 0];

% Allowable Stress (Pa)
maxGER=1.38e8; % 138 MPa (~20000 psi)

% Allowed Displacement (m)
maxYER=0.025; % 25 mm (~1.0 in)

% Khối lượng riêng (kg/m^3)
GS=2770; % 2770 kg/m^3 (~0.1 lb/in^3)

% Design variable bounds
LB=ones(1,12);
LB(1,1:4)=LB(1,1:4)*22 * 1e-4; % 22 cm2 (m2)
LB(1,5)=LB(1,5)*20; %m (x3)
LB(1,6)=LB(1,6)*-6; %m (y3)
LB(1,7)=LB(1,7)*13; %m (x5)
LB(1,8)=LB(1,8)*-6; %m (y5)
LB(1,9)=LB(1,9)*7; %m (x7)
LB(1,10)=LB(1,10)*-6; %m (y7)
LB(1,11)=LB(1,11)*1; %m (x9)
LB(1,12)=LB(1,12)*-6; %m (y9)

UB=ones(1,12);
UB(1,1:4)=UB(1,1:4)*116 * 1e-4; % 116 cm2 (m2)
UB(1,5)=UB(1,5)*31; %m (x3)
UB(1,6)=UB(1,6)*6; %m (y3)
UB(1,7)=UB(1,7)*25; %m (x5)
UB(1,8)=UB(1,8)*6; %m (y5)
UB(1,9)=UB(1,9)*18; %m (x7)
UB(1,10)=UB(1,10)*6; %m (y7)
UB(1,11)=UB(1,11)*12; %m (x9)
UB(1,12)=UB(1,12)*6; %m (y9)

% Base node coordinates (m)
% Coordinates for nodes 3,5,7,9 will be overwritten by optimizer
dnkoor = [
    31.75  6.35 0; % Node 1
    25.40  6.35 0; % Node 2
    19.69 -5.72 0; % Node 3 (var x3, y3 - using default initial limit)
    19.05  6.35 0; % Node 4
    13.34 -5.72 0; % Node 5 (var x5, y5)
```

```
    12.70  6.35 0; % Node 6
     6.99 -5.72 0; % Node 7 (var x7, y7)
     6.35  6.35 0; % Node 8
     0.64 -5.72 0; % Node 9 (var x9, y9)
     0.00  6.35 0; % Node 10
     0.00  0.00 0  % Node 11
];

% Bar grouping (4 areas, 8 node coordinates) -> 12 Design variables
% Area 1: bars 1,4,8,12,16 (Top chord)
% Area 2: bars 2,6,10,14,18 (Bottom chord)
% Area 3: bars 3,7,11,15 (Diagonals / verticals)
% Area 4: bars 5,9,13,17 (Diagonals)
groups={[1,4,8,12,16],[2,6,10,14,18],[3,7,11,15],[5,9,13,17]};

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,'E',E,...
    'yuk',yuk,'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
    'LB',LB,'UB',UB,'groups',{groups});
```

Chương 6. Tối ưu cấu trúc

Code 6.1: Chương trình tối ưu cấu trúc SIMP dựa trên mã 99 dòng của Sigmund [42] (Ví dụ 6.1, Hình 6.4)

```
% ------------------------------------------------------------------------
% Chương trình dựa trên mã 99 dòng của O. Sigmund: "A 99 line topology
% optimization code written in Matlab", Structural and Multidisciplinary
% Optimization 21(2), 120-127, 2001 (mã gốc của tác giả, DTU, phổ biến cho
% mục đích giáo dục). Bản này được nhóm tác giả sách chỉnh sửa và chú thích.
% ------------------------------------------------------------------------
% --- DONG LENH CHAY O TREN CUNG ---
top99(60,20,0.5,3.0,1.5,1)
% ---------------------------------

% Dinh nghia cac function o ben duoi
function top99(nelx,nely,volfrac,penal,rmin,nLoadCase)

% KHOI TAO
x(1:nely,1:nelx) = volfrac;
loop = 0;
change = 1;
passive = [];

% BAT DAU VONG LAP TOI UU
while change > 0.01
    loop = loop + 1;
    xold = x;

    % Phan tich phan tu huu han
    [U] = FE(nelx,nely,x,penal,nLoadCase);

    % Tinh toan ham muc tieu
    [KE] = localK;
    c = 0;
    dc = zeros(nely,nelx);
    for ely = 1:nely
        for elx = 1:nelx
            n1 = (nely+1)*(elx-1)+ely;
            n2 = (nely+1)* elx   +ely;
            Ue = U([2*n1-1;2*n1; 2*n2-1;2*n2; 2*n2+1;2*n2+2; 2*n1+1;2*n1+2],1);
            c = c + x(ely,elx)^penal*Ue'*KE*Ue;
            dc(ely,elx) = -penal*x(ely,elx)^(penal-1)*Ue'*KE*Ue;
        end
    end

    % Loc do nhay
    [dc] = check(nelx,nely,rmin,x,dc);

    % Cap nhat bien thiet ke
    [x] = OC(nelx,nely,x,volfrac,dc,passive);

    % In ket qua
    change = max(max(abs(x-xold)));
```

```
    disp([' It.: ' sprintf('%4i',loop) ' Obj.: ' sprintf('%10.4f',c) ...
          ' Vol.: ' sprintf('%6.3f',sum(sum(x))/(nelx*nely)) ...
          ' ch.: ' sprintf('%6.3f',change )])

    % Ve bieu do
    colormap(gray); imagesc(-x); axis equal; axis tight; axis off;drawnow;
end
end

%%%%%%%%% CAC HAM CON (LOCAL FUNCTIONS) %%%%%%%%%

function [dcn] = check(nelx,nely,rmin,x,dc)
dcn = zeros(nely,nelx);
for i = 1:nelx
  for j = 1:nely
    sum = 0;
    for k = max(i-floor(rmin),1):min(i+floor(rmin),nelx)
      for l = max(j-floor(rmin),1):min(j+floor(rmin),nely)
        fac = rmin-sqrt((i-k)^2+(j-l)^2);
        sum = sum+max(0,fac);
        dcn(j,i) = dcn(j,i) + max(0,fac)*x(l,k)*dc(l,k);
      end
    end
    dcn(j,i) = dcn(j,i)/(x(j,i)*sum);
  end
end
end

function [U] = FE(nelx,nely,x,penal,nLoadCase)
[KE] = localK;
K = sparse(2*(nelx+1)*(nely+1), 2*(nelx+1)*(nely+1));
F = sparse(2*(nely+1)*(nelx+1),nLoadCase);
U = sparse(2*(nely+1)*(nelx+1),nLoadCase);

for elx = 1:nelx
    for ely = 1:nely
        n1 = (nely+1)*(elx-1)+ely;
        n2 = (nely+1)* elx   +ely;
        edof = [2*n1-1; 2*n1; 2*n2-1; 2*n2; 2*n2+1; 2*n2+2; 2*n1+1; 2*n1+2];
        K(edof,edof) = K(edof,edof) + x(ely,elx)^penal*KE;
    end
end

% DIEU KIEN BIEN (MBB Beam)
F(2,1) = -1;
fixeddofs = union([1:2:2*(nely+1)],[2*(nelx+1)*(nely+1)]);

alldofs = [1:2*(nely+1)*(nelx+1)];
freedofs = setdiff(alldofs,fixeddofs);

U(freedofs,:) = K(freedofs,freedofs) \ F(freedofs,:);
U(fixeddofs,:) = 0;
end

function [KE] = localK
E = 1.;
nu = 0.3;
k = [ 1/2-nu/6   1/8+nu/8 -1/4-nu/12 -1/8+3*nu/8 ...
     -1/4+nu/12 -1/8-nu/8  nu/6       1/8-3*nu/8 ];
KE = E/(1-nu^2)*[ k(1) k(2) k(3) k(4) k(5) k(6) k(7) k(8)
                  k(2) k(1) k(8) k(7) k(6) k(5) k(4) k(3)
                  k(3) k(8) k(1) k(6) k(7) k(4) k(5) k(2)
                  k(4) k(7) k(6) k(1) k(8) k(3) k(2) k(5)
                  k(5) k(6) k(7) k(8) k(1) k(2) k(3) k(4)
                  k(6) k(5) k(4) k(3) k(2) k(1) k(8) k(7)
                  k(7) k(4) k(5) k(2) k(3) k(8) k(1) k(6)
                  k(8) k(3) k(2) k(5) k(4) k(7) k(6) k(1)];
end

function [xnew] = OC(nelx,nely,x,volfrac,dc,passive)
l1 = 0; l2 = 100000; move = 0.1;
while (l2-l1 > 1e-4)
    lmid = 0.5*(l2+l1);
    xnew = max(0.001,max(x-move,min(1,min(x+move,x.*sqrt(-dc./lmid)))));
    if ~isempty(passive)
```

```
        xnew(passive==1) = 0.001;
    end
    if sum(sum(xnew)) - volfrac*nelx*nely > 0
        l1 = lmid;
    else
        l2 = lmid;
    end
end
end
```

Chương 7. Tối ưu hóa đa mục tiêu

Code 7.2: Vẽ Hình 7.2 (Python)

```python
import matplotlib.pyplot as plt

# ===== Font math (Computer Modern) =====
plt.rcParams.update({
    "mathtext.fontset": "cm",
    "font.family": "serif",
})

# ===== Figure =====
fig, ax = plt.subplots(figsize=(6.8, 3.8))

ax.set_xlim(0, 10)
ax.set_ylim(0, 6)

# Ẩn khung và ticks
for s in ax.spines.values():
    s.set_visible(False)
ax.set_xticks([])
ax.set_yticks([])

# ===== Trục tọa độ =====
origin = (0, 0)
arrow_style = dict(
    arrowstyle="-|>",
    mutation_scale=12,
    lw=1.4,
    color="gray"
)

ax.annotate("", xy=(9.7, 0), xytext=origin, arrowprops=arrow_style)
ax.annotate("", xy=(0, 5.7), xytext=origin, arrowprops=arrow_style)

# Nhãn trục
ax.text(4.85, -0.55, r"$f_1$", fontsize=16, color="gray")
ax.text(-0.55, 2.85, r"$f_2$", fontsize=16, color="gray")

# ===== Dữ liệu điểm minh họa =====
# dominated: các điểm bị chi phối (vòng tròn rỗng); pareto: tập Pareto (điểm tô đen)
dominated = [
    (1.5, 4.6), (2.2, 4.6), (3.1, 5.0), (3.8, 4.7),
    (5.0, 5.0), (6.1, 4.8), (7.0, 5.2),

    (2.8, 3.8), (3.6, 3.9), (4.5, 4.2),
    (5.3, 3.8), (6.4, 4.1), (7.2, 4.0),

    (2.6, 2.9), (3.3, 3.1), (4.2, 3.0),
    (5.8, 3.1), (6.6, 3.0), (7.3, 3.2),

    (4.8, 2.2), (6.9, 2.4), (7.4, 2.0),
    (8.0, 1.6), (9.0, 2.2),
]

pareto = [
    (0.7, 5.2),
    (1.0, 3.9), (1.3, 3.6),
    (2.6, 2.5), (3.1, 2.3),
    (4.6, 1.7),
    (6.2, 1.0),
    (7.3, 0.9),
```

```
    (8.6, 0.8),
]

bx, by = zip(*dominated)
rx, ry = zip(*pareto)

# ===== Scatter =====
ax.scatter(bx, by, s=28, facecolors="none", edgecolors="black", linewidths=0.8)
ax.scatter(rx, ry, s=28, color="black", edgecolors="black")

# ===== Giữ tỷ lệ trục vuông =====
ax.set_aspect("equal")

import os
script_dir = os.path.dirname(os.path.abspath(__file__))

# ===== Export PNG độ phân giải cao =====
plt.savefig(
    os.path.join(script_dir, "pareto_scatter.png"),
    dpi=600,
    bbox_inches="tight",
    pad_inches=0.05
)

plt.show()
```

Code 7.3: Giải Ví dụ 7.1 và vẽ Hình 7.4 (Python)

```
"""
============================================================
Ví dụ 7.1 - Tối ưu đa mục tiêu giàn hai thanh
Phương pháp tổng trọng số (Weighted-Sum) → Biên Pareto
============================================================
Hai mục tiêu:
  f1 = khối lượng giàn (kg)
  f2 = |v_C| - chuyển vị đứng tại nút C (mm)

Ràng buộc (viết ở dạng chuẩn hóa, không thứ nguyên):
  g_i = 1 - |σ_i|/σ_allow ≥ 0   (i = 1, 2)
  A_min ≤ A_i ≤ A_max

Biến thiết kế: x = [A1, A2]  (mm²)
  (dùng mm² và ràng buộc chuẩn hóa để các đại lượng cùng bậc độ lớn,
   tránh SLSQP hội tụ sai do chênh lệch thang đo Pa ~1e8 / m² ~1e-4)
============================================================
"""
import sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")

import numpy as np
from scipy.optimize import minimize
import matplotlib
matplotlib.use("Agg")                    # backend không cần GUI
import matplotlib.pyplot as plt
matplotlib.rcParams.update({
    "font.size": 12,
    "font.family": "serif",
    "axes.grid": True,
    "grid.alpha": 0.3,
})

# ============================================================
# 1. DỮ LIỆU BÀI TOÁN
# ============================================================
L   = 2.0           # m   - khoảng cách AB
h   = 1.0           # m   - chiều cao nút C
P   = 50e3          # N   - tải trọng
E   = 200e9         # Pa  - mô-đun đàn hồi
rho = 7850          # kg/m³ - khối lượng riêng
sig_allow = 150e6   # Pa  - ứng suất cho phép
A_min = 200.0       # mm²
A_max = 4000.0      # mm²

# Tọa độ nút (m)
node_A = np.array([0.0, 0.0])
```

```python
node_B = np.array([L,   0.0])
node_C = np.array([L/2, h  ])

# Chiều dài thanh
L1 = np.linalg.norm(node_C - node_A)   # |AC|
L2 = np.linalg.norm(node_C - node_B)   # |BC|

# Cosin chỉ phương (từ gối đến nút C)
c1 = (node_C[0] - node_A[0]) / L1
s1 = (node_C[1] - node_A[1]) / L1
c2 = (node_C[0] - node_B[0]) / L2
s2 = (node_C[1] - node_B[1]) / L2

print(f"Chiều dài thanh: L1 = {L1:.4f} m, L2 = {L2:.4f} m")
print(f"Thanh AC: cosθ = {c1:.4f}, sinθ = {s1:.4f}")
print(f"Thanh BC: cosθ = {c2:.4f}, sinθ = {s2:.4f}")
print()

# ============================================================
# 2. PHÂN TÍCH KẾT CẤU (Phương pháp độ cứng trực tiếp)
# ============================================================
def analyze(A1, A2):
    """
    Phân tích giàn hai thanh bằng phương pháp độ cứng.
    A1, A2 : diện tích tiết diện (m²)
    Trả về: (u_C, v_C, σ1, σ2)
      u_C, v_C : chuyển vị nút C theo x, y (m)
      σ1, σ2   : ứng suất dọc trong thanh AC, BC (Pa)
    """
    # Ma trận độ cứng phần tử (2×2, đã rút gọn về DOF nút C)
    k1 = (E * A1 / L1) * np.array([[c1**2, c1*s1],
                                   [c1*s1, s1**2]])
    k2 = (E * A2 / L2) * np.array([[c2**2, c2*s2],
                                   [c2*s2, s2**2]])
    # Ma trận độ cứng tổng thể
    K = k1 + k2

    # Vector tải
    F = np.array([0.0, -P])

    # Giải hệ phương trình: K · d = F
    d = np.linalg.solve(K, F)
    u_C, v_C = d

    # Biến dạng dọc → ứng suất
    delta1 = c1 * u_C + s1 * v_C          # biến dạng thanh AC
    sigma1 = E * delta1 / L1

    delta2 = c2 * u_C + s2 * v_C          # biến dạng thanh BC
    sigma2 = E * delta2 / L2

    return u_C, v_C, sigma1, sigma2

# ============================================================
# 3. HÀM MỤC TIÊU VÀ RÀNG BUỘC
# ============================================================
# Biến thiết kế x tính bằng mm²; đổi sang m² trước khi phân tích
def f1_mass(x):
    """Khối lượng giàn (kg)."""
    A1, A2 = np.asarray(x) * 1e-6      # mm² → m²
    return rho * (L1 * A1 + L2 * A2)

def f2_disp(x):
    """Chuyển vị đứng tuyệt đối tại C (mm)."""
    A1, A2 = np.asarray(x) * 1e-6
    _, v_C, _, _ = analyze(A1, A2)
    return abs(v_C) * 1e3              # đổi m → mm

def stress_con1(x):
    """g1 = 1 - |σ1|/σ_allow ≥ 0  (dạng chuẩn hóa)"""
    A1, A2 = np.asarray(x) * 1e-6
    _, _, sig1, _ = analyze(A1, A2)
    return 1.0 - abs(sig1) / sig_allow

def stress_con2(x):
```

```
    """g2 = 1 - |σ2|/σ_allow ≥ 0  (dạng chuẩn hóa)"""
    A1, A2 = np.asarray(x) * 1e-6
    _, _, _, sig2 = analyze(A1, A2)
    return 1.0 - abs(sig2) / sig_allow

# ============================================================
# 4. CHUẨN HÓA MỤC TIÊU (utopia-nadir)
# ============================================================
bounds = [(A_min, A_max), (A_min, A_max)]
cons   = [{"type": "ineq", "fun": stress_con1},
          {"type": "ineq", "fun": stress_con2}]
x0     = np.array([1000.0, 1000.0])    # điểm xuất phát (mm²)

# Tối ưu riêng f1
res1 = minimize(f1_mass, x0, method="SLSQP",
                bounds=bounds, constraints=cons,
                options={"ftol": 1e-12, "maxiter": 500})
f1_min = f1_mass(res1.x)
f2_at_f1min = f2_disp(res1.x)

# Tối ưu riêng f2
res2 = minimize(f2_disp, x0, method="SLSQP",
                bounds=bounds, constraints=cons,
                options={"ftol": 1e-12, "maxiter": 500})
f2_min = f2_disp(res2.x)
f1_at_f2min = f1_mass(res2.x)

# Giá trị nadir (cận trên) cho chuẩn hóa
f1_max = f1_at_f2min
f2_max = f2_at_f1min

def max_stress_MPa(x):
    A1, A2 = np.asarray(x) * 1e-6
    _, _, s1, s2 = analyze(A1, A2)
    return max(abs(s1), abs(s2)) / 1e6

print("-- Kết quả tối ưu đơn mục tiêu --")
print(f"  min f1 → A = ({res1.x[0]:.1f}, {res1.x[1]:.1f}) mm², "
      f"f1 = {f1_min:.3f} kg, f2 = {f2_at_f1min:.4f} mm, "
      f"σmax = {max_stress_MPa(res1.x):.1f} MPa")
print(f"  min f2 → A = ({res2.x[0]:.1f}, {res2.x[1]:.1f}) mm², "
      f"f1 = {f1_at_f2min:.3f} kg, f2 = {f2_min:.4f} mm, "
      f"σmax = {max_stress_MPa(res2.x):.1f} MPa")
print(f"  Chuẩn hóa: f1 ∈ [{f1_min:.3f}, {f1_max:.3f}] kg")
print(f"             f2 ∈ [{f2_min:.4f}, {f2_max:.4f}] mm\n")

# ============================================================
# 5. GIẢI BẰNG PHƯƠNG PHÁP TỔNG TRỌNG SỐ
# ============================================================
def weighted_obj(x, w):
    """F(x;w) = w·f1_norm + (1-w)·f2_norm  (f chuẩn hóa)"""
    f1_norm = (f1_mass(x) - f1_min) / (f1_max - f1_min + 1e-30)
    f2_norm = (f2_disp(x) - f2_min) / (f2_max - f2_min + 1e-30)
    return w * f1_norm + (1 - w) * f2_norm

# Quét w từ 0 đến 1
n_pts = 51                                  # nhiều điểm hơn cho đường mượt
weights = np.linspace(0, 1, n_pts)

results = []                                # [(w, A1, A2, f1, f2)]
for w in weights:
    res = minimize(weighted_obj, x0, args=(w,), method="SLSQP",
                   bounds=bounds, constraints=cons,
                   options={"ftol": 1e-12, "maxiter": 500})
    A1_opt, A2_opt = res.x
    f1_val = f1_mass(res.x)
    f2_val = f2_disp(res.x)
    results.append((w, A1_opt, A2_opt, f1_val, f2_val))

results = np.array(results)

# Kiểm tra khả dĩ: mọi nghiệm phải thỏa |σ_i| ≤ σ_allow
sig_max_all = max(max_stress_MPa(r[1:3]) for r in results)
print(f"Ứng suất lớn nhất trên toàn biên Pareto: {sig_max_all:.2f} MPa "
      f"(σ_allow = {sig_allow/1e6:.0f} MPa)\n")
```

```

# In bảng kết quả (mỗi bước 0.1)
print("-- Bảng kết quả Pareto --")
print(f"{'w':>5s} {'A1(mm²)':>10s} {'A2(mm²)':>10s} "
      f"{'f1(kg)':>10s} {'f2(mm)':>10s}")
print("-" * 50)
for row in results:
    w, A1, A2, f1, f2 = row
    if np.isclose(w * 10 % 1, 0, atol=0.02) or w == weights[-1]:
        print(f"{w:5.2f} {A1:10.1f} {A2:10.1f} "
              f"{f1:10.3f} {f2:10.4f}")
print()

# ============================================================
# 6. VẼ ĐỒ THỊ
# ============================================================
fig, ax = plt.subplots(figsize=(8, 6))

# -- Biên Pareto ------------------------------------
f1_arr = results[:, 3]
f2_arr = results[:, 4]
sc = ax.scatter(f1_arr, f2_arr, c=results[:, 0], cmap="Greys",
                edgecolors="k", s=80, zorder=5)
ax.plot(f1_arr, f2_arr, "k--", lw=0.8, alpha=0.5)


cbar = fig.colorbar(sc, ax=ax, pad=0.02)
cbar.set_label("Trọng số w")
ax.set_xlabel("$f_1$ - Khối lượng (kg)")
ax.set_ylabel("$f_2$ - Chuyển vị $|v_C|$ (mm)")

plt.tight_layout()
import os
script_dir = os.path.dirname(os.path.abspath(__file__))
save_path = os.path.join(script_dir, "pareto_front_two_bar_truss.png")
plt.savefig(save_path, dpi=600, bbox_inches="tight")

print(f"✓ Đồ thị đã lưu: {save_path}")

# ============================================================
# 7. NHẬN XÉT BIÊN PARETO
# ============================================================
print("""
-- NHẬN XÉT --
• Đường Pareto cho thấy mối quan hệ đánh đổi giữa khối lượng
  và chuyển vị: giảm khối lượng → tăng chuyển vị và ngược lại.
• Vùng w nhỏ (ưu tiên chuyển vị): tăng thêm khối lượng chỉ
  giảm rất ít chuyển vị → "bão hòa" về chuyển vị.
• Vùng w lớn (ưu tiên khối lượng): giảm thêm một ít khối lượng
  sẽ khiến chuyển vị tăng mạnh → "bão hòa" về khối lượng.
• Vùng giữa (w ≈ 0.3-0.6): mỗi đơn vị khối lượng tiết kiệm
  được "đổi" lượng chuyển vị tăng hợp lý → vùng thiết kế hiệu quả.
• Phương án thiết kế đề xuất: chọn điểm "đầu gối" (knee point)
  trên đường Pareto, thường ở w ≈ 0.4-0.5, cân bằng tốt nhất
  giữa hai mục tiêu.
""")
```

Chương 8. Thuật toán tối ưu metaheuristic

Code 8.1: Giải Ví dụ 8.1 bằng thuật toán luyện kim mô phỏng (Python)

```
"""
Ví dụ 8.1: Tối ưu hóa dầm công-xôn bằng thuật toán luyện kim mô phỏng (SA)

Mô tả bài toán:
  - Dầm công-xôn dài L = 3m, chia thành 5 đoạn, mỗi đoạn dài 0.6m
  - Mặt cắt ngang hình tròn rỗng (r_outer, r_inner)
  - Vật liệu: Thép S270 (E = 210 GPa, σ_yield = 270 MPa)
  - Lực tập trung F = 500 kN tại đầu tự do

Mục tiêu:
  - Tối thiểu hóa Khối lượng dầm

```

```
Biến thiết kế (10 biến):
  - r_outer[1..5]: bán kính ngoài của 5 đoạn, 1 ≤ r_outer ≤ 30 cm (≤ L/10)
  - r_inner[1..5]: bán kính trong của 5 đoạn (cm)

Ràng buộc:
  1. Chuyển vị tại đầu dầm ≤ 2 cm
  2. r_outer[i] > r_inner[i]  (mặt cắt hợp lệ)
  3. r_outer[i] > r_inner[i+1] và r_outer[i+1] > r_inner[i]  (thành ống chồng lấn → hàn được)
  4. Ứng suất max < σ_yield = 270 MPa
  5. Bán kính ngoài ≤ L/10 = 30 cm
  6. Chiều dày thành ống: 0.1 cm ≤ (r_outer - r_inner) ≤ 5 cm
"""

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from scipy.interpolate import CubicSpline
import os
from matplotlib.ticker import FuncFormatter

# Nhãn trên hình dùng dấu phẩy thập phân theo quy ước của sách (chỉ ảnh hưởng hiển thị)
def vn(v, nd=2):
    return f"{v:.{nd}f}".replace('.', ',')

VN_TICKS = FuncFormatter(lambda v, pos: f"{v:g}".replace('.', ','))

# ============================================================
# 1. THÔNG SỐ BÀI TOÁN
# ============================================================

L = 3.0                    # Chiều dài dầm (m)
NUM_SEGMENTS = 5           # Số đoạn
SEG_LEN = L / NUM_SEGMENTS  # Chiều dài mỗi đoạn (m)

E = 210e9                  # Mô-đun đàn hồi Young (Pa)
RHO = 7850                 # Khối lượng riêng thép (kg/m^3)
SIGMA_YIELD = 270e6        # Giới hạn chảy (Pa)
MAX_DISP = 0.02            # Chuyển vị cho phép tại đầu dầm (m)
F = 500000                 # Lực tập trung tại đầu tự do (N) = 500 kN

# Giới hạn biến thiết kế (cm)
R_OUT_MAX = L * 100 / 10       # r_outer ≤ L/10 = 30 cm
R_OUT_MIN = 1.0                # Bán kính ngoài tối thiểu (cm)
T_MIN = 0.1                    # Chiều dày thành ống tối thiểu (cm) = 1 mm
T_MAX = 5.0                    # Chiều dày thành ống tối đa (cm) = 5 cm

# ============================================================
# 2. CÁC HÀM TÍNH TOÁN KẾT CẤU (Phần tử hữu hạn dầm)
# ============================================================

def area(r_out, r_in):
    """Diện tích mặt cắt tròn rỗng."""
    return np.pi * (r_out**2 - r_in**2)

def moment_of_inertia(r_out, r_in):
    """Mô-men quán tính mặt cắt tròn rỗng."""
    return np.pi * (r_out**4 - r_in**4) / 4

def beam_stiffness_matrix(r_outer, r_inner):
    """
    Lắp ghép ma trận độ cứng tổng thể cho dầm.
    Mỗi nút có 2 bậc tự do: chuyển vị đứng (v) và góc xoay (θ).
    Tổng cộng: (NUM_SEGMENTS + 1) nút × 2 = 12 bậc tự do.
    """
    n_dof = 2 * (NUM_SEGMENTS + 1)
    K = np.zeros((n_dof, n_dof))

    for i in range(NUM_SEGMENTS):
        I_i = moment_of_inertia(r_outer[i], r_inner[i])
        le = SEG_LEN  # chiều dài phần tử

        # Ma trận độ cứng phần tử dầm Euler-Bernoulli (4×4)
        k_e = (E * I_i) * np.array([
            [ 12/le**3,  6/le**2, -12/le**3,  6/le**2],
            [  6/le**2,  4/le,     -6/le**2,  2/le   ],
```

```
            [-12/le**3, -6/le**2,  12/le**3, -6/le**2],
            [  6/le**2,  2/le,     -6/le**2,  4/le   ]
        ])

        # Ghép vào ma trận tổng thể
        dofs = [2*i, 2*i+1, 2*i+2, 2*i+3]
        for j in range(4):
            for k in range(4):
                K[dofs[j], dofs[k]] += k_e[j, k]

    return K

def solve_displacement(r_outer, r_inner):
    """
    Giải hệ phương trình K·u = F để tính chuyển vị.
    Điều kiện biên: ngàm tại đầu trái (v₁ = 0, θ₁ = 0).
    Trả về: vector chuyển vị đầy đủ (bao gồm cả nút ngàm).
    """
    K = beam_stiffness_matrix(r_outer, r_inner)

    # Loại bỏ 2 bậc tự do đầu tiên (ngàm)
    K_red = K[2:, 2:]

    # Vector tải trọng: lực F tại đầu tự do.
    # Thứ tự bậc tự do mỗi nút: [v, θ] → chuyển vị đứng nút cuối là phần tử [-2]
    F_vec = np.zeros(K_red.shape[0])
    F_vec[-2] = F

    # Giải hệ phương trình
    u_red = np.linalg.solve(K_red, F_vec)

    # Ghép lại vector chuyển vị đầy đủ (thêm 2 giá trị = 0 tại ngàm)
    u_full = np.zeros(2 * (NUM_SEGMENTS + 1))
    u_full[2:] = u_red
    return u_full

def tip_displacement(r_outer, r_inner):
    """Chuyển vị đứng tại đầu tự do của dầm (nút cuối)."""
    u = solve_displacement(r_outer, r_inner)
    return u[-2]   # bậc tự do chuyển vị đứng tại nút cuối

def max_stress(r_outer, r_inner):
    """
    Tính ứng suất lớn nhất trong dầm theo công thức uốn: σ = M·c / I.
    Với dầm công-xôn chịu lực tập trung tại đầu: M(x) = F·(L - x).
    M(x) giảm dần về đầu tự do nên trong mỗi đoạn (tiết diện không đổi),
    mô-men và ứng suất lớn nhất nằm tại đầu trái của đoạn, x = i·SEG_LEN.
    """
    sigma_max = 0.0
    for i in range(NUM_SEGMENTS):
        x_left = i * SEG_LEN                # đầu trái đoạn (phía ngàm)
        M = F * (L - x_left)                # mô-men uốn lớn nhất của đoạn
        c = r_outer[i]                      # khoảng cách đến thớ ngoài cùng
        I_i = moment_of_inertia(r_outer[i], r_inner[i])
        sigma = abs(M * c / I_i)
        sigma_max = max(sigma_max, sigma)
    return sigma_max

# ============================================================
# 3. HÀM MỤC TIÊU VÀ RÀNG BUỘC
# ============================================================

def objective(params):
    """
    Hàm mục tiêu: Khối lượng dầm (kg).
    params = [r_out1..r_out5, r_in1..r_in5] (đơn vị: cm)
    """
    r_out = params[:NUM_SEGMENTS] / 100   # chuyển sang mét
    r_in = params[NUM_SEGMENTS:] / 100
    weight = 0.0
    for i in range(NUM_SEGMENTS):
        weight += area(r_out[i], r_in[i]) * SEG_LEN * RHO
    return weight

def is_feasible(params):
```

```
    """
    Kiểm tra tất cả ràng buộc. Trả về True nếu thiết kế thỏa mãn.

    Ràng buộc:
      (1) Chuyển vị đầu dầm ≤ MAX_DISP
      (2) r_outer[i] > r_inner[i]  ∀i
      (3) r_outer[i+1] > r_inner[i]  ∀i (ràng buộc hàn)
      (4) Ứng suất max < SIGMA_YIELD
      (5) r_outer[i] ≤ R_OUT_MAX (bán kính ngoài ≤ L/10)
      (6) T_MIN ≤ (r_outer[i] - r_inner[i]) ≤ T_MAX (chiều dày thành ống)
    """
    r_out = params[:NUM_SEGMENTS]
    r_in = params[NUM_SEGMENTS:]
    r_out_m = r_out / 100   # chuyển sang mét
    r_in_m = r_in / 100

    # (2) Ràng buộc bán kính: r_outer > r_inner
    if np.any(r_out <= r_in):
        return False

    # (5) Ràng buộc giới hạn trên bán kính ngoài
    if np.any(r_out > R_OUT_MAX) or np.any(r_out < R_OUT_MIN):
        return False

    # (6) Ràng buộc chiều dày thành ống
    thickness = r_out - r_in
    if np.any(thickness < T_MIN) or np.any(thickness > T_MAX):
        return False

    # (3) Ràng buộc hàn: thành ống hai đoạn kề nhau phải chồng lấn
    for i in range(NUM_SEGMENTS - 1):
        if r_out[i] <= r_in[i + 1]:       # r_outer[i] > r_inner[i+1]
            return False
        if r_out[i + 1] <= r_in[i]:       # r_outer[i+1] > r_inner[i]
            return False

    # (1) Ràng buộc chuyển vị
    disp = abs(tip_displacement(r_out_m, r_in_m))
    if disp > MAX_DISP:
        return False

    # (4) Ràng buộc ứng suất
    sigma = max_stress(r_out_m, r_in_m)
    if sigma > SIGMA_YIELD:
        return False

    return True

# ============================================================
# 4. THUẬT TOÁN SIMULATED ANNEALING (SA)
# ============================================================

def simulated_annealing(x0, n_iter=5000, T0=100.0, alpha=0.99):
    """
    Thuật toán Simulated Annealing (Luyện kim mô phỏng).

    Tham số:
      x0     : Nghiệm ban đầu (vector biến thiết kế)
      n_iter : Số vòng lặp tối đa
      T0     : Nhiệt độ ban đầu
      alpha  : Tốc độ làm nguội (0 < alpha < 1)

    Thuật toán:
      1. Bắt đầu với nghiệm x_current, tính f(x_current)
      2. Lặp:
         a. Tạo nghiệm lân cận x_new = x_current + nhiễu ngẫu nhiên
         b. Nếu x_new không thỏa ràng buộc → bỏ qua
         c. Tính Δf = f(x_new) - f(x_current)
         d. Nếu Δf < 0 → chấp nhận (nghiệm tốt hơn)
         e. Nếu Δf ≥ 0 → chấp nhận với xác suất p = exp(-Δf / T)
         f. Giảm nhiệt độ: T = T0 × alpha^k
      3. Trả về nghiệm tốt nhất tìm được
    """
    # --- Khởi tạo ---
    x_current = x0.copy()
```

```
    f_current = objective(x_current)
    x_best = x_current.copy()
    f_best = f_current

    # Tìm điểm xuất phát khả dĩ (nếu x0 vi phạm ràng buộc)
    if not is_feasible(x_current):
        print("Nghiệm ban đầu vi phạm ràng buộc. Tìm điểm khả dĩ...")
        for _ in range(10000):
            # Tạo nghiệm ngẫu nhiên trong giới hạn biến
            r_out_trial = np.random.uniform(R_OUT_MIN, R_OUT_MAX, NUM_SEGMENTS)
            r_out_trial = np.sort(r_out_trial)[::-1]  # giảm dần (hợp lý cho dầm công-xôn)
            t_trial = np.random.uniform(T_MIN, T_MAX, NUM_SEGMENTS)
            r_in_trial = r_out_trial - t_trial
            x_trial = np.concatenate([r_out_trial, r_in_trial])
            if is_feasible(x_trial):
                x_current = x_trial.copy()
                x_best = x_trial.copy()
                f_current = objective(x_trial)
                f_best = f_current
                print(f"  Đã tìm được điểm khả dĩ, khối lượng = {f_best:.2f} kg")
                break
        else:
            print("  CẢNH BÁO: Không tìm được điểm khả dĩ.")

    # --- Vòng lặp SA ---
    history = []            # lưu lịch sử f_best qua các vòng lặp
    step_size = 1.0         # kích thước bước nhiễu
    n_reject = 0            # đếm số lần liên tiếp bị từ chối

    for k in range(n_iter):
        # Nhiệt độ hiện tại: T_k = T0 × alpha^k
        T_k = T0 * (alpha ** k)

        # Tự điều chỉnh bước: giảm bước nếu bị từ chối quá nhiều
        if n_reject > 50:
            step_size *= 0.9
            n_reject = 0

        # Bước 2a: Tạo nghiệm lân cận
        perturbation = np.random.uniform(-step_size, step_size, len(x_current))
        x_new = x_current + perturbation

        # Giới hạn biên cho biến thiết kế
        x_new[:NUM_SEGMENTS] = np.clip(x_new[:NUM_SEGMENTS], R_OUT_MIN, R_OUT_MAX)
        x_new[NUM_SEGMENTS:] = np.clip(
            x_new[NUM_SEGMENTS:],
            x_new[:NUM_SEGMENTS] - T_MAX,   # r_inner ≥ r_outer - T_MAX
            x_new[:NUM_SEGMENTS] - T_MIN    # r_inner ≤ r_outer - T_MIN
        )
        x_new = np.maximum(x_new, 0.1)      # đảm bảo bán kính trong > 0

        # Bước 2b: Kiểm tra ràng buộc
        if not is_feasible(x_new):
            n_reject += 1
            history.append(f_best)
            continue

        # Bước 2c: Tính chênh lệch hàm mục tiêu
        f_new = objective(x_new)
        delta_f = f_new - f_current

        # Bước 2d-e: Quyết định chấp nhận hay từ chối
        if delta_f < 0:
            # Nghiệm tốt hơn → luôn chấp nhận
            accept = True
        else:
            # Nghiệm kém hơn → chấp nhận với xác suất p = exp(-Δf / T)
            p = np.exp(-delta_f / T_k)
            accept = (np.random.random() < p)

        if accept:
            x_current = x_new.copy()
            f_current = f_new
            n_reject = 0

```

```
            # Cập nhật nghiệm tốt nhất
            if f_current < f_best:
                x_best = x_current.copy()
                f_best = f_current
        else:
            n_reject += 1

        # Lưu lịch sử
        history.append(f_best)

        # In tiến trình mỗi 500 vòng
        if k % 500 == 0:
            print(f"  Vòng lặp {k:5d} | f_best = {f_best:.2f} kg | T = {T_k:.6f}")

    return x_best, f_best, history

# ============================================================
# 5. HIỂN THỊ KẾT QUẢ
# ============================================================

def plot_history(history, save_dir):
    """Vẽ biểu đồ lịch sử hội tụ của thuật toán SA."""
    plt.figure(figsize=(10, 6))
    plt.plot(history, linewidth=1.5)
    plt.xlabel('Vòng lặp')
    plt.ylabel('Khối lượng (kg)')
    plt.title('Quá trình hội tụ - thuật toán luyện kim mô phỏng')
    plt.gca().yaxis.set_major_formatter(VN_TICKS)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.savefig(os.path.join(save_dir, 'optimization_history.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: optimization_history.png")

def plot_deformed_beam(params, save_dir):
    """Vẽ hình dạng biến dạng của dầm sau tối ưu."""
    r_out_m = params[:NUM_SEGMENTS] / 100
    r_in_m = params[NUM_SEGMENTS:] / 100

    # Giải chuyển vị
    u_full = solve_displacement(r_out_m, r_in_m)

    # Lấy chuyển vị đứng tại các nút (đảo dấu để hiển thị võng xuống)
    v_nodes = -u_full[0::2]
    x_nodes = np.linspace(0, L, NUM_SEGMENTS + 1)

    # Nội suy mịn bằng spline bậc 3
    cs = CubicSpline(x_nodes, v_nodes)
    x_smooth = np.linspace(0, L, 200)
    v_smooth = cs(x_smooth)

    # --- Vẽ đồ thị ---
    plt.figure(figsize=(12, 5))

    # Dầm ban đầu (chưa biến dạng)
    plt.plot(x_nodes, np.zeros_like(x_nodes), 'k-', lw=2, label='Dầm ban đầu')

    # Dầm biến dạng
    plt.plot(x_smooth, v_smooth, 'r-', lw=2, label='Dầm biến dạng')
    plt.plot(x_nodes, v_nodes, 'ro', ms=6)

    # Ghi chú chuyển vị tại từng nút
    for x, v in zip(x_nodes, v_nodes):
        plt.annotate(f"{vn(abs(v)*100)} cm", (x, v),
                     textcoords="offset points", xytext=(0, -15),
                     ha='center', fontsize=9,
                     bbox=dict(boxstyle="round,pad=0.3",
                               fc="white", ec="gray", alpha=0.8))

    # Mũi tên lực
    plt.arrow(L, 0, 0, v_nodes[-1] / 2,
              head_width=0.05, head_length=abs(v_nodes[-1] / 4),
              fc='blue', ec='blue', lw=1.5)
    plt.text(L + 0.05, v_nodes[-1] / 2, f"F = {F/1000:.0f} kN",
             color='blue', fontsize=10, ha='left', va='center')

```

```
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.title('Chuyển vị dầm sau tối ưu')
    plt.xlabel('Chiều dài (m)')
    plt.ylabel('Chuyển vị (m)')
    plt.legend(loc='upper right')
    plt.gca().xaxis.set_major_formatter(VN_TICKS)
    plt.gca().yaxis.set_major_formatter(VN_TICKS)

    # Giới hạn trục
    v_min = min(v_nodes)
    y_range = abs(v_min) if abs(v_min) > 0.001 else 0.001
    plt.xlim(0, L + 0.2)
    plt.ylim(v_min * 1.3, y_range * 0.3)

    plt.savefig(os.path.join(save_dir, 'deformed_beam.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: deformed_beam.png")

def plot_beam_shape(params, save_dir):
    """
    Vẽ hình dạng dầm tối ưu:
      - Subplot 1: Sơ đồ dầm nhìn từ mặt bên (side view)
      - Subplot 2: Các mặt cắt ngang hình tròn rỗng (cross sections)
    """
    r_out = params[:NUM_SEGMENTS]   # cm
    r_in = params[NUM_SEGMENTS:]    # cm

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 9),
                                   gridspec_kw={'height_ratios': [1, 1.5]})
    plt.subplots_adjust(hspace=0.45)

    # ------------------------------------------------------------------
    # SUBPLOT 1: SƠ ĐỒ DẦM (SIDE VIEW)
    # ------------------------------------------------------------------
    colors = plt.cm.Blues(np.linspace(0.3, 0.7, NUM_SEGMENTS))
    current_x = 0
    for i in range(NUM_SEGMENTS):
        # Vùng thành ống (tô màu)
        rect_fill_top = patches.Rectangle(
            (current_x, r_in[i]), SEG_LEN, r_out[i] - r_in[i],
            facecolor=colors[i], edgecolor='none', alpha=0.4)
        rect_fill_bot = patches.Rectangle(
            (current_x, -r_out[i]), SEG_LEN, r_out[i] - r_in[i],
            facecolor=colors[i], edgecolor='none', alpha=0.4)
        ax1.add_patch(rect_fill_top)
        ax1.add_patch(rect_fill_bot)

        # Biên ngoài (nét liền)
        rect_outer = patches.Rectangle(
            (current_x, -r_out[i]), SEG_LEN, r_out[i] * 2,
            linewidth=2, edgecolor='black', facecolor='none')
        # Biên trong (nét đứt)
        rect_inner = patches.Rectangle(
            (current_x, -r_in[i]), SEG_LEN, r_in[i] * 2,
            linewidth=1, edgecolor='gray', linestyle='--', facecolor='none')

        ax1.add_patch(rect_outer)
        ax1.add_patch(rect_inner)

        # Nhãn đoạn
        ax1.text(current_x + SEG_LEN / 2, r_out[i] + 3,
                 f"Đoạn {i+1}", ha='center', fontsize=9, fontweight='bold')
        # Kích thước bán kính ngoài
        ax1.text(current_x + SEG_LEN / 2, -r_out[i] - 4,
                 f"R={vn(r_out[i], 1)}", ha='center', fontsize=8, color='blue')

        current_x += SEG_LEN

    # Vẽ ký hiệu ngàm (hatch)
    ax1.add_patch(patches.Rectangle(
        (-0.08, -max(r_out) * 1.1), 0.08, max(r_out) * 2.2,
        facecolor='gray', edgecolor='black', hatch='///', linewidth=2))

    # Mũi tên lực tại đầu tự do
    r_max = max(r_out)
```

```
    ax1.annotate('', xy=(L, 0), xytext=(L, r_max * 1.5),
                 arrowprops=dict(facecolor='red', edgecolor='red',
                                 shrink=0.05, width=3, headwidth=10))
    ax1.text(L + 0.05, r_max * 1.3, f'F = {F/1000:.0f} kN',
             color='red', fontsize=11, fontweight='bold', ha='left')

    # Đường trục dầm
    ax1.axhline(y=0, color='gray', linestyle='-.', linewidth=0.8, alpha=0.5)

    ax1.set_xlim(-0.15, L + 0.4)
    ax1.set_ylim(-r_max * 1.6, r_max * 1.8)
    ax1.set_title('Sơ đồ dầm công-xôn (nhìn từ mặt bên)', fontsize=13)
    ax1.set_xlabel('Chiều dài (m)')
    ax1.set_ylabel('Kích thước (cm)')
    ax1.grid(True, linestyle=':', alpha=0.5)
    ax1.xaxis.set_major_formatter(VN_TICKS)
    ax1.yaxis.set_major_formatter(VN_TICKS)

    # ------------------------------------------------------------------
    # SUBPLOT 2: MẶT CẮT NGANG (CROSS SECTIONS) -- hình tròn rỗng
    # ------------------------------------------------------------------
    spacing = max(r_out) * 2.8   # khoảng cách giữa các mặt cắt
    for i in range(NUM_SEGMENTS):
        cx = i * spacing   # tâm mặt cắt

        # Vòng tròn ngoài (tô xám)
        circle_out = patches.Circle(
            (cx, 0), r_out[i],
            facecolor='lightsteelblue', edgecolor='black', linewidth=1.5)
        # Vòng tròn trong (đục trắng)
        circle_in = patches.Circle(
            (cx, 0), r_in[i],
            facecolor='white', edgecolor='black', linewidth=1)

        ax2.add_patch(circle_out)
        ax2.add_patch(circle_in)

        # Nhãn mặt cắt
        t_i = r_out[i] - r_in[i]
        ax2.text(cx, -r_out[i] - 5,
                 f"Mặt cắt {i+1}\nt = {vn(t_i)} cm",
                 ha='center', fontsize=9, va='top')

        # Đường ghi kích thước bán kính ngoài
        ax2.plot([cx, cx + r_out[i]], [0, 0], 'b-', linewidth=0.8)
        ax2.plot(cx + r_out[i], 0, 'b|', markersize=8)
        ax2.text(cx + r_out[i] / 2, 1.5,
                 f"R={vn(r_out[i], 1)}", ha='center', fontsize=7, color='blue')

    total_width = (NUM_SEGMENTS - 1) * spacing
    ax2.set_xlim(-r_max * 1.5, total_width + r_max * 1.5)
    ax2.set_ylim(-r_max * 2, r_max * 1.8)
    ax2.set_aspect('equal')
    ax2.axis('off')
    ax2.set_title('Chi tiết các mặt cắt ngang - ống tròn rỗng', fontsize=13)

    plt.savefig(os.path.join(save_dir, 'beam_cross_sections.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: beam_cross_sections.png")

def print_results(params, f_best):
    """In kết quả tối ưu ra console."""
    r_out = params[:NUM_SEGMENTS]
    r_in = params[NUM_SEGMENTS:]
    r_out_m = r_out / 100
    r_in_m = r_in / 100

    print("\n" + "=" * 50)
    print("KẾT QUẢ TỐI ƯU")
    print("=" * 50)
    print(f"Khối lượng tối ưu: {f_best:.2f} kg")
    print(f"\nKích thước mặt cắt tối ưu (cm):")
    print(f"  {'Đoạn':<8} {'r_outer':>10} {'r_inner':>10} {'t (dày)':>10}")
    print(f"  {'-'*38}")
    for i in range(NUM_SEGMENTS):
```

```
        t_i = r_out[i] - r_in[i]
        print(f"  {i+1:<8} {r_out[i]:>10.2f} {r_in[i]:>10.2f} {t_i:>10.2f}")

    disp = abs(tip_displacement(r_out_m, r_in_m))
    sigma = max_stress(r_out_m, r_in_m)
    print(f"\nChuyển vị đầu dầm:  {disp*100:.2f} cm  (giới hạn: {MAX_DISP*100:.0f} cm)")
    print(f"Ứng suất lớn nhất:  {sigma/1e6:.2f} MPa (giới hạn: {SIGMA_YIELD/1e6:.0f} MPa)")
    print("=" * 50)

def save_results(params, f_best, save_dir):
    """Lưu kết quả ra file text."""
    r_out = params[:NUM_SEGMENTS]
    r_in = params[NUM_SEGMENTS:]
    r_out_m = r_out / 100
    r_in_m = r_in / 100

    path = os.path.join(save_dir, 'optimization_results.txt')
    with open(path, 'w', encoding='utf-8') as f:
        f.write(f"Khối lượng tối ưu: {f_best:.2f} kg\n\n")
        f.write(f"Kích thước mặt cắt tối ưu (cm):\n")
        for i in range(NUM_SEGMENTS):
            f.write(f"  Đoạn {i+1}: r_outer = {r_out[i]:.2f}, r_inner = {r_in[i]:.2f}\n")
        disp = abs(tip_displacement(r_out_m, r_in_m))
        sigma = max_stress(r_out_m, r_in_m)
        f.write(f"\nChuyển vị đầu dầm: {disp*100:.2f} cm\n")
        f.write(f"Ứng suất lớn nhất: {sigma/1e6:.2f} MPa\n")
    print(f"  Đã lưu: optimization_results.txt")

# ============================================================
# 6. CHƯƠNG TRÌNH CHÍNH
# ============================================================

def generate_random_initial(seed=None):
    """
    Tạo nghiệm ban đầu ngẫu nhiên trong miền khả dĩ.
    r_outer giảm dần (hợp lý cho dầm công-xôn), chiều dày trong [T_MIN, T_MAX].
    """
    if seed is not None:
        np.random.seed(seed)

    # Sinh r_outer ngẫu nhiên rồi sắp giảm dần
    r_out = np.random.uniform(R_OUT_MIN + T_MAX, R_OUT_MAX, NUM_SEGMENTS)
    r_out = np.sort(r_out)[::-1]

    # Sinh chiều dày ngẫu nhiên
    t = np.random.uniform(T_MIN, T_MAX, NUM_SEGMENTS)
    r_in = r_out - t

    return np.concatenate([r_out, r_in])

def main():
    save_dir = os.path.dirname(os.path.abspath(__file__))

    # --- Thông số chạy nhiều lần ---
    N_RUNS = 50          # số lần chạy SA
    N_ITER = 5000        # số vòng lặp mỗi lần
    T0 = 100.0           # nhiệt độ ban đầu
    ALPHA = 0.99         # tốc độ làm nguội

    # In thông tin bài toán
    print("=" * 60)
    print("TỐI ƯU HÓA DẦM CÔNG-XÔN - SIMULATED ANNEALING")
    print(f"Chạy {N_RUNS} lần từ các điểm xuất phát ngẫu nhiên khác nhau")
    print("=" * 60)
    print(f"Chiều dài dầm:  L = {L} m ({NUM_SEGMENTS} đoạn)")
    print(f"Lực tập trung:  F = {F/1000:.0f} kN")
    print(f"Vật liệu:       Thép S270 (σ_y = {SIGMA_YIELD/1e6:.0f} MPa)")
    print(f"Biến thiết kế:  {2*NUM_SEGMENTS} biến (r_outer, r_inner)")
    print(f"Giới hạn:       r_outer ≤ {R_OUT_MAX} cm, "
          f"chiều dày {T_MIN}-{T_MAX} cm")

    # --- Chạy SA nhiều lần ---
    all_results = []     # [(x_best, f_best, history), ...]
    all_f_best = []

```

```
    for run in range(N_RUNS):
        print(f"\n{'-'*60}")
        print(f"LẦN CHẠY {run + 1}/{N_RUNS}")
        print(f"{'-'*60}")

        # Tạo nghiệm ban đầu ngẫu nhiên
        x0 = generate_random_initial(seed=run * 42 + 7)

        # Kiểm tra khả dĩ
        w0 = objective(x0)
        feas = is_feasible(x0)
        print(f"  Nghiệm ban đầu: Khối lượng = {w0:.2f} kg, Khả thi: {feas}")

        # Chạy SA
        x_best, f_best, history = simulated_annealing(
            x0, n_iter=N_ITER, T0=T0, alpha=ALPHA
        )

        all_results.append((x_best, f_best, history))
        all_f_best.append(f_best)
        print(f"  → Kết quả: Khối lượng = {f_best:.2f} kg")

    # --- Thống kê tổng hợp ---
    all_f_best = np.array(all_f_best)
    idx_best = np.argmin(all_f_best)
    idx_worst = np.argmax(all_f_best)

    print(f"\n{'='*60}")
    print(f"THỐNG KÊ TỔNG HỢP ({N_RUNS} lần chạy)")
    print(f"{'='*60}")
    print(f"\n  {'Lần':<6} {'Khối lượng (kg)':>16}")
    print(f"  {'-'*24}")
    for i, f in enumerate(all_f_best):
        marker = " ← TỐT NHẤT" if i == idx_best else ""
        print(f"  {i+1:<6} {f:>16.2f}{marker}")

    print(f"\n  {'-'*30}")
    print(f"  Tốt nhất  (lần {idx_best+1}): {all_f_best[idx_best]:.2f} kg")
    print(f"  Kém nhất  (lần {idx_worst+1}): {all_f_best[idx_worst]:.2f} kg")
    print(f"  Trung bình:          {np.mean(all_f_best):.2f} kg")
    print(f"  Độ lệch chuẩn:      {np.std(all_f_best):.2f} kg")
    print(f"  Hệ số biến thiên:   {np.std(all_f_best)/np.mean(all_f_best)*100:.1f}%")
    print(f"{'='*60}")

    # --- In chi tiết nghiệm tốt nhất ---
    x_global_best, f_global_best, history_best = all_results[idx_best]
    print(f"\nCHI TIẾT NGHIỆM TỐT NHẤT (lần chạy {idx_best + 1}):")
    print_results(x_global_best, f_global_best)
    save_results(x_global_best, f_global_best, save_dir)

    # --- Vẽ đồ thị cho nghiệm tốt nhất ---
    print("\nĐang vẽ đồ thị cho nghiệm tốt nhất...")
    plot_history(history_best, save_dir)
    plot_deformed_beam(x_global_best, save_dir)
    plot_beam_shape(x_global_best, save_dir)

    # --- Vẽ biểu đồ so sánh các lần chạy ---
    plt.figure(figsize=(10, 5))
    plt.bar(range(1, N_RUNS + 1), all_f_best,
            color=['green' if i == idx_best else 'steelblue'
                   for i in range(N_RUNS)],
            edgecolor='black', linewidth=0.5)
    plt.axhline(y=np.mean(all_f_best), color='red', linestyle='--',
                linewidth=1.5, label=f'Trung bình = {vn(np.mean(all_f_best))} kg')
    plt.xlabel('Lần chạy')
    plt.ylabel('Khối lượng tối ưu (kg)')
    plt.title(f'So sánh {N_RUNS} lần chạy SA từ các điểm xuất phát khác nhau')
    plt.legend()
    plt.grid(True, axis='y', linestyle=':', alpha=0.5)
    plt.xticks([1] + list(range(5, N_RUNS + 1, 5)))   # tránh nhãn trục chồng nhau
    plt.gca().yaxis.set_major_formatter(VN_TICKS)
    plt.savefig(os.path.join(save_dir, 'multi_run_comparison.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: multi_run_comparison.png")
```

```
687
688     print("\nHoàn thành!")
689
690 if __name__ == "__main__":
691     main()
```

Kết quả chạy Code 8.1 (tệp `optimization_results.txt`)

```
Khối lượng tối ưu: 591.57 kg

Kích thước mặt cắt tối ưu (cm):
  Đoạn 1: r_outer = 29.83, r_inner = 27.60
  Đoạn 2: r_outer = 28.28, r_inner = 26.32
  Đoạn 3: r_outer = 27.45, r_inner = 25.91
  Đoạn 4: r_outer = 26.72, r_inner = 25.67
  Đoạn 5: r_outer = 25.87, r_inner = 25.33

Chuyển vị đầu dầm: 1.75 cm
Ứng suất lớn nhất: 269.98 MPa
```

Code 8.2: Giải Ví dụ 8.2 bằng luyện kim mô phỏng kết hợp SAP2000 OAPI (Python)

```
"""
Tối ưu hóa kết cấu khung thép bằng thuật toán Simulated Annealing (SA)
kết hợp với SAP2000 qua COM API (OAPI).

Bài toán:
    - Hàm mục tiêu: Cực tiểu hóa trọng lượng kết cấu
    - Biến thiết kế: Tiết diện thép W-section cho 8 nhóm thanh
    - Ràng buộc: Tất cả các thanh phải thỏa mãn tiêu chuẩn thiết kế thép AISC 360

Tài liệu tham khảo:
    - AISC 360-16, Chapter H, Equations H1-1a & H1-1b
    - Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983).
      "Optimization by Simulated Annealing". Science, 220(4598), 671-680.
"""

import os
import random
import math
import win32com.client
import matplotlib.pyplot as plt

# ============================================================================
# PHẦN 1: CẤU HÌNH VÀ THAM SỐ
# ============================================================================

# --- Danh sách tiết diện thép W-section (AISC) ---
# Mỗi tiết diện ký hiệu theo dạng: WdXw
#   d = chiều cao tiết diện (inch)
#   w = trọng lượng trên đơn vị chiều dài (lb/ft)
W_SECTIONS = [
    "W12X14", "W12X16", "W12X19", "W12X22", "W12X26", "W12X30", "W12X35", "W12X40",
    "W12X45", "W12X50", "W12X53", "W12X58", "W12X65", "W12X72", "W12X79", "W12X87",
    "W12X96", "W12X106", "W12X120", "W12X136", "W12X152", "W12X170", "W12X190", "W12X210",
    "W12X230", "W12X252", "W12X279", "W12X305", "W12X336", "W14X22", "W14X26", "W14X30",
    "W14X34", "W14X38", "W14X43", "W14X48", "W14X53", "W14X61", "W14X68", "W14X74", "W14X82",
    "W14X90", "W14X99", "W14X109", "W14X120", "W14X132", "W14X145", "W14X159", "W14X176",
    "W14X193", "W14X211", "W14X233", "W14X257", "W14X283", "W14X311", "W14X342", "W14X370",
    "W14X398", "W14X426", "W14X455", "W14X500", "W14X550", "W14X605", "W14X665", "W14X730",
    "W16X26", "W16X31", "W16X36", "W16X40", "W16X45", "W16X50", "W16X57", "W16X67", "W16X77",
    "W16X89", "W16X100", "W18X35", "W18X40", "W18X46", "W18X50", "W18X55", "W18X60", "W18X65",
    "W18X71", "W18X76", "W18X86", "W18X97", "W18X106", "W18X119", "W18X130", "W18X143", "W18X158",
    "W18X175", "W21X44", "W21X48", "W21X50", "W21X55", "W21X57", "W21X62", "W21X68", "W21X73",
    "W21X83", "W21X93", "W21X101", "W21X111", "W21X122", "W21X132", "W21X147", "W21X166",
    "W21X182", "W21X201", "W24X55", "W24X62", "W24X68", "W24X76", "W24X84", "W24X94",
    "W24X103", "W24X104", "W24X117", "W24X131", "W24X146", "W24X162", "W24X176", "W24X192",
    "W24X207", "W24X229", "W24X250", "W24X279", "W24X306", "W24X335", "W24X370", "W27X84",
    "W27X94", "W27X102", "W27X114", "W27X129", "W27X146", "W27X161", "W27X178", "W27X194",
    "W27X217", "W27X235", "W27X258", "W27X281", "W27X307", "W27X336", "W27X368", "W27X539",
    "W30X90", "W30X99", "W30X108", "W30X116", "W30X124", "W30X132", "W30X148", "W30X173",
    "W30X191", "W30X211", "W30X235", "W30X261", "W30X292", "W30X326", "W30X357", "W30X391",
    "W33X118", "W33X130", "W33X141", "W33X152", "W33X169", "W33X201", "W33X221",
    "W33X241", "W33X263", "W33X291", "W33X318", "W33X354", "W33X387"
```

```
]

# --- Tham số mô hình SAP2000 ---
# Tên file mô hình SAP2000 và đường dẫn đầy đủ (cùng thư mục với file code)
MODEL_FILE = "model_v24_2.sdb"
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(CURRENT_DIR, MODEL_FILE)
# Các nút gối tựa (dùng để tính trọng lượng)
JOINTS = ["1", "10", "19", "28", "37"]
# Số nhóm thanh cần tối ưu (G1 → G8)
NUM_ELEMENTS = 8

# --- Tham số thuật toán Simulated Annealing ---
INITIAL_TEMP = 100      # Nhiệt độ ban đầu T_0
COOLING_RATE = 0.98      # Hệ số làm nguội α (T_mới = α × T_cũ)
MIN_TEMP = 0.1           # Nhiệt độ dừng T_min
MAX_ITER = 80            # Số vòng lặp tối đa
PENALTY_FACTOR = 1000    # Hệ số phạt cho lời giải không khả thi
RESTART_THRESHOLD = 10   # Số vòng không cải thiện trước khi khởi động lại

# --- Thiết lập Random Seed để kết quả luôn giống nhau mỗi lần chạy ---
RANDOM_SEED = 42
random.seed(RANDOM_SEED)

# ============================================================================
# PHẦN 2: GIAO TIẾP VỚI SAP2000 (OAPI)
# ============================================================================

def start_sap():
    """
    Kết nối hoặc khởi tạo phiên làm việc SAP2000 qua COM API.

    Returns:
        sap_object: Đối tượng ứng dụng SAP2000
        sap_model:  Đối tượng mô hình SAP2000
    """
    try:
        # Thử kết nối đến SAP2000 đang mở
        try:
            sap_object = win32com.client.GetActiveObject("CSI.SAP2000.API.SapObject")
            print("Đã kết nối đến SAP2000 đang chạy")
        except:
            # Nếu chưa mở, khởi tạo SAP2000 mới
            print("Đang khởi tạo SAP2000 mới...")
            sap_object = win32com.client.Dispatch("CSI.SAP2000.API.SapObject")

        # Khởi động ứng dụng SAP2000
        ret = sap_object.ApplicationStart()
        if ret != 0:
            raise Exception("Không thể khởi động SAP2000")

        # Khởi tạo mô hình rỗng
        sap_model = sap_object.SapModel
        ret = sap_model.InitializeNewModel()
        if ret != 0:
            raise Exception("Không thể khởi tạo mô hình mới")

        return sap_object, sap_model

    except Exception as e:
        print(f"Lỗi khi khởi động SAP2000: {str(e)}")
        raise

def assign_sections(sap_model, solution):
    """
    Gán tiết diện W-section cho từng nhóm thanh trong mô hình.

    Mỗi phần tử trong lời giải là chỉ số (index) tương ứng với
    một tiết diện trong danh sách W_SECTIONS.

    Args:
        sap_model: Đối tượng mô hình SAP2000
        solution:  Mảng [s_1, s_2, ..., s_8], trong đó s_i là chỉ số tiết diện cho nhóm G_i
    """
    for i in range(len(solution)):
```

```
        group_name = f"G{i+1}"                          # Tên nhóm thanh: G1, G2, ..., G8
        section_name = W_SECTIONS[solution[i]]           # Tra tiết diện từ danh sách
        # ItemType=1: gán cho tất cả thanh trong nhóm
        sap_model.FrameObj.SetSection(group_name, section_name, ItemType=1)

def open_model(sap_model):
    """
    Mở file mô hình SAP2000 và lưu bản sao tạm thời.

    Returns:
        sap_model: Đối tượng mô hình SAP2000 đã mở
    """
    # Mở file mô hình gốc
    ret = sap_model.File.OpenFile(MODEL_PATH)
    if ret != 0:
        raise Exception(f"Không thể mở file mô hình: {MODEL_FILE}")

    # Lưu bản sao tạm thời (tránh ghi đè file gốc)
    temp_path = os.path.join(CURRENT_DIR, "model_temp3.sdb")
    sap_model.File.Save(temp_path)

    return sap_model

def run_analysis(sap_model):
    """
    Chạy phân tích kết cấu trong SAP2000.
    """
    sap_model.Analyze.RunAnalysis()

def check_design(sap_model):
    """
    Chạy kiểm tra thiết kế thép theo AISC 360.

    SAP2000 sẽ kiểm tra tất cả thanh theo tiêu chuẩn AISC:
    - Kiểm tra bền (strength)
    - Kiểm tra ổn định (stability)
    - Tỷ số demand/capacity (D/C ratio)

    Returns:
        num_not_passed: Số thanh không thỏa mãn (0 = tất cả đạt)
    """
    # Chạy thiết kế thép
    sap_model.DesignSteel.StartDesign()

    # Kiểm tra kết quả: có bao nhiêu thanh không đạt?
    ret, num_not_passed, failed, not_checked, failed_frames = \
        sap_model.DesignSteel.VerifyPassed()

    return num_not_passed

def get_weight(sap_model):
    """
    Tính tổng trọng lượng kết cấu từ phản lực gối tựa.

    Trọng lượng = Tổng phản lực đứng (F3) tại tất cả gối tựa,
    lấy từ trường hợp tải "weight" (trọng lượng bản thân).

    Returns:
        weight: Tổng trọng lượng kết cấu (kips)
    """
    # Chọn trường hợp tải "weight" để xuất kết quả
    sap_model.Results.Setup.SetCaseSelectedForOutput("weight")

    # Khởi tạo biến đầu ra cho hàm JointReact
    NumberResults = 0
    Obj = []; Elm = []; LoadCase = []; StepType = []; StepNum = []
    F1 = []; F2 = []; F3 = []; M1 = []; M2 = []; M3 = []

    # Tính tổng phản lực đứng tại tất cả gối tựa
    weight = 0
    for joint in JOINTS:
        result = sap_model.Results.JointReact(
            joint, 1, NumberResults, Obj, Elm, LoadCase,
            StepType, StepNum, F1, F2, F3, M1, M2, M3
        )
```

```
        # result[9] chứa mảng F3 (phản lực theo phương đứng)
        weight += result[9][0]

    return weight

# ============================================================================
# PHẦN 3: ĐỊNH NGHĨA BÀI TOÁN TỐI ƯU
# ============================================================================

def objective_function(sap_model, solution):
    """
    Đánh giá lời giải: tính trọng lượng và kiểm tra ràng buộc.

    Quy trình:
        1. Mở mô hình SAP2000
        2. Gán tiết diện theo lời giải
        3. Chạy phân tích kết cấu
        4. Kiểm tra thiết kế thép (ràng buộc)
        5. Tính trọng lượng kết cấu (hàm mục tiêu)

    Args:
        sap_model: Đối tượng mô hình SAP2000
        solution:  Mảng chỉ số tiết diện [s1, s2, ..., s8]

    Returns:
        weight:         Trọng lượng kết cấu (kips)
        num_not_passed: Số thanh không thỏa mãn ràng buộc
    """
    # Bước 1: Mở mô hình và lưu bản sao tạm
    sap_model = open_model(sap_model)

    # Bước 2: Gán tiết diện cho từng nhóm thanh
    assign_sections(sap_model, solution)

    # Bước 3: Chạy phân tích kết cấu
    run_analysis(sap_model)

    # Bước 4: Kiểm tra ràng buộc thiết kế thép (AISC 360)
    num_not_passed = check_design(sap_model)

    # Bước 5: Tính trọng lượng kết cấu
    weight = get_weight(sap_model)

    # In kết quả
    feasible = (num_not_passed == 0)
    sections = [W_SECTIONS[i] for i in solution]
    print(f"  Tiết diện: {sections}")
    print(f"  Trọng lượng: {weight:.2f} kips | Khả thi: {feasible}")

    return weight, num_not_passed

def penalized_cost(weight, num_not_passed):
    """
    Tính chi phí có phạt (penalized cost) cho lời giải.

    Phương pháp hàm phạt (penalty function):
        - Nếu lời giải khả thi (tất cả thanh đạt): chi phí = trọng lượng
        - Nếu lời giải KHÔNG khả thi: chi phí = trọng lượng × PENALTY_FACTOR

    Hệ số phạt lớn (= 1000) đảm bảo lời giải vi phạm ràng buộc
    hầu như luôn bị loại bỏ trong quá trình tối ưu.

    Args:
        weight:         Trọng lượng kết cấu
        num_not_passed: Số thanh vi phạm ràng buộc

    Returns:
        cost: Chi phí sau khi áp dụng hàm phạt
    """
    if num_not_passed > 0:
        return weight * PENALTY_FACTOR  # Phạt nặng nếu vi phạm
    else:
        return weight                   # Giữ nguyên nếu khả thi

# ============================================================================
```

```python
# PHẦN 4: THUẬT TOÁN SIMULATED ANNEALING (SA)
# ============================================================================

def generate_initial_solution():
    """
    Tạo lời giải ban đầu.

    Sử dụng tiết diện bảo thủ (W14X90, index = 41) cho tất cả nhóm thanh
    để đảm bảo lời giải ban đầu là khả thi, giúp thuật toán hội tụ nhanh hơn.

    Returns:
        Mảng [41, 41, ..., 41] có NUM_ELEMENTS phần tử
    """
    return [41 for _ in range(NUM_ELEMENTS)]

def generate_neighbor(current_solution):
    """
    Tạo lời giải lân cận bằng cách thay đổi ngẫu nhiên 1-2 tiết diện.

    Cơ chế tạo lân cận (neighborhood structure):
        1. Chọn ngẫu nhiên 1 hoặc 2 nhóm thanh
        2. Thay thế tiết diện của mỗi nhóm bằng một tiết diện ngẫu nhiên khác

    Thay đổi 1-2 phần tử giúp cân bằng giữa:
        - Khám phá (exploration): thay đổi đủ lớn để thoát cực trị địa phương
        - Khai thác (exploitation): thay đổi đủ nhỏ để cải thiện dần lời giải

    Args:
        current_solution: Lời giải hiện tại [s_1, s_2, ..., s_8]

    Returns:
        neighbor: Lời giải lân cận
    """
    neighbor = current_solution.copy()

    # Chọn ngẫu nhiên số phần tử cần thay đổi (1 hoặc 2)
    num_changes = random.randint(1, 2)

    for _ in range(num_changes):
        # Chọn ngẫu nhiên nhóm thanh cần thay đổi
        idx = random.randint(0, NUM_ELEMENTS - 1)

        # Chọn tiết diện mới (khác tiết diện hiện tại)
        old_section = neighbor[idx]
        new_section = old_section
        while new_section == old_section:
            new_section = random.randint(0, len(W_SECTIONS) - 1)

        neighbor[idx] = new_section

    return neighbor

def acceptance_probability(old_cost, new_cost, temperature):
    """
    Tính xác suất chấp nhận lời giải mới theo tiêu chuẩn Metropolis.

    Công thức Boltzmann:
        - Nếu f(x_mới) < f(x_cũ):  P = 1.0        (luôn chấp nhận lời giải tốt hơn)
        - Nếu f(x_mới) ≥ f(x_cũ):  P = exp(-(f_mới - f_cũ) / T)

    Ý nghĩa:
        - Nhiệt độ T cao → P lớn → dễ chấp nhận lời giải xấu hơn (khám phá)
        - Nhiệt độ T thấp → P nhỏ → chỉ chấp nhận lời giải tốt hơn (khai thác)

    Args:
        old_cost:    Chi phí lời giải hiện tại f(x_cũ)
        new_cost:    Chi phí lời giải mới f(x_mới)
        temperature: Nhiệt độ hiện tại T

    Returns:
        Xác suất chấp nhận P ∈ [0, 1]
    """
    # Trường hợp 1: Lời giải mới TỐT HƠN → luôn chấp nhận
    if new_cost < old_cost:
        return 1.0
```

```
353
354     # Trường hợp 2: Lời giải mới XẤU HƠN → chấp nhận theo xác suất Boltzmann
355     delta = new_cost - old_cost           # Δf = f_mới - f_cũ > 0
356     return math.exp(-delta / temperature)  # P = exp(-Δf / T)
357
358 def simulated_annealing(sap_model):
359     """
360     Vòng lặp chính của thuật toán Simulated Annealing.
361
362     Pseudocode:
363         1. Tạo lời giải ban đầu x₀, tính f(x₀)
364         2. Đặt T = T₀, x* = x₀ (lời giải tốt nhất)
365         3. Lặp cho đến khi T < T_min hoặc đạt MAX_ITER:
366             3.1. Tạo lời giải lân cận x' từ x hiện tại
367             3.2. Tính f(x') và chi phí có phạt
368             3.3. Nếu random() < P(f(x), f(x'), T):
369                     Chấp nhận: x ← x'
370                     Nếu f(x') < f(x*): cập nhật x* ← x'
371             3.4. Giảm nhiệt độ: T ← α × T
372         4. Trả về x* (lời giải tốt nhất)
373
374     Args:
375         sap_model: Đối tượng mô hình SAP2000
376
377     Returns:
378         best_solution: Mảng tiết diện tối ưu
379         best_weight:   Trọng lượng tối ưu
380         best_feasible: True nếu lời giải tốt nhất thỏa mãn ràng buộc
381     """
382
383     # ----- Bước 1: Khởi tạo lời giải ban đầu -----
384     print("=" * 60)
385     print("THUẬT TOÁN SIMULATED ANNEALING")
386     print("=" * 60)
387
388     current_x = generate_initial_solution()
389     print(f"\nLời giải ban đầu: {[W_SECTIONS[i] for i in current_x]}")
390
391     # Đánh giá lời giải ban đầu
392     current_w, current_violations = objective_function(sap_model, current_x)
393     current_cost = penalized_cost(current_w, current_violations)
394
395     print(f"\nTrọng lượng ban đầu: {current_w:.2f} kips")
396     print(f"Khả thi: {current_violations == 0}")
397
398     # ----- Bước 2: Khởi tạo lời giải tốt nhất -----
399     best_x = current_x.copy()
400     best_cost = current_cost
401     best_w = current_w
402     best_violations = current_violations
403
404     # ----- Khởi tạo biến theo dõi -----
405     temperature = INITIAL_TEMP             # Nhiệt độ hiện tại T
406     iteration = 0                           # Bộ đếm vòng lặp
407     no_improve_count = 0                    # Bộ đếm vòng không cải thiện
408     history = []                            # Lịch sử các lời giải tốt nhất
409
410     # Dữ liệu cho đồ thị hội tụ
411     weight_list = [current_w]
412     iter_list = [0]
413     feasible_list = [current_violations == 0]
414
415     # ----- Bước 3: Vòng lặp chính -----
416     print(f"\n{'='*60}")
417     print(f"Bắt đầu tối ưu hóa: T₀={INITIAL_TEMP}, α={COOLING_RATE}, MaxIter={MAX_ITER}")
418     print(f"{'='*60}")
419
420     while temperature > MIN_TEMP and iteration < MAX_ITER:
421         iteration += 1
422
423         # --- Bước 3.1: Tạo lời giải lân cận ---
424         neighbor_x = generate_neighbor(current_x)
425
426         # --- Bước 3.2: Đánh giá lời giải mới ---
427         print(f"\n--- Vòng lặp {iteration} | T = {temperature:.2f} ---")
```

```
        neighbor_w, neighbor_violations = objective_function(sap_model, neighbor_x)
        neighbor_cost = penalized_cost(neighbor_w, neighbor_violations)

        # --- Bước 3.3: Quyết định chấp nhận theo tiêu chuẩn Metropolis ---
        P = acceptance_probability(current_cost, neighbor_cost, temperature)
        r = random.random()    # Số ngẫu nhiên r ∈ [0, 1)

        if r < P:
            # CHẤP NHẬN lời giải mới
            old_cost = current_cost   # lưu lại để in thông tin (chỉ phục vụ hiển thị)
            current_x = neighbor_x
            current_cost = neighbor_cost
            current_w = neighbor_w
            current_violations = neighbor_violations

            result_str = "✓ CHẤP NHẬN"
            if neighbor_cost >= old_cost:   # chấp nhận lời giải không tốt hơn
                result_str += f" (P={P:.4f}, r={r:.4f})"

            # Cập nhật lời giải tốt nhất nếu cần
            if neighbor_cost < best_cost or \
               (neighbor_cost == best_cost and neighbor_violations < best_violations):
                best_x = neighbor_x.copy()
                best_cost = neighbor_cost
                best_w = neighbor_w
                best_violations = neighbor_violations
                no_improve_count = 0

                history.append((best_x.copy(), best_w, best_violations == 0))
                print(f"  ★ LỜI GIẢI TỐT NHẤT MỚI! W = {best_w:.2f} kips")
            else:
                no_improve_count += 1
        else:
            # TỪ CHỐI lời giải mới
            no_improve_count += 1
            result_str = f"× TỪ CHỐI (P={P:.4f}, r={r:.4f})"

        print(f"  {result_str}")

        # Lưu dữ liệu cho đồ thị
        weight_list.append(current_w)
        iter_list.append(iteration)
        feasible_list.append(current_violations == 0)

        # --- Bước 3.4: Giảm nhiệt độ ---
        temperature *= COOLING_RATE    # T ← α × T

        # Khởi động lại từ lời giải tốt nhất nếu không cải thiện quá lâu
        if no_improve_count >= RESTART_THRESHOLD:
            print(f"\n  ↻ Khởi động lại từ lời giải tốt nhất "
                  f"(không cải thiện {RESTART_THRESHOLD} vòng)")
            current_x = best_x.copy()
            current_cost = best_cost
            current_w = best_w
            current_violations = best_violations
            no_improve_count = 0

    # ----- Bước 4: Kết quả cuối cùng -----
    best_feasible = (best_violations == 0)

    print(f"\n{'='*60}")
    print(f"KẾT QUẢ TỐI ƯU HÓA")
    print(f"{'='*60}")
    print(f"Tiết diện tối ưu: {[W_SECTIONS[i] for i in best_x]}")
    print(f"Trọng lượng:      {best_w:.2f} kips")
    print(f"Khả thi:          {best_feasible}")
    print(f"Số vòng lặp:      {iteration}")

    # In lịch sử cải thiện
    if history:
        print(f"\nLịch sử cải thiện lời giải tốt nhất:")
        for idx, (sol, w, feasible) in enumerate(history):
            print(f"  {idx+1}. W = {w:.2f} kips | Khả thi: {feasible}")
            print(f"     Tiết diện: {[W_SECTIONS[i] for i in sol]}")
```

```
502
503     # Vẽ đồ thị hội tụ
504     plot_convergence(iter_list, weight_list, feasible_list)
505
506     return best_x, best_w, best_feasible
507
508 # ============================================================================
509 # PHẦN 5: KẾT QUẢ VÀ TRỰC QUAN HÓA
510 # ============================================================================
511
512 def plot_convergence(iter_list, weight_list, feasible_list):
513     """
514     Vẽ đồ thị quá trình hội tụ của thuật toán SA.
515
516     - Điểm xanh (●): lời giải khả thi
517     - Điểm đỏ  (×): lời giải không khả thi
518     - Đường xanh:    trọng lượng theo vòng lặp
519     """
520     fig, ax = plt.subplots(figsize=(10, 6))
521
522     # Vẽ đường trọng lượng
523     ax.plot(iter_list, weight_list, 'b-', linewidth=1, alpha=0.7, label='Trọng lượng')
524
525     # Phân loại điểm khả thi / không khả thi
526     f_iters = [iter_list[i] for i, f in enumerate(feasible_list) if f]
527     f_weights = [weight_list[i] for i, f in enumerate(feasible_list) if f]
528     nf_iters = [iter_list[i] for i, f in enumerate(feasible_list) if not f]
529     nf_weights = [weight_list[i] for i, f in enumerate(feasible_list) if not f]
530
531     # Vẽ điểm phân tán
532     ax.scatter(f_iters, f_weights, c='green', marker='o', s=30,
533                label='Khả thi', zorder=5)
534     ax.scatter(nf_iters, nf_weights, c='red', marker='x', s=30,
535                label='Không khả thi', zorder=5)
536
537     # Định dạng đồ thị
538     ax.set_xlabel('Vòng lặp', fontsize=12)
539     ax.set_ylabel('Trọng lượng (kips)', fontsize=12)
540     ax.set_title('Quá trình hội tụ - Simulated Annealing', fontsize=14)
541     ax.legend(fontsize=11)
542     ax.grid(True, alpha=0.3)
543
544     plt.tight_layout()
545     plt.savefig(os.path.join(CURRENT_DIR, 'optimization_progress.png'), dpi=150)
546     plt.show()
547     print(f"Đã lưu đồ thị: optimization_progress.png")
548
549 def print_summary(solution, weight, feasible):
550     """In bảng tóm tắt kết quả tối ưu."""
551     print(f"\n{'='*60}")
552     print(f"TÓM TẮT KẾT QUẢ TỐI ƯU HÓA")
553     print(f"{'='*60}")
554     print(f"{'Nhóm thanh':<12} {'Tiết diện':<15}")
555     print(f"{'-'*27}")
556     for i, idx in enumerate(solution):
557         print(f"  G{i+1:<9} {W_SECTIONS[idx]:<15}")
558     print(f"{'-'*27}")
559     print(f"Trọng lượng tối ưu: {weight:.2f} kips")
560     print(f"Tất cả thanh đạt:  {feasible}")
561     print(f"{'='*60}")
562
563 # ============================================================================
564 # PHẦN 6: CHƯƠNG TRÌNH CHÍNH
565 # ============================================================================
566
567 def main():
568     """
569     Chương trình chính: khởi động SAP2000, chạy tối ưu hóa, lưu kết quả.
570     """
571     try:
572         # Bước 1: Khởi động SAP2000
573         print("Đang kết nối SAP2000...")
574         sap_object, sap_model = start_sap()
575
576         # Bước 2: Chạy tối ưu hóa bằng Simulated Annealing
```

```
        best_solution, best_weight, best_feasible = simulated_annealing(sap_model)

        # Bước 3: Áp dụng lời giải tốt nhất và lưu mô hình
        if best_feasible:
            print("\nĐang áp dụng lời giải tối ưu vào mô hình...")
            objective_function(sap_model, best_solution)

            output_path = os.path.join(CURRENT_DIR, "model_optimized.sdb")
            sap_model.File.Save(output_path)
            print(f"Đã lưu mô hình tối ưu: {output_path}")
        else:
            print("\n! Cảnh báo: Lời giải tốt nhất KHÔNG khả thi. Không lưu mô hình.")

        # In tóm tắt
        print_summary(best_solution, best_weight, best_feasible)

    except Exception as e:
        print(f"\nLỗi: {str(e)}")

    finally:
        # Đóng SAP2000
        if 'sap_object' in locals():
            try:
                print("\nĐang đóng SAP2000...")
                sap_object.ApplicationExit(False)
                print("Đã đóng SAP2000 thành công")
            except:
                print("Lỗi khi đóng SAP2000")

if __name__ == "__main__":
    main()
```

Code 8.3: Dữ liệu giàn không gian 120 thanh (Bài 8.1)

```
function Truss_Data=TrussData_120bar
% ------------------------------------------------------------------------
% Nguồn: dữ liệu lấy từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity (psi)
E=30450000;
```

. . . (dòng 16–240: tọa độ nút, liên kết thanh và điều kiện biên; xem tệp đầy đủ trên GitHub)

```
% Loading     ----- > Load(dn_no,:)=[x y z] (lbs)
yuk=zeros(size(dnkoor));
yuk(1,:)=[0 0 -13490];
for i=2:14
yuk(i,:)=[0 0 -6744];
end
for i=15:37
yuk(i,:)=[0 0 -2248];
end

% Steel yield strength (psi)
fy=58000;

% Allowed Displacement (inch) = 5 mm
maxYER=0.1969;%inch

% Khối lượng riêng
GS=0.288;%lb/in^3

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk',yuk,'fy',fy,'maxYER',maxYER,'GS',GS);
```

Code 8.4: Dữ liệu giàn 37 thanh, ràng buộc tần số (Bài 8.2)

```
function Truss_Data=TrussData_37bar_modal
% ------------------------------------------------------------------------
% Nguồn: chuyển thể (quy đổi đơn vị, bổ sung chú thích và dữ liệu) từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity
E=2.10e11;
```

. . . (dòng 16–100: tọa độ nút, liên kết thanh và điều kiện biên; xem tệp đầy đủ trên GitHub)

```
% Fixed cross sections
A=zeros(size(eldn,1),1);
A(28:37)=4e-3; %m2

% Frequency limits (Hz)
limfre(1)=20;
limfre(2)=40;
limfre(3)=60;

% Khối lượng riêng
GS=7800;%kg/m^3

% added masses
add_mass=zeros(size(dnkoor,1),1);
add_mass(2:2:18)=10; %kg

% Bar grouping (14 symmetric groups for the 27 active members)
% Elements 28-37 are constant and excluded from active design groups
groups={[1,27], [4,25], [7,22], [10,19], [13,16], ... % Lower chords (5 groups)
        [2,26], [5,23], [8,20], [11,17], [14],    ... % Verticals (5 groups)
        [3,24], [6,21], [9,18], [12,15]};             % Diagonals (4 groups)

% Design variable bounds (19 variables total)
% Variables 1-5: Shape variables (Y-coordinates of upper nodes in meters)
% Variables 6-19: Sizing variables (Cross-sectional areas of the 14 groups in m^2)
LB = [ones(1,5)*0.5, ones(1,14)*1e-4];   % Y_min = 0.5m, A_min = 1 cm2
UB = [ones(1,5)*5.0, ones(1,14)*100e-4]; % Y_max = 5.0m, A_max = 100 cm2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,'E',E,...
    'GS',GS,'add_mass',add_mass,'limfre',limfre,'A',A,...
    'groups',{groups},'LB',LB,'UB',UB);
```

Code 8.5: Dữ liệu giàn không gian 52 thanh, ràng buộc tần số (Bài 8.3)

```
function Truss_Data=TrussData_52bar_modal
% ------------------------------------------------------------------------
% Nguồn: dữ liệu lấy từ tệp cùng tên trong
% bộ bài toán giàn chuẩn của H.T. Ozturk và H.T. Kahraman (2023),
% Applied Soft Computing 145, 110573.
% Copyright (c) 2024, Hamdi Tolga KAHRAMAN. All rights reserved.
% Phân phối lại theo giấy phép BSD: toàn văn giấy phép (các điều kiện và phần
% miễn trừ trách nhiệm) có trong tệp LICENSE_Kahraman_BSD.txt đi kèm thư mục này.
% Tên biến theo bộ dữ liệu gốc: dnkoor = tọa độ nút; eldn = nút đầu, nút cuối
% của thanh; MesKos = điều kiện biên (1 = cố định); yuk = tải trọng nút;
% maxGER = ứng suất cho phép; maxYER = chuyển vị cho phép; GS = khối lượng riêng.
% ------------------------------------------------------------------------

% Modulus of Elasticity
E=2.10e11;
```

. . . (dòng 16–93: liên kết thanh và điều kiện biên; xem tệp đầy đủ trên GitHub)

```
% Frequency limits (Hz)
limfre(1)=15.916;
limfre(2)=28.648;
```

```
97
98 % Khối lượng riêng
99 GS=7800;%kg/m^3
100
101 % added masses
102 add_mass=zeros(size(MesKos,1));
103 add_mass(1:13)=50; %kg
104
105 Truss_Data=struct('MesKos',MesKos,'eldn',eldn,'E',E,'GS',GS,'add_mass',add_mass,'limfre',limfre);
```

Chương 9. Ứng dụng tối ưu kết cấu trong công trình cầu

Các chương trình vẽ hình (`girder_figs.py`, `section_fig.py`, `tmd_figs.py`, `namo_figs.py`), chương trình kiểm chứng `tmd_check.py` và các tệp kết quả `*.json` không in lại ở đây, chỉ có trong kho mã. Các chương trình cần chạy theo thứ tự: mô hình, tối ưu, phân tích, vẽ hình.

Code 9.1: Mô hình kiểm toán dầm thép liên hợp theo TCVN 11823:2017 (Ví dụ 9.1, Python)

```
1 """
2 Code 9.1 - Tối ưu dầm thép chữ I liên hợp nhịp giản đơn theo TCVN 11823:2017.
3
4 Bài toán dựng lại theo Hùng và cs. (2021), Tạp chí KHCN Xây dựng NUCE 15(3V):55-68,
5 ví dụ dầm giữa, cầu nhịp 33 m (nhịp tính toán 32 m). Đơn vị: N, mm, MPa.
6 Biến thiết kế x = [b_tf, t_tf, b_bf, t_bf, t_w, D] (mm).
7
8 Hai mô hình:
9   'A' - gần với giả thiết của bài báo: mọi tải trọng do tiết diện liên hợp chịu
10        (thi công có chống đỡ), không kể trọng lượng bản thân dầm thép, mỏi với hệ số 0,75;
11  'B' - đầy đủ theo trình tự thi công không chống đỡ: dầm thép chịu tĩnh tải giai đoạn 1
12        (kể cả trọng lượng bản thân), kiểm toán giai đoạn thi công, mỏi I (hệ số 1,5).
13 """
14 import numpy as np
15
16 # ------------------------------------------------------------------------------ số liệu
17 E, Fy, fc = 2.0e5, 345.0, 30.0       # MPa
18 N_MOD = 8                            # hệ số quy đổi n (ngắn hạn); dài hạn 3n
19 BE, TS, HH = 2300.0, 241.6, 98.4     # bản quy đổi (mm); HH: khoảng vút hiệu chỉnh theo Bảng 3 bài báo
20 L = 32000.0                          # nhịp tính toán (mm)
21 L_GIRDER = 33000.0                   # chiều dài dầm để tính khối lượng (mm)
22 RHO = 7.85e-6                        # kg/mm^3
23 G_ACC = 9.81e-3                      # N/kg tính theo mm: rho*A*G_ACC -> N/mm
24 DC1_EXT = 13.35 + 1.75 + 0.2         # N/mm: bản + vút; dầm ngang + sườn; chốt + mối nối
25 DW = 3.62                            # N/mm: lớp phủ mặt đường
26 DFM, DFV = 0.598, 0.793              # hệ số phân bố mô-men / lực cắt
27 IM, IM_FAT = 0.33, 0.15
28 LANE = 9.3                           # N/mm
29 LB = 4000.0                          # chiều dài không giằng bản cánh trên khi thi công (dầm ngang cách
       ↪ 4 m, giả thiết)
30 DF_TH = 82.7                         # MPa, ngưỡng mỏi chi tiết loại C'
31 TRUCK = [(0.0, 35e3), (4300.0, 145e3), (8600.0, 145e3)]
32 TANDEM = [(0.0, 110e3), (1200.0, 110e3)]
33 FATIGUE_TRUCK = [(0.0, 35e3), (4300.0, 145e3), (13300.0, 145e3)]
34 SECTIONS = np.array([0.0, 1/8, 2/8, 3/8, 4/8]) * L          # MC1..MC5
35 BOUNDS = np.array([[50, 600], [5, 100], [50, 600], [5, 100], [5, 30], [500, 3000]], float)
36 NAMES = ['b_tf', 't_tf', 'b_bf', 't_bf', 't_w', 'D']
37 X_EMP = np.array([300, 16, 500, 22, 14, 1125], float)          # thiết kế kinh nghiệm (Hình 1b bài báo)
38 X_PAPER = np.array([228.64, 10.047, 240.60, 25.173, 9.1245, 1368.60])  # nghiệm công bố (Bảng 4)
39
40 MODELS = {
41     'A': dict(self_weight=False, staged=False, construction=False, gamma_fat=0.75),
42     'B': dict(self_weight=True, staged=True, construction=True, gamma_fat=1.5),
43 }
44
45 # ------------------------------------------------------------------------------ hoạt tải
46 def _envelope(axles, x, ordinate):
47     """Giá trị lớn nhất của đại lượng có đường ảnh hưởng 'ordinate' khi đoàn trục chạy qua."""
48     best = 0.0
49     for sgn in (1, -1):
50         for lead in np.linspace(-20000, L + 20000, 4001):
51             val = sum(P * ordinate(lead + sgn * a, x) for a, P in axles if 0 <= lead + sgn * a <= L)
52             best = max(best, val)
```

```
    return best

_M_IL = lambda p, x: (L - p) * x / L if p >= x else p * (L - x) / L
_V_IL = lambda p, x: (L - p) / L if p > x else (-p / L if p < x else 0.0)
_D_IL = lambda p, x: (p * (L - x) * (L**2 - p**2 - (L - x)**2) / (6 * L) if p <= x
                      else x * (L - p) * (L**2 - x**2 - (L - p)**2) / (6 * L))  # độ võng tại x (nhân 1/EI)

def _live_effects():
    out = []
    for x in SECTIONS:
        Mt = max(_envelope(TRUCK, x, _M_IL), _envelope(TANDEM, x, _M_IL))
        Vt = max(_envelope(TRUCK, x, _V_IL), _envelope(TANDEM, x, _V_IL))
        out.append(dict(M=(1 + IM) * Mt + LANE * x * (L - x) / 2,
                        V=(1 + IM) * Vt + LANE * (L - x)**2 / (2 * L),
                        Mfat=(1 + IM_FAT) * _envelope(FATIGUE_TRUCK, x, _M_IL)))
    dt = (1 + IM) * _envelope(TRUCK, L / 2, _D_IL)
    return out, max(dt, 0.25 * dt + LANE * 5 * L**4 / 384)

LIVE, DEFL_LL = _live_effects()

# ---------------------------------------------------------------------------- tiết diện
def steel_section(x):
    btf, ttf, bbf, tbf, tw, D = x
    parts = [(bbf * tbf, tbf / 2, bbf * tbf**3 / 12),
             (D * tw, tbf + D / 2, tw * D**3 / 12),
             (btf * ttf, tbf + D + ttf / 2, btf * ttf**3 / 12)]
    A = sum(p[0] for p in parts)
    yb = sum(p[0] * p[1] for p in parts) / A
    I = sum(p[2] + p[0] * (p[1] - yb)**2 for p in parts)
    return A, yb, I, tbf + D + ttf

def composite_section(x, nn):
    A, yb, I, d = steel_section(x)
    Ac, yc = BE * TS / nn, d + HH + TS / 2
    yt = (A * yb + Ac * yc) / (A + Ac)
    It = I + A * (yb - yt)**2 + BE * TS**3 / (12 * nn) + Ac * (yc - yt)**2
    return yt, It

def plastic(x):
    """Mô-men dẻo Mp, khoảng cách Dp từ mặt bản đến trục trung hòa dẻo, Dcp (phần bụng chịu nén)."""
    btf, ttf, bbf, tbf, tw, D = x
    Ps = 0.85 * fc * BE * TS
    Pc, Pw, Pt = Fy * btf * ttf, Fy * D * tw, Fy * bbf * tbf
    if Pt + Pw >= Pc + Ps:                          # trục trung hòa dẻo trong bản bụng
        Yb = D / 2 * ((Pt - Pc - Ps) / Pw + 1)
        Mp = (Pw / (2 * D) * (Yb**2 + (D - Yb)**2) + Ps * (Yb + ttf + HH + TS / 2)
              + Pc * (Yb + ttf / 2) + Pt * (D - Yb + tbf / 2))
        return Mp, TS + HH + ttf + Yb, Yb
    if Pt + Pw + Pc >= Ps:                          # trong bản cánh trên
        Yb = ttf / 2 * ((Pw + Pt - Ps) / Pc + 1)
        Mp = (Pc / (2 * ttf) * (Yb**2 + (ttf - Yb)**2) + Ps * (Yb + HH + TS / 2)
              + Pw * (ttf - Yb + D / 2) + Pt * (ttf - Yb + D + tbf / 2))
        return Mp, TS + HH + Yb, 0.0
    Yb = TS * (Pc + Pw + Pt) / Ps                   # trong bản bê tông
    c = TS - Yb + HH
    Mp = Yb**2 * Ps / (2 * TS) + Pc * (c + ttf / 2) + Pw * (c + ttf + D / 2) + Pt * (c + ttf + D + tbf / 2)
    return Mp, Yb, 0.0

def shear_C(D, tw, k):
    r = np.sqrt(E * k / Fy)
    lam = D / tw
    if lam <= 1.12 * r:
        return 1.0
    if lam <= 1.40 * r:
        return 1.12 * r / lam
    return 1.57 / lam**2 * (E * k / Fy)

def construction_capacity(x):
    """Ứng suất danh định bản cánh nén của dầm thép khi thi công (cục bộ + xoắn ngang, Cb = 1)."""
    btf, ttf, bbf, tbf, tw, D = x
    A, yb, I, d = steel_section(x)
    Dc = d - yb - ttf                               # chiều cao bụng chịu nén (đàn hồi)
    lam_f, lam_pf = btf / (2 * ttf), 0.38 * np.sqrt(E / Fy)
```

```
    lam_rf = 0.56 * np.sqrt(E / (0.7 * Fy))
    if lam_f <= lam_pf:
        F_flb = Fy
    else:
        F_flb = (1 - 0.3 * (lam_f - lam_pf) / (lam_rf - lam_pf)) * Fy
    rt = btf / np.sqrt(12 * (1 + Dc * tw / (3 * btf * ttf)))
    Lp, Lr = rt * np.sqrt(E / Fy), np.pi * rt * np.sqrt(E / (0.7 * Fy))
    if LB <= Lp:
        F_ltb = Fy
    elif LB <= Lr:
        F_ltb = (1 - 0.3 * (LB - Lp) / (Lr - Lp)) * Fy
    else:
        F_ltb = np.pi**2 * E / (LB / rt)**2
    k_w = 9 / (Dc / D)**2                          # oằn bụng do uốn
    F_crw = min(0.9 * E * k_w / (D / tw)**2, Fy)
    return min(F_flb, F_ltb), F_crw, Dc

# ------------------------------------------------------------------------------ mục tiêu và ràng buộc
def mass(x):
    return RHO * steel_section(x)[0] * L_GIRDER       # kg

def constraints(x, model='B'):
    """Dict {tên: g}; g <= 0 nghĩa là thỏa mãn (các ràng buộc đã chuẩn hóa không thứ nguyên)."""
    opt = MODELS[model]
    btf, ttf, bbf, tbf, tw, D = x
    A, yb, I, d = steel_section(x)
    y8, I8 = composite_section(x, N_MOD)
    y24, I24 = composite_section(x, 3 * N_MOD)
    Mp, Dp, Dcp = plastic(x)
    Dt = TS + HH + d
    wdc = DC1_EXT + (RHO * A * G_ACC if opt['self_weight'] else 0.0)
    # mô-đun chống uốn (thớ trên, thớ dưới) cho tải giai đoạn 1 và giai đoạn 2
    S1t, S1b = ((I / (d - yb), I / yb) if opt['staged'] else (I24 / (d - y24), I24 / y24))
    S2t, S2b = I24 / (d - y24), I24 / y24
    S3t, S3b = I8 / (d - y8), I8 / y8
    g = {}
    # (a) cấu tạo
    g['D/t_w <= 150'] = D / tw / 150 - 1
    g['b_tf/2t_tf <= 12'] = btf / (2 * ttf) / 12 - 1
    g['b_bf/2t_bf <= 12'] = bbf / (2 * tbf) / 12 - 1
    g['b_tf >= D/6'] = D / (6 * btf) - 1
    g['b_bf >= D/6'] = D / (6 * bbf) - 1
    g['t_tf >= 1,1t_w'] = 1.1 * tw / ttf - 1
    g['t_bf >= 1,1t_w'] = 1.1 * tw / tbf - 1
    r = (ttf * btf**3) / (tbf * bbf**3)
    g['I_yc/I_yt >= 0,1'] = 0.1 / r - 1
    g['I_yc/I_yt <= 10'] = r / 10 - 1
    # (b) cường độ CĐ1: uốn tại MC2..MC5, cắt tại MC1..MC5
    compact = 2 * Dcp / tw <= 3.76 * np.sqrt(E / Fy)
    Mn = Mp if Dp <= 0.1 * Dt else Mp * (1.07 - 0.7 * Dp / Dt)
    g['D_p <= 0,42D_t'] = Dp / (0.42 * Dt) - 1
    flex, shear = [], []
    Vp = 0.58 * Fy * D * tw
    for i, (xs, lv) in enumerate(zip(SECTIONS, LIVE)):
        Mdc, Mdw = wdc * xs * (L - xs) / 2, DW * xs * (L - xs) / 2
        Mll = DFM * lv['M']
        if i > 0:
            if compact:
                flex.append((1.25 * Mdc + 1.5 * Mdw + 1.75 * Mll) / Mn - 1)
            else:
                ft = 1.25 * Mdc / S1t + 1.5 * Mdw / S2t + 1.75 * Mll / S3t
                fb = 1.25 * Mdc / S1b + 1.5 * Mdw / S2b + 1.75 * Mll / S3b
                flex.append(max(ft, fb) / Fy - 1)
        Vu = 1.25 * wdc * (L / 2 - xs) + 1.5 * DW * (L / 2 - xs) + 1.75 * DFV * lv['V']
        if i == 0:      # khoang đầu dầm, d0 = 0,5D, không kể trường kéo
            Vn = shear_C(D, tw, 5 + 5 / 0.5**2) * Vp
        else:           # khoang trong, d0 = D, có trường kéo
            C = shear_C(D, tw, 10.0)
            Vn = Vp * (C + 0.87 * (1 - C) / np.sqrt(2.0))
        shear.append(Vu / Vn - 1)
    g['uốn CĐ1'] = max(flex)
    g['cắt CĐ1'] = max(shear)
    # (c) sử dụng SD2 tại giữa nhịp và độ võng hoạt tải
    Mdc, Mdw, Mll = wdc * L**2 / 8, DW * L**2 / 8, 1.3 * DFM * LIVE[-1]['M']
    g['SD2 bản cánh trên'] = (Mdc / S1t + Mdw / S2t + Mll / S3t) / (0.95 * Fy) - 1
```

```
    g['SD2 bản cánh dưới'] = (Mdc / S1b + Mdw / S2b + Mll / S3b) / (0.95 * Fy) - 1
    g['độ võng L/800'] = DFM * DEFL_LL / (E * I8) / (L / 800) - 1
    # (d) mỏi tại bản cánh dưới giữa nhịp (một làn: DF/1,2)
    dF = opt['gamma_fat'] * (DFM / 1.2) * LIVE[-1]['Mfat'] / S3b
    g['mỏi bản cánh dưới'] = dF / DF_TH - 1
    # (e) giai đoạn thi công: dầm thép chịu 1,25 x tĩnh tải giai đoạn 1 (kể cả bê tông ướt)
    if opt['construction']:
        Mc = 1.25 * wdc * L**2 / 8
        F_nc, F_crw, Dc = construction_capacity(x)
        g['thi công: bản cánh nén'] = Mc * (d - yb) / I / F_nc - 1
        g['thi công: bản cánh kéo'] = Mc * yb / I / Fy - 1
        g['thi công: oằn bụng'] = Mc * Dc / I / F_crw - 1
    return g

def report(x, model, title):
    gg = constraints(x, model)
    print(f'--- [{model}] {title}: khối lượng = {mass(x):.1f} kg, vi phạm lớn nhất = {max(gg.values())
    ↪ :+.4f}')
    for k, v in gg.items():
        tag = 'HOẠT ĐỘNG' if abs(v) < 5e-3 else ('VI PHẠM' if v > 0 else '')
        print(f'    {k:24s} {v:+.4f} {tag}')

if __name__ == '__main__':
    for m in ('A', 'B'):
        report(X_EMP, m, 'thiết kế kinh nghiệm')
        report(X_PAPER, m, 'nghiệm công bố')
```

Code 9.1 (tiếp): Giải bằng SQP và tiến hóa vi sai

```
"""Code 9.1 (phần giải): SQP đa điểm xuất phát và tiến hóa vi phân (DE) với hàm phạt."""
import time, json
import numpy as np
from scipy.optimize import minimize, differential_evolution
import girder as G

LB_, UB_ = G.BOUNDS[:, 0], G.BOUNDS[:, 1]
X_REF = G.X_EMP.copy()                    # đưa biến về thang đo ~1 để SQP ổn định

def gvec(x, model):
    return np.array(list(G.constraints(x, model).values()))

def solve_sqp(model, x0, extra=None):
    cons = [{'type': 'ineq', 'fun': lambda z: -gvec(z * X_REF, model)}]
    if extra:
        cons += extra
    res = minimize(lambda z: G.mass(z * X_REF) / 1000.0, x0 / X_REF, method='SLSQP',
                   bounds=list(zip(LB_ / X_REF, UB_ / X_REF)), constraints=cons,
                   options=dict(maxiter=500, ftol=1e-10))
    x = res.x * X_REF
    return x, G.mass(x), gvec(x, model).max(), res.nit

def penalized(x, model, alpha=1e4):
    """Hàm phạt dạng V(1 + sum alpha*beta) như công thức (7) của bài báo."""
    viol = np.maximum(gvec(x, model), 0.0).sum()
    return G.mass(x) * (1 + alpha * viol)

def solve_de(model, seed, integrality=None):
    rng = np.random.default_rng(seed)
    init = LB_ + rng.random((25, 6)) * (UB_ - LB_)          # NP = 25 như bài báo
    res = differential_evolution(penalized, G.BOUNDS, args=(model,), strategy='rand1bin',
                                 maxiter=500, mutation=0.7, recombination=0.6, init=init,
                                 tol=0, polish=False, seed=seed, integrality=integrality)
    return res.x, G.mass(res.x), gvec(res.x, model).max(), res.nfev

if __name__ == '__main__':
    out = {}
    for model in ('A', 'B'):
        t0 = time.time()
        best = None
        rng = np.random.default_rng(1)
        starts = [G.X_EMP] + [LB_ + rng.random(6) * (UB_ - LB_) for _ in range(30)]
        sq = []
        for s in starts:
            x, m, v, it = solve_sqp(model, s)
            if v <= 1e-6:
```

```
                sq.append((m, x, it))
        sq.sort(key=lambda t: t[0])
        t_sqp = time.time() - t0
        print(f'[{model}] SQP: {len(sq)}/{len(starts)} khả thi, tốt nhất {sq[0][0]:.1f} kg, '
              f'từ thiết kế kinh nghiệm {solve_sqp(model, G.X_EMP)[1]:.1f} kg, {t_sqp:.1f} s')
        print('      x* =', np.round(sq[0][1], 3))
        t0 = time.time()
        de = [solve_de(model, s) for s in range(20)]
        ms = np.array([d[1] for d in de]); feas = np.array([d[2] <= 1e-6 for d in de])
        ib = int(np.argmin(np.where(feas, ms, np.inf)))
        print(f'[{model}] DE 20 lần: tốt nhất {ms[feas].min():.1f}, kém nhất {ms[feas].max():.1f}, '
              f'TB {ms[feas].mean():.1f}, độ lệch {ms[feas].std():.2f} kg, khả thi {feas.sum()}/20, '
              f'{de[0][3]} lượt tính/lần, {time.time()-t0:.1f} s')
        print('      x* =', np.round(de[ib][0], 3))
        G.report(sq[0][1], model, 'nghiệm SQP')
        out[model] = dict(sqp_best=sq[0][0], sqp_x=sq[0][1].tolist(), sqp_feasible=len(sq),
                          sqp_starts=len(starts), de=ms.tolist(), de_feas=feas.tolist(),
                          de_x=de[ib][0].tolist())
    json.dump(out, open('girder_results.json', 'w'), indent=1)
```

Code 9.1 (tiếp): Nhân tử Lagrange, nghiệm rời rạc và đường đánh đổi

```
"""Code 9.1 (phần phân tích): nhân tử Lagrange, nghiệm rời rạc và đường đánh đổi khối lượng - chiều cao. """
import json
import numpy as np
from scipy.optimize import differential_evolution, nnls
import girder as G
from girder_opt import solve_sqp, gvec

MODEL = 'B'
res = json.load(open('girder_results.json'))
xB = np.array(res[MODEL]['sqp_x'])

# ---------------- nhân tử Lagrange: grad m = -sum(lambda_i grad g_i), lambda >= 0 (KKT, Chương 3)
names = list(G.constraints(xB, MODEL).keys())
g0 = gvec(xB, MODEL)
active = [i for i, v in enumerate(g0) if v > -5e-3]
h = 1e-4 * xB
grad_m = np.array([(G.mass(xB + h[j] * np.eye(6)[j]) - G.mass(xB - h[j] * np.eye(6)[j])) / (2 * h[j])
                   for j in range(6)])
J = np.array([[(gvec(xB + h[j] * np.eye(6)[j], MODEL)[i] - gvec(xB - h[j] * np.eye(6)[j], MODEL)[i]) /
               (2 * h[j])
               for j in range(6)] for i in active])
lam, resid = nnls(J.T, -grad_m)
print('Nhân tử Lagrange (kg trên một đơn vị nới lỏng ràng buộc chuẩn hóa):')
for i, l in zip(active, lam):
    print(f'   {names[i]:24s} lambda = {l:8.1f} kg   (nới 1% -> giảm ~{0.01*l:.1f} kg)')
print(f'   phần dư điều kiện dừng ||grad m + J^T lambda|| = {resid:.3e} (so với ||grad m|| = {np.linalg.norm(grad_m):.2f})')

# ---------------- nghiệm rời rạc: bề rộng bội số 10 mm, chiều dày theo danh mục thép tấm, D bội số 10 mm
THK = np.array([8, 10, 12, 14, 16, 18, 20, 22, 25, 28, 30, 32, 36, 40, 45, 50], float)
def decode(z):
    i = np.clip(np.round(z).astype(int), 0, None)
    return np.array([10 * i[0], THK[min(i[1], len(THK) - 1)], 10 * i[2], THK[min(i[3], len(THK) - 1)],
                     THK[min(i[4], len(THK) - 1)], 10 * i[5]], float)
def pen_discrete(z):
    x = decode(z)
    return G.mass(x) * (1 + 1e4 * np.maximum(gvec(x, MODEL), 0).sum())
bounds_d = [(5, 60), (0, len(THK) - 1), (5, 60), (0, len(THK) - 1), (0, 6), (50, 300)]
best = None
for seed in range(10):
    r = differential_evolution(pen_discrete, bounds_d, integrality=[True] * 6, seed=seed, maxiter=800,
                               popsize=10, mutation=0.7, recombination=0.6, tol=0, polish=False)
    x = decode(r.x)
    if gvec(x, MODEL).max() <= 0 and (best is None or G.mass(x) < G.mass(best)):
        best = x
print(f'Nghiệm rời rạc tốt nhất (10 lần DE): {best} -> {G.mass(best):.1f} kg')
G.report(best, MODEL, 'nghiệm rời rạc')

# ---------------- đường đánh đổi: min khối lượng với D <= D_max (phương pháp ràng buộc epsilon, Chương 7)
front = []
```

```
for Dmax in np.arange(900, 1801, 50):
    extra = [{'type': 'ineq', 'fun': lambda z, Dm=Dmax: Dm - z[5] * G.X_EMP[5]}]
    sols = []
    rng = np.random.default_rng(3)
    x0s = [np.minimum(xB, [600, 100, 600, 100, 30, Dmax])] + \
          [G.BOUNDS[:, 0] + rng.random(6) * (np.minimum(G.BOUNDS[:, 1], [600, 100, 600, 100, 30, Dmax])
    ↪ - G.BOUNDS[:, 0]) for _ in range(12)]
    for x0 in x0s:
        x, m, v, _ = solve_sqp(MODEL, x0, extra)
        if v <= 1e-6 and x[5] <= Dmax + 1e-6:
            sols.append((m, x))
    m, x = min(sols, key=lambda t: t[0])
    act = [k for k, val in G.constraints(x, MODEL).items() if abs(val) < 5e-3]
    front.append(dict(Dmax=float(Dmax), mass=float(m), x=x.tolist(), active=act))
    print(f'   D <= {Dmax:5.0f} mm: m = {m:8.1f} kg, D = {x[5]:7.1f}, hoạt động: {", ".join(act)}')
res[MODEL]['lambda'] = {names[i]: float(l) for i, l in zip(active, lam)}
res[MODEL]['discrete_x'] = best.tolist()
res[MODEL]['discrete_mass'] = float(G.mass(best))
res[MODEL]['front'] = front
json.dump(res, open('girder_results.json', 'w'), indent=1)
```

Code 9.2: Mô hình cầu – TMD chịu đoàn tàu HSLM-A (Ví dụ 9.2, Python)

```
"""
Code 9.2 - Tối ưu bộ giảm chấn khối lượng (TMD) cho cầu đường sắt cao tốc nhịp giản đơn
chịu các đoàn tàu chuẩn HSLM-A1...A10 (EN 1991-2, Phụ lục E).

Mô hình: dầm Euler-Bernoulli tựa đơn, chồng chất dạng dao động (3 dạng uốn), tải trọng là lực tập trung
di động (bỏ qua tương tác tàu - cầu); TMD gắn tại giữa nhịp. Phản ứng được tính trong miền tần số
(FFT) nên mỗi lần đánh giá hàm mục tiêu xét đủ 10 đoàn tàu x toàn bộ dải vận tốc.
"""
import numpy as np

# ------------------------------------------------------------------- cầu (ví dụ minh họa do tác giả lập)
L = 20.0              # m, nhịp tựa đơn
m = 15000.0           # kg/m, khối lượng phân bố (kể cả balát, ray, tà vẹt)
f1 = 6.0              # Hz, tần số riêng thứ nhất
EI = m * (2 * L**2 * f1 / np.pi)**2   # N m^2
zeta = 0.010          # tỷ số cản tối thiểu, cầu BTCT DƯL L >= 20 m (EN 1991-2, Bảng 6.6)
N_MODES = 3           # xét đến max(30 Hz; 1,5f1; f3) theo EN 1990 A2: dạng 3 (54 Hz)
A_LIM = 3.5           # m/s^2, giới hạn gia tốc mặt cầu đường balát (EN 1990 A2.4.4.2.1)
M1 = m * L / 2        # khối lượng dạng 1 (phi_1 = 1 tại giữa nhịp)
x_t = L / 2           # vị trí TMD
V_KMH = np.arange(150.0, 360.0 + 1e-9, 2.5)   # dải vận tốc: đến 1,2 x 300 km/h

# ------------------------------------------------------------------- đoàn tàu HSLM-A (Bảng E.1)
HSLM = {  # tên: (N toa trung gian, D chiều dài toa [m], d khoảng cách trục giá chuyển hướng [m], P [kN])
    'A1': (18, 18, 2.0, 170), 'A2': (17, 19, 3.5, 200), 'A3': (16, 20, 2.0, 180), 'A4': (15, 21, 3.0,
    ↪ 190),
    'A5': (14, 22, 2.0, 170), 'A6': (13, 23, 2.0, 180), 'A7': (13, 24, 2.0, 190), 'A8': (12, 25, 2.5,
    ↪ 190),
    'A9': (11, 26, 2.0, 210), 'A10': (11, 27, 2.0, 210)}

def hslm_axles(name):
    """Vị trí trục (m, tính từ trục đầu) và lực trục (N) của đoàn tàu HSLM-A theo Hình E.1."""
    N, D, d, P = HSLM[name]
    head = np.array([0.0, 3.0, 14.0, 17.0, 20.525, 23.525])  # đầu máy 3+11+3 m; cách 3,525 m tới giá tr
    ↪ ước của toa cuối
    c0 = 22.025 + D                                          # tâm giá chuyển hướng khớp nối đầu tiên
    centres = c0 + D * np.arange(N + 1)                      # N+1 giá khớp nối (toa cuối - N toa trung
    ↪ gian - toa cuối)
    body = np.concatenate([centres - d / 2, centres + d / 2])
    total = 44.05 + (N + 2) * D                              # đoàn tàu đối xứng: tổng chiều dài giữa hai
    ↪ trục ngoài cùng
    pos = np.sort(np.concatenate([head, body, total - head]))
    return pos, np.full(pos.size, P * 1e3)

# ------------------------------------------------------------------- lực dạng dao động và FFT (tính một l
    ↪ ần)
DT, NFFT = 0.004, 8192                                       # 250 Hz, 32,8 s (đủ cho đoàn tàu dài nhất ở
    ↪ 150 km/h)
FREQ = np.fft.rfftfreq(NFFT, DT)
OMEGA = 2 * np.pi * FREQ
WN = np.array([(n**2) * 2 * np.pi * f1 for n in range(1, N_MODES + 1)])
PHI_T = np.array([np.sin(n * np.pi * x_t / L) for n in range(1, N_MODES + 1)])   # dạng tại TMD và tại
```

```
    ↪ giữa nhịp

def modal_forces(name, v):
    pos, P = hslm_axles(name)
    t = np.arange(NFFT) * DT
    s = v * t[:, None] - pos[None, :]                       # vị trí từng trục trên cầu
    on = (s >= 0) & (s <= L)
    F = np.empty((N_MODES, NFFT))
    for k in range(N_MODES):
        F[k] = (P[None, :] * np.sin((k + 1) * np.pi * s / L) * on).sum(axis=1)
    return F

CASES = [(nm, v) for nm in HSLM for v in V_KMH / 3.6]

def precompute():
    return np.stack([np.fft.rfft(modal_forces(nm, v), axis=1) for nm, v in CASES])   # (ncase, nmode,
    ↪ nfreq)

FHAT = precompute()

# ------------------------------------------------------------------- phản ứng
def accel_spectra(mu=0.0, gamma=1.0, zt=0.0):
    """Hàm truyền gia tốc giữa nhịp H_n(omega) với mỗi lực dạng n, có/không TMD (ma trận động lực 4x4)."
    ↪ ""
    w = OMEGA
    nm = N_MODES
    Z = np.zeros((w.size, nm + 1, nm + 1), complex)
    for k in range(nm):
        Z[:, k, k] = M1 * (WN[k]**2 - w**2 + 2j * zeta * WN[k] * w)
    if mu > 0:
        mt = mu * M1
        wt = gamma * WN[0]
        kt, ct = mt * wt**2, 2 * zt * mt * wt
        zc = kt + 1j * w * ct
        for a in range(nm):
            for b in range(nm):
                Z[:, a, b] += zc * PHI_T[a] * PHI_T[b]
            Z[:, a, nm] = -zc * PHI_T[a]
            Z[:, nm, a] = -zc * PHI_T[a]
        Z[:, nm, nm] = -mt * w**2 + zc
    else:
        Z[:, nm, nm] = 1.0
    # gia tốc giữa nhịp do lực đơn vị ở dạng n: -w^2 * sum_j phi_j(L/2) * [Z^-1]_{j n}
    Zinv = np.linalg.inv(Z)
    return -(w**2)[:, None] * np.einsum('j,fjn->fn', PHI_T, Zinv[:, :nm, :nm])   # (nfreq, nmode)

def peak_accelerations(mu=0.0, gamma=1.0, zt=0.0):
    H = accel_spectra(mu, gamma, zt)                        # (nfreq, nmode)
    A = np.einsum('cnf,fn->cf', FHAT, H)
    a = np.fft.irfft(A, n=NFFT, axis=1)
    return np.abs(a).max(axis=1)                            # theo từng (đoàn tàu, vận tốc)

def a_max(theta, mu):
    gamma, zt = theta
    return peak_accelerations(mu, gamma, zt).max()

def den_hartog(mu):
    return 1 / (1 + mu), np.sqrt(3 * mu / (8 * (1 + mu)**3))

if __name__ == '__main__':
    import time
    print(f'EI = {EI:.4e} N m^2, M1 = {M1/1e3:.0f} t, so truc A1 = {hslm_axles("A1")[0].size}, chieu dai
    ↪  A1 = {hslm_axles("A1")[0][-1]:.1f} m')
    t0 = time.time(); a0 = peak_accelerations(); t1 = time.time()
    i = int(a0.argmax())
    print(f'Khong TMD: a_max = {a0.max():.2f} m/s^2 ({CASES[i][0]}, {CASES[i][1]*3.6:.1f} km/h); 1 lan
    ↪ danh gia {t1-t0:.2f} s')
    for mu in (0.01, 0.02, 0.03):
        g, z = den_hartog(mu)
        print(f'mu={mu:.2f}: Den Hartog gamma={g:.4f}, zeta_t={z:.4f} -> a_max={a_max((g, z), mu):.2f}')
```

Code 9.2 (tiếp): Den Hartog, SQP dạng biến phụ, GA và biên Pareto

```python
"""Code 9.2 (phần tối ưu): Den Hartog, bản đồ đẳng trị, SQP (dạng epigraph), GA và biên Pareto mu - a_max."""
import json, time
import numpy as np
from scipy.optimize import minimize
import tmd as T

BND = np.array([[0.80, 1.10], [0.005, 0.30]])          # gamma = f_t/f_1, zeta_t

def sqp(mu, x0):
    """min s  s.t.  a_c(gamma, zeta_t) <= s với mọi (đoàn tàu, vận tốc) c -- dạng epigraph trơn của min max."""
    def cons(z):
        return z[2] - T.peak_accelerations(mu, z[0], z[1])
    a0 = T.peak_accelerations(mu, *x0).max()
    res = minimize(lambda z: z[2], np.r_[x0, a0], method='SLSQP',
                   bounds=list(map(tuple, BND)) + [(0, None)],
                   constraints=[{'type': 'ineq', 'fun': cons}], options=dict(maxiter=100, ftol=1e-6))
    th = res.x[:2]
    return th, T.a_max(th, mu), res.nit, res.nfev

def ga(mu, pop=24, gens=40, seed=0, pc=0.9, pm=0.2):
    """GA số thực: chọn lọc đấu, lai ghép số học, đột biến Gauss, giữ cá thể tốt nhất (Chương 8)."""
    rng = np.random.default_rng(seed)
    lo, hi = BND[:, 0], BND[:, 1]
    X = lo + rng.random((pop, 2)) * (hi - lo)
    f = np.array([T.a_max(x, mu) for x in X])
    hist = [f.min()]
    for _ in range(gens):
        elite = X[f.argmin()].copy(); fe = f.min()
        def pick():
            i, j = rng.integers(pop, size=2)
            return X[i] if f[i] < f[j] else X[j]
        children = []
        while len(children) < pop - 1:
            p1, p2 = pick(), pick()
            if rng.random() < pc:
                w = rng.random()
                c1, c2 = w * p1 + (1 - w) * p2, (1 - w) * p1 + w * p2
            else:
                c1, c2 = p1.copy(), p2.copy()
            for c in (c1, c2):
                mask = rng.random(2) < pm
                c[mask] += rng.normal(0, 0.1, mask.sum()) * (hi - lo)[mask]
                children.append(np.clip(c, lo, hi))
        X = np.vstack([elite, children[:pop - 1]])
        f = np.r_[fe, [T.a_max(x, mu) for x in X[1:]]]
        hist.append(f.min())
    return X[f.argmin()], f.min(), hist, pop * (gens + 1)

if __name__ == '__main__':
    out = {}
    a0 = T.peak_accelerations()
    out['no_tmd'] = dict(a_max=float(a0.max()), case=T.CASES[int(a0.argmax())][0],
                         v=float(T.CASES[int(a0.argmax())][1] * 3.6))
    print('Không TMD:', out['no_tmd'])
    mu = 0.01
    dh = T.den_hartog(mu)
    a_dh = T.a_max(dh, mu)
    print(f'Den Hartog mu=1%: gamma={dh[0]:.4f}, zeta={dh[1]:.4f}, a_max={a_dh:.3f}')
    # bản đồ đẳng trị a_max(gamma, zeta_t) cho phương pháp đồ thị
    G_ = np.linspace(0.90, 1.04, 36); Z_ = np.linspace(0.01, 0.20, 32)
    t0 = time.time()
    grid = np.array([[T.a_max((g, z), mu) for g in G_] for z in Z_])
    print(f'Bản đồ {grid.size} điểm: {time.time()-t0:.0f} s; nhỏ nhất trên lưới {grid.min():.3f}')
    t0 = time.time()
    th_s, a_s, nit, nfev = sqp(mu, np.array(dh))
    print(f'SQP: gamma={th_s[0]:.4f}, zeta={th_s[1]:.4f}, a_max={a_s:.3f} ({nit} vòng lặp, {nfev} lượt, {time.time()-t0:.0f} s)')
    t0 = time.time()
    gas = [ga(mu, seed=s) for s in range(3)]
    bi = int(np.argmin([g[1] for g in gas]))
    print(f'GA (3 lần): ' + ', '.join(f'{g[1]:.3f}' for g in gas) + f'; tốt nhất gamma={gas[bi][0][0]:.4
```

```
        ↪ f}, zeta={gas[bi][0][1]:.4f}; {gas[bi][3]} lượt/lần; {time.time()-t0:.0f} s')
    out['mu1'] = dict(den_hartog=[float(v) for v in dh], a_dh=float(a_dh), sqp=th_s.tolist(), a_sqp=
        ↪ float(a_s),
                      sqp_nit=int(nit), sqp_nfev=int(nfev), ga=[g[0].tolist() for g in gas], a_ga=[float
        ↪ (g[1]) for g in gas],
                      ga_hist=[list(map(float, g[2])) for g in gas], ga_nfev=int(gas[0][3]),
                      grid=dict(gamma=G_.tolist(), zeta=Z_.tolist(), a=grid.tolist()))
    # biên Pareto: với mỗi mu, cực tiểu a_max theo (gamma, zeta_t)
    front = []
    x0 = np.array(T.den_hartog(0.0025))
    for mu_k in [0.0025, 0.005, 0.0075, 0.01, 0.015, 0.02, 0.025, 0.03, 0.04, 0.05]:
        start = x0 if T.a_max(x0, mu_k) < T.a_max(T.den_hartog(mu_k), mu_k) else np.array(T.den_hartog(
        ↪ mu_k))
        th, a, _, _ = sqp(mu_k, start)
        x0 = th
        front.append(dict(mu=mu_k, gamma=float(th[0]), zeta=float(th[1]), a_max=float(a),
                          a_dh=float(T.a_max(T.den_hartog(mu_k), mu_k))))
        print(f'   mu={mu_k*100:4.2f}%  m_t={mu_k*T.M1/1e3:5.2f} t  gamma={th[0]:.4f} zeta={th[1]:.4f}
        ↪ a_max={a:.3f} (Den Hartog {front[-1]["a_dh"]:.3f})')
    out['front'] = front
    # phản ứng theo vận tốc để vẽ hình (không TMD, Den Hartog, tối ưu) với mu = 1%
    curves = {}
    for lab, args in (('none', (0.0, 1.0, 0.0)), ('dh', (mu, *dh)), ('opt', (mu, *th_s))):
        a = T.peak_accelerations(*args).reshape(len(T.HSLM), len(T.V_KMH))
        curves[lab] = a.tolist()
    out['curves'] = dict(v=T.V_KMH.tolist(), trains=list(T.HSLM), a=curves)
    json.dump(out, open('tmd_results.json', 'w'), indent=1)
```

Code 9.3: Mô hình phần tử hữu hạn cầu Nam Ô (Ví dụ 9.3, Python)

```
"""
Code 9.3 - Mô hình phần tử hữu hạn không gian nhịp thứ nhất cầu giàn thép đường sắt Nam Ô (Đà Nẵng).

Số liệu theo Tran-Ngoc, Khatir, De Roeck, Bui-Tien, Nguyen-Ngoc, Abdel Wahab (2018),
Sensors 18(12):4131 (CC BY 4.0): tiết diện 9 nhóm thanh (Bảng 2), vật liệu (Bảng 3),
tần số đo (Bảng 5). Hình học dựng lại từ các hình của bài báo: nhịp 75 m chia 8 khoang,
cánh trên dạng parabol cao 13 m ở giữa nhịp, hai giàn chủ cách nhau 5 m.
Phần tử dầm không gian Euler-Bernoulli (6 bậc tự do/nút), ma trận khối lượng nhất quán.
"""
import numpy as np
from scipy.linalg import eigh

SPAN, NPANEL, H, B = 75.0, 8, 13.0, 5.0
RHO = 7850.0
M_NS = 247.2           # kg/m, khối lượng ray và bộ phận không kết cấu trên mỗi cánh dưới
Y_STR = 1.0            # m, vị trí dầm dọc (giả thiết, khổ đường 1 m)
# A (m2), Iz (m4, uốn ngoài mặt phẳng), Iy (m4, uốn trong mặt phẳng chính); J giả thiết (bài báo không
    ↪ cho)
SEC = {
    'upper':   (0.056, 6.70e-4, 3.10e-3), 'lower':   (0.020, 2.10e-4, 6.30e-4),
    'vertical': (0.010, 5.49e-5, 1.15e-4), 'diagonal': (0.014, 1.24e-4, 2.78e-4),
    'stringer': (0.020, 2.07e-4, 6.27e-4), 'transverse': (0.026, 2.03e-4, 3.61e-3),
    'strut':   (0.020, 6.25e-4, 2.80e-3), 'upper_brace': (0.0036, 8.00e-6, 1.09e-5),
    'lower_brace': (0.0049, 2.38e-6, 4.38e-6)}
J_FACTOR = 0.05        # J = J_FACTOR*(Iy + Iz), giả thiết
MEASURED = np.array([1.45, 3.11, 3.28, 4.62, 6.05])           # Hz, 5 dạng đầu (Bảng 5)
MEASURED_TYPE = ['L1', 'T1', 'L2', 'V1', 'T2']                 # ngang 1, xoắn 1, ngang 2, uốn đứng 1,
    ↪ xoắn 2
NOMINAL = dict(E=2.0e11, k_rot=8.0e5, k_v=1.3e10, k_h=1.5e7)  # giá trị ban đầu theo Bảng 7 bài báo

def geometry():
    dx = SPAN / NPANEL
    nodes, tag = [], {}
    def add(name, xyz):
        tag[name] = len(nodes); nodes.append(xyz)
    for side, y in (('A', -B / 2), ('B', B / 2)):
        for k in range(NPANEL + 1):
            add(f'L{side}{k}', (k * dx, y, 0.0))
        for k in range(1, NPANEL):
            x = k * dx
            add(f'U{side}{k}', (x, y, H * (1 - ((x - SPAN / 2) / (SPAN / 2))**2)))
    for s, y in (('S1', -Y_STR), ('S2', Y_STR)):
        for k in range(NPANEL + 1):
            add(f'{s}{k}', (k * dx, y, 0.0))
```

```
    E = []  # (n1, n2, nhóm, có lò xo xoay ở đầu nút)
    for side in 'AB':
        for k in range(NPANEL):
            E.append((f'L{side}{k}', f'L{side}{k+1}', 'lower', False))
        up = [f'L{side}0'] + [f'U{side}{k}' for k in range(1, NPANEL)] + [f'L{side}{NPANEL}']
        for a, b in zip(up[:-1], up[1:]):
            E.append((a, b, 'upper', False))                    # kể cả thanh xiên đầu dàn
        for k in range(1, NPANEL):
            E.append((f'U{side}{k}', f'L{side}{k}', 'vertical', True))
        for k in range(1, NPANEL // 2):                          # thanh xiên dốc về giữa nhịp
            E.append((f'U{side}{k}', f'L{side}{k+1}', 'diagonal', True))
            E.append((f'U{side}{NPANEL-k}', f'L{side}{NPANEL-k-1}', 'diagonal', True))
    for k in range(NPANEL + 1):                                  # dầm ngang
        E += [(f'LA{k}', f'S1{k}', 'transverse', False), (f'S1{k}', f'S2{k}', 'transverse', False),
              (f'S2{k}', f'LB{k}', 'transverse', False)]
    for s in ('S1', 'S2'):                                       # dầm dọc
        for k in range(NPANEL):
            E.append((f'{s}{k}', f'{s}{k+1}', 'stringer', False))
    for k in range(NPANEL):                                      # liên kết gió dưới (chữ X)
        E += [(f'LA{k}', f'LB{k+1}', 'lower_brace', False), (f'LB{k}', f'LA{k+1}', 'lower_brace', False)
    ↪ ]
    for k in range(1, NPANEL):                                   # thanh chống ngang trên tại mọi nút c
    ↪ ánh trên
        E.append((f'UA{k}', f'UB{k}', 'strut', False))
    for k in range(1, NPANEL - 1):                               # liên kết gió trên (chữ X) giữa các n
    ↪ út cánh trên
        E += [(f'UA{k}', f'UB{k+1}', 'upper_brace', False), (f'UB{k}', f'UA{k+1}', 'upper_brace', False)
    ↪ ]
    return np.array(nodes), tag, E

def frame_element(xi, xj, A, Iz, Iy, J, E, G, rho, extra_m=0.0):
    """Ma trận độ cứng và khối lượng nhất quán 12x12 trong hệ tổng thể."""
    d = xj - xi; L = np.linalg.norm(d); ex = d / L
    ref = np.array([0.0, 0.0, 1.0]) if abs(ex[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    if abs(ex[1]) > 0.9:                                         # thanh ngang cầu: trục y cục bộ theo X
    ↪  tổng thể
        ref = np.array([0.0, 0.0, 1.0])
    ey = np.cross(ref, ex); ey /= np.linalg.norm(ey)
    if abs(ex[1]) > 0.9:
        ey = np.array([1.0, 0.0, 0.0]) - ex * ex[0]; ey /= np.linalg.norm(ey)
    ez = np.cross(ex, ey)
    R = np.vstack([ex, ey, ez])
    # uốn trong mặt phẳng x-z cục bộ dùng I quanh trục y cục bộ (Iy, trục mạnh), x-y dùng Iz
    k = np.zeros((12, 12))
    EA, GJ = E * A / L, G * J / L
    k[0, 0] = k[6, 6] = EA; k[0, 6] = k[6, 0] = -EA
    k[3, 3] = k[9, 9] = GJ; k[3, 9] = k[9, 3] = -GJ
    def bend(I, i1, i2, sgn):
        c = E * I / L**3
        idx = [i1, i2, i1 + 6, i2 + 6]
        kb = c * np.array([[12, sgn * 6 * L, -12, sgn * 6 * L],
                           [sgn * 6 * L, 4 * L**2, -sgn * 6 * L, 2 * L**2],
                           [-12, -sgn * 6 * L, 12, -sgn * 6 * L],
                           [sgn * 6 * L, 2 * L**2, -sgn * 6 * L, 4 * L**2]])
        for a in range(4):
            for b in range(4):
                k[idx[a], idx[b]] += kb[a, b]
    bend(Iz, 1, 5, +1)       # v, theta_z
    bend(Iy, 2, 4, -1)       # w, theta_y
    mbar = rho * A + extra_m
    mm = np.zeros((12, 12))
    mm[0, 0] = mm[6, 6] = mbar * L / 3; mm[0, 6] = mm[6, 0] = mbar * L / 6
    Jp = rho * (Iy + Iz)
    mm[3, 3] = mm[9, 9] = Jp * L / 3; mm[3, 9] = mm[9, 3] = Jp * L / 6
    def bmass(i1, i2, sgn):
        c = mbar * L / 420
        idx = [i1, i2, i1 + 6, i2 + 6]
        mb = c * np.array([[156, sgn * 22 * L, 54, -sgn * 13 * L],
                           [sgn * 22 * L, 4 * L**2, sgn * 13 * L, -3 * L**2],
                           [54, sgn * 13 * L, 156, -sgn * 22 * L],
                           [-sgn * 13 * L, -3 * L**2, -sgn * 22 * L, 4 * L**2]])
        for a in range(4):
            for b in range(4):
                mm[idx[a], idx[b]] += mb[a, b]
    bmass(1, 5, +1); bmass(2, 4, -1)
```

```
    T = np.kron(np.eye(4), R)
    return T.T @ k @ T, T.T @ mm @ T

class NamO:
    def __init__(self):
        self.X, self.tag, self.E = geometry()
        # nút phụ cho lò xo xoay ở hai đầu thanh đứng và thanh xiên
        self.nodes = [tuple(p) for p in self.X]
        self.elems, self.springs = [], []
        for a, b, grp, semi in self.E:
            ia, ib = self.tag[a], self.tag[b]
            if semi:
                ja = len(self.nodes); self.nodes.append(self.nodes[ia])
                jb = len(self.nodes); self.nodes.append(self.nodes[ib])
                self.springs += [(ia, ja), (ib, jb)]
                ia, ib = ja, jb
            self.elems.append((ia, ib, grp))
        self.nodes = np.array(self.nodes)
        self.ndof = 6 * len(self.nodes)
        self.bearings = {'LA0': 'rocker', 'LB0': 'rocker', f'LA{NPANEL}': 'pin', f'LB{NPANEL}': 'pin'}

    def matrices(self, E, k_rot, k_v, k_h):
        n = self.ndof
        K, M = np.zeros((n, n)), np.zeros((n, n))
        G = E / (2 * 1.3)
        for ia, ib, grp in self.elems:
            A, Iz, Iy = SEC[grp]
            em = M_NS if grp == 'lower' else 0.0
            ke, me = frame_element(self.nodes[ia], self.nodes[ib], A, Iz, Iy, J_FACTOR * (Iy + Iz), E, G
    ↪ , RHO, em)
            dofs = np.r_[6 * ia:6 * ia + 6, 6 * ib:6 * ib + 6]
            K[np.ix_(dofs, dofs)] += ke; M[np.ix_(dofs, dofs)] += me
        big = 1e13
        for i, j in self.springs:                    # tịnh tiến liên kết cứng, xoay qua lò xo k_rot
            for c in range(6):
                kk = big if c < 3 else k_rot
                a, b = 6 * i + c, 6 * j + c
                K[a, a] += kk; K[b, b] += kk; K[a, b] -= kk; K[b, a] -= kk
        for name, kind in self.bearings.items():     # gối: lò xo đứng k_v, ngang cầu cứng, dọc: cứng (g
    ↪ ối cố định) / k_h (gối di động)
            i = self.tag[name]
            K[6 * i + 2, 6 * i + 2] += k_v
            K[6 * i + 1, 6 * i + 1] += big
            K[6 * i + 0, 6 * i + 0] += big if kind == 'pin' else k_h
        return K, M

    def modes(self, E=None, k_rot=None, k_v=None, k_h=None, nmodes=30):
        p = dict(NOMINAL); p.update({k: v for k, v in dict(E=E, k_rot=k_rot, k_v=k_v, k_h=k_h).items()
    ↪ if v is not None})
        K, M = self.matrices(**p)
        # khử bậc tự do xoay quanh trục thanh vô định ở nút phụ: thêm độ cứng rất nhỏ để ma trận xác định
        K += np.eye(self.ndof) * 1e-3
        w2, V = eigh(K, M, subset_by_index=[0, nmodes - 1])
        self.M_last = M
        return np.sqrt(np.abs(w2)) / (2 * np.pi), V

    def classify(self, V, M=None, cutoff=0.5):
        """Phân loại dạng tổng thể: L (ngang), V (uốn đứng), T (xoắn); dạng cục bộ (X) khi các nút
        cánh giãn chiếm < cutoff phần động năng của dạng."""
        out, count = [], {'L': 0, 'V': 0, 'T': 0, 'X': 0}
        chord = np.array([self.tag[n] for n in self.tag if n[0] in 'LU'])
        cd = np.r_[6 * chord, 6 * chord + 1, 6 * chord + 2]
        lo = {s: [self.tag[f'L{s}{k}'] for k in range(1, NPANEL)] for s in 'AB'}
        up = {s: [self.tag[f'U{s}{k}'] for k in range(1, NPANEL)] for s in 'AB'}
        for i in range(V.shape[1]):
            v = V[:, i]
            uy = lambda ids: v[6 * np.array(ids) + 1]
            uz = lambda ids: v[6 * np.array(ids) + 2]
            zA, zB = uz(lo['A']), uz(lo['B'])
            yL = (uy(lo['A']) + uy(lo['B'])) / 2; yU = (uy(up['A']) + uy(up['B'])) / 2
            vert_sym = np.linalg.norm(zA + zB) / 2
            vert_anti = np.linalg.norm(zA - zB) / 2
            lat_same = np.linalg.norm(yL + yU) / 2
            lat_opp = np.linalg.norm(yL - yU) / 2
            scores = {'V': vert_sym, 'L': lat_same, 'T': vert_anti + lat_opp}
```

```
            kind = max(scores, key=scores.get)
            if M is not None:
                tr = np.zeros(self.ndof, bool); tr[np.r_[0:self.ndof:6, 1:self.ndof:6, 2:self.ndof:6]] = True
                e_all = v[tr] @ (M[np.ix_(tr, tr)] @ v[tr])
                sel = np.zeros(self.ndof, bool); sel[cd] = True
                e_ch = v[sel] @ (M[np.ix_(sel, sel)] @ v[sel])
                if e_ch / e_all < cutoff:
                    kind = 'X'

            count[kind] += 1
            out.append(f'{kind}{count[kind]}')
        return out

    def paired(self, f, labels):
        """Ghép tần số tính toán với 5 dạng đo theo loại và bậc."""
        lookup = dict(zip(labels, f))
        return np.array([lookup.get(t, np.nan) for t in MEASURED_TYPE])

if __name__ == '__main__':
    import time
    mdl = NamO()
    print(f'{len(mdl.nodes)} nút ({len(mdl.X)} nút hình học), {len(mdl.elems)} phần tử, {len(mdl.springs)} lò xo xoay, {mdl.ndof} bậc tự do')
    t0 = time.time(); f, V = mdl.modes(); t1 = time.time()
    lab = mdl.classify(V, mdl.M_last)
    for i, (fi, li) in enumerate(zip(f, lab)):
        print(f'  dạng {i+1:2d}: {fi:6.3f} Hz  {li}')
    fp = mdl.paired(f, lab)
    print('ghép với đo:', np.round(fp, 3), ' sai số %:', np.round(100 * (fp - MEASURED) / MEASURED, 1), f'({t1-t0:.2f} s)')
    tot_mass = sum(RHO * SEC[g][0] * np.linalg.norm(mdl.nodes[a] - mdl.nodes[b]) + (M_NS * np.linalg.norm(mdl.nodes[a] - mdl.nodes[b]) if g == 'lower' else 0) for a, b, g in mdl.elems)
    print(f'khối lượng nhịp: {tot_mass/1e3:.1f} t')
```

Code 9.3 (tiếp): Hiệu chỉnh mô hình bằng bình phương tối thiểu có cận và GA

```
"""Code 9.3 (phần hiệu chỉnh): bình phương tối thiểu có cận (Gauss-Newton/vùng tin cậy) và GA."""
import json, time
import numpy as np
from scipy.optimize import least_squares
import namo as N

MDL = N.NamO()
W = np.ones(5)                                          # trọng số các dạng

# Bộ tham số 1 (như bài báo): E, k_rot, k_v, k_h; Bộ 2 (chọn theo độ nhạy): E, k_rot, m_ns, alpha_lb
SETS = {
    'paper': dict(names=['E (GPa)', 'log10 k_rot', 'k_v (1e10 N/m)', 'k_h (1e7 N/m)'],
                  lo=np.array([190, 5.0, 1.0, 1.0]), hi=np.array([220, 9.0, 2.0, 2.0]),
                  x0=np.array([200, np.log10(8e5), 1.3, 1.5])),
    'sens': dict(names=['E (GPa)', 'log10 k_rot', 'm_ns (kg/m)', 'alpha_lb'],
                 lo=np.array([190, 5.0, 100, 0.5]), hi=np.array([220, 9.0, 350, 3.0]),
                 x0=np.array([200, np.log10(8e5), 247.2, 1.0])),
}

def frequencies(theta, which):
    if which == 'paper':
        E, lk, kv, kh = theta
        f, V = MDL.modes(E=E * 1e9, k_rot=10**lk, k_v=kv * 1e10, k_h=kh * 1e7)
    else:
        E, lk, mns, alb = theta
        old_m, old_b = N.M_NS, N.SEC['lower_brace']
        N.M_NS = mns
        A, Iz, Iy = old_b
        N.SEC['lower_brace'] = (alb * A, alb * Iz, alb * Iy)
        try:
            f, V = MDL.modes(E=E * 1e9, k_rot=10**lk)
        finally:
            N.M_NS, N.SEC['lower_brace'] = old_m, old_b
    return MDL.paired(f, MDL.classify(V, MDL.M_last))

def residual(theta, which):
    fp = frequencies(theta, which)
    r = W * (fp - N.MEASURED) / N.MEASURED
```

```
    return np.where(np.isnan(r), 1.0, r)                # dạng không tìm thấy -> phạt

def objective(theta, which):
    return float(np.sum(residual(theta, which)**2))

def solve_ls(which, x0):
    s = SETS[which]
    scale = s['hi'] - s['lo']
    res = least_squares(lambda z: residual(s['lo'] + z * scale, which), (x0 - s['lo']) / scale,
                        bounds=(0, 1), diff_step=1e-3, method='trf', xtol=1e-8, ftol=1e-10)
    th = s['lo'] + res.x * scale
    return th, objective(th, which), res.nfev

def ga(which, pop=20, gens=25, seed=0):
    s = SETS[which]; rng = np.random.default_rng(seed)
    lo, hi = s['lo'], s['hi']
    X = lo + rng.random((pop, lo.size)) * (hi - lo)
    f = np.array([objective(x, which) for x in X]); hist = [f.min()]
    for _ in range(gens):
        e, fe = X[f.argmin()].copy(), f.min()
        def pick():
            i, j = rng.integers(pop, size=2); return X[i] if f[i] < f[j] else X[j]
        kids = []
        while len(kids) < pop - 1:
            p1, p2 = pick(), pick(); w = rng.random(lo.size)
            c = w * p1 + (1 - w) * p2
            mask = rng.random(lo.size) < 0.25
            c[mask] += rng.normal(0, 0.1, mask.sum()) * (hi - lo)[mask]
            kids.append(np.clip(c, lo, hi))
        X = np.vstack([e, kids]); f = np.r_[fe, [objective(x, which) for x in X[1:]]]
        hist.append(f.min())
    return X[f.argmin()], f.min(), hist, pop * (gens + 1)

if __name__ == '__main__':
    out = {}
    for which in ('paper', 'sens'):
        s = SETS[which]
        f0 = frequencies(s['x0'], which)
        t0 = time.time()
        th_ls, J_ls, nfev = solve_ls(which, s['x0'])
        t_ls = time.time() - t0
        t0 = time.time()
        th_ga, J_ga, hist, nfev_ga = ga(which)
        t_ga = time.time() - t0
        f_ls, f_ga = frequencies(th_ls, which), frequencies(th_ga, which)
        print(f'== Bộ tham số "{which}": J0 = {objective(s["x0"], which):.4f}')
        print(f'   ban đầu  f = {np.round(f0, 3)}  sai số % = {np.round(100*(f0-N.MEASURED)/N.MEASURED,
    ↪ 1)}')
        print(f'   BPTT     f = {np.round(f_ls, 3)}  sai số % = {np.round(100*(f_ls-N.MEASURED)/N.
    ↪ MEASURED, 1)}  J = {J_ls:.4f}, {nfev} lượt, {t_ls:.0f} s')
        print(f'            theta = {dict(zip(s["names"], np.round(th_ls, 3)))}')
        print(f'   GA       f = {np.round(f_ga, 3)}  sai số % = {np.round(100*(f_ga-N.MEASURED)/N.
    ↪ MEASURED, 1)}  J = {J_ga:.4f}, {nfev_ga} lượt, {t_ga:.0f} s')
        print(f'            theta = {dict(zip(s["names"], np.round(th_ga, 3)))}', flush=True)
        out[which] = dict(names=s['names'], x0=s['x0'].tolist(), lo=s['lo'].tolist(), hi=s['hi'].tolist
    ↪ (),
                          f0=f0.tolist(), ls=th_ls.tolist(), f_ls=f_ls.tolist(), J_ls=J_ls, nfev_ls=int(
    ↪ nfev),
                          ga=th_ga.tolist(), f_ga=f_ga.tolist(), J_ga=float(J_ga), ga_hist=list(map(
    ↪ float, hist)),
                          nfev_ga=int(nfev_ga), J0=objective(s['x0'], which))
    out['measured'] = N.MEASURED.tolist()
    json.dump(out, open('namo_results.json', 'w'), indent=1)
```